Руководство пользователя панели 3X-UI

🇬🇧 English · 🇷🇺 Русский

Версия 3X-UI: 3.11.0. Руководство составлено по этой версии и актуально для неё. Сводка изменений 3.11.0 относительно 3.10.2 — в разделе «Что нового в 3.11.0».

> Подробное русскоязычное руководство по веб-панели **3X-UI** (управление Xray-core): функции, настройка и эксплуатация, с разбором каждого поля и переключателя в интерфейсе.
>
> Названия и подписи соответствуют интерфейсу панели; русские термины приведены так, как они отображаются в интерфейсе. Слова *inbound* / *outbound* не переводятся.

Содержание

- Что нового в 3.11.0
- 1. Введение, требования и установка
  - 1.1. Что такое 3X-UI
  - 1.2. Поддерживаемые операционные системы и архитектуры
  - 1.3. Способы установки
  - 1.4. Первый запуск и учётные данные по умолчанию
  - 1.5. Расположение файлов
  - 1.6. Команда управления `x-ui` (меню скрипта)
  - 1.7. Подкоманды `x-ui` (без интерактивного меню)
  - 1.8. Миграция SQLite → PostgreSQL
- 2. Вход в панель и безопасность доступа
  - 2.1. Форма входа
  - 2.2. Двухфакторная аутентификация (2FA / TOTP)
  - 2.3. Ограничение попыток входа (login limiter / защита от перебора)
  - 2.4. Смена логина и пароля администратора
  - 2.5. Секретный путь (URI-путь / webBasePath) и порт панели
  - 2.6. Время жизни сессии (таймаут)
  - 2.7. LDAP (синхронизация и аутентификация)
- 3. Обзор / Дашборд
  - 3.1. Общие принципы сбора данных
  - 3.2. ЦП (CPU)
  - 3.3. Память (RAM)
  - 3.4. Подкачка (Swap)
  - 3.5. Диск (Storage)
  - 3.6. Время работы системы (Uptime)
  - 3.7. Загрузка системы (Load average)
  - 3.8. Сеть: скорость и общий объём трафика
  - 3.9. IP-адреса сервера
  - 3.10. TCP/UDP соединения
  - 3.11. Статус Xray и управление процессом
  - 3.12. Обновление панели (3X-UI)
  - 3.13. Обновление гео-файлов (GeoIP / GeoSite)
  - 3.14. Резервное копирование и восстановление базы данных
  - 3.15. Дополнительные элементы интерфейса
  - 3.16. Что изменилось на дашборде в 3.7.0
  - 3.17. Командная строка (Ctrl+K)
- 4. Inbounds: создание и общие параметры
  - 4.1. Общие поля формы
  - 4.2. Sniffing (Сниффинг)
  - 4.3. Allocate (стратегия распределения портов)
  - 4.4. External Proxy (Внешний прокси)
  - 4.5. Fallbacks (Fallback'и)
  - 4.6. Периодический сброс трафика

    - 4.7. JSON входящего (advanced)
    - 4.8. Действия с inbound: QR / Edit / Reset / Delete и статистика
- 5. Протоколы
    - 5.1. Список поддерживаемых протоколов
    - 5.2. Какие протоколы поддерживают TLS / REALITY / транспорт
    - 5.3. VLESS
    - 5.4. VMess
    - 5.5. Trojan
    - 5.6. Shadowsocks
    - 5.7. Dokodemo-door / Tunnel (прозрачный форвардер)
    - 5.8. SOCKS / HTTP (протокол `mixed`)
    - 5.9. WireGuard (inbound)
    - 5.10. AmneziaWG
    - 5.11. Hysteria (по умолчанию v2)
    - 5.12. MTProto (прокси для Telegram)
    - 5.13. TUIC v5
    - 5.14. Краткая шпаргалка по выбору протокола
    - 5.15. MASQUE
- 6. Транспорт (Stream Settings)
    - 6.1. Выбор сети передачи
    - 6.2. RAW / TCP (`tcpSettings`)
    - 6.3. mKCP (`kcpSettings`)
    - 6.4. WebSocket (`wsSettings`)
    - 6.5. gRPC (`grpcSettings`)
    - 6.6. HTTPUpgrade (`httpupgradeSettings`)
    - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
    - 6.8. Транспорт Hysteria (`hysteriaSettings`)
    - 6.9. Сопутствующие параметры
    - 6.10. XDRIVE (`xdriveSettings`)
    - 6.11. Транспорт MASQUE (`masqueSettings`)
    - 6.12. FinalMask: обфускация транспортного уровня
- 7. Безопасность соединения: TLS, XTLS и REALITY
    - 7.1. В чём разница: TLS vs XTLS vs REALITY
    - 7.2. Режим «Нет» (`none`)
    - 7.3. Режим TLS
    - 7.4. Режим REALITY
    - 7.5. Практические рекомендации по настройке
- 8. Клиенты
    - 8.1. Поля клиента
    - 8.2. Привязка к inbound
    - 8.3. Операции над клиентом
    - 8.4. Массовые операции
    - 8.5. Поиск, фильтры и сортировка
    - 8.6. Значки и статусы
- 9. Группы клиентов
    - 9.1. Что такое группа клиентов и зачем она нужна

    - 9.2. Связь группы с клиентами, inbound, нодами и протоколами
    - 9.3. Справочник групп и «пустые» группы
    - 9.4. Поля и столбцы группы
    - 9.5. Создание группы
    - 9.6. Переименование группы
    - 9.7. Добавление клиентов в группу
    - 9.8. Удаление клиентов из группы (без удаления самих клиентов)
    - 9.9. Сброс трафика группы
    - 9.10. Удаление группы и удаление клиентов группы
    - 9.11. Связь со страницей «Клиенты»
    - 9.12. Сводка эндпоинтов API
    - 9.13. Трафик по группе
- 10. Подписки (Subscription)
    - 10.1. Что такое subId и как формируется ссылка
    - 10.2. Настройки сервера подписок
    - 10.3. Балансировщики подписки (3.7.0)
    - 10.4. Форматы вывода
    - 10.5. Страница информации о подписке и QR-коды
    - 10.6. Кастомные шаблоны страницы подписки
    - 10.7. Интеграция с клиентом Happ
- 11. Xray: маршрутизация, outbounds, DNS и расширения
    - 11.1. Структура редактора: вкладки/режимы
    - 11.2. Основные настройки (General)
    - 11.3. Правила маршрутизации (routing)
    - 11.4. Outbounds (исходящие подключения)
    - 11.5. Балансировщики (Balancers)
    - 11.6. DNS
    - 11.7. Fake DNS
    - 11.8. WireGuard / WARP / NordVPN
    - 11.9. Reverse-прокси и TUN
    - 11.10. Логи и статистика (Stats, metrics)
    - 11.11. Сохранение, перезапуск и автоматические преобразования
    - 11.12. Outbound из подписки (с авто-обновлением)
    - 11.13. Ротация IP в WARP
    - 11.14. Lua-скрипты маршрутизации и DNS
- 12. Узлы (мультипанель, master/slave)
    - 12.1. Сводка вверху списка
    - 12.2. Добавление и редактирование узла
    - 12.3. Проверка TLS (для https-узлов)
    - 12.4. Что показывается по каждому узлу
    - 12.5. Действия над узлом
    - 12.6. История метрик
    - 12.7. Как синхронизируются inbounds и клиенты
    - 12.8. Цепочки узлов (подузлы / транзитивные узлы)
    - 12.9. Узлы: новое в 3.7.0
    - 12.10. Исправления 3.5.0 (стабильность узлов)

    - 12.11. Узлы: новое в 3.3.0
- 13. Настройки панели
    - 13.1. Сохранение и перезапуск панели
    - 13.2. Общие настройки (вкладка «Панель» / *General*)
    - 13.3. Доступ к панели: IP, порт, путь, домен, сертификат
    - 13.4. Сессия, прокси панели и доверенные прокси (вкладка «Прокси и сервер» / *Proxy and Server*)
    - 13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)
    - 13.6. Дата и время (вкладка «Дата и время» / *Date and Time*)
    - 13.7. Внешний трафик и поведение Xray (вкладка «Внешний трафик» / *External Traffic*)
    - 13.8. Прочее: шаблон конфигурации Xray и проверочный URL
    - 13.9. Учётная запись администратора и API-токены
    - 13.10. Изменения API в 3.3.0 (важно для интеграций)
    - 13.11. Discord-бот (вкладка «Discord бот» / *Discord Bot*)
- 14. Telegram-бот
    - 14.1. Включение и настройка бота
    - 14.2. Главное меню и кнопки
    - 14.3. Команды бота
    - 14.4. Управление клиентом (только администратор)
    - 14.5. Уведомления и отчёты
    - 14.6. Бэкап и логи
    - 14.7. Особенности работы
    - 14.8. Discord-бот
- 15. Гео-базы (geoip / geosite и кастомные)
    - 15.1. Что такое geoip.dat и geosite.dat
    - 15.2. Стандартные гео-файлы и их обновление
    - 15.3. Авто-обновление гео-данных средствами Xray (Geodata Auto-Update)
    - 15.4. Просмотр содержимого гео-баз (3.7.0)
    - 15.5. Валидация и ограничения
    - 15.6. Автопроверка при старте панели
    - 15.7. Использование гео-баз в правилах маршрутизации
- 16. Эксплуатация: бэкапы, логи, обновление, CLI
    - 16.1. Резервное копирование и восстановление базы данных
    - 16.2. Просмотр логов
    - 16.3. Уровень и настройка логирования Xray
    - 16.4. Управление Xray: остановка и перезапуск
    - 16.5. Перезапуск и обновление панели
    - 16.6. Периодические задачи (cron)
    - 16.7. Консольное меню и CLI (`x-ui`)
    - 16.8. Что изменилось в эксплуатации в 3.7.0
    - 16.9. Удаление панели
    - 16.10. Команда `x-ui migrateDB`

Что нового в 3.11.0

Версия 3.11.0 — крупный выпуск после 3.10.2. Главное: **Telegram-бот переписан** — один «живой» экран вместо потока сообщений, меню из четырёх разделов, списки через inline-поиск, уведомления-карточки, темы форума (раздел 14); **AmneziaWG** получил режим ядра и keepalive диапазоном; у **хостов подписки** — пул SNI, случайные IP из CIDR, ручной порядок и привязка клиента к своим хостам; в **FinalMask** — маска Filter, Lua-скрипт у Noise, DoH у xDNS и пресеты фрагментации tlshello; у **TLS** — пресеты наборов шифров и помощник динамического ECH; **XDRIVE** теперь только VLESS с шифрованием; новая область API-токена `node-admin` и порог уведомлений о недоступности узла. Ядро Xray — **26.10.21**, интерфейс — на 14 языках (добавлен словацкий). Ниже — изменения относительно 3.10.2 по разделам руководства.

Изменения в разделе 1 — Введение, требования и установка

- **Первый пользователь со случайными данными** (1.4): на новой базе (Docker, ручной запуск) логин и пароль создаются при старте сервера и один раз печатаются в лог; фиксированного `admin` / `admin` больше нет. `x-ui setting -show` показывает `hasDefaultCredential: true`, пока пользователя нет.
- **Образы Docker для ARM и 386** (1.3): в образ попадают правильные бинарники Xray и `mtg` — ARMv6 получает ARMv6-сборку, 386 — 32-битную; неизвестная архитектура останавливает сборку.
- **Ядро Xray 26.10.21** (1.3): 26.10.10 отозван в апстриме.
- **Словацкий интерфейс** (1.1): панель доступна на 14 языках, добавлен `sk-SK`.

Изменения в разделе 2 — Вход в панель и безопасность доступа

- **Адрес клиента за прокси** (2.3): `X-Forwarded-For` читается справа налево с пропуском доверенных прокси, `X-Real-IP` принимается только при совпадении; прокси должен перезаписывать `X-Real-IP` (для Caddy — `header_up X-Real-IP {remote_host}`). Иначе при расхождении ключом блокировки входа станет адрес прокси.
- **Отказ CSRF виден в логе** (2.1): запрос с неверным токеном получает 403 и оставляет предупреждение с методом, путём и адресом клиента (#6692).

Изменения в разделе 3 — Обзор / Дашборд

- **Метрики LXC-контейнера** (3.1): в LXC с LXCFS ЦП, подкачка и время работы берутся из контейнера, а не с хоста; неизмеримые показатели выводятся прочерком, а не нулём (#6920).
- **Публичные IP обновляются** (3.9): адреса перепроверяются раз в 5 минут, неудачные запросы повторяются с нарастающей паузой, а временный сбой не стирает последний известный адрес (#6933).
- **Обновление панели по API** (3.12): `POST /panel/api/server/updatePanel` доступен токенам `admin` и новой области `node-admin`.
- **Доступность** (3.15): панель проверена на работу со скринридерами (#6907).

Изменения в разделе 4 — Inbounds: создание и общие параметры

- **Fallback: заголовок PROXY protocol берётся от дочернего inbound'а** (#6835, #6880; 4.5): если у строки fallback выбран дочерний inbound, пустой **Dest** и не задан **Xver**, а дочерний inbound принимает PROXY protocol (`sockopt` или транспорт RAW/WebSocket/HTTPUpgrade), в конфиг попадает `xver 1`. Раньше включение Proxy Protocol у дочернего inbound'а ломало fallback: мастер не слал заголовок, и дочерний

отвергал все соединения. Правка адреса, порта или флага PROXY у дочернего inbound'а пересобирает зависящие от него мастера.
- **Периодический сброс включает обратно отключённых по квоте** (#6833, #6834; 4.6): клиенты inbound'а, которых отключила исчерпанная квота, и сам inbound, выключенный собственным лимитом трафика, после сброса возвращаются в работу. Раньше клиент показывал нулевой расход, но оставался вне работающего ядра, а inbound оставался выключенным навсегда. Выключенный вручную или по сроку inbound остаётся выключенным; inbound на узле автоматически не включается.
- **Расширенный JSON-редактор сохраняет форматирование** (#6857; 4.7): перевод строки или пробелы в редакторе **Всё** больше не заменяются собственным форматированием панели и не сдвигают курсор.
- **Удаление inbound'а и очистка исчерпавших клиентов идут через общую очередь записи трафика** (#6912; 4.8): на PostgreSQL исчезает взаимная блокировка с опросом трафика, а удаляемый во время правки inbound больше не «воскресает».

Изменения в разделе 5 — Протоколы

- **AmneziaWG: опциональный режим ядра** (#6877; 5.10): переключатель **«Режим ядра»** переводит отдельный inbound со встроенного движка на модуль ядра Linux (`xui-awg<id>` , NAT в nftables, трафик мимо Xray). Нужны root, модуль `amneziawg` , `awg` , `ip` , `nft` ; панель показывает причины, по которым хост не готов, и команду установки `x-ui install-awg-kernel` . Не поддерживаются маршрутизация и сниффинг Xray, IPv6 пиров, проброс портов и закреплённый IP прослушивания; в Docker режим недоступен. Трафик и квоты учитываются так же, как у других протоколов; при штатной остановке панели интерфейсы убираются, после аварии подхватываются.
- **AmneziaWG: диапазон persistent keepalive** (#6881; 5.10): в **Keepalive (секунды)** клиента можно указать диапазон `15-25` (0–65535). `.conf` и `vpn://` несут диапазон, Clash/Mihomo получает нижнюю границу, обычный WireGuard остаётся на числе. Старым узлам диапазоны не отправляйте.
- **AmneziaWG: быстрее загрузка клиента через встроенный движок и безопасный запасной порт моста** (5.10): приёмный TCP-буфер подстраивается до 8 МБ (при RTT 50 мс одиночный поток вырос примерно с 310 до 500 Мбит/с), а если порт `64900` локального SOCKS5-моста занят, запасной порт выбирается ниже 62789 и не занимает порты самой панели.
- **Конфиги WireGuard/AmneziaWG проверяются построчно** (5.9): значение с управляющим символом делает `.conf` пустым, профиль `vpn://` с хуками wg-quick отклоняется, а форма inbound проверяет **Allowed IPs** клиентов при создании и изменении.
- **TUIC: маршрутизация Xray по пользователю** (#6875; 5.13): SOCKS5-мост различает клиентов по email, поэтому правила `user` срабатывают для TCP и UDP. Добавление, удаление и переименование клиента заменяют только мост этого inbound'а, без перезапуска Xray.

Изменения в разделе 6 — Транспорт (Stream Settings)

- **XDRIVE: только VLESS с шифрованием, имя сервиса `google-drive`** (Xray-core#7182; 6.10): под требования xray-core 26.10.17 XDRIVE предлагается только у VLESS, сохранение без **«Расшифрование»** отклоняется, flow клиента на XDRIVE не выдаётся (и недоступен в форме, 5.3), в outbound выбор XDRIVE включает **Mux**. Сохранённые ранее inbound'ы и outbound'ы, которые ядро теперь отклоняет, не попадают в конфиг (с предупреждением в логе) и не останавливают остальные. Сервис `Google Drive` теперь называется `google-drive` (старое имя читается как псевдоним).

- **XMUX клиента у XDRIVE и MASQUE** (6.10, 6.11): необязательный объект `xmux` панель больше не отбрасывает при сохранении inbound'а, а JSON-подписка передаёт его клиентам; в форме поля нет — задаётся в расширенном JSON.
- **FinalMask: маска Filter (source IPs)** (Xray-core#7206; 6.12): в редакторе inbound (TCP и UDP) списки **Allowed Source IPs** и **Blocked Source IPs** отсекают подключения по IP источника; ссылки и JSON-подписка эту маску не передают.
- **FinalMask: пресеты фрагментации tlshello** (6.12): в клиентских редакторах (outbound, хосты, Final Mask JSON-подписки) список **«Пресеты»** заменяет TCP-маски готовыми `tlshello-0-len` или `tlshello-6-98`; проверка длин Fragment допускает ноль в элементах `lengths`, кроме последнего.
- **FinalMask: скрипт Lua у Noise и DoH у xDNS** (Xray-core#7124, Xray-core#7140; 6.12): поле **Script** заменяет список шума, а резолвер xDNS принимает `https://`-адреса с полями **Workers**, **DoH TLS** и **DoH QUIC** — раньше сохранение теряло всё, кроме `addrs`.

Изменения в разделе 7 — Безопасность соединения: TLS, XTLS и REALITY

- **Наборы шифров: пресеты и предупреждение об отпечатке** (7.3): в поле «Наборы шифров» появилась группа «Пресеты» (`semi-python`, `real-firefox`) — готовые упорядоченные списки клиентских наборов; пресет заменяет список целиком. Если наборы заданы при отпечатке uTLS, отличном от `unsafe`, форма предупреждает: клиенты применяют наборы только с `unsafe` и игнорируют их при любом другом отпечатке.
- **Помощник динамического ECH** (#6869, 7.3): под полем «Список конфигураций ECH» (бывшее «ECH config») собирает значение `домен+URL DoH` из готовых комбинаций или из своего домена и резолвера, принимает только схемы `https`, `h2c` и `udp`, а перед заменой локально созданного ключа ECH спрашивает подтверждение. Форма также объясняет, что локальные ключи ECH не работают за CDN (Cloudflare за оранжевым облаком) — там нужна динамическая форма. Тот же помощник есть в хостах (10.4).
- **Исправлено описание подсказки ML-KEM в ссылках REALITY** (7.4): ссылка `vless://` несёт `support-x25519mlkem768=true` (раньше раздел утверждал, что параметра в ссылке нет); у хоста с переключателем «Mihomo X25519» подсказка убирается.

Изменения в разделе 8 — Клиенты

- **Клиент привязывается к своим хостам** (8.1): поле **Переопределение правила хоста** во вкладке **Основные** ограничивает подписки клиента выбранными управляемыми хостами. Если привязанных хостов не осталось, клиент в этом формате не получает ссылок, но остаётся в учёте трафика (#6620).
- **AmneziaWG: keepalive диапазоном** (8.1): в **Keepalive (секунды)** можно ввести `15-25`; `.conf` и `vpn://` несут диапазон, Clash/Mihomo получает нижнюю границу (#6881).
- **IP клиентов пишутся и без Fail2ban** (8.1): при `XUI_ENABLE_FAIL2BAN=false` отключается только блокировка по лимиту, **Журнал IP** остаётся актуальным (#6830).
- **Поиск клиента по ID подписки через API** (8.3): `GET /panel/api/clients/get/subId/:subId` (#6908).
- **Сброс трафика возвращает клиентов в работу** (8.3): периодический сброс inbound'а и **Сбросить трафик всех клиентов** включают обратно клиентов, отключённых по квоте; inbound, выключенный своим лимитом, включается периодическим сбросом (#6833, #6834).

Изменения в разделе 10 — Подписки (Subscription)

- **Хост: пул SNI** (#6923, 10.4): в поле SNI хоста можно задать несколько значений — каждая конфигурация получает случайный SNI из пула, а переключатель «Один случайный SNI для всех конфигураций» выбирает один SNI на весь хост в каждом запросе подписки. Не нужен отдельный хост на каждый домен; хост с одним SNI выдаётся как раньше.
- **Хост: случайные IP из CIDR-диапазона** (#6784, 10.4): адресом хоста может быть CIDR (IPv4 или IPv6), а поле «Количество случайных IP» (1–100, по умолчанию 1) задаёт, сколько уникальных адресов диапазона получит каждый запрос подписки. Диапазон разворачивается во всех форматах, в обоих порядках подписки, в QR-кодах и экспорте конфигов туннелей.
- **Порядок хостов в подписке и перетаскивание** (#6894, #6895, 10.2): новая настройка «Порядок узлов подписки» (`subHostSortMode`) с режимом «Порядок хостов» выстраивает raw-, JSON- и Clash-подписки по порядку страницы Hosts сквозь все inbound (по умолчанию порядок прежний — внутри каждого inbound). Строки таблицы хостов можно перетаскивать, а не только двигать кнопками «Вверх»/«Вниз».
- **Привязка клиента к хостам** (#6620, 10.4): клиент получает в подписках только привязанные к нему хосты (поле «Переопределение правила хоста» в форме клиента, API: `clientHostRuleIds`). Привязка закрывается при ошибке — клиент без подходящих хостов не получает ссылок, но остаётся в учёте трафика. Удаление хоста закрывает привязку только на его inbound'е, а не на всех inbound'ах клиента.
- **Внешний прокси inbound'а переносится на импортированные хосты** (#6831, 10.4): правка `externalProxy` при обновлении inbound'а теперь доходит до подписки — раньше обновление возвращало 200, но подписки продолжали отдавать прежние адрес, порт и защиту. Хосты, которые администратор менял, выключал, удалял или делал общими, не затрагиваются.
- **Наборы шифров в ссылках: `cs` и `fp=unsafe`** (#6825, 10.4): share-ссылки хоста или TLS-inbound несут параметр `cs` , а при пустом отпечатке к ним добавляется `fp=unsafe` — иначе наборы не действуют в v2rayN и других клиентах на Xray. Раньше наборы шифров доходили только до JSON- и Clash-подписок.
- **Переключатель хоста «Mihomo X25519» убирает ML-KEM** (#6803, 10.4): для REALITY-хоста Clash-прокси получает `support-x25519mlkem768: false` , а ссылка `vless://` — без подсказки. Раньше переключатель сохранялся, но не действовал, а все REALITY-прокси несли `true` ; сохранённые значения при обновлении один раз сброшены, чтобы подписки не поменялись молча. Отпечаток REALITY-хоста теперь доходит до Clash и JSON, а пустые SNI, отпечаток, Host и путь одного хоста не наследуются от другого.
- **Shadowsocks с HTTP-заголовком TCP в Clash/Mihomo** (#6838, 10.4): такой узел выдаётся с `plugin: obfs` (`mode: http` и заголовок Host), если заголовок принимает путь `/` ; раньше он пропускался.
- **Числа XMUX в Clash** (#6843, 10.4): числовые параметры XMUX inbound'а, например `maxConnections` (в панели по умолчанию `3`), теперь доходят до `reuse-settings` — раньше передавались только строковые значения.
- **DNS из удалённых правил Clash** (#6868, 10.4): новый переключатель «DNS из удалённых правил» (`subClashRemoteDNS` , по умолчанию выключен) переносит в профиль разделы `dns` и `sniffer` шаблона, на который указывает ссылка в глобальных правилах; включайте только для шаблона, которому доверяете.
- **Запрет кэширования на всех маршрутах подписки** (#6862, 10.4): raw-, JSON-, Clash-, `/mihomo/` - и `/clash-legacy/` -ответы отдаются с `Cache-Control: no-cache, no-store, must-revalidate` , поэтому CDN или прокси перед сервером подписок не отдают устаревшие трафик, срок и правила. Кроме того, настройки inbound'ов без списка клиентов кэшируются на 10 с, и на установках со 100 тысячами клиентов запрос подписки читает их в десятки раз быстрее (#6758).

Изменения в разделе 11 — Xray: маршрутизация, outbounds, DNS и расширения

- **Lua-скрипты: что панель сохраняет** (11.14): шаблон с `routing.script` и `dns.script` вместе с резервными/параллельными переключателями или серверами с доменами отклоняется при сохранении; такие скрипты в старых шаблонах при сборке конфигурации отбрасываются с предупреждением (xray-core v26.10.21; Xray-core#7124, Xray-core#7136).
- **XDRIVE в outbound — только VLESS** (11.4): выбор XDRIVE включает Mux, `flow` не пишется, непринимаемые ядром outbound'ы пропускаются с предупреждением; сервис хранилища называется `google-drive` (Xray-core#7182).
- **Частный DoH-сервер работает** (11.6): авто-правило для приватного DNS-сервера открывает порт из URL (443 по умолчанию) у `https://` и `h2c://`, а не 53 (#6918).
- **Проверка TCP понимает IPv6** (11.4): outbound с IPv6-адресом без скобок больше не помечается недоступным (#6850).

Изменения в разделе 12 — Узлы (мультипанель, master/slave)

- **Токен «Администратор узла»** (12.2): область `node-admin` включает права `node-sync` и позволяет мастеру обновлять панель узла; `node-sync` обновление не разрешает (#6666).
- **Порог уведомлений о недоступности узла** (12.7): `node.down` уходит после заданного числа неудачных проверок подряд, по умолчанию после одной (#6768).
- **Отключение узлом не залипает** (12.7): если мастер разрешает клиента, случайное «выключен» от узла не гоняет клиента между состояниями (#6870).
- **Пересозданный клиент не пропадает с узла** (12.7): повторное добавление inbound'а узла с только что удалённым email снимает пометку удаления.

Изменения в разделе 13 — Настройки панели

- **Прокси и адрес клиента** (13.4): описано, как панель выбирает адрес клиента из `X-Forwarded-For` и `X-Real-IP`, и добавлен пример Caddy с `header_up X-Real-IP {remote_host}`.
- **Тема форума для уведомлений бота** (13.5): в «User ID администратора бота» можно указать `chatId:topicId`, и уведомления пойдут в указанную тему форум-супергруппы.
- **Предупреждения о настройке бота** (13.5): тестовое сообщение проверяет возможности бота и предупреждает, если выключен inline-режим или включён режим приватности.
- **Порог для уведомления об узле** (13.5): `nodeDownThreshold` (по умолчанию 1) откладывает `node.down` до заданного числа неудачных проверок подряд (#6768).
- **Область токена `node-admin`** (13.9): токен узла, которому разрешено дополнительно обновлять панель узла; поиск токена по индексу снижает нагрузку (#6666, #6915).

Изменения в разделе 14 — Telegram-бот

- **Бот перестроен вокруг одного «живого» экрана** (14.2): нажатия правят одно сообщение с картинкой и клавиатурой, над текстом стоит путь «Panel › …», у каждого экрана есть «⬅️ Back», длинный текст листается страницами («Page X of Y», лимит подписи 1024 символа). Главное меню администратора — четыре раздела (🖥 Сервер, 👥 Клиенты, 🚦 Трафик, ⚙️ Обслуживание) и «🔄 Обновить» (#6760).
- **Списки inbound и клиентов открываются поиском в inline-режиме** (14.2): кнопки «🔍 Find inbound…» / «🔍 Find client…». Inline-режим нужно один раз включить в @BotFather командой `/setinline` (14.1);

пока он выключен, бот показывает «⚠️ Inline mode is off» с инструкцией и кнопкой «Check again», а настройки панели после кнопки «Отправить тестовое сообщение» — предупреждение «Inline mode is disabled» (запрос `POST /panel/api/setting/tgBotCapabilities`) (#6760).

- **Бот работает только в личных сообщениях** (14.1): в группах и каналах он отвечает «🤖 Бот работает только в личных сообщениях…», а режим приватности группы теперь лишь информационное предупреждение «Режим приватности включён» (#6760).
- **Меню «/» содержит только `/start`; кнопка «Команды» убрана** (14.3): остальные команды работают, если набрать их вручную. Действия сгруппированы по разделам, добавлены кнопки ♻️ Restart Xray и 📣 Рассылка (#6760).
- **Уведомления — живые карточки** (14.5): повтор события за 30 минут правит одно сообщение и считает «🔁 repeated N× · last HH:MM:SS», у каждого сообщения есть «🙈 Hide»; отдельный ограничитель частоты убран; успешный и неудачный входы — разные карточки. Периодический отчёт заменяет предыдущий вместо накопления (#6760).
- **Мастер «➕ Новый клиент» начинается с черновика** (14.4): inbound выбираются кнопкой «Привязать инбаунд (N)» со списком отметок и «✅ Готово», можно привязать клиента к нескольким inbound сразу; после создания открывается карточка клиента (#6760).
- **QR-коды — одна картинка-лист** с нумерованной легендой «◼ QR-коды:» вместо нескольких сообщений (14.2); бэкап БД приходит одним сообщением с подписью и `config.json` (14.6) (#6760).
- **Уведомления администратору можно направить в тему форума** (14.1): в поле «User ID администратора бота» к ID чата добавляется `:topicId`, например `-1001234567890:42`. Тема 1 равна General, две разные темы одного чата — ошибка (#6879).
- **Поиск клиента по Telegram ID идёт по индексированной таблице клиентов** (14.3): бот быстрее определяет уровень доступа на панелях с большим числом клиентов (#6887).
- **Ограничения 3.11.0** (14.4, 14.6): ссылки и QR-коды чужих клиентов бот администратору больше не выдаёт (админ-меню ссылок удалено, в карточке клиента таких кнопок нет) — их берут в панели; «📄 Лог банов» присылает только предыдущий журнал `3xipl-banned.prev.log` (#6984, #6990).

Изменения в разделе 16 — Эксплуатация: бэкапы, логи, обновление, CLI

- **Чистая остановка службы** (16.5): юниты используют `KillMode=mixed`, панель сама останавливает Xray и вспомогательные процессы, ложная ошибка завершения в журнал не пишется (#6884).
- **IP клиентов при выключенном fail2ban** (16.3): панель продолжает записывать адреса, блокировка по лимиту по-прежнему требует fail2ban (#6830).
- **Область `node-admin` в CLI** (16.7): `x-ui setting -getApiToken -tokenScope node-admin`.
- **Модуль ядра AmneziaWG** (16.7): команда `x-ui install-awg-kernel` собирает и ставит модуль через DKMS для режима ядра AmneziaWG.

1. Введение, требования и установка

1.1. Что такое 3X-UI

3X-UI — это открытая веб-панель управления для серверов Xray-core. Панель предоставляет единый многоязычный веб-интерфейс для развёртывания, настройки и мониторинга широкого набора прокси- и VPN-протоколов: от одиночного VPS до распределённых конфигураций из нескольких узлов (нод).

3X-UI — это расширенный форк оригинального проекта X-UI. По сравнению с ним добавлены поддержка большего числа протоколов, повышенная стабильность, поучётный (по каждому клиенту) учёт трафика и множество удобных функций.

Основные возможности:

- **Inbound разных протоколов** — VLESS, VMess, Trojan, Shadowsocks, WireGuard, AmneziaWG, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN, **MTProto** (прокси Telegram, добавлен в 3.3.0) и **TUIC v5** (нативный сервер внутри процесса панели, см. 5.13).
- **Современные транспорты и шифрование** — TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade и XHTTP, защищаемые TLS, XTLS и REALITY.
- **Fallback** — обслуживание нескольких протоколов на одном порту (например, VLESS и Trojan на 443) средствами fallback в Xray.
- **Управление по каждому клиенту** — квоты трафика, даты истечения, лимиты IP, отображение статуса «онлайн», ссылки-приглашения в один клик, QR-коды и подписки.
- **Статистика трафика** — по каждому inbound, клиенту и outbound, с возможностью сброса.
- **Поддержка нескольких узлов (нод)** — управление и масштабирование на несколько серверов из одной панели.
- **Outbound и маршрутизация** — WARP, NordVPN, пользовательские правила маршрутизации, балансировщики нагрузки, цепочки прокси.
- **Встроенный сервер подписок** с несколькими форматами вывода.
- **Telegram-бот** для удалённого мониторинга и управления; **Discord-бот** — уведомления, отчёты, бэкапы и команды (см. 14.8).
- **REST API** со встроенной документацией Swagger.
- **Гибкое хранилище** — SQLite (по умолчанию) или PostgreSQL.
- **14 языков интерфейса**, тёмная и светлая темы.
- **Интеграция с Fail2ban** для применения лимитов IP по каждому клиенту.

> Важно: проект предназначен только для личного использования. Не рекомендуется применять его в незаконных целях или в продакшен-окружении.

1.2. Поддерживаемые операционные системы и архитектуры

Операционные системы

Установочный скрипт определяет дистрибутив по полю `ID` из `/etc/os-release` (или `/usr/lib/os-release`). Официально поддерживаются:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, а также Windows.

На системах семейства Alpine используется служба OpenRC (`rc-service` / `rc-update`), на остальных — systemd. Для CentOS 7 пакеты устанавливаются через `yum` , для более новых выпусков — через `dnf` . Если дистрибутив не распознан, скрипт по умолчанию пытается использовать менеджер пакетов `apt-get` .

Архитектуры процессора

Архитектура определяется по выводу `uname -m` и приводится к одному из поддерживаемых значений:

Значение `uname -m`	Архитектура 3X-UI
`x86_64` , `x64` , `amd64`	`amd64`
`i*86` , `x86`	`386`
`armv8*` , `arm64` , `aarch64`	`arm64`
`armv7*` , `arm`	`armv7`
`armv6*`	`armv6`
`armv5*`	`armv5`
`s390x`	`s390x`

Если архитектура не входит в этот список, скрипт выводит сообщение «Unsupported CPU architecture!» и прекращает установку.

TUIC и архитектура. С 3.9.0 inbound TUIC v5 обслуживает нативный сервер **внутри процесса панели** — отдельный сайдкар `tuic-server` больше не нужен. Поэтому TUIC доступен на всех архитектурах, где работает сама панель (включая `armv6` , `armv5` и `s390x`), и никаких готовых сборок сайдкара докачивать не требуется.

Базовые зависимости

Перед установкой панели скрипт автоматически устанавливает базовый набор пакетов (имена различаются по дистрибутивам): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Способы установки

Способ 1. Установочный скрипт (рекомендуется)

Установка выполняется одной командой от имени root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Скрипт обязательно требует прав root: при запуске не от root выводится «Please run this script with root privilege» и завершается с ошибкой.

Что делает установщик пошагово:

1. Определяет ОС и архитектуру.
2. Устанавливает базовые зависимости.
3. Скачивает архив релиза `x-ui-linux-<arch>.tar.gz`, сверяет его SHA-256 с опубликованной контрольной суммой (см. ниже) и распаковывает в каталог `/usr/local/x-ui`.
4. **(С 3.9.0.)** TUIC встроен в панель — отдельный сайдкар `tuic-server` больше не скачивается; при обновлении установщик удаляет оставшийся от прежних версий бинарник `bin/tuic-server*` и каталог `bin/tuic`.
5. Скачивает управляющий скрипт `x-ui.sh` **той же версии**, что и панель (из тега релиза, для dev-сборки — из ветки `main`), и устанавливает его как команду `/usr/bin/x-ui`. Из того же тега берутся init-скрипт `x-ui.rc` (Alpine) и systemd-юниты, если их нет в архиве.
6. Создаёт каталог логов `/var/log/x-ui`.
7. Запускает первичную настройку: выбор базы данных, генерация учётных данных, выбор порта, опциональная настройка SSL.
8. Устанавливает и запускает службу автозапуска (systemd-юнит `x-ui.service` или init-скрипт OpenRC для Alpine). **(С 3.10.0.)** systemd-юнит работает в песочнице, а рядом генерируется drop-in `x-ui.service.d/10-xui-sandbox.conf` со списком каталогов, доступных панели на запись (см. 16.5, «Песочница службы systemd»).

Проверка контрольной суммы. Рядом с каждым архивом релиза (`x-ui-linux-<arch>.tar.gz`, `x-ui-windows-amd64.zip`) публикуется файл `<архив>.sha256`. Установщик (как и `update.sh`) скачивает его и до распаковки сравнивает с SHA-256 полученного архива; при совпадении выводится «Checksum verified: <хеш>». Несовпадение («Checksum mismatch for …») или неудачная загрузка файла суммы («Failed to download the checksum for … (HTTP <код>)») прерывают установку, скачанный архив удаляется. Исключение — ответ HTTP 404: у релизов, выпущенных до появления файлов сумм, их нет, поэтому выводится предупреждение «No checksum published for this release, skipping verification», и установка продолжается.

Выбор базы данных при установке. Установщик предлагает:

- `1) SQLite` (по умолчанию, рекомендуется при числе клиентов < 500) — один файл `/etc/x-ui/x-ui.db`, не требует настройки.
- `2) PostgreSQL` (рекомендуется при большом числе клиентов или нескольких нодах). PostgreSQL можно установить локально (создаётся выделенный пользователь и БД с именем `xui`) либо указать DSN к уже существующему серверу. Параметры подключения записываются в файл окружения службы (`/etc/default/x-ui`, `/etc/conf.d/x-ui` или `/etc/sysconfig/x-ui` в зависимости от дистрибутива) в виде переменных `XUI_DB_TYPE` и `XUI_DB_DSN`.

Какая версия PostgreSQL ставится локально (с 3.10.0). При локальной установке (и здесь, и из меню `x-ui` → «PostgreSQL Management») скрипт подключает официальный репозиторий PostgreSQL (PGDG) и ставит **PostgreSQL 18**: на Ubuntu/Debian/Armbian — через `apt` (пакет `postgresql-18`), на RHEL-семействе 8+ (RHEL, AlmaLinux, Rocky, Oracle Linux, CentOS Stream, Virtuozzo) — через `dnf` с отключённым модулем `postgresql` из AppStream; там служба называется `postgresql-18`, а данные лежат в `/var/lib/pgsql/18/data`. Раньше ставилась версия дистрибутива (Ubuntu 22.04 — 14, Debian 12 — 15, EL9 — 13), серверы оказывались на разных мажорных версиях, а бэкап `pg_dump` с более нового сервера не восстанавливается на более старый. Где у PGDG нет сборки (Ubuntu 20.04, Debian 11, архитектуры armhf/i386/s390x, CentOS 7, Fedora, Amazon Linux, старые point-релизы EL), добавленный репозиторий откатывается, выводится «PostgreSQL 18 is not available for this system; installing the distribution's PostgreSQL instead.», и ставится

пакет дистрибутива, как прежде. Если на хосте уже есть кластер PostgreSQL, он сохраняется — действующие установки до 18 не обновляются.

Пример: запись параметров PostgreSQL в файл окружения службы. После выбора PostgreSQL и указания DSN установщик добавит в файл окружения примерно такие строки:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Здесь `xui` — имя пользователя и базы, `127.0.0.1:5432` — адрес и порт сервера, `sslmode=disable` подходит для локального подключения (для удалённого сервера обычно используют `require`).

Установка конкретной (старой) версии. Можно явно указать тег версии — установщик скачает соответствующий релиз:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Минимально допустимая версия для такой установки — `v2.3.5`; при указании более старой выводится «Please use a newer version (at least v2.3.5)». Управляющий скрипт `x-ui.sh` (на Alpine — и `x-ui.rc`) берётся из того же тега. Ещё до остановки текущей панели установщик проверяет, что эти файлы для тега опубликованы, и если нет — прерывается с сообщением «<файл> is not available for <тег> (HTTP <код>)», не затрагивая текущую установку. Поэтому на Alpine нельзя поставить версию раньше `v2.8.4`, где `x-ui.rc` ещё не было. Файлов контрольных сумм у старых релизов нет — их проверка сводится к предупреждению (см. выше).

Установка dev-сборки. Помимо тега версии установщик принимает аргумент `dev-latest` (псевдоним `dev`) — это устанавливает rolling dev-сборку по последнему коммиту ветки `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Dev-сборка — это per-commit предрелиз (тег `dev-latest`), а не стабильная версия, поэтому проверка минимально допустимой версии для неё не выполняется. При запуске выводится предупреждение «Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.». Без аргумента установщик ставит последний стабильный релиз. Использовать dev-сборку имеет смысл только для проверки ещё не вышедших исправлений; в обычной эксплуатации устанавливайте стабильные версии.

Способ 2. Docker

Запуск с базой SQLite по умолчанию:

```
docker compose up -d
```

Для запуска со встроенным сервисом PostgreSQL нужно раскомментировать строки `XUI_DB_*` в `docker-compose.yml` и запустить с профилем:

```
docker compose --profile postgres up -d
```

Образ включает Fail2ban (по умолчанию активен) для применения лимитов IP по клиентам. Fail2ban блокирует нарушителей через `iptables`, что требует возможности `NET_ADMIN`. В `docker-compose.yml` она уже предоставлена через `cap_add`. С 3.5.0 в контейнере починено **автопродление сертификатов**, выпущенных через SSL-меню: entrypoint запускает `crond` и заново регистрирует cron-задание acme.sh, а `docker-compose.yml` получил отдельный том для состояния acme.sh — продление переживает пересоздание контейнера (раньше сертификаты молча истекали через ~90 дней). При ручном запуске через `docker run` возможности нужно добавить самостоятельно, иначе блокировки будут только логироваться, но не применяться:

Пример: полная команда `docker run`. Минимальный вариант с прокидыванием порта панели, сетевыми возможностями и постоянным томом для базы:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Том `/etc/x-ui` сохраняет файл `x-ui.db` между перезапусками контейнера, иначе настройки и аккаунты будут теряться.

Inbound TUIC v5 слушает **UDP** (сервер встроен в панель, отдельного сайдкара в образе больше нет), поэтому при публикации портов контейнера его порт пробрасывается как UDP, например `-p 8443:8443/udp` (см. 5.13).

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

В Docker панель является основным процессом контейнера: автозапуск регулируется политикой перезапуска контейнера (например, `restart: unless-stopped`), а не службой внутри контейнера.

Платформы образа. Образ публикуется для пяти платформ: `linux/amd64`, `linux/arm64`, `linux/arm/v7`, `linux/arm/v6` и `linux/386`. При сборке под каждую из них скрипт `DockerInit.sh` берёт свои бинарники Xray и `mtg` и кладёт их под теми именами, которые ищет панель (`xray-linux-386`, `xray-linux-arm32`, `mtg-linux-386`, `mtg-linux-arm` и т. д.): ARMv6 получает ARMv6-сборку, а не ARMv7, а 386 — 32-битную. Неизвестная архитектура останавливает сборку образа, а не подменяется молча на amd64.

WireGuard-outbound в контейнере (с 3.10.0). `docker-compose.yml` выдаёт контейнеру `NET_ADMIN` (для Fail2ban), а с этой возможностью ядро Xray по умолчанию поднимает для WireGuard-outbound'а TUN-интерфейс ядра. Docker же оставляет `/proc/sys` только для чтения и не пробрасывает `/dev/net/tun`, поэтому раньше один такой outbound не давал Xray запуститься вовсе — вместе со всеми inbound'ами (#6796). Теперь панель один раз проверяет, может ли ядро создать TUN ядра, и если нет — включает у всех WireGuard-outbound'ов `noKernelTun` (пользовательский TUN на gVisor). Там, где TUN ядра доступен, он сохраняется.

Обновление на 3.10.0: что учесть

- **В комплекте — ядро Xray 26.10.10** (вместо 26.9.30). Сборка из исходников требует **Go 1.27.2**; установка готовыми пакетами и скриптом этого не касается. Новое ядро проверяет TLS-сертификаты серверов outbound'ов по встроенному набору корневых сертификатов Mozilla, а не по системному хранилищу: для outbound'а к серверу с сертификатом частного CA включите в его настройках TLS новый переключатель «Использовать системные CA» (`useSystemCA`).
- **Перед обновлением сделайте бэкап базы.** При первом старте 3.10.0 однократно выполняются преобразования (каждое отмечается в `history_of_seeders` и не повторяется):
  - `XdnsFinalmaskNamesAddrsFix` — маски XDNS (Final Mask) в inbound'ах, хостах, шаблоне, маске JSON-подписки и кэше подписочных outbound'ов переводятся в формат Xray 26.10.10: домен — список `names` вместо одного `name` , резолвер — список `addrs` из URL `udp://` / `tcp://` вместо `{type, settings:{addr}}` . Старые ключи новое ядро молча игнорирует, и без миграции XDNS-туннель ничего бы не передавал. Протокол на проводе не изменился, но клиент на более старом ядре новый формат конфигурации не прочитает;
  - `UDPHopClientMaskFix` — устаревший `finalmask.quicParams.udpHop` , который ядро игнорирует с 26.9.9, в клиентских масках (хосты, глобальная маска JSON-подписки, кэш подписочных outbound'ов, outbound'ы шаблона) заменяется UDP-маской `udphop` , иначе клиенты Hysteria2/XHTTP молча переставали переключать порты. У inbound'ов `udpHop` остаётся как есть — это данные для `mport` в ссылке;
  - `HostVerifyPeerCertByNameBoolFix` — у хостов очищается поле «Проверять сертификат пира по имени» (`verifyPeerCertByName`), если в нём со времён, когда оно было флажком, осталось `0` , `1` , `false` или `true` ; настоящие имена не трогаются. С 3.10.0 JSON-подписка передаёт это поле на уровень Xray, и оставшийся `0` уходил бы клиенту как имя сертификата — Happ и другие клиенты отвергали бы сертификат сервера. Кроме того, при каждом старте Shadowsocks-inbound'ы с методом `2022-blake3-chacha20-poly1305` переводятся на `2022-blake3-aes-256-gcm` (с пользователями ядро принимает только AES-методы и с chacha20 не запускалось целиком; ключи той же длины остаются действительными), а серверный ключ Shadowsocks-2022 неверной длины заменяется случайным (такой inbound и раньше не работал). **Ссылки этих inbound'ов меняются** — клиентам нужно выдать их заново; в лог пишется «Inbound "<тег>": switched … to 2022-blake3-aes-256-gcm; its share links changed» или «… replaced a … server key of the wrong size; its share links changed».
- **Миграция базы — до запуска службы.** `update.sh` теперь выполняет `x-ui migrate` до старта службы, поэтому служба больше не падает при обновлении из-за параллельных миграций (#6729), а **настройки TLS обновление не трогает** (см. 16.5). `update.sh` берётся из ветки `main` , поэтому оба исправления действуют уже при переходе на 3.10.0.
- **Служба systemd работает в песочнице** (#6718): `/usr` , `/etc` , `/boot` , `/efi` и домашние каталоги для панели и Xray — только для чтения, кроме каталогов панели. Если Xray или ваши скрипты, запускаемые панелью, пишут файлы в эти деревья, а также если хранилища панели перенесены переменными `XUI_*_FOLDER` , прочитайте 16.5 («Песочница службы systemd»). На Alpine, в Docker и на Windows ничего не меняется.
- **Новый локальный PostgreSQL — версии 18** из репозитория PGDG; уже установленный сервер PostgreSQL обновление не трогает (см. выше «Выбор базы данных при установке»).
- При переходе с версии младше 3.9.0 дополнительно выполняются миграции 3.9.0: стратегия доменов WireGuard-outbound переносится в `sockopt` (`WireguardDomainStrategyFix`), а старые маски XDNS

переводятся сразу в формат 26.10.10; при переустановке удаляется больше не нужный сайдкар `tuic-server` (подробнее — «Что нового в 3.9.0» в руководстве к версии 3.9.0).

Обновление на 3.11.0: что учесть

- **В комплекте — ядро Xray 26.10.21** (вместо 26.10.10): релиз 26.10.10 отозван в апстриме, поэтому Docker-образ и релизные сборки переведены на 26.10.21.
- **Первый пользователь новой базы — со случайными логином и паролем** (в том числе в Docker и при ручном запуске): они создаются при старте сервера и один раз печатаются в лог (см. 1.4). Существующие базы и пользователи не меняются.
- **Образы Docker для ARM и 386** теперь содержат правильные бинарники Xray и `mtg` (см. «Способ 2. Docker» выше).
- **Служба systemd** в поставляемых юнитах запускается с `KillMode=mixed`: при остановке панель сама останавливает Xray и вспомогательные процессы (см. 16.5).
- **Токены узлов.** Для обновления ПО узла центральной панелью нужен токен с областью `admin` или новой `node-admin`; `node-sync` по-прежнему обновить панель узла не может (см. 13.9).
- **Миграции базы выполняются сами.** Добавляются колонки `nodes.down_notified` и `nodes.unavailable_metrics` и индекс `idx_api_tokens_token`; узлы, которые на момент обновления уже офлайн, считаются объявленными, поэтому повторного `node.down` по ним не будет.

1.4. Первый запуск и учётные данные по умолчанию

При первой установке (когда ещё используются учётные данные по умолчанию) установщик **генерирует случайные значения** имени пользователя, пароля и веб-пути, а также порта:

Параметр	Как формируется при установке	Примечание
Имя пользователя (Username)	случайная строка из 10 символов	генерируется автоматически
Пароль (Password)	случайная строка из 10 символов	генерируется автоматически
Веб-путь панели (WebBasePath)	случайная строка из 18 символов	защищает панель от обнаружения по корневому URL
Порт панели (Port)	по умолчанию случайный порт в диапазоне 1024–62000; при желании можно задать вручную	«заводское» значение `webPort` равно `2053`, но установщик его перезаписывает

Пути сервера подписок задаёт сама панель при создании новой базы: `subPath`, `subJsonPath` и `subClashPath` получают случайные значения из 16 цифр и строчных латинских букв (вида `/k3v9x0q2m7b1c8zd/`), поэтому адреса подписок у каждой установки свои; обновление существующей панели пути не меняет. Посмотреть и изменить их можно в настройках сервера подписок (см. 10.2).

В конце установки скрипт выводит итоговую сводку: имя пользователя, пароль, порт, веб-путь, API-токен и готовую ссылку для входа (Access URL) вида:

```
https://<домен-или-IP>:<порт>/<веб-путь>
```

Если SSL-сертификат не настроен, ссылка будет по `http://`, и скрипт выведет предупреждение о необходимости настроить SSL (пункт меню 20).

> Обязательная смена учётных данных. Поскольку логин и пароль генерируются случайно, их следует **сохранить сразу после установки**. Сменить их в любой момент можно пунктом меню «Reset Username & Password» (см. ниже) либо из веб-интерфейса в настройках панели. После сброса скрипт напоминает: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Запуск без установщика (Docker, ручной запуск бинарника). Фиксированной пары `admin` / `admin` нет. Если в базе нет ни одного пользователя, панель при старте сервера создаёт первого: логин из 10 символов и пароль из 16 случайных символов, и **один раз** печатает их в лог сервера строкой «No panel user existed, so one was generated. Username: … Password: … (shown once; change it in Panel Settings)». В Docker её видно в `docker logs <контейнер>`, под systemd — в журнале (`journalctl -u x-ui`). Повторно пароль не показывается — сохраните его сразу, а позже сменьте в настройках панели (см. 2.4). Пока пользователя нет, `x-ui setting -show` выводит `hasDefaultCredential: true`, поэтому установщик на свежей установке по-прежнему задаёт собственные учётные данные.

После установки для открытия меню управления используется команда `x-ui` (см. раздел 1.6).

1.5. Расположение файлов

Путь	Назначение
`/usr/local/x-ui/`	каталог установки панели (бинарь `x-ui`, скрипт `x-ui.sh`)
`/usr/local/x-ui/bin/xray-linux-<arch>`	бинарь Xray-core (на armv5/armv6/armv7 переименовывается в `xray-linux-arm32`)
`/usr/bin/x-ui`	управляющий скрипт (команда `x-ui`)
`/etc/x-ui/`	каталог данных панели; при создании получает права `0700`
`/etc/x-ui/x-ui.db`	файл базы данных SQLite (по умолчанию); при каждом старте панели он и файлы `x-ui.db-wal` / `x-ui.db-shm` получают права `0600`, ведь в базе хранятся UUID клиентов, приватные ключи REALITY и хеш пароля администратора
`/var/log/x-ui/`	каталог логов панели
`/etc/systemd/system/x-ui.service`	systemd-юнит службы (не для Alpine)
`/etc/systemd/system/x-ui.service.d/10-xui-sandbox.conf`	drop-in песочницы службы (с 3.10.0): каталоги, доступные панели на запись, и фильтр системных вызовов; перегенерируется при каждой установке и обновлении, свои правки держите в отдельном drop-in (например, `20-x-ui-local.conf`)
`/etc/init.d/x-ui`	init-скрипт OpenRC (только Alpine)
`/etc/default/x-ui` · `/etc/conf.d/x-ui` · `/etc/sysconfig/x-ui`	файл переменных окружения службы (путь зависит от дистрибутива); сюда пишутся `XUI_DB_TYPE` / `XUI_DB_DSN`

Каталог базы данных можно переопределить переменной окружения `XUI_DB_FOLDER` (по умолчанию `/etc/x-ui`), а каталог бинарей Xray — переменной `XUI_BIN_FOLDER` (по умолчанию `bin` относительно каталога панели). Имя файла БД — `x-ui.db`.

Пример: вынос базы на отдельный диск. Чтобы хранить `x-ui.db` не в `/etc/x-ui`, а, например, на смонтированном диске `/data`, задайте переменную в файле окружения службы и перезапустите панель:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Полный путь к базе станет `/data/x-ui/x-ui.db`.

С 3.10.0 служба работает в песочнице systemd (см. 16.5): каталог вне `/usr`, `/etc`, `/boot`, `/efi` и домашних каталогов, как `/data` в примере, доступен на запись и так. Если же переносите хранилище внутрь этих деревьев, после правки файла окружения выполните `x-ui update` (или переустановку) — скрипт перегенерирует drop-in `10-xui-sandbox.conf` с новым путём, иначе панель не сможет писать в новый каталог.

Основные переменные окружения

Переменная	Назначение	По умолчанию
`XUI_DB_TYPE`	бэкенд БД: `sqlite` или `postgres`	`sqlite`
`XUI_DB_DSN`	строка подключения PostgreSQL (при `XUI_DB_TYPE=postgres`)	—
`XUI_DB_FOLDER`	каталог файла БД SQLite	`/etc/x-ui`
`XUI_INIT_WEB_BASE_PATH`	начальный URI-путь веб-панели (только при первой инициализации)	`/`
`XUI_DB_MAX_OPEN_CONNS`	максимум открытых соединений (пул PostgreSQL)	—
`XUI_DB_MAX_IDLE_CONNS`	максимум простаивающих соединений (пул PostgreSQL)	—
`XUI_ENABLE_FAIL2BAN`	включить применение лимитов IP через Fail2ban	`true`
`XUI_LOG_LEVEL`	уровень логирования (`debug`, `info`, `warning`, `error`)	`info`
`XUI_DEBUG`	режим отладки	`false`

Пример: временно включить подробное логирование. Для диагностики проблемы поднимите уровень логов до `debug` и перезапустите службу:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log    # просмотр отладочного лога
```

После диагностики верните значение `info`, чтобы лог не разрастался.

Начальный путь веб-панели через окружение. Переменная `XUI_INIT_WEB_BASE_PATH` задаёт URI-путь веб-панели (`webBasePath`) при первичной инициализации настроек. Это удобно при развёртывании в Docker или через systemd, чтобы сразу зафиксировать путь входа в панель. Значение нормализуется

автоматически — ведущий и замыкающий слэши добавляются при необходимости, а пустое или состоящее из пробелов значение игнорируется (тогда применяется путь по умолчанию `/`). Переменная влияет **только на первичную инициализацию**: если настройки уже созданы, путь меняется в веб-интерфейсе или пунктом меню «Reset Web Base Path».

1.6. Команда управления `x-ui` (меню скрипта)

После установки команда `x-ui` (запускается от root) открывает интерактивное меню «3X-UI Panel Management Script». Выбор пункта производится вводом его номера (диапазон 0–28). Многие пункты доступны и в виде подкоманд для скриптов (см. раздел 1.7).

Меню разбито на тематические блоки.

Установка и обновление

- **1. Install** — установка панели (запускает `install.sh`). Перед установкой проверяется, что панель ещё не установлена.
- **2. Update** — обновление всех компонентов x-ui до последней версии. Данные при этом не теряются; после обновления панель перезапускается автоматически. Требует подтверждения. С 3.10.0 миграция базы выполняется до запуска службы, а настройки TLS обновление не трогает (см. 16.5).
- **3. Update to Dev Channel (latest commit)** — обновление до rolling-сборки `dev-latest` по последнему коммиту ветки `main`; требует подтверждения (см. 16.5).
- **4. Update Menu** — обновление только управляющего скрипта (`x-ui.sh` / команды `x-ui`) без переустановки панели. Скрипт берётся из тега **установленной** версии панели (`v<версия>`), чтобы меню соответствовало бинарю; если для этой версии он не опубликован (например, на dev-сборке), используется `main` с предупреждением «No x-ui.sh published for the installed version (…), using main».
- **5. Legacy Version** — установка указанной (старой) версии панели. Скрипт запрашивает номер версии (например, `2.4.0`) и скачивает соответствующий релиз.
- **6. Uninstall** — полное удаление панели **вместе с Xray**. Останавливается и отключается служба, удаляются каталоги `/etc/x-ui/` и `/usr/local/x-ui/`, файл окружения службы и сам управляющий скрипт. Требует подтверждения (по умолчанию «нет»).

Учётные данные и настройки

- **7. Reset Username & Password** — сброс имени пользователя и пароля панели. Можно ввести свои значения или оставить пустыми для случайной генерации (случайное имя — 10 символов, случайный пароль — 18 символов). Дополнительно предлагается отключить двухфакторную аутентификацию (2FA), если она настроена. После сброса панель перезапускается.
- **8. Reset Web Base Path** — сброс веб-пути панели: генерируется новый случайный путь (18 символов), панель перезапускается. Используется, если прежний путь скомпрометирован или забыт.
- **9. Reset Settings** — сброс всех настроек панели к значениям по умолчанию. **Учётные данные (имя пользователя и пароль) и данные аккаунтов при этом не теряются.** Пути сервера подписок (`subPath`, `subJsonPath`, `subClashPath`) получают новые случайные значения, поэтому ранее выданные ссылки подписок перестают открываться. Требует подтверждения; после сброса панель перезапускается.
- **10. Change Port** — смена порта веб-панели. Запрашивается номер порта (1–65535); после установки требуется перезапуск, чтобы порт вступил в силу.

- **11. View Current Settings** — просмотр текущих настроек (`/usr/local/x-ui/x-ui setting -show`). Показывает в том числе используемый бэкенд БД (SQLite или PostgreSQL с замаскированным паролем в DSN) и готовую ссылку доступа (Access URL). Если SSL не настроен, предлагает выпустить сертификат Let's Encrypt для IP-адреса.

Управление службой

- **12. Start** — запуск службы панели. Если панель уже работает, выводится сообщение, что повторный запуск не нужен.
- **13. Stop** — остановка службы панели.
- **14. Restart** — перезапуск службы панели.
- **15. Restart Xray** — перезапуск только ядра Xray-core без перезапуска самой панели (через `systemctl reload x-ui` , в Docker — сигналом `USR1` процессу панели).
- **16. Check Status** — проверка состояния службы (`systemctl status x-ui` или `rc-service x-ui status`).
- **17. Logs Management** — управление логами: просмотр отладочного лога (Debug Log, через `journalctl`) и, кроме Alpine, очистка всех логов (Clear All logs).

Автозапуск

- **18. Enable Autostart** — включить автозапуск панели при загрузке ОС (`systemctl enable x-ui` или `rc-update add`).
- **19. Disable Autostart** — отключить автозапуск при загрузке ОС.

В Docker автозапуск регулируется политикой перезапуска контейнера, поэтому эти пункты лишь выводят соответствующую подсказку.

Безопасность и сеть

- **20. SSL Certificate Management** — управление SSL-сертификатами через acme.sh: выпуск сертификата для домена, отзыв, принудительное обновление, просмотр существующих доменов, указание путей к сертификату для панели, а также выпуск краткоживущего (~6 дней, с автообновлением) сертификата для IP-адреса.
- **21. Cloudflare SSL Certificate** — выпуск SSL-сертификата через DNS-валидацию Cloudflare.
- **22. IP Limit Management** — управление лимитами числа IP по клиентам (на базе Fail2ban): просмотр и снятие блокировок и т. п.
- **23. Firewall Management** — управление межсетевым экраном (открытие/закрытие портов и просмотр правил).
- **24. SSH Port Forwarding Management** — настройка проброса SSH-портов, чтобы открывать панель с локальной машины через SSH-туннель.
- **25. PostgreSQL Management** — управление встроенным/связанным экземпляром PostgreSQL (включение и сопутствующие операции).

Производительность и обслуживание

- **26. Enable BBR** — включение/отключение алгоритма управления перегрузкой TCP BBR (подменю с пунктами Enable BBR / Disable BBR).
- **27. Update Geo Files** — обновление geo-баз (файлы `.dat`) с выбором источника: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) или All (все сразу). Панель перезапускается, только если хотя бы один файл действительно обновился; если всё актуально, выводится «… are already up to date, restart is not needed.».
- **28. Speedtest by Ookla** — запуск теста скорости сети через Speedtest by Ookla.
- **0. Exit Script** — выход из меню.

1.7. Подкоманды `x-ui` (без интерактивного меню)

Для использования в скриптах команда `x-ui` поддерживает прямые подкоманды (вывод `x-ui` без аргументов открывает меню):

Команда	Действие
`x-ui`	открыть меню управления
`x-ui start`	запустить панель
`x-ui stop`	остановить панель
`x-ui restart`	перезапустить панель
`x-ui restart-xray`	перезапустить Xray
`x-ui status`	текущее состояние службы
`x-ui settings`	текущие настройки
`x-ui enable`	включить автозапуск при загрузке ОС
`x-ui disable`	отключить автозапуск
`x-ui log`	просмотр логов
`x-ui banlog`	просмотр логов блокировок Fail2ban
`x-ui update`	обновить панель
`x-ui update-all-geofiles`	обновить все geo-файлы
`x-ui migrateDB [file]`	конвертация `.db` ? `.dump` (SQLite)
`x-ui legacy`	установить старую версию
`x-ui install`	установить панель
`x-ui uninstall`	удалить панель

1.8. Миграция SQLite → PostgreSQL

Существующую установку на SQLite можно перенести на PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# затем задать XUI_DB_TYPE и XUI_DB_DSN в /etc/default/x-ui и перезапустить:
systemctl restart x-ui
```

Исходный файл SQLite остаётся нетронутым — удалите его вручную только после проверки работы нового бэкенда.

Пример: проверка переключения на PostgreSQL. После миграции убедитесь, что панель действительно работает на новом бэкенде, командой просмотра настроек — в выводе должен быть указан PostgreSQL (пароль в DSN маскируется):

```
x-ui settings | grep -i -E 'db|dsn'
```

Если панель открывается, а аккаунты на месте, исходный `x-ui.db` можно удалить.

---

2. Вход в панель и безопасность доступа

Этот раздел описывает всё, что относится к аутентификации администратора панели 3X-UI: форму входа, двухфакторную аутентификацию (TOTP), защиту от перебора пароля, смену учётных данных, изменение секретного пути и порта панели, время жизни сессии, а также синхронизацию/аутентификацию через LDAP.

2.1. Форма входа

Страница входа отдаётся по корню секретного пути панели (`webBasePath`). Если пользователь уже авторизован, он автоматически перенаправляется на `…/panel/` . На странице есть переключатель темы, выбор языка интерфейса и собственно форма.

Поля формы:

Поле	Подсказка/ заголовок (RU)	Обязательно	Описание
Имя пользователя	«Имя пользователя»	Да	Логин администратора. Пустое значение отклоняется ещё на клиенте, а на сервере — сообщением «Введите имя пользователя».
Пароль	«Пароль»	Да	Пароль администратора. Пустое значение отклоняется сообщением «Введите пароль».
Код 2FA	«Код 2FA»	Только при включённой 2FA	Поле появляется **только** если на панели включена двухфакторная аутентификация. 6-значный код из приложения-аутентификатора.

Кнопка **«Войти»** отправляет форму на `POST /login` .

Поведение и сообщения:

- При успешном входе показывается «Вход выполнен успешно» и происходит переход на `…/panel/` .
- При любой ошибке учётных данных или неверном коде 2FA сервер возвращает **единое** сообщение: «Неверные данные учетной записи.» (англ.: *Invalid username or password or two-factor code.*). Это сделано намеренно — панель не подсказывает, что именно неверно (логин, пароль или код), чтобы не облегчать перебор.
- Поле «Код 2FA» панель показывает или скрывает на основании запроса `POST /getTwoFactorEnable` , который возвращает текущий статус 2FA ещё до авторизации.
- Если истекла серверная сессия, при следующем запросе показывается «Сессия истекла. Войдите в систему снова», и пользователь перенаправляется на страницу входа.

> Примечание о CSRF: перед отправкой формы клиент получает CSRF-токен (`GET /csrf-token`); запросы `/login` и `/logout` защищены CSRF-проверкой. Запрос, отклонённый из-за отсутствующего или неверного токена, получает ответ 403 и оставляет предупреждение в логе панели (см. 16.2).

Пример: вход через API. Когда 2FA выключена, достаточно логина и пароля; при включённой 2FA добавляется поле `twoFactorCode` :

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

При успехе сервер вернёт `Set-Cookie` с сессионной cookie — её и нужно передавать в последующих запросах к `/panel/api/…`.

Ответ API без авторизации. Эндпоинты `/panel/api/…` принимают сессионную cookie или API-токен в заголовке `Authorization: Bearer <токен>` (см. 13.9). Запрос без действующей сессии и без токена получает **404**, как будто API по этому пути нет, — так панель не выдаёт себя сканерам. Если же заголовок `Authorization: Bearer …` передан, но токен неверный, отключён или истёк, ответ — **401 Unauthorized** (до 3.8.0 — тоже 404): скрипт может отличить ошибку токена от ошибки в пути, ведь неверный базовый путь (`webBasePath`) по-прежнему даёт 404. Запросы самого веб-интерфейса (`X-Requested-With: XMLHttpRequest`) без сессии тоже получают 401.

2.2. Двухфакторная аутентификация (2FA / TOTP)

2FA в 3X-UI реализована по стандарту **TOTP** и совместима с любым приложением-аутентификатором (Google Authenticator, Aegis, FreeOTP и т. п.). Параметры жёстко заданы: алгоритм **SHA1**, **6** цифр, период **30** секунд, эмитент (issuer) `3x-ui`, метка `Administrator`.

Пример: otpauth-URI, который кодирует QR-код. Если приложение-аутентификатор не сканирует камеру, токен можно добавить вручную по такой ссылке (подставьте свой Base32-секрет вместо `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Параметры `algorithm=SHA1`, `digits=6`, `period=30` соответствуют жёстко заданным значениям панели — менять их не нужно.

Настройки находятся в разделе **Настройки** → **Учетная запись**, вкладка **«Двухфакторная аутентификация»**.

Элемент	Текст (RU)	Описание
Переключатель	«Включить 2FA»	Включает/отключает двухфакторную аутентификацию.
Описание	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Подсказка под переключателем.

Как включить 2FA

При включении переключателя панель **локально генерирует новый секрет** — случайную строку в кодировке Base32 (алфавит `A–Z` и `2–7`). Открывается окно «Включить двухфакторную аутентификацию» с пошаговой инструкцией:

1. **«Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение»**. Под QR-кодом отображается сам секрет в текстовом виде — по клику на QR-код секрет копируется в буфер обмена (всплывает «Скопировано»).
2. **«Введите код из приложения»** — нужно ввести 6-значный код, который сгенерировало приложение. Код проверяется **на стороне браузера**: панель сама вычисляет текущий TOTP по только что сгенерированному секрету и сравнивает с введённым. Если код неверный — «Неверный код»; поле принимает только ровно 6 цифр.

Только после успешного подтверждения секрет и флаг включения сохраняются. При сохранении показывается «Двухфакторная аутентификация была успешно установлена».

Важно: изменения в разделе настроек применяются общей кнопкой **«Сохранить»**, после чего, как правило, требуется перезапуск панели («Сохраните изменения и перезапустите панель для их применения»). При первом включении 2FA сервер дополнительно **инвалидирует все активные сессии** (увеличивает «login epoch»), поэтому после применения настройки потребуется повторный вход — уже с кодом 2FA.

Как отключить 2FA

Повторное нажатие переключателя открывает окно «Отключить двухфакторную аутентификацию» с подсказкой «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». После ввода верного кода (с 3.4.2 он проверяется **на сервере**) флаг и секрет очищаются, показывается «Двухфакторная аутентификация была успешно удалена».

Проверка кода при входе

При входе сервер берёт сохранённый секрет и сравнивает текущий TOTP с присланным кодом 2FA. Несовпадение трактуется как неуспешный вход, но пользователю показывается то же объединённое сообщение «Неверные данные учетной записи.».

Восстановление доступа (recovery)

Отдельного механизма «кодов восстановления» в 3X-UI **нет**. Если доступ к приложению-аутентификатору потерян, восстановить вход через интерфейс панели нельзя. Единственный путь — отключить 2FA напрямую в базе данных на сервере: сбросить ключ `twoFactorEnable` в `false` (и при необходимости очистить `twoFactorToken`) в таблице настроек, после чего перезапустить панель. Поэтому секрет (Base32-токен) при включении 2FA рекомендуется сохранить в надёжном месте.

Пример: аварийное отключение 2FA на сервере. Получив доступ к серверу по SSH, остановите панель, сбросьте ключи в таблице настроек и запустите панель снова:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

После этого вход выполняется только по логину и паролю, а 2FA при желании настраивается заново.

> Связь со сменой учётных данных: при смене логина/пароля (см. 2.4) 2FA **автоматически отключается** на сервере, чтобы старый секрет не блокировал доступ под новой учётной записью.

Замена секрета требует кода (3.7.0). Перезаписать уже сохранённый секрет 2FA можно только с действующим кодом из приложения-аутентификатора. Иначе угнанная сессия просто перевешивала двухфакторную аутентификацию на свой секрет — и владелец терял доступ, а нападающий его сохранял.

2.3. Ограничение попыток входа (login limiter / защита от перебора)

Панель содержит встроенный лимитер неудачных входов (аналог fail2ban на уровне приложения). Параметры заданы в коде и **не настраиваются** через интерфейс:

Параметр	Значение	Назначение
Максимум неудач	**5**	Сколько неудачных попыток допускается в пределах окна.
Окно подсчёта	**5 минут**	Скользящее окно, в котором накапливаются неудачи (более старые отбрасываются).
Блокировка (cooldown)	**15 минут**	На сколько блокируется ключ после превышения порога.

Как это работает:

- Ключ блокировки строится из **связки «IP + логин»** (логин приводится к нижнему регистру, пробелы обрезаются). То есть блокировка применяется к конкретной паре «адрес + имя пользователя», а не ко всей панели.
- При каждой неудачной попытке (неверные логин/пароль или неверный код 2FA) счётчик растёт. Достигнув **5** неудач за **5 минут**, ключ блокируется на **15 минут**. Во время блокировки любые попытки этой пары немедленно отклоняются тем же сообщением «Неверные данные учетной записи.», даже если данные верны.
- **Успешный вход немедленно сбрасывает** счётчик и снимает блокировку для этой пары.
- IP-адрес клиента определяется с учётом доверенных прокси (см. `trustedProxyCIDRs`): заголовки `X-Real-IP` и `X-Forwarded-For` принимаются только если запрос пришёл с доверенного адреса. Иначе используется реальный адрес соединения, а при невозможности его извлечь — строка `unknown`. Адрес клиента читается со стороны прокси, а не со стороны клиента: `X-Forwarded-For` разбирается **справа налево** с пропуском адресов доверенных прокси, и берётся первый недоверенный адрес — всё, что клиент дописал слева, игнорируется. `X-Real-IP` принимается, только если совпадает с этим адресом или если `X-Forwarded-For` нет вовсе. При расхождении панель не верит ни одному из заголовков и использует адрес самого прокси, поэтому за неверно настроенным прокси все клиенты попадут под

общий ключ блокировки. Прокси должен **перезаписывать** `X-Real-IP` адресом, который увидел сам, а не пересылать присланное клиентом значение (пример для Caddy — в 13.4).

Все попытки логируются. Для неудачных пишется предупреждение в лог сервера с именем пользователя, IP, причиной и, при блокировке, временем `blocked_until`. Если включено уведомление о входе через Telegram-бота (`tgNotifyLogin` — «Уведомление о входе»), администратор дополнительно получает имя пользователя, IP и время как успешных, так и неуспешных и заблокированных попыток.

Строки входа и выхода пишутся в лог панели в едином формате `ключ=значение`, с настоящим именем пользователя в кавычках (спецсимволы экранируются). Успешный вход записывается один раз, с уровнем INFO, неудачи — с уровнем WARNING. Причина неудачи — `invalid credentials`, `invalid 2FA code` или, для уже заблокированной пары, `too many failed attempts`; когда пара «IP + логин» заблокирована, добавляется поле `blocked_until`:

```
logged in successfully: username="admin", IP="203.0.113.45"
failed login: username="admin", IP="203.0.113.45", reason="invalid 2FA code"
failed login: username="admin", IP="203.0.113.45", reason="too many failed attempts",
blocked_until=2026-06-10T14:47:07+03:00
logged out successfully: username="admin"
```

Пример: уведомление о входе в Telegram. При включённом `tgNotifyLogin` после каждой попытки администратор получает сообщение примерно такого вида:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Для заблокированной пары «IP + логин» в статусе будет указано, что попытка отклонена лимитером.

2.4. Смена логина и пароля администратора

Раздел **Настройки** → **Учетная запись**, вкладка **«Учетные данные администратора»**. Поля:

Поле	Текст (RU)	Описание
Текущий логин	«Текущий логин»	Действующее имя пользователя. Должно совпасть с текущим логином, иначе изменение отклоняется.
Текущий пароль	«Текущий пароль»	Действующий пароль для подтверждения личности.
Новый логин	«Новый логин»	Новое имя пользователя. Не может быть пустым.
Новый пароль	«Новый пароль»	Новый пароль. Не может быть пустым.

Изменение применяется кнопкой **«Подтвердить»** и отправляется на `POST /panel/setting/updateUser`.

Логика и сообщения сервера:

- Если «Текущий логин» не совпадает с реальным или «Текущий пароль» неверен — «Произошла ошибка при изменении учетных данных администратора.» с пояснением «Неверное имя пользователя или пароль».
- Если новый логин или новый пароль пуст — пояснение «Новое имя пользователя и новый пароль должны быть заполнены».
- При успехе — «Вы успешно изменили учетные данные администратора.». Пароль хранится в виде bcrypt-хеша.

Подтверждение кодом 2FA (с 3.4.2). Если на панели включена двухфакторная аутентификация, смена логина/пароля дополнительно требует **ввести текущий код** из приложения-аутентификатора. Открывается окно **«Смена учётных данных»** с подсказкой «Введите код из приложения, чтобы изменить учётные данные администратора.»; код (`twoFactorCode`) проверяется на сервере, при неверном или пустом коде изменение не применяется. То же подтверждение теперь требуется и при отключении 2FA (см. 2.2).

Пример: смена учётных данных через API. Запрос требует действующей сессионной cookie (получается при входе) и подтверждения текущих логина/пароля:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйС
ложныйПароль'
```

После успеха текущая сессия аннулируется — потребуется войти заново уже с новыми данными.

Важные эффекты смены учётных данных:

- **Все существующие сессии аннулируются** (увеличивается счётчик `login_epoch` у пользователя), поэтому после смены панель автоматически выполняет выход и перенаправляет на страницу входа — нужно войти заново.
- Если на момент смены была включена **2FA, она автоматически отключается** (флаг и секрет сбрасываются). Двухфакторную аутентификацию после смены логина/пароля придётся настроить заново.

Если 2FA включена, перед отправкой формы открывается окно «Изменить учетные данные» с подсказкой «Введите код из приложения, чтобы изменить учетные данные администратора.» — менять учётные данные можно только подтвердив текущий код 2FA.

2.5. Секретный путь (URI-путь / webBasePath) и порт панели

Эти параметры находятся в разделе **Настройки** → **Панель** и напрямую влияют на «скрытность» и доступность панели. Применяются после сохранения и **перезапуска панели**.

Поле	Текст (RU)	Значение по умолчанию	Описание
Порт панели	«Порт панели» (`panelPort`), подсказка «Порт, на котором работает панель»	**2053**	TCP-порт веб-интерфейса.

Поле	Текст (RU)	Значение по умолчанию	Описание
URI-путь	«URI-путь» (`panelUrlPath`), подсказка «Должен начинаться с '/' и заканчиваться '/'»	`/`	Секретный базовый путь (`webBasePath`). Панель доступна только по нему (например, `/мой-секрет/`).
IP-адрес для управления панелью	«IP-адрес для управления панелью» (`panelListeningIP`), подсказка «Оставьте пустым для подключения с любого IP»	пусто	Адрес, на котором слушает панель. Пусто = все интерфейсы.
Домен панели	«Домен панели» (`panelListeningDomain`), подсказка «Оставьте пустым для подключения с любых доменов и IP.»	пусто	Ограничение доступа по домену (Host).
Путь к публичному ключу сертификата панели	`publicKeyPath` , подсказка «Введите полный путь, начинающийся с '/'»	пусто	Сертификат TLS для HTTPS-доступа к панели.
Путь к приватному ключу сертификата панели	`privateKeyPath` , та же подсказка	пусто	Приватный ключ TLS.

Поведение базового пути (`webBasePath`):

- Значение нормализуется автоматически: если не начинается с `/` , символ добавляется в начало; если не заканчивается на `/` , добавляется в конец. То есть фактически путь всегда вида `/…/` .
- Базовый путь применяется к самой панели, к ассетам и к cookie сессии (cookie выдаётся только для этого пути).

> Рекомендации безопасности (раздел «Предупреждения безопасности»): панель сама показывает предупреждения, если конфигурация «слишком публична»:
>
> - «Панель работает по обычному HTTP — настройте TLS для продакшна.»
> - «Стандартный порт 2053 широко известен — измените его на случайный.»
> - «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»
>
> Иными словами, для боевого сервера следует задать **нестандартный порт**, **нетривиальный URI-путь** и **TLS-сертификат**.

Пример: «скрытная» конфигурация панели для продакшна. В разделе **Настройки** → **Панель** задайте примерно такие значения:

Поле	Значение
Порт панели	`34571` (случайный, вместо 2053)

Поле	Значение
URI-путь	`/aXf9Qm2/` (нетривиальный, начинается и заканчивается на `/`)
Путь к публичному ключу сертификата панели	`/etc/letsencrypt/live/panel.example.com/fullchain.pem`
Путь к приватному ключу сертификата панели	`/etc/letsencrypt/live/panel.example.com/privkey.pem`

После сохранения и перезапуска панель будет доступна только по `https://panel.example.com:34571/aXf9Qm2/`, а предупреждения безопасности исчезнут.

2.6. Время жизни сессии (таймаут)

Поле **«Продолжительность сессии»** (`sessionMaxAge`) находится среди настроек панели/интервалов.

Поле	Текст (RU)	Значение по умолчанию	Единица	Описание
Продолжительность сессии	«Продолжительность сессии», подсказка «Продолжительность сессии в системе (значение: минута)»	**360**	минуты	Срок жизни cookie-сессии администратора.

Поведение:

- Значение задаётся в **минутах** (по умолчанию 360 минут = 6 часов) и при настройке cookie переводится в секунды.
- Если значение **больше 0**, у cookie сессии выставляется соответствующий `MaxAge`. По истечении этого срока cookie перестаёт действовать и при следующем запросе пользователь получает «Сессия истекла. Войдите в систему снова».
- Сессия также становится недействительной досрочно при смене учётных данных или первом включении 2FA (за счёт механизма `login_epoch`, см. 2.4 и 2.2) и при явном выходе (`POST /logout`).
- Cookie сессии помечается `HttpOnly`, с политикой `SameSite=Lax`; флаг `Secure` выставляется при прямом HTTPS-доступе к панели.

Помимо самого таймаута есть связанное уведомление: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`, подсказка «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», по умолчанию `0`) — позволяет получать предупреждение заранее.

2.7. LDAP (синхронизация и аутентификация)

Раздел LDAP даёт две возможности: (1) аутентифицировать вход администратора по LDAP, если локальный пароль не подошёл, и (2) периодически синхронизировать состояние клиентов (включён/выключен VLESS-флаг) из каталога.

Как используется при входе: сервер сначала проверяет локальный bcrypt-хеш пароля. Если он **не совпал** и LDAP включён, панель пытается аутентифицировать пользователя в каталоге: при заданном `Bind DN` выполняется служебный bind, затем по фильтру и атрибуту ищется запись пользователя, и делается попытка

bind под найденным DN с введённым паролем. Успех означает вход. (После успешной LDAP-аутентификации, если включена 2FA, всё равно проверяется код TOTP.)

Поля раздела:

Поле	Текст (RU)	Значение по умолчанию	Описание
Включить LDAP-синхронизацию	«Включить LDAP-синхронизацию» (`enable`)	**false**	Главный выключатель LDAP-интеграции.
LDAP-хост	«LDAP-хост» (`host`)	пусто	Адрес LDAP-сервера.
Порт LDAP	«Порт LDAP» (`port`)	**389**	Порт. Для LDAPS обычно 636.
Использовать TLS (LDAPS)	«Использовать TLS (LDAPS)» (`useTls`)	**false**	При включении используется схема `ldaps://` (по умолчанию — с проверкой сертификата сервера).
Пропускать проверку TLS-сертификата	«Пропускать проверку TLS-сертификата» (`ldapInsecureSkipVerify`)	**false**	Добавлено в 3.4.2. Отключает проверку сертификата сервера при LDAPS — для внутренних/самоподписанных CA. Подсказка: «Небезопасно — отключает проверку сертификата сервера. Используйте только с внутренними/недоверенными CA». Переключатель доступен **только** при включённом «Использовать TLS (LDAPS)».
Bind DN	«Bind DN» (`bindDn`)	пусто	DN служебной учётной записи для первичного bind/поиска. Если пусто — bind не выполняется (анонимный поиск).
Пароль bind	подсказки: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено — введите новое значение для замены»	пусто	Пароль для `Bind DN` . Хранится отдельно; чтобы оставить прежний, поле оставляют пустым.
Base DN	«Base DN» (`baseDn`)	пусто	Корень поддерева, в котором ведётся поиск (поиск рекурсивный, по всему поддереву).
Фильтр пользователя	«Фильтр пользователя» (`userFilter`)	`(objectClass=person)`	LDAP-фильтр выбора учётных записей. При аутентификации логин подставляется в фильтр с экранированием.

Поле	Текст (RU)	Значение по умолчанию	Описание
Атрибут пользователя (username/email)	«Атрибут пользователя (username/email)» (`userAttr`)	`mail`	Атрибут, сопоставляемый с логином/идентификатором клиента (например, `mail` или `uid`).
Атрибут VLESS-флага	«Атрибут VLESS-флага» (`vlessField`)	`vless_enabled`	Атрибут, определяющий, должен ли VLESS-доступ клиента быть включён.
Общий атрибут флага (опц.)	«Общий атрибут флага (опц.)» (`flagField`), подсказка «Если задано, переопределяет флаг VLESS — напр. shadowInactive.»	пусто	Если задан, используется вместо `vless_enabled` .
Truthy-значения	«Truthy-значения» (`truthyValues`), подсказка «Через запятую; по умолчанию: true,1,yes,on»	`true,1,yes,on`	Список значений атрибута флага, трактуемых как «включено».
Инвертировать флаг	«Инвертировать флаг» (`invertFlag`), подсказка «Включите, когда атрибут означает «отключено» (напр. shadowInactive).»	**false**	Инвертирует смысл флага.
Расписание синхронизации	«Расписание синхронизации» (`syncSchedule`), подсказка «Строка типа cron, напр. @every 1m»	`@every 1m`	Периодичность синхронизации (ключ настройки `ldapSyncCron`) в cron-подобном формате. С 3.10.0 принимаются `@every <интервал>` , макросы `@hourly` / `@daily` / `@weekly` / `@monthly` и crontab как из пяти полей (`30 8 * * *` — каждый день в 8:30), так и из шести с секундами (`0 30 8 * * *`). Нераспознаваемое значение отклоняется при сохранении настроек с ошибкой `ldapSyncCron: cron schedule <…> is not valid: …` ; пустое поле допустимо и означает `@every 1m` . Расписание читается при запуске панели.
Теги входящих	«Теги входящих» (`inboundTags`), подсказка «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	пусто	Ограничивает, на каких inbound разрешены авто-операции. Если inbound нет: «Входящие не найдены. Сначала создайте входящий.»

Поле	Текст (RU)	Значение по умолчанию	Описание
Авто-создание клиентов	«Авто-создание клиентов» (`autoCreate`)	**false**	Создавать клиента в указанных inbound, если он появился в каталоге.
Авто-удаление клиентов	«Авто-удаление клиентов» (`autoDelete`)	**false**	Удалять клиента, если он исчез из каталога.
Объём по умолчанию (ГБ)	«Объём по умолчанию (ГБ)» (`defaultTotalGb`)	**0**	Лимит трафика для авто-создаваемых клиентов (0 = без лимита).
Срок по умолчанию (дни)	«Срок по умолчанию (дни)» (`defaultExpiryDays`)	**0**	Срок действия для авто-создаваемых клиентов (0 = бессрочно).
Лимит IP по умолчанию	«Лимит IP по умолчанию» (`defaultIpLimit`)	**0**	Ограничение числа одновременных IP (0 = без ограничения).

Особенности логики флага синхронизации: при чтении атрибута флага (`flagField` , по умолчанию `vless_enabled`) значение считается «включено», если оно входит в список truthy-значений; при включённой инверсии результат меняется на противоположный. Атрибут пользователя (`userAttr`) используется как ключ сопоставления (email/имя) — записи без значения этого атрибута пропускаются.

> Безопасность: рекомендуется включать **TLS (LDAPS)**, чтобы пароли bind и проверяемые пароли не передавались открытым текстом, а для `Bind DN` использовать учётную запись с минимально необходимыми правами на чтение.

Пример: типовая конфигурация LDAP-синхронизации (Active Directory). Заполнение полей раздела для каталога, где статус доступа хранится в атрибуте `userAccountControl` -подобного флага, а сопоставление идёт по почте:

Поле	Значение
LDAP-хост	`ldap.example.com`
Порт LDAP	`636`
Использовать TLS (LDAPS)	включено
Bind DN	`CN=svc-3xui,OU=Service,DC=example,DC=com`
Base DN	`OU=Users,DC=example,DC=com`
Фильтр пользователя	`(objectClass=person)`
Атрибут пользователя (username/email)	`mail`
Атрибут VLESS-флага	`vless_enabled`
Truthy-значения	`true,1,yes,on`
Расписание синхронизации	`@every 5m`

При такой настройке каждые 5 минут панель пройдёт по поддереву `OU=Users`, сопоставит клиентов по `mail` и включит/выключит VLESS-доступ по значению `vless_enabled`.

> До 3.10.0 планировщик панели понимал crontab только из шести полей (с секундами): привычное пятипольное выражение вроде `30 8 * * *` сохранялось без ошибки, но задача синхронизации при запуске не регистрировалась и молча никогда не выполнялась. Теперь такое значение работает, а сохранённое ранее нераспознаваемое значение больше не отключает синхронизацию: панель пишет в лог предупреждение «LDAP sync: cron schedule <…> is not valid: …; using default @every 1m» и синхронизирует раз в минуту, пока расписание не исправлено.

---

3. Обзор / Дашборд

Дашборд («Дашборд», в английском интерфейсе — *Overview*) — это стартовая страница панели. Она в режиме реального времени показывает состояние сервера и процесса Xray. Все показатели приходят со стороны сервера. Фоновый планировщик пересобирает снимок **каждые 2 секунды** и рассылает его всем открытым вкладкам через WebSocket; раз в минуту накопленные ряды метрик сбрасываются на диск. HTTP-эндпоинт `GET /status` отдаёт последний кэшированный снимок.

Ниже разобран каждый показатель и каждый элемент управления на странице.

В боковом меню панели (и в мобильной шторке) с версии 3.4.2 есть кнопка **«Документация»** (иконка книги) — открывает официальную документацию `https://docs.sanaei.dev/`. Также в 3.4.2 проведено сквозное улучшение **доступности**: значки-кнопки и кликабельные элементы получили aria-подписи и активацию по Enter/Space (для скринридеров и навигации с клавиатуры). Позднее доступность проверена по всей панели: у вкладок на узких экранах сохраняются имена для скринридера, а подсказки полей (например, к сроку действия inbound) связаны с самими полями (#6907).

3.1. Общие принципы сбора данных

- Снимок собирается библиотекой `gopsutil`. Если конкретный замер не удался, поле остаётся нулевым, а в лог пишется предупреждение (`get cpu percent failed`, `get uptime failed` и т. п.) — это не валит весь дашборд, просто соответствующая плитка покажет 0/N-A.
- «Мгновенные» скорости (CPU %, сеть, диск I/O) вычисляются как разница между текущим и предыдущим снимком, делённая на интервал в секундах. Поэтому при первой загрузке страницы значения скоростей могут быть нулевыми, пока не накопится второй замер.
- Историю можно посмотреть в разделе «История системы» (*System History*) — графики строятся по тем же рядам данных, что описаны ниже (см. п. 3.12).

Контейнеры LXC (LXCFS)

Если панель работает в LXC-контейнере с LXCFS, показатели относятся к самому контейнеру, а не к хосту. На плитке ЦП вместо «ядра / потоки · МГц» выводится `N vCPU`, а у времени работы подпись `CT` вместо `OS`:

- ЦП — загрузка считается по счётчику cgroup v2 контейнера и нормируется на видимые vCPU и на квоту, если она задана; топология и частота процессора хоста не показываются.
- Подкачка и время работы берутся из виртуализированных файлов LXCFS, а не с хоста, поэтому плитки больше не сочетают память контейнера с подкачкой хоста (#6920).
- История ЦП и подкачки контейнера ведётся отдельно от более ранних замеров хоста: на графиках не смешиваются значения разных областей.
- Показатель, который нельзя измерить (первый замер ЦП после запуска или раскладка, которую нельзя проверить, например cgroup v1), показывается прочерком «—», а не нулём. То же касается списка узлов, истории ЦП узла и времени работы в уведомлениях Telegram и Discord; узлы хранят такие признаки в колонке `nodes.unavailable_metrics`. В статусе (`GET /panel/api/server/status`) это поля `resourceScope` (`container`) и `unavailableMetrics`.

3.2. ЦП (CPU)

Плитка «ЦП» (*CPU*) показывает текущую загрузку процессора в процентах, а также параметры самого процессора.

Показатель	Описание
Загрузка ЦП, %	Доля занятого процессорного времени за последний интервал. Сглаживается экспоненциальным средним (EMA, коэффициент `alpha = 0.3`), чтобы скачки не дёргали индикатор. Значение всегда зажато в диапазоне 0–100 %. При самом первом замере возвращается 0 (инициализация базовой точки).
Логические процессоры	Количество логических ядер — то есть с учётом Hyper-Threading.
Физические ядра	Число физических ядер.
Частота	Базовая частота процессора в МГц. Запрашивается лениво и кэшируется: первый успешный замер сохраняется, повторная попытка делается не чаще чем раз в 5 минут, а сам запрос ограничен таймаутом 1,5 с (на некоторых системах запрос частоты отвечает медленно).

CPU-загрузка алгоритмически считается так: при наличии нативной платформенной реализации используется она, иначе — расчёт по дельтам счётчиков процессорного времени (busy / total). Guest- и GuestNice-время исключаются, чтобы не учитывать его дважды.

3.3. Память (RAM)

Плитка «Память» (*RAM*) показывает использовано и всего. Отображается в виде «использовано / всего» и/или процента заполнения. В историю записывается процент.

3.4. Подкачка (Swap)

Плитка «Подкачка» (*Swap*) показывает использовано и всего. Если файл/раздел подкачки не настроен (всего = 0), показатель равен нулю; в исторический ряд при отсутствии swap пишется 0.

3.5. Диск (Storage)

Плитка «Диск» (*Storage*) показывает использовано и всего, при этом учитывается **только корневой раздел** `/`. В историю «Использование диска» (*Disk Usage*) пишется процент заполнения. Отдельно собирается дисковый ввод-вывод (чтение / запись, байт/с) как дельта счётчиков за интервал — он отображается на вкладке «Диск I/O» истории.

3.6. Время работы системы (Uptime)

Показатель «Время работы системы» (*Uptime*). Это время с момента загрузки **всего сервера** (в секундах), а не время работы панели или Xray. Отдельно хранится аптайм процесса Xray (см. п. 3.9), а также число потоков панели (в переводе — «Потоки» / *Threads*).

Память, занимаемая панелью

Рядом с показателями процесса панели выводится объём оперативной памяти, который занимает сам процесс 3X-UI. Это значение берётся из реального RSS процесса (как его видит операционная система) и совпадает с тем, что показывают системные утилиты. Число снижается по мере освобождения памяти. Раньше панель выводила внутренний счётчик Go, который завышал расход памяти (например, ~300 МБ на простаивающем сервере с одним клиентом) и никогда не уменьшался — теперь этого артефакта нет. Дополнительно периодический фоновый процесс возвращает неиспользуемую память операционной системе, чтобы показатель отражал фактическое потребление.

3.7. Загрузка системы (Load average)

Блок «Нагрузка на систему» (*System Load*) — массив из трёх чисел `[Load1, Load5, Load15]`. Подпись-подсказка: «Средняя загрузка системы за последние 1, 5 и 15 минут» (*System load average for the past 1, 5, and 15 minutes*). График истории называется «Средняя нагрузка системы (1 / 5 / 15 мин)». В исторические ряды значения пишутся раздельно: `load1`, `load5`, `load15`.

Это стандартный Unix-показатель: среднее число процессов, находящихся в очереди на выполнение. Ориентир — сравнивать с числом ядер: загрузка, устойчиво превышающая количество физических ядер, говорит о перегрузке.

3.8. Сеть: скорость и общий объём трафика

Учитываются **только физические интерфейсы**. Виртуальные и туннельные интерфейсы исключаются: это `lo` / `lo0`, а также всё, что начинается с `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Значения суммируются по всем оставшимся интерфейсам.

Общая скорость (*Overall Speed*) — мгновенная скорость, дельта счётчиков за интервал:

Показатель	Описание
Загрузка / отдача (подпись «Загрузка» / *Upload*)	Исходящая скорость, байт/с.
Скачивание / приём (подпись «Скачать» / *Download*)	Входящая скорость, байт/с.

Общий объём трафика (*Total Data*) — накопленные счётчики с момента старта системы:

Показатель	Описание
Отправлено (подпись «Отправлено» / *Sent*)	Всего отправлено байт.
Получено (подпись «Получено» / *Received*)	Всего получено байт.

Дополнительно собираются скорости пакетов (пакетов/с) и суммарные счётчики пакетов — они показываются на вкладке «Сетевые пакеты» (*Network Packets*) истории. Ряды истории по сети: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. IP-адреса сервера

Блок «IP-адреса сервера» (*IP Addresses*) показывает `IPv4` и `IPv6`. Внешние адреса определяются через сторонние сервисы (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` для IPv4 и аналогичные для IPv6). Список перебирается по очереди до первого успешного ответа; таймаут на каждый запрос — 3 с.

Особенности:

- Результат **кэшируется и периодически обновляется**: адрес перепроверяется раз в 5 минут, поэтому после смены внешнего IP сервера дашборд подхватывает новый адрес без перезапуска панели (#6933). Опрос идёт в фоне — сбор показателей его не ждёт.
- Если ни один сервис не ответил, в поле остаётся `N/A`; временный сбой не стирает последний известный адрес. Неудачный запрос IPv4 повторяется через минуту, а для IPv6 пауза между попытками растёт — 1, 2, 5 и затем 15 минут, чтобы не тратить время на сетях без IPv6.
- Рядом есть кнопка-«глаз» для скрытия/показа адресов — подсказка «Скрыть или показать IP-адреса сервера» (*Toggle visibility of the IP*). Это только визуальное скрытие в интерфейсе (например, для скриншотов), на сами адреса оно не влияет.

3.10. TCP/UDP соединения

Блок «Количество соединений» (*Connection Stats*) показывает общее число активных TCP- и UDP-соединений на сервере (по всей системе, не только Xray). График истории — «Активные соединения (TCP / UDP)» (*Active Connections*), ряды `tcpCount`, `udpCount`.

3.11. Статус Xray и управление процессом

Карточка «Xray» показывает состояние процесса Xray-core и даёт им управлять.

Состояния

Значение	Подпись	Перевод	Когда выставляется
`running`	«Запущен»	*Running*	Процесс Xray запущен.
`stop`	«Остановлен»	*Stopped*	Процесс не запущен, и при этом нет зафиксированной ошибки запуска.
`error`	«Ошибка»	*Error*	Процесс не запущен, но зафиксирована ошибка запуска. Текст ошибки показывается во всплывающем окне с заголовком «Ошибка при запуске Xray» (*An error occurred while running Xray*).
—	«Неизвестно»	*Unknown*	Отображается, пока статус ещё не получен.

Рядом со статусом выводится **версия Xray**.

Кнопки управления

- **Стоп** (*Stop*). Вызывает `POST /stopXrayService`. При успехе панель рассылает по WebSocket новое состояние `stop` и уведомление «Xray успешно остановлен» (*Xray service has been stopped*), при ошибке

— состояние `error` с текстом. Важно: если панель доступна *через* сам Xray, остановка Xray может оборвать связь с панелью — при прямом подключении к панели проблемы нет.
- **Перезапуск** (*Restart*). Вызывает `POST /restartXrayService`. Перед действием показывается подтверждение «Перезапустить xray?» с пояснением «Перезагружает сервис xray с сохранённой конфигурацией». При успехе — состояние `running` и уведомление «Xray успешно перезапущен» (*Xray service has been restarted successfully*). Перезапуск применяет текущую сохранённую конфигурацию — используйте его после изменения настроек.

> Примечание. В этом форке для дашборда добавлено полноценное управление Start / Stop / Restart для всех типов авторизации; в исходном UI 3x-ui отдельной кнопки «старт» нет — запуск выполняется перезапуском.

Кнопка просмотра логов Xray

На карточке Xray есть кнопка просмотра логов Xray (*Logs*). Она появляется только тогда, когда в конфигурации Xray настроен access-лог: встроенный просмотрщик читает именно этот файл, поэтому без access-лога кнопка скрыта. Видимость кнопки привязана к отдельному признаку `accessLogEnable` и больше не зависит от лимита IP — онлайн-список и лимит IP-адресов продолжают работать даже без access-лога (см. п. 8).

Выбор версии Xray

Раздел «Выбор версии» (*Version*) позволяет переключить Xray-core на другой релиз. Список версий загружается через `GET /getXrayVersion`:

- Источник — GitHub API репозитория `XTLS/Xray-core` (`/releases`). Запросы кэшируются на **15 минут**; при сбое GitHub отдаётся последний удачно полученный список, чтобы пикер не пустел.
- В список попадают только релизы вида `X.Y.Z` версии **26.6.27 и новее**.

Подсказки: «Выберите нужную версию» (*Choose the version you want to switch to.*) и предупреждение «Важно: старые версии могут не поддерживать текущие настройки» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Переключение: `POST /installXray/:version`. Сценарий:

Пример. Переключиться на конкретную версию Xray-core (cookie сессии должен быть уже получен авторизацией):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v26.10.21' \
  -b cookie.txt
```

Здесь `v26.10.21` — тег из списка, который отдаёт `GET /getXrayVersion`. Версия обязана присутствовать в этом списке, иначе панель ответит отказом. С версии 3.4.2 минимально допустимая для установки версия Xray поднята до **26.6.27**, а в комплекте 3.5.0 идёт ядро **Xray 26.7.11**. Попутные изменения ядра: шифры Shadowsocks `none` / `plain` и VMess `none` / `zero` удалены (сохранённые конфиги переписываются автоматически: SS — на поддерживаемый шифр, VMess — на `auto`), а незашифрованный VLESS/Trojan-

outbound на публичный адрес отклоняется при сохранении — ядро с таким конфигом не запустилось бы. В комплекте 3.9.0 шло ядро **Xray 26.9.30**, в комплекте 3.10.0 — **Xray 26.10.10**, в комплекте 3.11.0 — **Xray 26.10.21** (26.10.10 отозван в апстриме); ключи, которые новое ядро отвергает или читает иначе, панель переписывает при первом старте (маски XDNS, стратегия доменов WireGuard-outbound, `udpHop` в клиентских масках, см. 1.3 и 11.11).

1. Выбранная версия проверяется на наличие в актуальном списке релизов (иначе — отказ).
2. Xray останавливается.
3. Под текущую ОС и архитектуру скачивается архив `Xray-<os>-<arch>.zip` с GitHub (поддерживаются amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; для Windows — `xray.exe`). Размер архива и бинарника ограничены 200 МБ.
4. Бинарник заменяется атомарно (через временный файл + переименование) и помечается исполняемым.
5. Xray запускается заново.

Перед переключением показывается диалог «Переключить версию Xray» (*Do you really want to change the Xray version?*) с описанием «Это изменит версию Xray на #version#». При успехе — уведомление «Xray успешно обновлён» (*Xray updated successfully*).

3.12. Обновление панели (3X-UI)

Блок проверки обновлений панели. Данные приходят через `GET /getPanelUpdateInfo`:

Поле	Описание
Текущая версия панели	Версия установленной панели.
Последняя версия панели	Последний релиз 3x-ui, полученный с GitHub.
Доступно обновление	Признак того, что последняя версия новее текущей. Если обновление не нужно — показывается «Панель обновлена» / «Обновлено».

Кнопка **«Обновить панель»** (*Update Panel*) запускает `POST /updatePanel`. Подсказка: «Это обновит 3X-UI до последнего релиза и перезапустит сервис панели». Перед запуском — подтверждение «Вы действительно хотите обновить панель?» с текстом «Это обновит 3X-UI до версии #version# и перезапустит сервис панели».

Особенности и ограничения:

- Самообновление поддерживается **только на Linux** (на других ОС возвращается ошибка).
- По API (`POST /panel/api/server/updatePanel`) обновление запускает токен с областью `admin` или `node-admin`; `monitor` и `node-sync` получают HTTP 403 (см. 13.9).
- Скрипт-обновлятор скачивается из официального репозитория (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, лимит 2 МБ) и запускается через `bash`, по возможности изолированно через `systemd-run`. Сам `update.sh` проверяет SHA-256 архива релиза и берёт `x-ui.sh` и юниты службы из тега этого релиза (см. 16.5).

- При успешном запуске показывается «Обновление панели началось» (*Panel update started*); если проверка обновления не удалась — «Проверка обновления панели не удалась». Во время установки выводится предупреждение «Установка в процессе. Не обновляйте страницу».
- С 3.10.0 окно подтверждения закрывается сразу после нажатия «Подтвердить», и дальше страница показывает только индикатор «Установка в процессе. Не обновляйте страницу» с номером устанавливаемой версии (#6766); раньше подтверждение оставалось открытым до конца обновления. После успешного обновления страница перезагружается сама, при сбое выводится «Не удалось обновить панель», а если панель не сообщила результат вовремя — «Не удалось подтвердить завершение обновления».

3.13. Обновление гео-файлов (GeoIP / GeoSite)

Кнопка/диалог обновления гео-баз вызывает `POST /updateGeofile` (все файлы) или `POST /updateGeofile/:fileName` (один файл). Обновление работает по строгому белому списку имён и источников:

Файл	Источник
`geoip.dat`, `geosite.dat`	`Loyalsoldier/v2ray-rules-dat` (latest)
`geoip_IR.dat`, `geosite_IR.dat`	`chocolate4u/Iran-v2ray-rules` (latest)
`geoip_RU.dat`, `geosite_RU.dat`	`runetfreedom/russia-v2ray-rules-dat` (latest)

Поведение:

- Имя файла валидируется: запрещены `..`, слэши, абсолютные пути; допустимы только `[a-zA-Z0-9._-]+.dat`. Файлы вне белого списка не скачиваются.
- Используется условный запрос `If-Modified-Since`: если на сервере-источнике файл не менялся (HTTP 304), повторно он не качается, лишь обновляется отметка времени.
- Каждый скачанный файл сверяется с SHA-256 из файла `<файл>.sha256sum`, опубликованного источником в том же релизе. Файлы одного источника сначала складываются во временный каталог и ставятся на место, только если проверку прошли все; иначе прежние файлы остаются, а в ошибке будет «Error downloading Geofile '...': does not match the published SHA-256 checksum...». Сбой одного источника не мешает обновить файлы остальных.
- Xray **перезапускается** (чтобы подхватить новые базы), только если хотя бы один файл действительно заменён; если все источники ответили 304, перезапуска и обрыва соединений нет.

Пример. Обновить только российские гео-базы, не трогая остальные файлы:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b
cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b
cookie.txt
```

Чтобы обновить сразу все файлы из белого списка — вызовите `POST /updateGeofile` без имени файла.

- Диалоги: «Вы действительно хотите обновить геофайл?» с «Это обновит файл #filename#» для одного файла и «Это обновит все геофайлы» для кнопки «Обновить все». Успех — «Геофайлы успешно обновлены».

3.14. Резервное копирование и восстановление базы данных

Блок «Бэкап и восстановление» (*Backup & Restore*). Поведение зависит от используемой СУБД (SQLite по умолчанию или PostgreSQL).

Экспорт базы (Резервная копия)

Кнопка «Экспорт базы данных» / «Резервная копия» (*Back Up*) вызывает `GET /getDb`. Файл отдаётся как вложение:

- **SQLite**: сначала выполняется checkpoint (сброс WAL), затем скачивается файл `x-ui.db`. Подсказка: «Нажмите, чтобы скачать файл .db, содержащий резервную копию вашей текущей базы данных…».
- **PostgreSQL**: скачивается дамп `x-ui.dump` в кастомном формате (`pg_dump --format=custom --no-owner --no-privileges`). На сервере должны быть установлены клиентские инструменты PostgreSQL; иначе — ошибка про отсутствие `pg_dump`.

Импорт базы (Восстановление)

Кнопка «Импорт базы данных» / «Восстановление» (*Restore*) загружает файл через `POST /importDB` (поле формы `db`). Подсказка: «Нажмите, чтобы выбрать и загрузить файл .db… для восстановления базы данных из резервной копии».

Флажок «Keep this machine's settings» (3.7.0, включён по умолчанию). Импорт больше не перетирает сетевую идентичность этой панели: адреса и порты прослушивания, базовый путь, пути к сертификатам и идентичность узла берутся у текущей установки, а не из загруженного файла. Это то, чего почти всегда хотят при восстановлении на другой машине — иначе панель после импорта поднималась на чужом порту, с чужим базовым путём и становилась недоступна. Снимите флажок, чтобы вернуть прежнее поведение — полный клон файла.

Сценарий для **SQLite** безопасный, с откатом:

1. Файл проверяется на формат SQLite и сохраняется во временный файл, затем проверяется целостность.
2. Xray останавливается, текущая БД закрывается и переименовывается в `*.backup` (фолбэк).
3. Новый файл встаёт на место рабочей БД, выполняется инициализация и миграция. Если что-то пошло не так — восстанавливается фолбэк.
4. Xray запускается заново.

Для **PostgreSQL** с 3.5.0 «Восстановление» принимает **три вида файлов** (тип определяется по содержимому, не по расширению): архив `pg_dump` (`PGDMP`) — применяется через `pg_restore --clean --if-exists --single-transaction …`; **SQLite-базу** `.db` (обычный бэкап) и **SQLite-миграционный** `.dump` — они проверяются, при необходимости пересобираются и импортируются в PostgreSQL **одной транзакцией** тем же движком, что и `x-ui migrate-db --dsn` (при ошибке PostgreSQL остаётся нетронутым). Диалог выбора файла принимает `.dump,.db` на обеих СУБД; загрузка `pg_dump`-архива на SQLite-панель даёт понятную ошибку. Подсказка прямо предупреждает: «Это заменит все текущие данные».

После успешного импорта (на обеих СУБД) сервер через 3 секунды перезапускает **панель целиком** — так же, как кнопка «Перезапустить панель», — чтобы вступили в силу параметры, которые читаются только при

старте, например пути подписок (`subPath` , `subJsonPath` , `subClashPath`) из загруженного файла. Страница показывает «Перезапустить панель…», ждёт 5 секунд и перезагружается; если после этого откроется страница входа, используйте учётные данные из загруженной базы.

Сообщения: «База данных успешно импортирована», «Произошла ошибка при импорте базы данных», «… при чтении базы данных», «…при получении базы данных».

Файл миграции (между SQLite и PostgreSQL)

Кнопка «Скачать файл миграции» (*Download Migration*) вызывает `GET /getMigration` . С 3.5.0 она показывается **только на PostgreSQL-панелях** и скачивает `x-ui.db` — готовую SQLite-базу, собранную из данных PostgreSQL (путь «обратно на SQLite»). На SQLite-панелях кнопка убрана как избыточная: обычный бэкап `.db` теперь и так восстанавливается прямо в PostgreSQL-панель.

3.15. Дополнительные элементы интерфейса

- **Индикатор онлайн-клиентов.** Дашборд ведёт ряд `online` (*Online Clients* / «Клиенты онлайн») — число клиентов с активным соединением. Считается при работающем Xray (иначе 0) и записывается в историю на том же 2-секундном такте. График — вкладка «Онлайн».
- **История системы (графики).** Кнопка/раздел «Графики» → «История системы» с вкладками: «Пропускная способность», «Пакеты», «Диск I/O», «Онлайн», «Нагрузка», «Соединения», «Использование диска». Данные тянутся по `GET /history/:metric/:bucket` ; допустимые интервалы агрегации (bucket, сек): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, на вкладку приходит до 60 точек. В самом селекторе диапазона на странице доступны кнопки **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (bucket'ы `2, 60, 180, 360, 720, 1440, 2880, 10080` соответственно). На длинных диапазонах **2d** и **7d** подписи времени по оси дополнены датой в формате `MM-DD HH:MM` . Хранение организовано трёхуровневым прореживанием (rollup): свежие данные держатся с шагом 2 с за последний **час**, далее усредняются до шага 1 мин за **48 часов** и до шага 10 мин за **7 суток**. Поэтому графики (CPU, RAM, трафик, пакеты, соединения, диск, онлайн, нагрузка) можно просматривать за период **до 7 суток** (раньше — до 48 часов), причём чем дальше в прошлое, тем грубее детализация. Допустимые метрики: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15` . Подпись «Последние 2 минуты» соответствует bucket = 2 (режим реального времени).

Пример. Получить ряд загрузки ЦП за последние ~2 минуты (bucket = 2 с, до 60 точек) и тот же ряд, агрегированный по 5 минут (bucket = 300 с):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Метрику можно заменить на любую допустимую (`mem` , `netUp` , `tcpCount` , `load1` и т. д.). Bucket вне белого списка `2, 30, 60, 180, 360, 720, 1440, 2880, 10080` будет отвергнут.

- **Метрики Xray** — отдельный блок с потреблением памяти и сборкой мусора Xray (ряды `xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs`) и «Обсерваторией» (состояние исходящих соединений). Работают, только если в конфигурации Xray задан блок `metrics` (`listen 127.0.0.1:11111` , тег `metrics_out`); иначе показывается «Конечная точка метрик Xray не настроена».

В окне метрик Xray свой селектор диапазона с кнопками **2m, 1h, 3h, 6h, 12h** (bucket'ы `2, 60, 180, 360, 720`).

Пример блока, который включает плитку метрик Xray. В разделе настроек Xray должны присутствовать одновременно `metrics` (с тегом) и inbound, который этот тег слушает:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Адрес `127.0.0.1:11111` намеренно не выставляется наружу — панель опрашивает его локально.

- **Переключатель тёмной темы.** Находится в общем меню/шапке, а не в самом дашборде. Варианты: «Тема» (*Theme*) с вариантами «Темная» и «Очень темная» (*Ultra Dark*). Это чисто визуальная настройка оформления, на работу панели не влияет.
- **Прочие ссылки** в окружении дашборда (из меню/нижней панели): «Логи», «Конфигурация» — просмотр итогового JSON Xray (`GET /getConfigJson`), «Документация».

3.16. Что изменилось на дашборде в 3.7.0

- **Боковую панель можно закрепить.** Рельс из иконок раскрывался при наведении и схлопывался обратно; теперь есть «Pin sidebar» / «Unpin sidebar» — раскрытое меню остаётся раскрытым.
- **Панель устанавливается как приложение (PWA).** Установка работает только при доступной сети: офлайн-режима нет, кэширующего service worker'а нет — приложение просто открывается в своём окне.
- **Плитка «Uptime»** — время работы.
- **Локализованы уровни логов и события доступа**: Debug / Info / Notice / Warning / Error и DIRECT / BLOCKED / PROXY выводятся на языке интерфейса, а не строками из ядра.
- **Логи AmneziaWG** — в инструментах появился просмотр состояния туннелей: последнее рукопожатие, интерфейс, inbound, endpoint, простой и события. Пусто — «No AmneziaWG peers are currently up».

> Про виртуализацию таблиц: в 3.7.0 её добавляли (#6187), но **в релизе она выключена** — прокрутка списков осталась обычной.

3.17. Командная строка (Ctrl+K)

Командная строка (*Command Palette*) — окно быстрого поиска и команд, доступное на любой странице панели. Открывается сочетанием **Ctrl+K** или **⌘K** (панель принимает оба модификатора на любой ОС) либо кнопкой **«Поиск…»** в боковом меню и в мобильной шторке; на кнопке показана подсказка `Ctrl K` (на устройствах Apple — `⌘ K`), а в свёрнутом меню — всплывающая подпись «Командная строка». Закрывается повторным сочетанием, клавишей **Esc** или щелчком вне окна. ↑/↓ выбирают строку, **Enter** выполняет её.

Поле ввода подписано «Введите команду или найдите клиентов, инбаунды, страницы…». Результаты сгруппированы:

Группа	Что показывается	Что происходит при выборе
Клиенты	до 8 клиентов, найденных на сервере по введённой строке (`GET /panel/api/clients/list/paged?search=…`, запрос уходит после паузы в наборе 300 мс): email, израсходованный / общий трафик (`∞` — без лимита), комментарий, значок «включён / выключен»	переход на страницу «Клиенты» с поиском по email. Кнопка **«Копировать подписку»** в строке копирует ссылку подписки клиента; она есть, если у клиента задан `subId` и включён сервер подписок
Входящие	до 8 inbound, у которых примечание, тег, протокол или порт содержат введённую строку, с метками протокола, транспорта и безопасности	переход на страницу «Входящие» с поиском по примечанию (или порту)
Навигация	разделы «Дашборд», «Входящие», «Клиенты», «Группы», «Узлы», «Хосты», «Исходящие», «Маршрутизация», «Настройки панели», «Конфигурации Xray», «Документация API»; ищутся по названию и по английским ключевым словам (`cpu`, `vless`, `warp`, `swagger` и т. п.)	переход в раздел
Настройки	вкладки настроек панели («Панель», «Учетная запись», «Telegram-бот», «Email», «Discord бот», «Подписка», «Форматы подписки», «Балансировщики подписки») и разделы конфигурации Xray («Основное», «Эндпоинт метрик», «Ограничения соединения», «Лог», «Балансировщик», DNS, «Исходящие», «Базовые соединения», «Расширенный шаблон»)	переход сразу к нужной вкладке
Действия	«Перезапустить службу Xray», «Тема», «Создать подключение», «Добавить клиента»	см. ниже

Пока поле пустое, в списке только навигация, настройки и действия; клиенты и inbound появляются по мере ввода.

- **«Перезапустить службу Xray»** выполняется сразу (`POST /panel/api/server/restartXrayService`), **без окна подтверждения**, в отличие от кнопки на карточке Xray (см. 3.11); при успехе — «Служба Xray успешно перезапущена». Как и любой перезапуск Xray, он обрывает текущие соединения клиентов.
- **«Тема»** переключает оформление по кругу: светлая → тёмная → очень тёмная.
- **«Создать подключение»** и **«Добавить клиента»** только открывают страницу «Входящие» или «Клиенты»; форму создания нужно открыть уже там.

---

4. Inbounds: создание и общие параметры

Раздел **«Входящие»** (англ. *Inbounds*) — это список всех точек входа Xray, через которые подключаются клиенты. Каждый inbound хранит как «панельные» поля (примечание, лимит трафика, расписание сброса), так и сырые JSON-блоки конфигурации Xray (`settings` , `streamSettings` , `sniffing`).

Создание выполняется кнопкой **«Создать подключение»** (*Add Inbound*), редактирование — **«Изменить подключение»** (*Modify Inbound*). Обе операции отправляются на API-эндпоинты `POST /add` и `POST /update/:id` .

Ниже разобраны все поля формы, **не** относящиеся к настройкам конкретного протокола (клиенты, шифрование, REALITY/TLS) и **не** относящиеся к транспорту/потоку (вкладки **«Поток»**, **«Безопасность»**) — это темы отдельных разделов.

4.1. Общие поля формы

Remark (Примечание)

Параметр	Значение
Поле	`remark`
Тип	строка
По умолчанию	пусто

Человекочитаемое имя inbound, отображаемое в списке и в заголовках диалогов («Удалить подключение "{remark}"?» и т. п.). Подпись поля — **«Примечание»**. На работу Xray не влияет, нужно только для удобства администрирования; рекомендуется задавать уникальные осмысленные имена, так как они подставляются в названия экспортируемых файлов и в подтверждения массовых операций.

В списке inbound рядом с примечанием в скобках выводятся примечания привязанных к нему групп управляемых хостов (у группы без примечания — её первый адрес), чтобы несколько точек входа — IPv4, IPv6, CDN — были видны без открытия inbound. Учитываются только включённые группы; если их больше двух, показываются первые две и «+N», а полный перечень — во всплывающей подсказке. Поле поиска над списком находит inbound и по этим примечаниям.

Protocol (Протокол)

Параметр	Значение
Поле	`protocol`
Подпись	**«Протокол»**
Валидация	`required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto amneziawg tuic`

Выпадающий список протокола inbound. Допустимые значения:

Значение	Примечание
`vmess`	
`vless`	
`trojan`	
`shadowsocks`	
`wireguard`	
`hysteria`	Hysteria v2 — это `hysteria` с `streamSettings.version = 2`, отдельного протокола нет
`http`	
`mixed`	socks/http на одном порту
`tunnel`	
`tun`	принимается валидатором, отдельной константы протокола нет
`mtproto`	прокси Telegram; обслуживается процессом `mtg`, см. 5.12
`amneziawg`	WireGuard с обфускацией; встроенный движок панели, см. 5.10
`tuic`	TUIC v5; нативный сервер внутри процесса панели, см. 5.13

Поле обязательное (`required`). Выбор протокола определяет, какие поля настроек клиентов и какой транспорт будут доступны (см. протокол-специфичные разделы).

> Важно: при сохранении сервис нормализует `streamSettings`. Транспортные настройки оставляются только для протоколов `vmess`, `vless`, `trojan`, `shadowsocks`, `hysteria`, а также `wireguard` и `tunnel` (им блок нужен для `sockopt`, WireGuard — ещё и для масок Finalmask); для остальных (`http`, `mixed`, `tun`, `mtproto`, `amneziawg`, `tuic`) поле `streamSettings` **принудительно очищается**.

Для inbound типа `tunnel`/TProxy, у которых блок `streamSettings` не содержит ключа `security` (бестранспортный вариант), форма открывается и сохраняется без ошибки валидации `streamSettings.security Invalid input`.

Listen IP (IP прослушивания)

Параметр	Значение
Поле	`listen`
Тип	строка
По умолчанию	пусто → Xray слушает на `0.0.0.0` (все IP)

IP-адрес, на котором inbound принимает соединения. Подсказка к полю:

> «Оставьте пустым для прослушивания всех IP-адресов».

При генерации конфигурации Xray пустое значение заменяется на `0.0.0.0`. Помимо IP, поле принимает **путь Unix-сокета** — подсказка:

> «Можно также указать путь Unix-сокета (например, /run/xray/in.sock) или имя абстрактного сокета с префиксом @ (например, @xray/in.sock), чтобы слушать сокет вместо TCP-порта — в этом случае задайте порт 0».

Таким образом, поле принимает две формы Unix-сокета: путь в файловой системе (`/run/xray/in.sock`) и абстрактное имя сокета с префиксом `@` (`@xray/in.sock`). В обоих случаях задайте `Port` равным `0`.

Меняют это поле, когда нужно ограничить inbound одним интерфейсом (например, `127.0.0.1` для inbound, работающего только как fallback-цель за Nginx) либо когда inbound слушает Unix-сокет.

Пример. Inbound, который слушает только локальный интерфейс (типичная fallback-цель за Nginx) и Unix-сокет:

```
listen = 127.0.0.1   порт = 8443
listen = /run/xray/in.sock   порт = 0
```

Port (Порт)

Параметр	Значение
Поле	`port`
Подпись	**«Порт»**
Валидация	`gte=0,lte=65535`
По умолчанию	— (задаётся пользователем)

TCP/UDP-порт прослушивания. Допустимы значения от `0` до `65535`. Значение `0` используется только в паре с прослушиванием на Unix-сокете (см. выше).

При сохранении сервис проверяет конфликт порта: два inbound не могут одновременно занимать пересекающиеся `listen:port` для одного и того же транспорта (TCP/UDP). Транспорт вычисляется из протокола и `streamSettings` / `settings` : например, `hysteria` , `wireguard` , `amneziawg` и `tuic` всегда занимают UDP, `mtproto` — TCP, `kcp` / `quic` — UDP, а большинство остальных — TCP. При конфликте сохранение отклоняется с ошибкой.

С 3.8.5 та же проверка конфликта порта выполняется не только при сохранении, но и при **включении** уже существующего inbound (переключатель «Enable»): раньше отключённый inbound мог держать порт незамеченным, и конфликт всплывал только при попытке его включить. Теперь включение отклоняется с той же ошибкой, что и при сохранении, — она называет строку-владельца порта, — а флаг `enable` не

переключается. Отдельно (и при сохранении, и при включении) отклоняется inbound на порту, который уже форвардит пир **AmneziaWG** (клиентская переадресация портов), — ошибка называет email этого клиента.

С 3.9.0 два inbound'а на одном номере порта допускаются, если они относятся к **разным семействам адресов**. IPv6-inbound, слушающий на `::` с включённым **`v6only`** в `streamSettings.sockopt`, занимает только IPv6 и может сосуществовать с IPv4-inbound (`0.0.0.0` или конкретный IPv4) на том же порту. Голый `0.0.0.0` или `::` без `v6only` ядро открывает как двухстековый сокет — он по-прежнему занимает порт целиком и конфликтует с любым inbound'ом на нём. Два inbound'а одного семейства, как и две одинаковые строки прослушивания, конфликтуют, как раньше.

Отдельно панель не даёт занять **зарезервированный порт внутреннего Xray API** (тег `api`, по умолчанию `62789` на `127.0.0.1`): локальный TCP-inbound, чей адрес прослушивания пересекается с этим портом на loopback, отклоняется с той же ошибкой конфликта порта. Реальный порт API читается из шаблона конфигурации Xray (с запасным значением `62789`). На узлах (nodes) это ограничение не действует — у них собственный Xray.

> **С 3.9.0 сохранение настроек inbound'а не изменяет его клиентов.** Форма редактирования inbound'а больше не загружает и не отправляет список клиентов, а переключатель «Enable» показывается только при добавлении inbound'а (у существующего состояние переключается из списка). Поля жизненного цикла клиента (`enable`, срок, квота, автопродление, счётчики трафика) меняются только отдельными операциями над клиентом. Поэтому клиент, добавленный, удалённый, продлённый, сброшенный или исчерпавший квоту параллельно (ботом, API, LDAP или другим администратором, пока была открыта форма), больше не откатывается сохранением inbound'а. Импорт inbound'а, выгруженного из панели 2.x, теперь принимает устаревший строковый `tgId` и другие старые типы полей клиента (#6663).

> Тег Xray (`Tag`, уникальный) генерируется автоматически из порта и транспорта в формате `in-<порт>-<tcp|udp|tcpudp|any>`; для inbound, развёрнутого на узле, добавляется префикс `n<nodeId>-`. При коллизии к тегу дописывается `-2`, `-3` и т. д. Пользователь обычно тег не редактирует.

Total traffic (Всего трафика, GB)

Параметр	Значение
Поле	`total` (в **байтах**)
Подпись	**«Общий расход»**
По умолчанию	`0`

Суммарный лимит трафика inbound. В форме значение вводится в гигабайтах, в БД хранится в байтах. Подсказка к полю:

> «= Безлимит. (единица: ГБ)».

То есть `0` **означает безлимит**. Это лимит на уровень всего inbound (а не отдельных клиентов); фактический израсходованный трафик хранится в полях `up` (отправлено) и `down` (получено) и сравнивается с `total`.

Expiry date / Duration (Дата окончания / срок)

Параметр	Значение
Поле	`expiryTime` (Unix-метка времени)
Подпись	**«Дата окончания»** (англ. *Duration*)
По умолчанию	пусто / `0`

Срок действия inbound. Подсказка:

> «Оставьте пустым, чтобы было бесконечным».

Пустое значение (`0`) означает бессрочный inbound. Значение хранится как Unix-метка; форма позволяет задавать как конкретную дату, так и срок в днях (относительный отсчёт от текущего момента — англ. подпись поля *Duration*).

Enabled (Включить)

Параметр	Значение
Поле	`enable`
Подпись	**«Включить»** (англ. *Enabled*)
По умолчанию	задаётся при создании

Признак активности inbound. Переключение этого флага в списке обрабатывается отдельным «лёгким» эндпоинтом `POST /setEnable/:id`, а не полным обновлением — это сделано специально, чтобы не пересериализовывать весь блок `settings` (всех клиентов) при каждом щелчке по переключателю на inbound с тысячами клиентов. При выключении inbound удаляется из работающего Xray, при включении — добавляется обратно.

Node / Deploy to (Узел / Развернуть на)

Параметр	Значение
Поле	`nodeId`
Подпись	**«Развернуть на»**, **«Локальная панель»**
По умолчанию	пусто (локальная панель)

Выбор, где физически работает inbound: на локальной панели или на одном из зарегистрированных узлов. С 3.5.0 список «Развернуть на» (как и фильтр «Узлы» над списком inbound) поддерживает **поиск по вводу**, а над самим списком inbound появилось **поле поиска** (по примечанию, порту и протоколу; работает вместе с

фильтром узлов). Особенность реализации: `nodeId = 0` нормализуется в `nil`, так как `0` — это не валидный id узла, а артефакт привязки формы; `nil` / `0` означает локальную панель. При сохранении inbound на офлайн-узле возможен тост «изменение синхронизируется, когда узел переподключится». С версии 3.4.2 этот тост показывается только когда узел действительно офлайн или выключен (раньше он мог появляться и при сохранении на онлайн-узел).

Стратегия адреса для ссылок (Share address strategy)

Параметр	Значение
Поле	стратегия + (опционально) пользовательский адрес
Подпись	**«Стратегия адреса для ссылок»** (англ. *Share address strategy*)
По умолчанию	**«Адрес прослушивания inbound»** (*Inbound listen*)

Выпадающий список определяет, какой адрес подставляется в **экспортируемые ссылки-шеринга и QR-коды** этого inbound. Значения:

Значение	Подпись	Что подставляется
`node`	**«Адрес узла»** (*Node address*)	адрес узла, на котором работает inbound
`listen`	**«Адрес прослушивания inbound»** (*Inbound listen*)	адрес прослушивания самого inbound
`custom`	**«Пользовательская»** (*Custom*)	свой адрес из поля **«Пользовательский адрес для ссылок»** (*Custom share address*)

При выборе **«Пользовательская»** появляется поле **«Пользовательский адрес для ссылок»**; в него вводят хост или IP **без схемы и порта** (значение валидируется). Вариант **«Адрес узла»** показывается в списке, только если существует включённый узел, на котором этот inbound может работать; иначе он скрыт, а значение приводится к **«Адрес прослушивания inbound»**.

Та же стратегия задаёт адрес сервера и в выдаче подписки (см. 10.4); привязанные к inbound управляемые хосты имеют приоритет — адрес по стратегии используется только для хостов без собственного адреса. Для MTProto поле не показывается: адрес его ссылок задаётся хостами (см. 5.12).

Порядок в подписке (Subscription sort order)

Параметр	Значение
Поле	`subSortIndex`
Подпись	**«Порядок в подписке»**
По умолчанию	`1`

Позиция ссылок этого inbound в выдаче подписки. Подсказка к полю:

> «Позиция ссылок этого входящего в выдаче подписки (страница подписки и клиентские приложения). Меньшие значения идут первыми; при равных значениях сохраняется порядок создания. Не влияет на список входящих в панели».

Начиная с 3.8.0 значение может быть **отрицательным**: чтобы вывести один inbound первым, достаточно задать ему, например, `-1`, не перенумеровывая остальные. `0` и пустое значение приводятся к `1` — при сохранении, а уже записанные в базу значения — при запуске панели. Если хотя бы у одного inbound значение отличается от `1`, в списке появляется сортируемая колонка **«Порядок»**. Как порядок применяется к форматам подписки — в 10.1.

Отключение XTLS flow и узкая правка порядка (3.7.0)

«Исключить из подписок» (`excludeFromSub`, с 3.9.0) — переключатель в общих параметрах inbound'а (рядом с «Порядок в подписке»): скрывает ссылки этого inbound'а из вывода подписки (raw, JSON, Clash), **оставляя сам inbound включённым и рабочим** (#6463). В отличие от выключения inbound'а (которое останавливает обслуживание в Xray), трафик и аутентификация продолжают работать, а клиенты этого inbound'а по-прежнему учитываются в заголовке `Subscription-Userinfo` (расход, квота, срок). Полезно, чтобы отдать клиенту работающий сервер, не публикуя его ссылку.

«Disable XTLS flow» (`disableFlow`, только VLESS) — переключатель, который **запрещает панели самой подставлять** `xtls-rprx-vision` этому inbound'у, даже когда транспорт формально позволяет. Нужен там, где Vision мешает: например, туннелируемый XHTTP-inbound с шифрованием VLESS. На остальных ваших inbound'ах в той же подписке Vision продолжает работать — отключение точечное.

Порядок в подписке правится отдельным запросом. Поле `subSortIndex` (см. 10.1) теперь можно менять узким эндпоинтом `POST /panel/api/inbounds/:id/subSortIndex`. Раньше позиция менялась только через `/update/:id`, куда приходилось отправлять inbound целиком вместе со всем списком клиентов: двое, правящих один inbound одновременно, затирали изменения друг друга.

4.2. Sniffing (Сниффинг)

Вкладка **«Сниффинг»** редактирует блок `sniffing` конфигурации Xray, который хранится как сырой JSON. Sniffing позволяет Xray «подсматривать» реальное доменное имя/протокол внутри соединения для целей маршрутизации.

Подполе	Подпись	Назначение
`enabled`	(переключатель вкладки)	Включает/выключает сниффинг для inbound
`destOverride`	—	Список протоколов, для которых перехватывается адрес назначения: `http`, `tls`, `quic`, `fakedns`
`metadataOnly`	**«Только метаданные»**	Использовать только метаданные соединения, без чтения полезной нагрузки
`routeOnly`	**«Только маршрутизация»**	Применять результат сниффинга только для маршрутизации, не переписывая адрес назначения

Подполе	Подпись	Назначение
`domainsExcluded`	**«Исключённые домены»**	Домены, исключаемые из сниффинга
(исключённые IP)	**«Исключённые IP»**	IP-адреса, исключаемые из сниффинга

- **`destOverride`** — набор сниферов: `http` (определяет домен из HTTP-заголовка Host), `tls` (из SNI), `quic` (из QUIC ClientHello), `fakedns` (сопоставление с FakeDNS-пулом). Обычно для определения домена включают `http` и `tls`.

Пример блока `sniffing` (определять домен по HTTP и TLS, использовать результат только для маршрутизации, не трогая локальную сеть):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — когда включён, Xray не считывает содержимое первого пакета и опирается только на метаданные; полезно, чтобы не нарушать протоколы, у которых нельзя «подсматривать» данные.
- **`routeOnly`** — результат сниффинга используется только правилами маршрутизации; адрес соединения в outbound при этом не переписывается на распознанный домен.

> Примечание: панель хранит `sniffing` как непрозрачный JSON-блок и при сохранении ничего к нему не добавляет — все значения по умолчанию для этих галочек формируются на стороне приложения-клиента. В сыром виде блок можно отредактировать через раздел «JSON входящего» (см. ниже).

4.3. Allocate (стратегия распределения портов)

Блок `allocate` в `streamSettings` управляет тем, как Xray распределяет порты прослушивания. Это часть конфигурации Xray; панель хранит и передаёт его как часть `streamSettings`/JSON inbound. Параметры (по терминологии Xray-core):

Подполе	Назначение	Значения / по умолчанию
`strategy`	Стратегия выделения портов	`always` — всегда слушать заданный порт (по умолчанию); `random` — периодически менять прослушиваемые порты внутри диапазона
`refresh`	Интервал смены портов (минуты) при `random`	целое число минут (рекомендуется 5; минимум — 2)
`concurrency`	Сколько портов держать открытыми одновременно при `random`	целое (по умолчанию 3; не более трети ширины диапазона портов)

`strategy: always` оставляет inbound на одном порту (стандартный режим). `strategy: random` нужен для сценариев anti-blocking, когда inbound периодически «прыгает» по диапазону портов; в этом случае имеют смысл `refresh` и `concurrency`. Менять эти значения нужно только при сознательном использовании режима случайных портов.

Пример блока `allocate` в `streamSettings` (режим случайных портов: держать 3 порта открытыми, менять каждые 5 минут):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Чтобы это работало, `port` inbound задают диапазоном (например, `20000–20100`).

4.4. External Proxy (Внешний прокси)

Поле **«External Proxy»** относится к настройкам формирования ссылок-приглашений и хранится в `streamSettings` inbound. Оно задаёт список альтернативных внешних адресов (хост/порт, при необходимости с принудительным TLS — **«Принудительный TLS»**), которые подставляются в клиентские ссылки вместо реального `listen:port` inbound.

Используется, когда клиенты должны подключаться не напрямую к серверу, а через внешний прокси/реверс/CDN: тогда в общих ссылках указывается публичный адрес такого фронтенда. На сам процесс приёма соединений Xray это не влияет — это «косметика» генерируемых ссылок. Связанные поля формы: **«Принудительный TLS»**, **«Fingerprint»**, подписи каждой записи.

4.5. Fallbacks (Fallback'и)

Раздел **«Fallback'и»** задаёт правила перенаправления соединений, не совпавших ни с одним клиентом inbound. Доступен для master-inbound на TLS-транспорте (VLESS/Trojan TCP-TLS). Управляется через эндпоинты `GET /:id/fallbacks` / `POST /:id/fallbacks`.

Подсказка к разделу:

> «Когда соединение на этом инбаунде не совпадает ни с одним клиентом, оно перенаправляется в другое место. Выберите дочерний инбаунд ниже, чтобы поля маршрутизации (SNI / ALPN / Path / xver) заполнились автоматически из его транспорта, либо оставьте выбор пустым и задайте Dest напрямую (например, 8080 или 127.0.0.1:8080), чтобы перенаправить на внешний сервер, такой как Nginx. Каждый дочерний инбаунд должен слушать на 127.0.0.1 с security=none».

Раздел fallback'ов показывается только для inbound VLESS/Trojan поверх RAW (TCP) с безопасностью TLS или REALITY. Новый inbound стартует с `security=none`, поэтому раздел сначала может выглядеть отсутствующим. В этом состоянии (VLESS/Trojan, RAW/TCP, безопасность ещё не настроена) вместо раздела

выводится встроенная подсказка: fallback'и станут доступны после выбора TLS или Reality на вкладке **«Безопасность»**.

Поля строки fallback

Поле	По умолчанию	Описание
(дочерний inbound)	—	Выбор дочернего inbound (подпись **«Выберите инбаунд»**). Если выбран, поля Name/Alpn/Path/Dest могут авто-заполниться из его транспорта
Name	пусто (= любой)	Условие совпадения по имени (SNI/имени). Подпись «любой» — **«любой»**
Alpn	пусто	Условие совпадения по ALPN
Path	пусто	Условие совпадения по пути (для WS/HTTP-транспортов дочернего inbound)
Dest	авто	Куда перенаправлять. Плейсхолдер **«авто (listen:порт дочернего)»**. Можно указать порт (`8080`) или `host:port` (`127.0.0.1:8080`)
Xver	`0` (авто)	Версия PROXY-протокола (**«Xver»**): `0` — явно не задана, `1` или `2` — соответствующая версия PROXY protocol. У строки с дочерним inbound'ом без явного **Dest** при `0` версия выводится из дочернего inbound'а (см. ниже)
(порядок)	по позиции	Порядок применения правил; задаётся кнопками **«Вверх»**/**«Вниз»**

Логика сохранения: весь список фолбэков мастера заменяется атомарно. Строка, у которой нет ни выбранного дочернего inbound (`childId <= 0`), ни заданного `Dest` , **пропускается**. Если выбранный дочерний inbound равен id самого мастера, он обнуляется. При генерации итогового JSON: если `Dest` пуст, он вычисляется из дочернего inbound как `listen:port` , причём `0.0.0.0` / `::` / `::0` заменяются на `127.0.0.1` ; пустые поля `name` / `alpn` / `path` в выходной JSON не попадают; `xver` добавляется только если он больше 0.

PROXY protocol связанного fallback'а (#6835, #6880). У строки с выбранным дочерним inbound'ом и пустым **Dest** адрес берётся от дочернего inbound'а, а при незаданном **Xver** (`0`) к нему добавляется заголовок PROXY protocol версии 1, если дочерний inbound его принимает: `acceptProxyProtocol` включён в `sockopt` или в настройках его транспорта (RAW, WebSocket или HTTPUpgrade; RAW хранится как `tcp`). Поэтому Proxy Protocol у дочернего inbound'а можно включать в любом порядке — до или после добавления fallback'а, из панели или через API: строка с `xver` `0` больше не остаётся без заголовка, который дочерний inbound уже ждёт и из-за которого отвергал все соединения. Явно заданный **Xver** (`1` или `2`) не переопределяется; у строки с явным **Dest** заголовок автоматически не добавляется. Правка **IP прослушивания**, порта или флага PROXY protocol у дочернего inbound'а пересобирает конфигурацию всех мастеров, чей fallback выводится из него.

Пример итогового `settings.fallbacks` (трафик с `alpn=h2` уходит на WS-цель по пути `/ws` , всё остальное — на локальный Nginx на порту 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Последняя строка без `name` / `alpn` / `path` — это правило «по умолчанию», ловящее всё остальное.

Кнопки и подсказки раздела fallbacks

- **«Добавить фолбэк»** — добавить строку; **«Фолбэков пока нет»** — пустое состояние.
- **«Быстро добавить все подходящие»** / **«Добавить все»** — добавляет строку fallback для каждого подходящего inbound, ещё не подключённого. Результат: «Добавлено {n} фолбэк(ов)» или «Нет новых подходящих инбаундов».
- **«Заполнить из дочернего»** — переподтянуть поля маршрутизации (SNI/ALPN/Path/xver) из транспорта выбранного дочернего inbound; после выполнения — «Заполнено из дочернего».
- **«Изменить поля маршрутизации»** / **«Скрыть расширенные»** — показать/скрыть тонкие поля строки.
- Подписи **«Маршрутизирует, когда»** и **«По умолчанию — ловит всё остальное»** поясняют условие срабатывания каждой строки.

После сохранения фолбэков сервер вызывает рестарт Xray, чтобы новые `settings.fallbacks` вступили в силу.

4.6. Периодический сброс трафика

Блок **«Сброс трафика»** настраивает автоматический сброс счётчиков трафика inbound по расписанию. Описание:

> «Автоматический сброс счетчика трафика через указанные интервалы».

Параметр	Значение
Поле	`trafficReset`
Валидация	`omitempty,oneof=never hourly daily weekly monthly`
По умолчанию	`never`
Сопутствующее поле	`lastTrafficResetTime` — метка последнего сброса (подпись **«Последний сброс»**)

Выпадающий список:

Значение	Подпись
`never`	**«Никогда»**
`hourly`	**«Ежечасно»**
`daily`	**«Ежедневно»**

Значение	Подпись
`weekly`	**«Еженедельно»**
`monthly`	**«Ежемесячно»**

Для каждого периода зарегистрирован cron-джоб, запускающийся по соответствующему расписанию (`@hourly` , `@daily` , `@weekly` , `@monthly`). Джоб выбирает все inbound с заданным `trafficReset` и для каждого сбрасывает счётчики самого inbound (`up=0` , `down=0`) **и** трафик всех его клиентов. То есть периодический сброс затрагивает и inbound, и его клиентов.

Отключённые по квоте снова включаются. Клиент, исчерпавший квоту, отключается (в таблице трафика, в своей записи и в `settings` inbound'а), а inbound с заданным общим лимитом трафика выключается, когда `up` + `down` достигает лимита. Периодический сброс не только обнуляет счётчики, но и возвращает в работу:

- всех клиентов inbound'а, отключённых из-за исчерпания квоты (#6833): они снова попадают в работающее ядро, а не остаются отключёнными при нулевом расходе;
- сам inbound, если он выключен собственным лимитом трафика (#6834). Inbound, выключенный вручную до достижения лимита или по истечении срока, остаётся выключенным. Автоматическое включение действует только для inbound'ов этой панели, не для inbound'ов на узлах.

Пример значения поля. Чтобы счётчики обнулялись первого числа каждого месяца, в форме выбирают **«Ежемесячно»**, что сохраняется как:

```
{ "trafficReset": "monthly" }
```

Значение `never` (по умолчанию) полностью отключает автосброс.

4.7. JSON входящего (advanced)

Раздел **«Разделы JSON входящего»** даёт прямой доступ к сырым JSON-блокам inbound. Описание:

> «Полный JSON входящего и отдельные редакторы для settings, sniffing и streamSettings».

Доступны редакторы:

Вкладка	Подпись	Что редактирует
Всё	«Полный объект входящего со всеми полями в одном редакторе»	весь объект Inbound
Настройки	«Обёртка блока settings Xray»	поле `settings`
Sniffing	«Обёртка блока sniffing Xray»	поле `sniffing`
Stream	«Обёртка блока stream Xray»	поле `streamSettings`

Эти поля сериализуются как вложенные JSON-объекты: пустые блоки отдаются как `null` , а текст, не являющийся валидным JSON, оборачивается в строку, чтобы данные не терялись. Ошибки парсинга при сохранении выводятся с префиксом **«Расширенный JSON»**.

Редактор **Всё** не переформатирует текст, пока он остаётся корректным JSON: перевод строки, пробелы и отступы, не меняющие содержимого, остаются как введены, а курсор не прыгает (#6857). Изменения, сделанные в других полях формы, попадают в редактор как обычно; смена тега правит в тексте только тег; значение, которое форма всё равно не сохранит (например, нечисловой порт или удалённый раздел), заменяется тем, что реально будет сохранено, — редактор не показывает JSON, который панель не отправит.

Окно просмотра «JSON входящего», как и окно импорта inbound, использует полноценный редактор кода с подсветкой синтаксиса JSON (вместо обычного текстового поля): просмотр конфигурации — в подсвеченном режиме только для чтения, а импорт — в редактируемом, что упрощает чтение и правку.

4.8. Действия с inbound: QR / Edit / Reset / Delete и статистика

В списке и в карточке inbound доступны следующие действия (меню **«Меню»**):

Статистика трафика

Отображается агрегированный трафик inbound: **«Отправлено/получено»** (поля `up` / `down`), **«Всего трафика»**, **«Всего подключений»**. В карточке также — **«Создано»**, **«Обновлено»**, **«Дата окончания»**.

В списке inbounds есть колонка **Speed** с текущей скоростью трафика по каждому inbound (отдача/загрузка), вычисляемой из приращений счётчиков между опросами; та же живая скорость показывается в окне статистики inbound. Когда очередной опрос не даёт приращения, значение скорости сбрасывается.

В сводке клиентов на странице inbounds статус определяется по приоритету «исчерпан/завершён»: клиенты, у которых истёк срок или исчерпан трафик (и которым авто-задание сняло `enable`), относятся к статусу **«Исчерпан/завершён»** (*Depleted/Ended*), а не к серому **«Отключён»** (*Disabled*), и не считаются дважды. Классификация совпадает с той, что показана в карточке самого клиента, и корректно учитывает клиентов, привязанных к нескольким inbound.

QR-код и копирование ссылок

- **«Подробнее»** — раскрывает ссылки подключения и подписки.
- QR-код клиента: подсказка **«Нажмите на QR-код, чтобы скопировать»**.
- **«Копировать ссылку»** (англ. *Copy URL*), **«Экспорт ссылок»**. С версии 3.4.2 страничное действие **«Экспортировать все ссылки»** (`GET /panel/api/inbounds/allLinks` ; окно «Экспорт всех ссылок inbound», файл «All-Inbounds») формирует ссылки через движок подписок — применяет шаблон ремарки к каждому клиенту и предпочитает управляемые Host-эндпоинты (раньше использовалась фиксированная ремарка `inbound–email`).

Edit (Изменить)

«Изменить подключение» — открывает форму редактирования (`POST /update/:id`). При обновлении сервис перечитывает существующую строку, переносит изменённые поля, при необходимости перегенерирует тег (если старый тег был авто-сгенерирован) и синхронизирует рантайм Xray. Успех — тост **«Подключение успешно обновлено»**.

Reset Traffic (Сбросить трафик)

«Сбросить трафик» — обнуляет счётчики `up` / `down` именно этого inbound (`POST /:id/resetTraffic` , ставит `up=0, down=0`). Подтверждение:

> «Сбросить трафик "{remark}"?» / «Сбрасывает счётчики отправки/получения этого подключения до 0».

Сброс трафика inbound **не** трогает счётчики его клиентов (для них есть отдельные действия «Сброс трафика клиентов»). Ручной сброс не включает обратно inbound, выключенный по достижении собственного лимита трафика, — включите его переключателем; автоматически такой inbound возвращает только периодический сброс. После сброса инициируется рестарт Xray. Успех — тост **«Входящий трафик сброшен»**. Есть и массовый вариант — **«Сброс трафика всех подключений»** (`POST /resetAllTraffics`).

Delete (Удалить)

«Удалить подключение» (`POST /del/:id`). Подтверждение:

> «Удалить подключение "{remark}"?» / «Подключение и все его клиенты будут удалены. Это действие нельзя отменить».

Удаление снимает inbound с работающего Xray (при необходимости с рестартом). Успех — тост **«Подключение успешно удалено»**. Массовое удаление — `POST /bulkDel` , с пер-элементной отчётностью и не более чем одним рестартом Xray.

Прочие действия с клиентами inbound

В меню также доступны: **«Клонировать»** (копия inbound с новым портом и пустым списком клиентов), **«Удалить всех клиентов»** (`POST /:id/delAllClients` — удаляет всех клиентов, сам inbound сохраняется), **«Удалить отключенных клиентов»**, **«Привязать/Отвязать клиентов»**, **«Импортировать»**/**«Экспорт подключений»** (`POST /import`). Детали клиентских операций относятся к разделу о клиентах.

---

5. Протоколы

При создании входящего соединения (inbound) первым выбирается **Протокол** («Protocol»). Протокол определяет, какой способ аутентификации и шифрования трафика Xray-core будет применять к этому inbound, какой набор полей в `settings` потребуется заполнить, а также какие транспорты (`network`) и виды безопасности (TLS / REALITY) для него доступны.

Поле протокола задаётся один раз при создании inbound и **не меняется при редактировании** (в форме редактирования выпадающий список заблокирован). Чтобы сменить протокол, нужно создать новый inbound.

5.1. Список поддерживаемых протоколов

Сервер принимает следующий набор значений поля `Protocol` :

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
amneziawg tuic masque
```

> Начиная с версии **3.3.0** в перечень добавлено значение `mtproto` (прокси Telegram), с **3.7.0** — `amneziawg` (WireGuard с обфускацией против DPI), с **3.8.0** — `tuic` (TUIC v5 поверх QUIC), с **3.10.0** — `masque` (IP-туннель CONNECT-IP поверх HTTP/3 или HTTP/2, см. 5.15).

Значение в конфиге	Назначение	Клиентская модель
`vless`	Основной прокси-протокол (по умолчанию при создании inbound)	Клиенты с UUID, поддержка flow и пост-квантового шифрования
`vmess`	Классический прокси-протокол Xray	Клиенты с UUID и параметром `security`
`trojan`	Прокси, маскирующийся под обычный HTTPS	Клиенты с паролем
`shadowsocks`	Прокси Shadowsocks (включая SIP022 / 2022-blake3)	Клиенты с паролем; с 3.10.0 inbound всегда многопользовательский
`wireguard`	Inbound WireGuard	Peer'ы (а не клиенты)
`hysteria`	Inbound Hysteria (по умолчанию версия 2)	Клиенты с токеном `auth`
`http`	Классический HTTP-прокси (forward proxy)	Аккаунты user/pass, без учёта трафика
`mixed`	Совмещённый SOCKS + HTTP-прокси	Аккаунты user/pass
`tunnel`	Прозрачный форвардер (xray `dokodemo-door`)	Без клиентов
`tun`	TUN-интерфейс Xray на самом сервере	Без клиентов
`mtproto`	Прокси Telegram (MTProto), добавлен в 3.3.0; обслуживается отдельным процессом `mtg` , а не Xray	Без клиентов (доступ по секрету)

Значение в конфиге	Назначение	Клиентская модель
`amneziawg`	WireGuard с обфускацией против DPI, добавлен в 3.7.0; поднимается встроенным движком в процессе панели, а не Xray	Клиенты (один включённый клиент = один peer)
`tuic`	Прокси TUIC v5 поверх QUIC, добавлен в 3.8.0; с 3.9.0 — нативный сервер внутри процесса панели (трафик проходит через маршрутизацию Xray)	Клиенты с UUID и паролем
`masque`	MASQUE — IP-туннель (CONNECT-IP) поверх HTTP/3 или HTTP/2 с обязательным TLS, добавлен в 3.10.0	Клиенты с email и паролем (HTTP Basic)

> Замечание о `tun`: это inbound TUN-интерфейса самого Xray на сервере — без клиентов, без вкладок «Поток» и «Безопасность» (`streamSettings` при сохранении очищается). На вкладке «Протокол» задаются «Имя интерфейса» (`name`, по умолчанию `xray0`), MTU (`1500`), «Gateway», DNS, «Уровень пользователя» (`userLevel`), «Авто-маршруты системы» (`autoSystemRoutingTable`) и «Авто-интерфейс исходящих» (`autoOutboundsInterface`, по умолчанию `auto`); см. также 11.9.
>
> С 3.10.0 в форме TUN есть три параметра xray-core 26.9, которые раньше задать было нельзя (поле `desc`, введённое через JSON, ещё и пропадало при следующем сохранении):
>
> | Поле в UI | Ключ | Описание |
> |---|---|---|
> | «Тип адаптера» | `desc` | Только Windows: тип туннеля, с которым создаётся новый адаптер Wintun. Пусто — `Wintun` |
> | «Системный DNS на шлюз» | `autoSystemDnsToGateway` | Только Linux: направляет systemd-resolved на шлюз TUN, чтобы системные DNS-запросы шли через Xray. Нужен заполненный «Gateway»; если настроить не удалось, TUN не запускается |
> | «Блокировка утечек (WFP)» | `autoSystemWfpBlockLeak` | Только Windows, вместе с «Авто-маршрутами системы»; мультивыбор: `dns` удерживает DNS внутри TUN (нужны DNS-серверы), `misconfigtun` блокирует версию IP, которую ни один маршрут не ведёт в TUN |
>
> Если условие параметра не выполнено, ядро отказывается собрать или запустить TUN — поэтому подсказки полей называют платформу и требования.

> Замечание о Hysteria 2: отдельного протокола «hysteria2» нет. Это протокол `hysteria` с полем `streamSettings.version = 2`. Схема ссылки `hysteria2://` при генерации share-ссылок выбирается автоматически, когда версия потока равна 2.

Не все протоколы поддерживают распределение по узлам (nodes). На узлы можно разворачивать `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`, а также протоколы, которые запускает панель самого узла, — при достаточной версии его панели: `mtproto` (v3.5.0+), `amneziawg` (v3.7.0+), `tuic` (v3.8.0+) и, с 3.10.0, `masque` (проверка требует v3.9.1+, то есть из выпущенных версий — 3.10.0). Протоколы `http`, `mixed`, `tunnel`, `tun` работают только на локальной панели (см. 12.7).

5.2. Какие протоколы поддерживают TLS / REALITY / транспорт

Возможность включить тот или иной слой безопасности и транспорт зависит от протокола и выбранной сети (`streamSettings.network`):

Возможность	Доступна для протоколов	Допустимые сети (`network`)
TLS	`vmess` , `vless` , `trojan` , `shadowsocks` (а также всегда для `hysteria` и `masque` ; у `masque` TLS обязателен)	`tcp` , `ws` , `http` , `grpc` , `httpupgrade` , `xhttp`
REALITY	`vless` , `trojan`	`tcp` , `http` , `grpc` , `xhttp`
flow (`xtls-rprx-vision`)	только `vless`	только `tcp` , при `security = tls` или `reality`
Stream / транспорт (вкладка «Поток»)	`vmess` , `vless` , `trojan` , `shadowsocks` , `hysteria` , `masque`	—

Для протоколов `http` , `mixed` , `tun` , `mtproto` , `amneziawg` и `tuic` вкладка транспорта недоступна — у них нет stream-настроек Xray; у `wireguard` и `tunnel` она урезана (только `sockopt` , у WireGuard ещё Finalmask — см. 5.9 и 5.7). У `amneziawg` , `mtproto` и `tuic` недоступен и блок sniffing: соединения обслуживает не Xray. У `tuic` нет и TLS/REALITY Xray — его собственный обязательный TLS настраивается на вкладке **«Протокол»** (см. 5.13). У `masque` (с 3.10.0) выбор сети скрыт — транспорт всегда `masque` : на вкладке «Поток» задаются только «Путь», `sockopt` и Finalmask, а на вкладке «Безопасность» доступен только TLS (см. 5.15).

5.3. VLESS

Назначение: основной современный прокси-протокол. Поддерживает XTLS-Vision (`flow`), REALITY, а также пост-квантовое шифрование на уровне самого VLESS (поля `decryption` / `encryption`). Используется по умолчанию для новых inbound.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
`clients`	`[]`	Список клиентов. У каждого: `id` (UUID), `email` (обязателен), `flow` , лимиты (`limitIp` , `totalGB` , `expiryTime`), `enable` , `tgId` , `subId` , `comment` , `reset`
`decryption`	`none`	Параметр расшифрования на стороне сервера. Подпись в UI: «Расшифрование» (англ. «Decryption»)
`encryption`	`none`	Парный параметр шифрования (попадает в клиентскую ссылку). Подпись: «Шифрование» (англ. «Encryption»)
`fallbacks`	`[]`	Список fallback'ов (см. раздел про fallback'и); доступен, когда `network = tcp` и `security` = TLS или REALITY

Поле	Значение по умолчанию	Описание
`testseed`	(4 числа: 900, 500, 900, 256)	«Vision testseed» — 4 положительных целых для XTLS-Vision padding. Применяется только к клиентам с flow `xtls-rprx-vision`, иначе игнорируется

flow (`xtls-rprx-vision`)

`flow` задаётся **на клиенте**, а не на inbound, и принимает одно из трёх значений:

Значение	Смысл
`` (пусто)	Без XTLS-flow (по умолчанию)
`xtls-rprx-vision`	XTLS-Vision — рекомендуемый режим для VLESS поверх TCP+TLS/REALITY
`xtls-rprx-vision-udp443`	Тот же Vision, но с обработкой UDP/443 (QUIC)

Поле `flow` доступно для выбора у протокола `vless`, если выполнено одно из условий: `network = tcp` и `security` = `tls` либо `reality` (классический XTLS-Vision) **или** у inbound включено VLESS-шифрование (`decryption` / `encryption` не `none`, см. ниже) — тогда на любом транспорте, кроме XDRIVE (с 3.10.0; раньше только на XHTTP). Поле **Vision testseed** в форме показывается при тех же условиях, если хотя бы у одного клиента выбран flow `xtls-rprx-vision`. Как Vision соотносится с TLS и REALITY — в 7.1.

> **Vision внутри VLESS-шифрования — на любом транспорте (с 3.10.0).** xray-core выполняет Vision внутри VLESS-шифрования (vlessenc) независимо от транспорта, поэтому при включённых `encryption` / `decryption` flow `xtls-rprx-vision` допустим на любой сети, а не только на `xhttp`, как было раньше, и при любом слое безопасности, в том числе с REALITY. Исключение — транспорт XDRIVE (6.10): xray-core не принимает flow на нём даже при VLESS-шифровании, поэтому поле `flow` там недоступно, а панель не пишет flow клиента ни в серверный конфиг, ни в JSON-подписку. Панель передаёт `xtls-rprx-vision` в share-ссылки и подписки (включая формат Clash/Mihomo), так что клиент получает конфигурацию именно с Vision.
>
> **Flow снимается там, где транспорт его не несёт (с 3.10.0).** Если inbound перевели на транспорт, где Vision невозможен (например, с TCP+TLS на WebSocket без VLESS-шифрования), а у клиентов остался `xtls-rprx-vision`, панель теперь убирает flow из серверного конфига Xray. Раньше он там оставался (учитывался только переключатель отключения flow), сервер требовал Vision и отклонял всех клиентов. Та же проверка, повторяющая правила ядра, действует для ссылок, подписок и формы outbound'а (в ней от действующего flow зависит и доступность Mux); значение flow в записи клиента при этом не стирается.

Расшифрование / Шифрование (пост-квантовая аутентификация VLESS)

Поля `decryption` и `encryption` — это аутентификация на уровне самого VLESS (отдельно от транспортного TLS/REALITY). По умолчанию оба равны `none`. В форме под этими полями расположен блок **«Генерация ключей»** — выпадающий список режима и кнопка **«Сгенерировать»** (рядом — кнопка **«Очистить»**). Выпадающий список содержит шесть вариантов: **X25519 (native)**, **X25519 (xorpub)**, **X25519**

(random), **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** — то есть два типа ключа (классический X25519 и пост-квантовый ML-KEM-768), каждый в трёх режимах:

- **native** — базовая пара ключей выбранного типа;
- **xorpub** — производный режим с дополнительной обработкой публичной части;
- **random** — производный режим со случайной составляющей.

Выберите нужный режим в списке и нажмите **«Сгенерировать»**: панель заполнит **оба** поля (`decryption` и `encryption`) готовой парой значений под этот режим. Кнопка **«Очистить»** сбрасывает оба поля обратно в `none` .

Под блоком выводится строка состояния **«Выбрано: …»**, которая распознаёт по сгенерированной строке как тип ключа (X25519 или ML-KEM-768), так и режим (native / xorpub / random) и показывает их. Пустые поля или `none` отображаются как «None».

Технически кнопки обращаются к `GET /panel/api/server/getNewVlessEnc` (генерация ключей через `xray vlessenc`) и заполняют **оба** поля парными значениями вида `mlkem768x25519plus.native.<rtt>.<role>` (например, `decryption = mlkem768x25519plus.native.600s.server–x25519` , `encryption = mlkem768x25519plus.native.0rtt.client–x25519`). Параметр `decryption` остаётся на сервере, `encryption` уходит в клиентскую ссылку.

> Важно: при генерации конфигурации inbound для Xray панель удаляет лишнее: если в `settings` остаётся `encryption` (которое относится к клиентской стороне), оно **вырезается** из серверного конфига. На самом сервере остаётся только `decryption` .

Когда выбирать VLESS: это рекомендуемый вариант по умолчанию для нового inbound, особенно в связке с REALITY (без собственного сертификата) или с TLS + XTLS-Vision.

Пример: блок `settings` VLESS-inbound с одним клиентом и XTLS-Vision. Поле `flow` стоит на клиенте, `decryption` остаётся на сервере:

```
{
  "clients": [
    {
      "id": "d342d11e–d424–4583–b36e–524ab1f0afa4",
      "email": "user1",
      "flow": "xtls–rprx–vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Для связки REALITY соответствующий блок `streamSettings` (вкладка «Transport» → Security: REALITY) выглядит так:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Назначение: классический прокси-протокол Xray. Аутентификация по UUID, на клиенте дополнительно настраивается метод шифрования полезной нагрузки (`security`).

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
`clients`	`[]`	Список клиентов

У каждого клиента VMess (помимо общих полей `email` ,лимитов, `enable` , `tgId` , `subId` , `comment` , `reset`):

Поле клиента	Значение по умолчанию	Описание
`id`	—	UUID клиента
`security`	`auto`	Метод шифрования полезной нагрузки VMess. Допустимые значения: `aes-128-gcm` , `chacha20-poly1305` , `auto` , `none` , `zero`

Значение `security` :

- `auto` — Xray сам выбирает шифр в зависимости от платформы (рекомендуется);
- `aes-128-gcm` , `chacha20-poly1305` — фиксированные AEAD-шифры;
- `none` — без шифрования полезной нагрузки (имеет смысл только поверх TLS);
- `zero` — без шифрования и без аутентификации полезной нагрузки.

> Историческая совместимость: старые записи могли хранить `security: ""` — при чтении пустая строка приводится к `auto` . При генерации серверного конфига поле `security` у клиентов VMess **удаляется** из `settings` ,так как для inbound оно не требуется.

Когда выбирать VMess: для совместимости со старыми клиентами или существующими конфигурациями. Для новых развёртываний обычно предпочтительнее VLESS.

5.5. Trojan

Назначение: прокси, имитирующий обычный HTTPS-трафик. Аутентификация по паролю. Как и VLESS, поддерживает fallback'и и (при `network = tcp`) REALITY/TLS.

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
`clients`	`[]`	Список клиентов
`fallbacks`	`[]`	Список fallback'ов (доступен при `network = tcp` и TLS/REALITY)

У каждого клиента Trojan ключевое поле:

Поле клиента	Значение по умолчанию	Описание
`password`	—	Пароль клиента (обязателен, минимум 1 символ)
`email`	—	Уникальный идентификатор клиента

Остальные поля клиента — общие (`limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`).

Когда выбирать Trojan: когда нужна маскировка под HTTPS на 443 порту, в том числе с fallback'ами на веб-сервер (Nginx) для непрошеных подключений.

Пример: блок `settings` Trojan с fallback на локальный веб-сервер. Непрошеные подключения (без валидного пароля) уходят на Nginx, который слушает `127.0.0.1:8080`:

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Для fallback нужны `network = tcp` и Security = TLS или REALITY; иначе поле fallbacks недоступно.

5.6. Shadowsocks

Назначение: лёгкий прокси Shadowsocks. Поддерживает как устаревшие AEAD-шифры, так и новые методы SIP022 (`2022-blake3-*`). Inbound Shadowsocks всегда многопользовательский: доступ дают клиенты (см. раздел 8). Однопользовательского режима с 3.10.0 больше нет — он существовал только для метода `2022-blake3-chacha20-poly1305`, который из inbound'ов убран (см. ниже).

С 3.10.2 изменения клиентов (добавление, удаление, продление, сброс трафика, автоотключение) применяются **на лету** через API Xray, без пересоздания inbound'а: порт не закрывается, и живые соединения

остальных пользователей не рвутся. Раньше любая такая правка — в том числе пришедшая на узел от главной панели — пересоздавала inbound и обрывала все соединения на нём.

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
`method`	`2022-blake3-aes-256-gcm`	Метод шифрования inbound. Подпись в UI: «Метод шифрования» (англ. «Encryption method»)
`password`	``	Пароль inbound (для 2022-методов генерируется автоматически под выбранный метод)
`network`	`tcp,udp`	Транспорт. Подпись: «Сеть» (англ. «Network»). Варианты: `tcp,udp` (TCP, UDP), `tcp`, `udp`
`clients`	`[]`	Список клиентов
`ivCheck`	`false` (выкл.)	Переключатель «ivCheck» — защита от повторного использования IV

Методы шифрования (`method`)

Допустимый набор:

Метод	Категория
`aes-256-gcm`	Устаревший AEAD
`chacha20-poly1305`	Устаревший AEAD
`chacha20-ietf-poly1305`	Устаревший AEAD
`xchacha20-ietf-poly1305`	Устаревший AEAD
`2022-blake3-aes-128-gcm`	SS 2022 (рекомендуется)
`2022-blake3-aes-256-gcm`	SS 2022 (по умолчанию)
`2022-blake3-chacha20-poly1305`	SS 2022 — **только для outbound**; в форме inbound с 3.10.0 не предлагается

Логика панели по методам:

- **2022-методы** (`2022-blake3-aes-128-gcm`, `2022-blake3-aes-256-gcm`) считаются «SS 2022» и допускают несколько клиентов. Поле пароля (с кнопкой генерации, подгоняющей длину ключа под метод) показывается в форме именно для 2022-методов.
- **Устаревшие шифры** (`aes-*`, `chacha20-*`) работают по классической схеме «один метод + один пароль».

> Нормализация перед запуском Xray: для устаревших шифров каждый клиент должен нести `method`, совпадающий с методом inbound (иначе Xray падает с «unsupported cipher method:»). Для 2022-методов наоборот — поле `method` у клиента должно быть **пустым** (иначе Xray отвергает inbound с «users must have empty method»). Панель сама приводит данные в порядок при переключении метода.

> Перегенерация клиентских ключей при смене размера ключа: для Shadowsocks-2022 при смене метода шифрования на метод с другим размером ключа (например между `2022-blake3-aes-256-gcm` и `2022-blake3-aes-128-gcm`) панель автоматически перегенерирует клиентские PSK под новую длину при сохранении inbound. Иначе старые ключи остались бы прежней длины, и Xray отверг бы их. Следствие: затронутым клиентам нужно заново получить подписку — прежние ссылки перестанут подключаться.

> **`2022-blake3-chacha20-poly1305` убран из inbound'ов (с 3.10.0).** xray-core собирает Shadowsocks-2022-inbound с пользователями только для AES-методов (многопользовательский заголовок идентификации есть только у AES), а inbound панели всегда несёт клиентов. Поэтому выбор chacha20 не давал запуститься всему ядру, а ссылки на клиенте падали с «multi-key is not supported». Теперь форма этот метод не предлагает, а добавление и сохранение inbound'а (в том числе через API) отклоняется с ошибкой `shadowsocks inbound method … does not support multiple users, use a 2022-blake3-aes method`. При первом запуске 3.10.0 сохранённые inbound'ы с этим методом автоматически переводятся на `2022-blake3-aes-256-gcm`: длина ключа у него та же (32 байта), поэтому все ключи остаются действительными, но метод входит в ссылку — **клиентам нужно заново получить подписку или ссылку**. В outbound метод остаётся: одноключевое подключение к такому серверу работает. Вместе с однопользовательским режимом из списка inbound'ов исчезли и связанные с ним элементы — отдельный QR-код в строке inbound'а и блок ссылки уровня inbound в окне информации.

> **Проверка ключа сервера (с 3.10.0).** Пароль inbound'а для 2022-метода — это ключ ровно нужной длины: 16 байт для `2022-blake3-aes-128-gcm` и 32 байта для `2022-blake3-aes-256-gcm`, в base64 (проверка идёт тем же разбором, что в ядре, поэтому «сырая» строка, которую принимает ядро, тоже допустима). Раньше проверялись только ключи клиентов, и inbound с ключом не той длины (например, 32-байтный ключ на `aes-128` через API) сохранялся, после чего Xray не запускался вовсе — падали все inbound'ы. Теперь добавление и сохранение такого inbound'а отклоняются с ошибкой вида `shadowsocks inbound password: 2022-blake3-aes-128-gcm needs a 16-byte key, base64 encoded`. Уже сохранённый ключ неверной длины при первом запуске 3.10.0 заменяется новым случайным (в лог пишется «replaced a … server key of the wrong size; its share links changed»): такой inbound всё равно не работал, но ссылки на него нужно раздать заново.

Когда выбирать Shadowsocks: для простых развёртываний без TLS-маскировки; современный выбор — 2022-blake3-методы.

Пример: блок `settings` Shadowsocks для метода 2022-blake3 (многопользовательский режим). У inbound — свой пароль (ключ base64 нужной длины), у каждого клиента — свой пароль, поле `method` клиента **пустое**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMi1ieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Для legacy-шифров (`aes-256-gcm` и т. п.) — наоборот: пароль один на inbound, а `method` клиента должен совпадать с методом inbound.

5.7. Dokodemo-door / Tunnel (прозрачный форвардер)

Назначение: прозрачный форвардер (в панели — протокол `tunnel` , реализующий поведение `dokodemo-door`). Принимает трафик и переадресует его на заданный адрес/порт, без аутентификации и клиентов.

Поля блока `settings` :

Поле	Значение по умолчанию	Описание
`rewriteAddress`	(нет)	«Переписать адрес» (англ. «Rewrite address») — целевой адрес, на который перенаправляется трафик
`rewritePort`	(нет)	«Переписать порт» (англ. «Rewrite port») — целевой порт (0–65535)
`allowedNetwork`	`tcp,udp`	«Разрешённая сеть» (англ. «Allowed network»). Варианты: `tcp,udp` , `tcp` , `udp`
`portMap`	`{}`	«Сопоставление портов» — карта порт→порт (Record<string,string>)
`followRedirect`	`false` (выкл.)	«Следовать redirect» (англ. «Follow redirect») — использовать исходный адрес назначения из перехваченного соединения

> Вкладка «Transport» для Tunnel: у inbound этого типа доступна вкладка **«Transport»**, ограниченная настройкой `sockopt` — этого достаточно для режима **TProxy** (прозрачное проксирование/redirect через `sockopt.tproxy`). Выпадающий список выбора транспорта (`network`) и вкладка «Security» для Tunnel скрыты, так как TLS/REALITY этим типом не поддерживаются.

Когда выбирать: для прозрачного проксирования/перенаправления портов на внутренние сервисы.

Поле «Переписать порт» (`rewritePort`) можно оставлять пустым: при очистке значение просто исключается из настроек inbound, а не вызывает ошибку сохранения. (Раньше очистка этого поля приводила к ошибке валидации `settings.rewritePort` и блокировала сохранение, в том числе через вкладку JSON.)

5.8. SOCKS / HTTP (протокол `mixed`)

В этой сборке отдельного протокола `socks` нет — SOCKS и HTTP-прокси объединены в протокол **`mixed`** (совмещённый SOCKS + HTTP). Кроме того, есть отдельный чистый `http`-прокси.

5.8.1. Mixed (SOCKS + HTTP)

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
`auth`	`password`	«Auth» — режим аутентификации. Варианты: `password` (по логину/паролю) или `noauth` (без авторизации)
`accounts`	(опционально)	«Аккаунты» — список пар user/pass. При `auth = noauth` поле не пишется в конфиг
`udp`	`false` (выкл.)	Переключатель «UDP» — поддержка UDP через SOCKS
`ip`	`127.0.0.1`	«UDP IP» — локальный адрес для UDP-ассоциаций. Поле показывается только при включённом `udp`

Аккаунты добавляются кнопкой «Добавить»; при добавлении генерируются случайные логин (8 символов) и пароль (12 символов), которые можно отредактировать.

5.8.2. HTTP (чистый прокси)

Назначение: классический форвард-прокси HTTP. На уровне Xray не отслеживает клиентов как «биллинговых» (нет email/лимитов) — есть только список аккаунтов.

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
`accounts`	`[]`	«Аккаунты» — список пар user/pass (оба поля обязательны)
`allowTransparent`	`false` (выкл.)	«Разрешить прозрачный» (англ. «Allow transparent») — пересылать запросы с оригинальным заголовком Host

Когда выбирать SOCKS/HTTP: для локального или служебного прокси-доступа без сложной маскировки. `mixed` удобен тем, что один порт обслуживает и SOCKS, и HTTP-клиентов.

5.9. WireGuard (inbound)

Назначение: inbound WireGuard — VPN-туннель WireGuard, а не маскируемый прокси. Транспорт и TLS/REALITY к нему неприменимы.

С версии 3.4.2 WireGuard переведён на мультиклиентную модель (как VLESS/VMess/Trojan/Shadowsocks/Hysteria). Сам inbound хранит только серверную часть; устройства-пиры теперь добавляются как обычные **клиенты** (см. раздел 8) — с автоматической выдачей адреса в туннеле, поддержкой подписок, лимитов трафика/срока и групп. Прежний инлайн-список «Peer'ы / Добавить peer» из формы inbound убран: новый inbound создаётся без пиров, а каждому клиенту адрес выдаётся автоматически.

Поля inbound (блок `settings`):

Поле	Значение по умолчанию	Описание
`secretKey`	—	Приватный ключ сервера (обязателен). Рядом кнопка генерации; публичный ключ (`Public Key`) выводится из него автоматически (только для чтения; отдельно хранившийся `pubKey` убран)
`mtu`	`1420`	MTU интерфейса
`dns`	`1.1.1.1, 1.0.0.1`	**Новое в 3.4.2.** DNS, записываемый в строку `DNS =` клиентского `.conf`
`noKernelTun`	`false` (выкл.)	«TUN без kernel» — userspace-TUN вместо ядерного
`domainStrategy`	(опционально)	«Domain Strategy»: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`

Параметры WireGuard у клиента — вкладка **«Учётные данные»** в окне добавления/редактирования клиента (показываются, когда привязанный inbound — WireGuard):

Поле	Описание
«Приватный ключ WireGuard»	Приватный ключ клиента (редактируемое, есть кнопка «Перегенерировать»); при вводе из него автоматически выводится публичный ключ
«Публичный ключ WireGuard»	Только для чтения; вычисляется из приватного
«Общий ключ WireGuard» (PSK)	Необязательный дополнительный общий ключ; кнопка рядом с полем генерирует случайный ключ
«Разрешённые IP WireGuard»	С 3.5.0 — **редактируемое** (плейсхолдер `10.0.0.2/32` , подсказка «Оставьте пустым для авто-выдачи; записи разделяйте запятыми»). Пусто — адрес выдаётся автоматически; сервер проверяет каждую запись (IP или CIDR), одиночные адреса нормализует до `/32` и отклоняет адрес, занятый другим клиентом этого inbound
«Keepalive (секунды)»	Интервал `PersistentKeepalive` в конфиге клиента: у новых клиентов `25` ; `0` — keepalive не отправляется и строка в `.conf` не пишется (подробнее — 8.1). Только целое число: диапазон вида `15–25` умеет AmneziaWG (5.10)

Адрес в туннеле выдаётся сервером автоматически из подсети inbound (по умолчанию `10.0.0.0/24` : сервер занимает `.1` , клиенты — начиная с `.2`); если существующие клиенты используют другую `/24` , новые адреса выдаются в той же подсети.

Domain Strategy и вкладка Transport

Помимо перечисленных, у WireGuard-inbound есть поле **Domain Strategy** (стратегия резолвинга доменов: `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`). Поле необязательно и пишется в конфиг, только если задано.

> Поле **Workers** (`workers`, число рабочих потоков) убрано из форм WireGuard (как inbound, так и outbound): начиная с xray-core v26.6.22 движок его больше не использует и полагается на внутренний механизм wireguard-go. Ранее сохранённые конфиги работают без изменений — при разборе поле просто отбрасывается, миграция не нужна.

Для WireGuard также доступна вкладка **«Transport»** — но в урезанном виде: на ней настраиваются только `sockopt` и обфускация **Finalmask**. Выпадающий список выбора транспорта (`network`) скрыт, поскольку WireGuard всегда слушает по UDP. В шумовых записях (noise) Finalmask отдельным полем задаётся **Rand Range** (диапазон байт 0–255, с валидацией), а в качестве метода обфускации для WireGuard и Hysteria доступен **Salamander**.

Действия inbound и экспорт (с 3.5.0)

Меню строки WireGuard-inbound теперь содержит полный «мультиклиентный» набор действий — **«Экспортировать все ссылки»**, экспорт по подписке, **привязка/отвязка клиентов**, добавление в группу и удаление всех клиентов (раньше были только экспорт/сброс/клонирование/удаление), а в списке отображается счётчик клиентов. Для WireGuard окно «Экспорт всех ссылок» разделено на две вкладки: **Config** (склеенные `.conf`-блоки, по одному на клиента) и **Links** (персональные ссылки `wireguard://`); «Копировать» и «Скачать» действуют на открытую вкладку. Для inbound, размещённого на узле, `Endpoint` в клиентском `.conf`/QR теперь указывает **адрес узла**, а не главной панели.

Конфигурация и шеринг клиента WireGuard

У WireGuard-клиента в окнах «Информация»/QR доступна **сворачиваемая карточка конфигурации**: полный `.conf` (`[Interface]` с `PrivateKey` / `Address` / `DNS` / `MTU` и `[Peer]` с `PublicKey` / `PresharedKey` / `AllowedIPs = 0.0.0.0/0, ::/0` / `Endpoint` / `PersistentKeepalive`) с кнопками **Копировать**, **Скачать** и **QR**. Поддерживается также ссылка вида `wireguard://` / `wg://`, которую клиент-приложение разбирает в готовый `.conf`.

Клиент, привязанный сразу к нескольким inbound WireGuard/AmneziaWG, — это несколько независимых peer'ов: у каждого inbound свои ключи, общий ключ и адрес клиента в туннеле, и в конфигурацию inbound попадают именно они. Окна «Информация» и QR показывают конфиги всех таких inbound (подписи и имена файлов различаются по inbound), а сохранение формы такого клиента не переписывает ключи и адреса всех его peer'ов одним набором значений. С 3.10.0 исправлен случай клиента, привязанного одновременно к WireGuard- и AmneziaWG-inbound'у (#6751): peer WireGuard получал публичный ключ и keepalive из AmneziaWG-туннеля и не мог установить соединение. Теперь peer WireGuard берёт публичный ключ и keepalive из записи клиента в своём WireGuard-inbound'е; если keepalive там не задан, peer WireGuard его не получает (значение AmneziaWG не наследуется).

Проверка сгенерированных конфигов. Клиентские `.conf` WireGuard и AmneziaWG (подписка, страница клиентов, окно inbound'а, страница подписки) собираются построчно, и значение с управляющим символом (например, переводом строки) делает конфиг пустым: панель не выводит файл, в котором оно могло бы добавить лишнюю строку, а страница подписки скрывает блок, оказавшийся пустым. Профиль `vpn://` AmneziaWG, содержащий директиву-хук wg-quick (`PreUp` , `PostUp` , `PreDown` , `PostDown`), при разборе отклоняется — панель такие профили не создаёт. При создании и изменении inbound форма проверяет **Allowed IPs** каждого клиента WireGuard/AmneziaWG, в том числе уже сохранённых клиентов, которых оставляет смена протокола, — так же, как это делают эндпоинты клиентов.

Когда выбирать WireGuard: когда нужен именно VPN-туннель WireGuard, а не маскируемый прокси.

5.10. AmneziaWG

Назначение: **AmneziaWG** — это WireGuard с обфускацией, устойчивой к DPI. Криптография и модель «сервер + пиры» те же, что у WireGuard, но пакеты дополняются мусорными данными, заголовки маскируются, а тайминги рукопожатий сдвигаются, поэтому трафик не опознаётся по сигнатуре WireGuard. Транспорт и TLS/REALITY к нему, как и к WireGuard, неприменимы.

Появился в 3.7.0. Панель поднимает AmneziaWG **внутри собственного процесса** — встроенным движком `amneziawg-go` поверх сетевого стека gVisor. Это значит: по умолчанию не нужен модуль ядра (DKMS), не нужен `awg-quick` , не нужен Docker и не нужна отдельная панель Amnezia. Если на Linux-сервере есть модуль ядра AmneziaWG, отдельный inbound можно по желанию перевести на ядерный датапас переключателем **«Режим ядра»** — см. «Режим ядра» ниже. Один включённый клиент = один peer; выключенный клиент или клиент без ключа/адреса в туннель не попадает.

Поля inbound (блок `settings.server`):

Поле	Значение по умолчанию	Описание
`privateKey` / `publicKey`	генерируются	Ключевая пара сервера; публичный ключ выводится из приватного. Если при обновлении inbound (например, через API) ключи не переданы, сохраняется прежняя пара — новая генерируется только для inbound без сохранённых ключей
`subnetIp` + `subnetCidr`	`10.8.1.0` / `24`	Подсеть туннеля: сервер занимает `.1` , клиентам адреса выдаются автоматически
`mtu`	пусто → `1420` минус `S4`	MTU интерфейса. Пока поле пустое, панель вычитает из стандартных 1420 значение `S4` (но не опускается ниже 1280): мусор `S4` добавляется к каждому транспортному пакету, и при `S4` больше 20 полноразмерные пакеты с MTU 1420 не помещались бы в канал 1500 байт. Явно заданное значение используется как есть; итоговое MTU всегда прописывается и в конфиге клиента
`primaryDns` / `secondaryDns`	`8.8.8.8` / `8.8.4.4`	Попадают в строку `DNS =` клиентского конфига
`externalInterface`	пусто (автоопределение)	Сетевая карта хоста для NAT (`PostUp` / `PostDown`)

Поле	Значение по умолчанию	Описание
`ipv6Enabled`, `ipv6Subnet`, `ipv6ExternalInterface`	выкл.	IPv6 в туннеле; при включении подсеть обязательна (например `fd86:ea04:1115::/64`). Отдельный интерфейс нужен, если адреса IPv6 пиров назначаются не на ту же карту
`kernelMode`	выкл.	Переключатель **«Режим ядра»**: inbound работает на модуле ядра AmneziaWG вместо встроенного движка (см. «Режим ядра»)

Поле **IP прослушивания** (см. 4.1) учитывается: UDP-сокет AmneziaWG привязывается к этому адресу, поэтому на сервере с несколькими IP ответы уходят с того же адреса, на который пришёл клиент. Пустое значение, `0.0.0.0` или `::` — все адреса; адрес, который на этом хосте использовать нельзя, заменяется прослушиванием всех адресов с предупреждением в логе. В режиме ядра закреплённый адрес не поддерживается: сохранение отклоняется, пока поле не очищено или не заменено на `0.0.0.0` / `::`.

С 3.10.0 UDP-ответы, которые встроенный движок пишет обратно в туннель пиру, учитывают **MTU туннеля** (#6780): датаграмма больше MTU фрагментируется (и для IPv4, и для IPv6), а не уходит одним пакетом сверх MTU, как раньше. Контрольные суммы UDP при этом сохраняются.

Приёмный TCP-буфер встроенного движка подстраивается автоматически и может вырасти до 8 МБ на соединение. Соединение начинается с 1 МБ и растёт, только пока ретранслятор вычитывает данные быстрее, чем приходит окно за RTT, поэтому простаивающие пиры память не расходуют. Раньше окно было фиксированным и на каналах с заметной задержкой упиралось в скорость загрузки клиента: в замере разработчиков при RTT 50 мс одиночный поток вырос примерно с 310 до 500 Мбит/с. Настройки у этого нет.

Трафик встроенного движка уходит в Xray через локальный SOCKS5-мост на `127.0.0.1:64900`. Если порт занят, панель берёт случайный свободный порт loopback из диапазона 40000–62788 и пишет предупреждение в лог: выше лежат порты самой панели (API Xray, узловой egress, ретрансляторы TUIC и AmneziaWG), и порт из этой области мешал бы их запуску.

Параметры обфускации — отдельный блок формы с кнопкой **«Regenerate»**. Значения по умолчанию **генерируются случайно для каждого inbound'а**: статичный набор дал бы всем установкам одинаковый отпечаток для DPI.

Параметр	Что задаёт
`Jc`	Сколько мусорных пакетов слать перед рукопожатием
`Jmin` / `Jmax`	Минимальный и максимальный размер мусорного пакета
`S1` / `S2`	Размер мусорной добавки в init- и response-пакете
`S3` / `S4`	Набивка cookie-reply и транспортных пакетов
`H1` – `H4`	«Магические заголовки» — число или диапазон `низ–верх`; пусто = классические 1/2/3/4. **«Regenerate»** задаёт по одному числу на заголовок: широкий диапазон при включённом `RandomTrailers` заставляет часть обычных пакетов распознаваться как рукопожатия и отбрасываться, что резко снижает скорость. Сохранённые ранее диапазоны автоматически не меняются

Параметр	Что задаёт
`I1` – `I5`	Необязательные сигнатурные пакеты; пусто — не отправляются
`HeaderProtectionKey`	Base64-ключ 32 байта для защиты заголовков; должен совпадать на всех клиентах. Пусто — защита выключена; очистка поля выключает её и у уже работающего интерфейса. **Если ключ задан, все `S1` – `S4` обязаны быть ≥ 12** — панель проверяет это при сохранении, а не только ядро при запуске
`ContentPaddingAddition`	Набивка содержимого: число или диапазон байт; пусто — выключено
`RekeyAfterTime`, `RekeyTimeout`, `RejectAfterTime`, `KeepaliveTimeout`, `MaxHandshakeAttempts`	Тайминги протокола: число или диапазон; пусто — значение WireGuard по умолчанию
`RandomTrailers`	Добавляет случайные байты к каждому пакету. **Требует AmneziaWG 3.1+ с обеих сторон**
`DisableCookies`	Никогда не отправлять cookie-reply: убирает один признак для DPI, но ослабляет защиту от флуда

При сохранении значения проверяются в границах, которые принимает сам `amneziawg-go`. С 3.9.0 набивка inbound'а ограничена так, чтобы рукопожатие помещалось в приёмный буфер iOS-пиров (1700 байт): **`S1` ⩽ 1552, `S2` ⩽ 1608, `S3` ⩽ 1636** (прежде `S1` / `S2` допускали до 65535, а `S3` был ошибочно ограничен 64 и блокировал валидные конфиги); `S4` — до 32; сохраняется запрет `S1` + 56 = `S2`. `Jc`, `Jmin`, `Jmax` — от 0 до 4294967295 (`Jmin` не больше `Jmax`). Каждый из `H1` – `H4` — число или диапазон `низ-верх`; **пересекающиеся диапазоны H отклоняются** (#6642) (ядро и `amneziawg-go` не поднимают интерфейс при пересечении). В `I1` – `I5` проверяется структура `<тег значение>` (теги `b`, `t`, `r`, `rc`, `rd`, `d`, `ds`, `dz`). Недопустимое значение отклоняется ещё формой. У **outbound** AmneziaWG с 3.9.0 открыты необязательные сигнатурные поля **I2–I5** (#6611).

Параметры клиента — во вкладке учётных данных клиента (показываются, когда привязанный inbound — AmneziaWG): **AmneziaWG Private Key**, **Public Key**, **Pre-Shared Key** (с кнопкой генерации), **Allowed IPs** (пусто — адрес назначается автоматически; записи разделяются запятыми), **Keepalive (секунды)** (у новых клиентов `25`, `0` — не отправлять; допустим и диапазон, см. ниже) и **Forwarded Ports** — порты и диапазоны, пробрасываемые (DNAT) на этого клиента, например `80, 443, 8000-8100`; пусто — проброса нет. Подробнее о полях клиента — в 8.1. С 3.9.0 панель отклоняет клиента, чьи **Allowed IPs пересекаются** с диапазоном другого клиента того же inbound'а (сравниваются маскированные диапазоны, а не точные строки), — так запись вида `10.10.2.9/24` больше не «захватывает» всю подсеть; маршрут по умолчанию `/0` ничего не резервирует, а уже сохранённые пересекающиеся клиенты работают до следующего редактирования (#6623).

Диапазон keepalive (#6881). В поле **Keepalive (секунды)** клиента AmneziaWG можно ввести не только число, но и упорядоченный диапазон `мин-макс`, например `15-25`: случайный интервал труднее опознать по таймингу. Значения — целые от 0 до 65535, минимум не больше максимума. Подсказка в форме: «Укажите секунды числом или диапазоном (например, 15-25). Значение должно быть от 0 до 65535; 0 отключает keepalive.»; при ошибке — «Введите целое число или диапазон от 0 до 65535.». Не переданное значение сохраняет прежнее, явный `0` отключает keepalive. Скачиваемый `.conf` и профиль `vpn://` несут диапазон как есть; Clash/Mihomo понимает только число, поэтому туда уходит нижняя граница (`0-25` даёт `0` — keepalive отключён); у обычного WireGuard keepalive остаётся числом. Диапазон хранится в отдельной

колонке (`wg_keep_alive_range`), так что целочисленное значение и откат на старую сборку панели продолжают работать. Узлы обновите до того, как отправлять им диапазоны: старые узлы читают keepalive как целое число.

Готовый конфиг клиента доступен в его карточке — пункт **«AmneziaWG config»** (тот же `.conf` , что понимают приложения Amnezia). Состояние туннеля видно в **`Инструменты → AmneziaWG Logs`** : последнее рукопожатие, интерфейс, inbound, endpoint, простой и события.

Когда выбирать AmneziaWG: когда нужен VPN-туннель, а обычный WireGuard уже распознаётся и режется по сигнатуре.

Режим ядра (Kernel mode)

По умолчанию inbound AmneziaWG работает на встроенном движке. На Linux-сервере его можно по желанию перевести на **модуль ядра AmneziaWG**: переключатель **«Режим ядра»** (`settings.server.kernelMode` , по умолчанию выключен) находится в блоке параметров inbound'а под `DisableCookies` . Подсказка поля: «Запускать этот inbound на модуле ядра AmneziaWG вместо встроенного стека в пространстве пользователя. Обычно быстрее, особенно на каналах с большой задержкой, но маршрутизация Xray, sniffing, IPv6 и проброс портов к его пирам не применяются. Требуются модуль ядра AmneziaWG и amneziawg-tools на сервере.» Режим выбирается для каждого inbound'а отдельно (#6877); остальные inbound'ы остаются на встроенном движке.

Как это работает. Панель создаёт интерфейс `xui-awg<id>` (`<id>` — номер inbound'а), настраивает его командой `awg setconf` и выпускает трафик пиров в сеть хоста через маскарад (NAT) в собственной таблице nftables `xui_awg<id>` . Трафик пиров **идёт мимо Xray**: маршрутизация, сниффинг и каскады Xray на него не действуют. Все шаги — прямые вызовы `ip` , `awg` и `nft` , без `awg-quick` и без оболочки; конфигурация интерфейса лежит в закрытом каталоге панели (на стандартной установке `/etc/x-ui/amneziawg`) и не пересекается с вашими собственными туннелями `awg-quick` . Панель включает пересылку IPv4 и, если установлен `iptables` , добавляет на интерфейс два разрешающих правила цепочки `FORWARD` с комментарием `3x-ui:xui-awg<id>` — чтобы туннель работал и на хосте с политикой `FORWARD` = `DROP` (так её настраивают Docker и ufw). Брандмауэр, который сбрасывает пересылаемый трафик в собственной таблице nftables (например, firewalld), нужно настроить самостоятельно.

Требования и проверка хоста. Нужны Linux, панель, запущенная от root, установленный модуль ядра `amneziawg` и команды `awg` , `ip` и `nft` . В Docker режим ядра не поддерживается (встроенный движок в контейнере работает как прежде). Пока хост не готов, под переключателем показывается предупреждение **«Этот сервер пока не может работать в режиме ядра»** (с пометкой предупреждения, если режим уже включён) со списком причин и готовой командой установки с кнопкой копирования. Состояние отдаёт `GET /panel/api/inbounds/amneziawgKernelReadiness` . Причины и подписи:

Причина	Подпись в форме
Не root	«Панель запущена не от root, поэтому не может создать сетевой интерфейс»
Нет команд	«Не найдена нужная команда; установите amneziawg-tools и iproute2» (в скобках — имя команды)
Нет модуля	«Модуль ядра amneziawg не установлен»

Причина	Подпись в форме
Нет nftables	«nftables не установлен; он нужен для правил NAT туннеля»
IPv6 пиров	«Выход по IPv6 для отдельных пиров в режиме ядра пока не поддерживается»
Проброс портов	«Проброс портов для клиентов в режиме ядра пока не поддерживается»
`routeThroughXray`	«routeThroughXray не реализован в режиме ядра: у туннеля не останется пути наружу»
Закреплённый IP прослушивания	код `listenAddress` и адрес в скобках (русской подписи у этой причины нет)

Установить модуль и инструменты можно командой `sudo /usr/local/x-ui/x-ui install-awg-kernel` (панель подставляет путь к своему исполняемому файлу). Команда собирает через DKMS модуль и `amneziawg-tools` 3.x из релизов, закреплённых тегом и проверяемых по коммиту, и работает только на Debian и Ubuntu; на других системах поставьте `amneziawg-linux-kernel-module` и `amneziawg-tools` 3.x вручную. Нужны заголовки ядра, а при включённом Secure Boot — подпись модулей. Команда выполняется от root и меняет хост, поэтому просмотрите скрипт до запуска.

Ограничения. В режиме ядра не поддерживаются маршрутизация и сниффинг Xray, IPv6 для отдельных пиров (`ipv6Enabled`), **Forwarded Ports** клиентов и закреплённый **IP прослушивания** (4.1); сохранение inbound'а с такими настройками отклоняется. Требования к хосту (root, модуль, команды) проверяются, только когда сохранение запускает ядерный датапас: новый inbound, переключение со встроенного режима или включение выключенного inbound'а. Поэтому на хосте, где модуль пропал (например, ядро обновили без пересборки DKMS), inbound по-прежнему можно выключить или вернуть на встроенный движок. Ограничения по возможностям (IPv6, проброс портов, закреплённый адрес) проверяются при каждом сохранении.

Узлы. Inbound в режиме ядра можно разворачивать на узле: узел сообщает о поддержке заголовком `X-3x-Node-Caps: awg-kernel` , главная панель не отправляет такой inbound на узел более старой версии (иначе узел молча отбросил бы неизвестное поле и запустил встроенный движок, пока в форме главной панели режим выглядит включённым), а требования к хосту при применении проверяет сам узел (12.7).

Трафик, квоты и учёт. Раз в 10 секунд задача AmneziaWG читает счётчики каждого пира со всех управляемых интерфейсов и начисляет приращения (не итоговые значения: счётчики обнуляются при каждой пересборке интерфейса) в тот же учёт, что и у других протоколов, — расход клиента и inbound'а, квота и срок. Клиент, исчерпавший лимит, отключается, а при следующем согласовании его peer снимается с интерфейса — так его отсекают от сети. Счётчики читаются до согласования, поэтому клиент, отключённый по квоте, и удалённый inbound оплачивают свой последний интервал. Для **«Лимита IP»** адрес пира, с которого он подключался (по последнему рукопожатию), учитывается в течение 30 минут: журнал доступа Xray этот трафик не видит.

Остановка и перезапуск. При штатной остановке панель записывает финальные счётчики и удаляет интерфейсы — как и встроенный движок: остановленная панель не применяет квоты и сроки, поэтому ничего не должно пересылаться. Перезапуск только панели (`SIGHUP`) оставляет интерфейсы работающими. Если процесс панели упал, следующий запуск подхватывает уцелевшие интерфейсы без пересборки, а трафик между сбоем и первым чтением счётчиков после него не начисляется. Интерфейсы и конфигурации, принадлежность которых панель не может доказать (одноимённый интерфейс с другим ключом, файл без метки панели, не обычный файл или символическая ссылка), она не изменяет и не удаляет.

5.11. Hysteria (по умолчанию v2)

Назначение: inbound Hysteria поверх QUIC. Панель по умолчанию работает с версией 2. Каждый клиент аутентифицируется токеном `auth` вместо UUID/пароля. TLS для Hysteria доступен всегда (см. таблицу возможностей в 5.2).

С 3.9.0 добавление, удаление и правка клиента Hysteria2 применяются «вживую» (AddUser/RemoveUser) без пересоздания UDP-слушателя: соединения остальных клиентов этого inbound'а больше не рвутся при изменении одного клиента (#6606).

Поля блока `settings`:

Поле	Значение по умолчанию	Описание
`version`	`2`	Версия протокола (минимум 1; панель по умолчанию 2)
`clients`	`[]`	Список клиентов

У каждого клиента ключевое поле — `auth` (токен, обязателен) плюс общие поля (`email`, лимиты, `enable`, `tgId`, `subId`, `comment`, `reset`).

Дополнительные параметры задаются в `streamSettings.hysteriaSettings`:

Поле	Значение / варианты	Описание
`version`	зафиксировано 2 (поле заблокировано)	«Версия» (англ. «Version»)
`udpIdleTimeout`	(целое ⩾ 1, сек.)	«UDP idle timeout (с)» — тайм-аут простоя UDP
`masquerade`	выключен по умолчанию	«Masquerade» — маскировка под обычный веб-сервер при «непрошеных» запросах

При включении `masquerade` доступен выбор типа (`type`):

- `` — default (страница 404);
- `proxy` — обратный прокси (поля «Upstream URL», «Переписать Host», «Заголовки X-Forwarded», «Пропустить TLS verify»);
- `file` — раздача каталога (поле «Директория», например `/var/www/html`);
- `string` — фиксированный ответ (поля «Код статуса», «Body», «Заголовки»).

Ссылка `hysteria2://`. Из маски Finalmask **Salamander** в ссылку попадают только стандартные параметры: `obfs=salamander` и `obfs-password`, а в режиме **Gecko** — `obfs=gecko` вместе с `minPacketSize` / `maxPacketSize` (размеры 1–2048). Прочие настройки маски в URI не выражаются: клиент, собранный по такой ссылке, не пройдёт рукопожатие, и панель пишет об этом предупреждение в лог. Диапазон **Hop Ports** из блока **UDP Hop** (QUIC-параметры Finalmask) объявляется клиентам параметром `mport`; на сервере этот блок ни на что не влияет — начиная с xray-core 26.9.9 перескок портов выполняет клиентская UDP-маска `udphop`, а inbound слушает только свой порт.

Когда выбирать Hysteria: при потребности в QUIC-транспорте и устойчивости на нестабильных/мобильных каналах; маскировка повышает скрытность точки входа.

Исправления Hysteria в 3.7.0

- Обновление inbound'а, у которого **клиент остался без** `auth`, теперь отклоняется. Раньше такое сохранение проходило и тихо ломало доступ этому клиенту.
- Клиент **Hysteria2, исчерпавший лимит трафика или срок, принудительно отключается**. Раньше уже открытая сессия продолжала качать до перезапуска Xray — лимит фактически не работал до рестарта.

5.12. MTProto (прокси для Telegram)

> Добавлен в версии **3.3.0**. Значение протокола — `mtproto`.

MTProto — протокол собственного прокси Telegram. В 3X-UI такой inbound **обслуживается не Xray, а отдельным процессом** `mtg`, которым управляет сама панель. Панель периодически сверяет включённые MTProto-inbound с запущенными процессами `mtg`: поднимает недостающие, останавливает лишние и снимает счётчики трафика с метрик `mtg`. Поэтому **учёт трафика** по такому inbound работает как у обычных протоколов.

С версии 3.5.0 MTProto — мультиклиентный протокол (движок сменён на форк `mtg-multi`): один inbound обслуживает много пользователей, каждый из которых — обычный **клиент** (см. раздел 8) со своим FakeTLS-секретом, квотой трафика, сроком, переключателем включения, необязательным ad-tag и персональной ссылкой `tg://proxy`. Прежняя модель «один inbound = один секрет» упразднена: поле «Secret» на уровне inbound из формы убрано, а при обновлении панели существующий одиночный секрет автоматически превращается в первого клиента этого inbound (ручных действий не требуется).

Следствия:

- Вкладка **«Транспорт» (Stream Settings) к этому inbound не применяется**; клиенты же теперь управляются как у остальных протоколов — привязка/отвязка, лимиты, группы, подписки.
- Изменения клиентов (добавление, удаление, смена секрета, включение/выключение, ad-tag) применяются **«на лету»** через management-API `mtg`, не разрывая Telegram-сессии остальных пользователей; лишь структурные изменения inbound (адрес, domain fronting, лимит соединений, лимит рукопожатий на IP, публичные IP) перезапускают процесс.
- С 3.9.0 MTProto-inbound можно развернуть и **на узле** (nodes): процесс `mtg` для него запускает панель самого узла, поэтому узел должен работать на версии не ниже **v3.5.0** — узел старее (или ещё не сообщивший свою версию) сохранение отклоняет. Главная панель такой inbound у себя не запускает (inbound'ы с заданным `NodeID` она пропускает); см. 12.7.
- Вкладка **«Sniffing»** для MTProto скрыта — этот протокол обслуживается процессом `mtg`, а не Xray, поэтому сниффинг к нему неприменим.

Поля. Хранятся в `settings` inbound:

Поле в UI	Ключ	Описание
Remark	`remark`	Метка inbound.

Поле в UI	Ключ	Описание
Listen IP	`listen`	IP для прослушивания (пусто = все интерфейсы).
Port	`port`	Порт прокси.
Домен прикрытия (FakeTLS)	`settings.fakeTlsDomain`	По умолчанию `www.cloudflare.com`. С 3.5.0 — **домен по умолчанию для генерации секретов новых клиентов** (подсказка: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); каждый клиент может прикрываться собственным доменом через свой секрет.
Max connections	`settings.throttleMaxConnections`	**Новое в 3.5.0.** Ограничение одновременных соединений на всех пользователей с честным делением; `0` — без ограничения.
Макс. незавершённых рукопожатий на IP	`settings.pendingHandshakesMaxPerIp`	**Новое в 3.10.2.** Подсказка: «Новые подключения с IP, у которого уже столько незавершённых рукопожатий, закрываются. Клиенты за общим IP (NAT оператора) делят этот лимит. 0 — отключено.» Защищает от флуда рукопожатиями. По умолчанию `0`; при значении больше нуля в `mtg-<id>.toml` пишется секция `[defense.pending-handshakes]`. Требует `mtg-multi` 1.16.0 или новее.
Public IPv4 / Public IPv6	`settings.publicIpv4` / `settings.publicIpv6`	**Новое в 3.5.0.** Публичный адрес сервера для ad-tag middle proxy (плейсхолдеры `1.2.3.4` / `2001:db8::1`); пусто — `mtg` определит сам.

Формат секрета (FakeTLS). Секрет теперь живёт **у каждого клиента** (поле **«MTProto secret»** в форме клиента, с кнопкой перегенерации; рядом — необязательный **«Ad-tag (спонсорский канал)»**, ровно 32 hex-символа). Вид секрета прежний: `ee` + 32 hex-символа + hex-код домена прикрытия (`ee<hex32><hex(домен)>`); при привязке нового клиента к MTProto-inbound секрет генерируется автоматически от домена inbound по умолчанию. Квота трафика и срок действия клиента с 3.5.0 **реально применяются** (через лимиты mtg-multi): исчерпанный или истёкший клиент отключается, а сброс трафика возвращает доступ сразу.

Domain-fronting и расширенные опции mtg

У MTProto-inbound есть дополнительные параметры процесса `mtg`. Поля **Domain fronting IP**, **Domain fronting port** и **Domain fronting PROXY protocol** задают, куда `mtg` отправляет не-Telegram трафик (например, на фейковый сайт NGINX): если IP оставить пустым, используется FakeTLS-домен через DNS, порт по умолчанию — `443`. Дополнительно доступны **Accept PROXY protocol** (для слушателя), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) и **Debug logging**. Каждое значение записывается в файл `mtg-<id>.toml`, только если оно задано.

Маршрутизация Telegram-трафика через Xray

Переключатель **«Route through Xray»** (по умолчанию выключен) и необязательное поле **Outbound** позволяют подчинить egress Telegram маршрутизатору Xray. При включении панель встраивает в конфиг Xray локальный SOCKS-мост с тегом самого inbound, и `mtg` отправляет Telegram-трафик через него. После этого трафик можно сопоставлять правилами на вкладке «Routing» либо принудительно направить в выбранный outbound или балансировщик через поле **Outbound** (если поле пусто, решают правила маршрутизации).

Как раздать пользователю. Для MTProto-inbound панель генерирует ссылку-приглашение:

Пример: секрет FakeTLS и готовая ссылка. Если домен прикрытия — `www.cloudflare.com`, секрет собирается как `ee` + 32 hex-символа + hex-код домена, например:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Готовая ссылка-приглашение (её и QR-код отправляют пользователю в Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f75646
66c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(эквивалент — `https://t.me/proxy?server=…&port=…&secret=…`). С 3.5.0 ссылка **персональная** — строится из секрета конкретного клиента, а фрагмент `#remark` из неё убран (нестрогие парсеры Telegram склеивали его с секретом и ломали импорт); примечание показывается отдельной меткой в окне информации. Эту ссылку и QR-код нужно отправить пользователю Telegram — при открытии прокси сразу добавляется в приложение. Ссылка также отдаётся через сервер подписок.

Адрес в ссылках задают управляемые хосты. Начиная с 3.8.0 все ссылки MTProto — в подписке, в окнах клиента, при копировании, в QR-коде и при экспорте — строятся по управляемым хостам, привязанным к inbound: по ссылке на каждый адрес включённых групп, не исключённых из RAW-подписки. Так клиентам объявляется правильный публичный адрес и порт, когда прокси стоит за обратным прокси или пробросом портов и внешний порт отличается от порта inbound; если порт у хоста не указан, берётся порт inbound. Без хостов адрес подставляется как раньше. Поле **«Стратегия адреса для ссылок»** для MTProto в форме больше не показывается, а прежний пользовательский адрес для ссылок при обновлении панели автоматически переносится в новый хост этого inbound — если среди его включённых хостов такого адреса ещё нет.

Когда использовать. Штатный способ обхода блокировок Telegram; FakeTLS-маскировка (домен прикрытия) делает трафик похожим на обычный визит на указанный сайт.

5.13. TUIC v5

> Добавлен в версии **3.8.0**. Значение протокола — `tuic`.

TUIC v5 — прокси-протокол поверх QUIC (UDP) с обязательным TLS, мультиплексированием потоков и выбираемым алгоритмом контроля перегрузок; хорошо работает на каналах с потерями пакетов. С 3.9.0 TUIC работает **нативным Go-сервером прямо внутри процесса 3x-ui** — отдельный сайдкар `tuic-server` больше не используется (#6577). Расшифрованный трафик панель направляет в ядро Xray через локальный SOCKS5-мост, поэтому к TUIC применяются маршрутизация, geo-правила и каскады Xray (TUIC → VLESS / Shadowsocks / WARP). Публичный UDP-порт inbound'а слушает сам сервер панели; для хранения учтённого трафика при остановке ведётся журнал (таблица `tuic_traffic_receipts`), который дозаписывается при следующем старте без двойного счёта.

Путь трафика. Публичный UDP-порт inbound'а слушает встроенный TUIC-сервер панели; расшифрованный поток уходит в ядро Xray по локальному SOCKS5-мосту. Поэтому к TUIC применяются маршрутизация, сниффинг, geo-правила и каскадирование на любые outbound'ы Xray — в отличие от прежнего сайдкара, который выпускал трафик мимо Xray. Трафик inbound'а и каждого клиента учитывается точно на любой ОС, **«Общий расход»** и **«Дата окончания»** inbound'а срабатывают как у других протоколов.

С 3.10.0 сниффинг на этом мосту работает в режиме `routeOnly` (#6753): домен, распознанный по HTTP/TLS/QUIC, используется только для правил маршрутизации, а адрес назначения, который запросил клиент, не подменяется. Так подключение TUIC-клиента к IP-адресу, у которого в TLS другой или нерезолвящийся SNI, больше не уводится на чужой адрес.

Маршрутизация Xray по пользователю (#6875). Локальный SOCKS5-мост принимает по одному аккаунту на email клиента, поэтому Xray знает, какому клиенту принадлежит соединение, и правила маршрутизации по пользователю (`user`) срабатывают для TCP и для обоих режимов UDP. Раньше мост не требовал логина и такие правила для TUIC не срабатывали (#6874). Допуск клиентов остаётся за нативным TUIC-сервером. Email длиннее 255 байт (предел SOCKS5) исключаются из моста с предупреждением в логе.

Скорость клиентов (с 3.10.0, #6723): на странице клиентов снова отображается текущая скорость TUIC-клиентов — встроенный сервер раз в 10 секунд передаёт в панель свои счётчики по клиентам. Общий трафик inbound'а по-прежнему считает Xray на локальном мосту, и скорость TUIC в него повторно не начисляется.

Развёртывание. Отдельный бинарник ставить не нужно — TUIC встроен в панель. TUIC-inbound можно развернуть и **на узле** (его запускает панель самого узла; узел должен работать на версии не ниже **v3.8.0**, см. 12.7). В форме есть вкладки **«Основное»**, **«Протокол»** и **«Расширенный шаблон»**, а **«Поток»**, **«Безопасность»** и **«Сниффинг»** отсутствуют. Порт занимает только UDP, поэтому при проверке конфликтов TUIC может использовать тот же номер порта, что и TCP-inbound (например, VLESS на 443/TCP).

Поля вкладки «Протокол» (хранятся в `settings.server`):

Поле в UI	Ключ	По умолчанию	Описание
SNI	`sni`	пусто	Имя сервера для клиентов: попадает в ссылку и Clash-конфиг, самому TUIC-серверу не передаётся. Пока пути сертификата пусты или лежат в `/root/cert/`, ввод SNI заполняет их как `/root/cert/<SNI>/fullchain.pem` и `/root/cert/<SNI>/privkey.pem`; кнопка **«Автозаполнение»** подставляет эти пути принудительно

Поле в UI	Ключ	По умолчанию	Описание
«Публичный ключ»	`certificate`	пусто	Путь к файлу сертификата (полная цепочка). Подсказки ввода: `/root/cert/<SNI>/fullchain.pem`, `/etc/letsencrypt/live/<SNI>/fullchain.pem`, `/root/cert.pem`
«Приватный ключ»	`private_key`	пусто	Путь к файлу закрытого ключа; подсказки — те же каталоги с `privkey.pem`
«Контроль перегрузок»	`congestion_control`	`bbr`	Алгоритм контроля перегрузок QUIC: BBR, CUBIC или New Reno. Значение попадает в клиентский профиль; на сервере встроенный движок умеет только `bbr` и `new_reno`, поэтому выбор `cubic` обслуживается как `new_reno`. На сервере смена алгоритма действует после переподключения
ALPN	`alpn`	`h3`, `spdy/3.1`	Список ALPN, ввод тегами
«Режим ретрансляции UDP»	`udp_relay_mode`	`native`	Как клиент передаёт UDP-трафик: **Native (Recommended)** — датаграммами QUIC, **QUIC** — потоками QUIC. Параметр клиентский: попадает в ссылку и Clash-конфиг, на сервер не влияет

Кнопка **«Установить сертификат панели»** подставляет пути к сертификату и ключу, настроенные для веб-интерфейса панели (если их нет — предупреждение «Для панели не настроен сертификат. Сначала установите его в Настройках.»); **«Очистить»** стирает оба пути. С 3.10.0 кнопки под путями сертификата переносятся на новую строку и не вылезают за край формы на узком экране (#6723).

Сворачиваемая группа **«Дополнительные настройки»**:

Поле в UI	Ключ	По умолчанию	Описание
«0-RTT рукопожатие»	`zero_rtt_handshake`	вкл.	Рукопожатие 0-RTT при повторном подключении клиента; в Clash-конфиге передаётся как `reduce-rtt`
«Уровень логирования»	`log_level`	`info`	Info, Warn, Error или Debug; вывод TUIC-сервера пишется в лог панели с префиксом `tuic:`. Подсказка поля — «Минимальный уровень сообщений в логах этого инбаунда» (статус онлайн от уровня лога больше не зависит)
«Макс. время простоя (сек)»	`max_idle_time`	`15`	Через сколько секунд бездействия закрывается QUIC-соединение
«Таймаут аутентификации (сек)»	`authentication_timeout`	`3`	Сколько секунд сервер ждёт аутентификации клиента

Поле в UI	Ключ	По умолчанию	Описание
«Макс. размер пакета UDP»	`max_udp_relay_packet_size`	`1500`	Максимальный размер ретранслируемого UDP-пакета в байтах. Допускается до `65245` (большие значения усекаются)

Сертификат обязателен. QUIC работает только с TLS, поэтому пути к сертификату и ключу нужно указать. При сохранении они не проверяются (проверка сертификатов при сохранении касается только TLS на вкладке «Безопасность»): с пустым или неверным путём TUIC-сервер для этого inbound не поднимется, панель повторит попытку при следующем применении конфигурации, а причина останется в логе панели.

Клиенты. TUIC — мультиклиентный протокол: клиенты создаются и привязываются как у остальных (см. раздел 8). У клиента обязательны **Email**, **UUID** и **пароль** — без них сохранение отклоняется; при привязке клиента к TUIC-inbound недостающие UUID и пароль генерируются автоматически. Встроенному серверу передаются только включённые клиенты с UUID и паролем; пока таких нет, сервер для этого inbound не запускается и UDP-порт не слушается.

- **Срок действия** и **включение/выключение** клиента работают: истёкший или выключенный клиент исключается из списка пользователей встроенного сервера.
- **Персональная квота трафика** (`totalGB`) **поддерживается и применяется:** встроенный сервер учитывает расход каждого клиента, счётчики растут, а по достижении квоты клиент отключается, как у других протоколов (прежняя подсказка «TUIC не поддерживает персональные квоты» убрана).
- **«Лимит IP»** на TUIC-клиентов не действует: он строится на статистике онлайн-IP Xray, а Xray видит соединения TUIC только от локального моста, а не с адреса клиента.
- **Статус онлайн**, «последний онлайн» и отсчёт срока «после первого использования» определяются встроенным сервером напрямую и больше не зависят от уровня логирования.

Правки клиентов применяются на лету. Добавление, удаление, выключение клиента (в том числе автоматическое — по сроку или квоте), смена UUID или пароля обновляют реестр пользователей встроенного сервера мгновенно, **не обрывая соединения** остальных клиентов и не переоткрывая UDP-порт. (Изменение сетевых полей самого inbound'а по-прежнему пересоздаёт слушатель.) Добавление, удаление или переименование (смена email) клиента заменяет лишь SOCKS5-мост этого inbound'а, без перезапуска Xray: новые сессии ждут за ограниченным по времени шлюзом, а идущие потоки и UDP-ассоциации продолжаются; после замены аккаунты проверяются и при сбое откатываются, а сбой одного моста не мешает остальным и не блокирует несвязанные живые правки.

Ссылка и конфиги. Для клиента панель формирует ссылку:

```
tuic://<uuid>:<пароль>@<адрес>:<порт>?
congestion_control=<алгоритм>&alpn=<ALPN>&sni=<SNI>&udp_relay_mode=<режим>&allow_insecur
e=0#<название>
```

`sni` добавляется, только если SNI задан. Адрес подставляется по стратегии адреса для ссылок; если к inbound привязаны управляемые хосты, ссылка строится на каждый хост с его SNI и ALPN, а при включённом в хосте Allow insecure — с `allow_insecure=1` . Ссылка показывается в окне **«Информация о подключении»** и отдаётся в сырой подписке; в Clash/Mihomo-подписке TUIC-inbound выдаётся как прокси `type: tuic` , а в JSON-подписку не попадает. В окне QR-кода клиента есть вкладка **«Конфиг TUIC (Clash)»** с готовым YAML для Clash / Mihomo (см. 8.3).

Когда выбирать TUIC: нужен QUIC-прокси с быстрым установлением соединения и контролем перегрузок (например, BBR) для каналов с потерями, а клиентские приложения (например, Mihomo или sing-box) поддерживают TUIC v5. Трафик учитывается и по inbound'у, и по каждому клиенту: с 3.9.0 персональные квота (`totalGB`) и срок действия клиента применяются — исчерпавший или истёкший клиент исключается из списка пользователей встроенного сервера. Не действует только «Лимит IP».

5.14. Краткая шпаргалка по выбору протокола

- **VLESS** — выбор по умолчанию; лучший вариант с REALITY или TLS + XTLS-Vision, поддерживает пост-квантовую аутентификацию.
- **Trojan** — маскировка под HTTPS с fallback'ами на веб-сервер.
- **VMess** — совместимость со старыми клиентами.
- **Shadowsocks** — простой прокси без TLS; современный выбор — методы `2022-blake3-*` .
- **Hysteria** — QUIC, устойчивость на плохих каналах.
- **mixed / http** — служебные SOCKS/HTTP-прокси.
- **WireGuard** — полноценный VPN-туннель.
- **tunnel** — прозрачное перенаправление портов.
- **MTProto** — прокси для обхода блокировок Telegram (FakeTLS); отдельный процесс `mtg` .
- **TUIC v5** — QUIC-прокси с контролем перегрузок для каналов с потерями; нативный сервер внутри панели, с маршрутизацией Xray и персональными квотами клиентов.
- **MASQUE** (с 3.10.0) — IP-туннель CONNECT-IP поверх HTTP/3 или HTTP/2 с обязательным TLS; клиенты получают конфигурацию только через JSON-подписку (см. 5.15).

5.15. MASQUE

> Добавлен в версии **3.10.0** (поддержка в xray-core v26.10.10 из комплекта). Значение протокола — `masque` .

MASQUE передаёт IP-пакеты поверх HTTP/3 (или HTTP/2) методом **CONNECT-IP**: клиент получает полноценный туннель третьего уровня, который снаружи выглядит как обычный HTTPS-трафик. В отличие от WireGuard/AmneziaWG, inbound обслуживает само ядро Xray, поэтому к нему применяются маршрутизация, сниффинг, учёт трафика и лимиты, как у VLESS или Trojan. В панели MASQUE — это протокол inbound'а и outbound'а, а также одноимённый транспорт `masque` , на котором работают оба (транспорт — в разделе 6, MASQUE-outbound и WARP через MASQUE — в 11.4).

Вкладки формы. При выборе протокола панель сама выставляет `network = masque` , `security = tls` , путь по умолчанию и TLS-настройки «как у Hysteria» (ALPN `h3`). Выбор сети скрыт; есть вкладки **«Основное»**, **«Протокол»**, **«Поток»**, **«Безопасность»** (только TLS — режимов «Нет» и REALITY нет), **«Сниффинг»** и **«Расширенный шаблон»**.

Поля вкладки «Протокол» (хранятся в `settings`):

Поле в UI	Ключ	По умолчанию	Описание
«Пул адресов» (обязательное)	`address`	`10.14.0.1/24`, `fd14::1/64`	Префиксы, из которых клиентам выдаются адреса туннелей, — не более одного IPv4 и одного IPv6 (подсказка поля: «Адреса туннелей выдаются из этих префиксов — не более одного IPv4 и одного IPv6.»). Ввод тегами, разделители — запятая или пробел. Размер префикса ограничивает число одновременно выдаваемых адресов
MTU	`mtu`	пусто	MTU туннеля, 1280–65535 (плейсхолдер `1280`); пусто — значение ядра по умолчанию, ключ в конфиг не пишется

Вкладка «Поток»: поле **«Путь»** (`streamSettings.masqueSettings.path`, по умолчанию `/.well-known/masque/ip/*/*/`) — путь запроса, на который отвечает сервер, плюс `sockopt` и Finalmask. Сервер сверяет запрос только по пути; host, учётные данные и заголовки — клиентские параметры и попадают к клиенту через JSON-подписку.

TLS обязателен (вкладка «Безопасность», как в 7.3). От **ALPN** зависит, какие слушатели поднимает ядро, и по нему же панель проверяет конфликты портов:

ALPN	Слушатель	Занимаемый порт
`h3` (по умолчанию) или без `h2`	HTTP/3 поверх QUIC	только UDP
`h2`	HTTP/2	только TCP
`h2` и `h3`	оба	TCP и UDP

Поэтому MASQUE-inbound с ALPN по умолчанию может делить номер порта с TCP-inbound'ом (например, VLESS на 443/TCP).

Клиенты. MASQUE — мультиклиентный протокол: клиенты создаются, привязываются, массово добавляются и отвязываются как у остальных (см. раздел 8). Клиент входит по HTTP Basic auth: имя пользователя — его **Email**, пароль — поле **«Пароль»** (подсказка поля клиента теперь называет и MASQUE). Особенности:

- **Пароль обязателен.** Если при привязке к MASQUE-inbound'у он пуст, панель генерирует его сама (32 hex-символа, как у Trojan); inbound с клиентом без пароля не сохраняется (`empty client ID`) — ядро отказалось бы запускать весь inbound. В серверный конфиг пароль уходит под ключом `pass`, который читает ядро; в базе панели он хранится как `password`.
- **Email не может содержать «:»** — он служит именем пользователя Basic auth, и ядро отвергает весь inbound (а с ним и запуск Xray) из-за одного такого клиента. Создание, привязка, массовые операции, импорт и сохранение inbound'а отклоняют такого клиента с ошибкой `MASQUE: client email "…" cannot contain ":"`.
- **Квота трафика, срок действия, включение/выключение и «Лимит IP»** работают как у других протоколов Xray.
- **Правки клиентов применяются на лету** — по каждому пользователю отдельно: добавление нового клиента не обрывает туннели уже подключённых.

Узлы. MASQUE-inbound можно разворачивать на узле; проверка требует панель узла не ниже **v3.9.1** (из выпущенных версий — 3.10.0), иначе сохранение отклоняется (см. 12.7).

Как раздать клиенту. Формата share-ссылки у MASQUE нет, поэтому ссылки, QR-кода и записи Clash/Mihomo для такого inbound'а не создаётся, а в сырую подписку он не попадает. Клиент получает готовую конфигурацию из **JSON-подписки** (см. 10.4): outbound `masque` с адресом и портом сервера (с учётом управляемых хостов), `security: tls` и транспортом, где `user` — email клиента, `pass` — его пароль, а `path` — путь сервера, если он задан.

Пример: блоки `settings` и `streamSettings` MASQUE-inbound'а (так их хранит панель; в конфиг Xray `password` клиента уходит как `pass`):

```
{
  "clients": [
    { "email": "user1", "password": "5f1c0e7a9b2d4c6e8f0a1b3c5d7e9f01", "limitIp": 0, "t
otalGB": 0, "expiryTime": 0, "enable": true }
  ],
  "address": ["10.14.0.1/24", "fd14::1/64"]
}
```

```
{
  "network": "masque",
  "security": "tls",
  "masqueSettings": { "path": "/.well-known/masque/ip/*/*/" },
  "tlsSettings": {
    "alpn": ["h3"],
    "certificates": [
      { "certificateFile": "/root/cert/example.com/fullchain.pem", "keyFile": "/root/
cert/example.com/privkey.pem" }
    ]
  }
}
```

Когда выбирать MASQUE: нужен полноценный IP-туннель (как у WireGuard), но замаскированный под HTTP/3- или HTTPS-трафик и с маршрутизацией, лимитами и учётом Xray; клиентом выступает Xray-core v26.10.10+ (конфигурация из JSON-подписки).

---

6. Транспорт (Stream Settings)

Транспорт (в интерфейсе панели — поле **«Транспорт»**, англ. *Transmission*) определяет способ, которым Xray-core передаёт данные внутри inbound: какой сетевой протокол используется поверх TLS/Reality и как именно кадрируется трафик. Эти параметры сохраняются в объект `streamSettings` конфигурации Xray и задаются на вкладке транспорта в редакторе inbound. Шифрование (TLS / Reality) рассматривается в отдельном разделе — здесь описывается только выбор сети и её параметры.

6.1. Выбор сети передачи

Сеть выбирается в выпадающем списке **«Транспорт»** (`streamSettings.network`). Значение по умолчанию — `tcp` (в списке отображается как **RAW**). Доступны следующие варианты:

Значение в списке	Поле `network`	Транспорт
RAW	`tcp`	Обычный TCP (в новых версиях Xray переименован в RAW), опционально с HTTP-обфускацией
mKCP	`kcp`	Надёжный UDP-транспорт mKCP
WebSocket	`ws`	WebSocket поверх HTTP(S)
gRPC	`grpc`	gRPC-туннель (HTTP/2)
HTTPUpgrade	`httpupgrade`	HTTP Upgrade
XHTTP	`xhttp`	XHTTP / SplitHTTP — современный мультиплексируемый транспорт
XDRIVE	`xdrive`	Туннель через файлы в общем хранилище: Google Drive, локальная папка или HTTP API по шаблону (см. п. 6.10). Пункт показывается только у inbound'а VLESS

При смене значения панель очищает блок настроек предыдущей сети и заполняет блок новой сети значениями по умолчанию из её схемы, поэтому каждое поле под-формы всегда имеет осмысленное начальное значение.

> **Важно.** В данной сборке панели **транспорт HTTP/2 (`h2`) в списке отсутствует** — он был исключён из набора сетей; для двунаправленного HTTP/2-подобного туннеля используется gRPC, а для современного HTTP-маскированного транспорта — XHTTP. Транспорт **Hysteria** (`hysteria`) не выбирается через этот список: он жёстко привязан к протоколу Hysteria и появляется автоматически, когда сам inbound создаётся с протоколом Hysteria (см. п. 6.8). Так же устроен транспорт **MASQUE** (`masque`): его нет в списке, он задаётся автоматически для протокола MASQUE (см. п. 6.11).

Ниже каждая сеть и каждое её поле разобраны отдельно.

6.2. RAW / TCP (`tcpSettings`)

Базовый TCP-транспорт. По умолчанию трафик передаётся «как есть»; опционально его можно замаскировать под обычный HTTP/1.1-обмен.

Поле	Значение по умолчанию	Описание
Proxy Protocol (`acceptProxyProtocol`)	`false` (выкл.)	Принимать заголовок PROXY protocol от вышестоящего балансировщика/прокси
HTTP-обфускация (`header.type`)	`none` (выкл.)	Включает маскировку трафика под HTTP/1.1

Proxy Protocol

Переключатель **«Proxy Protocol»** (`acceptProxyProtocol`). Когда включён, Xray ожидает на входящем соединении заголовок PROXY protocol и извлекает из него реальный IP клиента. Включают только если перед панелью стоит обратный прокси/балансировщик (например, HAProxy или nginx со `send-proxy`), который этот заголовок добавляет. По умолчанию выключен.

HTTP-обфускация (camouflage)

Переключатель **«HTTP Обфускация»**. Управляет полем `header` :

- **Выключен** → `header.type = "none"` (на проводе поле `header` просто отсутствует). Чистый TCP.
- **Включён** → `header.type = "http"` . Трафик кадрируется под видом HTTP/1.1-запроса и ответа. При включении панель сразу заполняет под-объекты `request` и `response` значениями по умолчанию.

При включённой HTTP-обфускации появляются поля настройки имитируемых запроса и ответа.

Заголовок запроса (`header.request`):

Поле	Ключ	Значение по умолчанию	Описание
Версия запроса	`request.version`	`1.1`	Версия HTTP в стартовой строке запроса
Метод запроса	`request.method`	`GET`	HTTP-метод имитируемого запроса
Путь запроса	`request.path`	`/`	Путь(и). Вводится списком значений через запятую; на проводе это массив строк. Если оставить пустым, подставляется `/`
Заголовки запроса	`request.headers`	`{}` (пусто)	Таблица «Имя/Значение» HTTP-заголовков. Хранится как карта `имя → [значения]` (одному имени может соответствовать несколько значений)

Заголовок ответа (`header.response`):

Поле	Ключ	Значение по умолчанию	Описание
Версия ответа	`response.version`	`1.1`	Версия HTTP в стартовой строке ответа
Статус ответа	`response.status`	`200`	HTTP-код статуса имитируемого ответа
Причина ответа	`response.reason`	`OK`	Текстовое описание статуса (reason-phrase)
Заголовки ответа	`response.headers`	`{}` (пусто)	Таблица «Имя/Значение» заголовков ответа (карта `имя → [значения]`)

Поля заголовков редактируются построчно — каждая строка задаёт имя заголовка (`Имя`) и его значение (`Значение`). Эти параметры используют только для маскировки внешнего вида трафика; на криптографию они не влияют. Значения по умолчанию (`GET / HTTP/1.1`, ответ `200 OK`) подходят для большинства сценариев — менять их стоит, только если требуется имитировать конкретный сайт/сервис.

Пример `streamSettings` для RAW с HTTP-обфускацией:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Обратите внимание: `path` на проводе — массив строк, а каждый заголовок — массив значений (`Host → ["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP — надёжный транспорт поверх UDP. Полезен на каналах с потерями пакетов и высокой задержкой, но создаёт повышенный служебный трафик. Все значения по умолчанию совпадают с рекомендованными в xray-core.

Поле	Ключ	По умолчанию	Допустимо	Описание
MTU	`mtu`	`1350`	576–1460	Максимальный размер пакета (байт). Уменьшают при проблемах фрагментации
TTI (мс)	`tti`	`20`	10–100	Интервал передачи (ms). Меньше — ниже задержка, но выше накладные расходы
Uplink (МБ/с)	`uplinkCapacity`	`5`	⩾ 0	Расчётная пропускная способность на отдачу (МБ/с)
Downlink (МБ/с)	`downlinkCapacity`	`20`	⩾ 0	Расчётная пропускная способность на приём (МБ/с)
Множитель CWND	`cwndMultiplier`	`1`	⩾ 1	Множитель окна перегрузки (congestion window)
Макс. окно отправки	`maxSendingWindow`	`2097152`	⩾ 0	Максимальный размер окна отправки

Замечания по полям:

- **Uplink / Downlink capacity** задают, насколько агрессивно mKCP занимает канал. Их выставляют в соответствии с реальной шириной канала: завышенные значения ведут к лишнему трафику, заниженные — к недоиспользованию канала.
- **TTI** напрямую влияет на компромисс «задержка ? накладные расходы»: меньшие значения снижают задержку, но увеличивают объём служебных пакетов.
- **MTU** ограничивает размер одного пакета mKCP; снижение помогает на каналах, где крупные UDP-пакеты режутся или теряются.

> В данной сборке поле «seed» (пароль обфускации mKCP) и выпадающий список **типа заголовка/обфускации** (`none` , `srtp` , `utp` , `wechat-video` , `dtls` , `wireguard`) в под-форме mKCP **отсутствуют как отдельные поля** — обфускация транспортного уровня вынесена в общий механизм «FinalMask» (включая режим `mkcp-legacy`), описываемый в п. 6.12. Параметр «congestion» как отдельная галочка также не выведен; управление перегрузкой задаётся через `cwndMultiplier` и `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

WebSocket-транспорт поверх HTTP(S). Хорошо проходит через CDN и обратные прокси, маскируется под обычный веб-трафик.

Поле	Ключ	По умолчанию	Описание
Proxy Protocol	`acceptProxyProtocol`	`false`	Принимать заголовок PROXY protocol от вышестоящего прокси (см. п. 6.2)
Хост	`host`	`""` (пусто)	Значение HTTP-заголовка `Host` . Указывают при работе через CDN/домен-фронтинг

Поле	Ключ	По умолчанию	Описание
Путь	`path`	`/`	Путь в строке запроса WebSocket-рукопожатия
Период heartbeat	`heartbeatPeriod`	`0`	Интервал отправки heartbeat-кадров (в секундах). `0` отключает heartbeat
Заголовки	`headers`	`{}` (пусто)	Дополнительные HTTP-заголовки рукопожатия. Карта «Имя → Значение» (только строковые значения, без массивов)

Замечания:

- **Путь** должен совпадать на сервере (inbound) и у клиента. Часто за этим путём на стороне веб-сервера маскируют точку входа.
- **Хост** имеет смысл задавать, если inbound стоит за CDN или используется фронтинг по домену; иначе можно оставить пустым.
- **Период heartbeat** держит соединение «живым» при прохождении через прокси/CDN, агрессивно разрывающие неактивные сессии. По умолчанию (`0`) heartbeat выключен.
- В отличие от RAW, таблица заголовков WebSocket использует «плоский» формат `имя → значение` (одна строка значения на заголовок).
- **WebSocket + TLS: ALPN `http/1.1`.** Рукопожатие WebSocket идёт по HTTP/1.1, а с ALPN по умолчанию `["h2", "http/1.1"]` сервер согласовывал h2 и рукопожатие срывалось (#6782). Поэтому при включении TLS на WebSocket-inbound, а также при переключении TLS-inbound на WebSocket, панель ставит ALPN = `["http/1.1"]` (#6787; см. п. 7.3). Уже сохранённые inbound не меняются.

Пример `streamSettings` для WebSocket за CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Значения `host` и `path` должны совпадать у клиента; в отличие от RAW значение заголовка здесь — обычная строка, а не массив.

6.5. gRPC (`grpcSettings`)

Самый «лёгкий» по числу параметров транспорт. Туннелирует трафик внутри gRPC-вызовов (поверх HTTP/2); хорошо совместим с CDN, поддерживающими gRPC. Обфускации заголовков нет.

Поле	Ключ	По умолчанию	Описание
Имя сервиса (`Service Name`)	`serviceName`	`""` (пусто)	Имя gRPC-сервиса (фактически — «путь» туннеля). Должно совпадать у сервера и клиента
Authority	`authority`	`""` (пусто)	Значение псевдо-заголовка `:authority` (аналог `Host` для HTTP/2). Указывают при работе через CDN/домен
Multi Mode	`multiMode`	`false` (выкл.)	Включает мультиплексирование нескольких параллельных gRPC-потоков внутри одного соединения

Замечания:

- **Service Name** — основной идентификатор gRPC-канала; он должен быть одинаковым на обеих сторонах. Пустое значение допустимо, но обычно задают неочевидную строку для маскировки.
- **Authority** влияет на то, какой `:authority` отправляется в HTTP/2-кадрах; нужен в первую очередь при проксировании через CDN.
- **Multi Mode** позволяет нескольким логическим потокам идти через одно физическое соединение; включают для повышения производительности, когда и сервер, и клиент это поддерживают.

Пример `streamSettings` для gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (здесь `GunService`) играет роль «пути» туннеля и должен совпадать у сервера и клиента.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Транспорт на основе механизма HTTP `Upgrade` (как у WebSocket, но без самого WebSocket-протокола). Также хорошо проходит через прокси и CDN. Набор полей повторяет WebSocket, но **без** периода heartbeat.

Поле	Ключ	По умолчанию	Описание
Proxy Protocol	`acceptProxyProtocol`	`false`	Принимать заголовок PROXY protocol от вышестоящего прокси
Хост	`host`	`""` (пусто)	Значение HTTP-заголовка `Host`
Путь	`path`	`/`	Путь HTTP-запроса с заголовком `Upgrade`
Заголовки	`headers`	`{}` (пусто)	Дополнительные HTTP-заголовки. «Плоская» карта `имя → значение` (как у WebSocket)

Назначение полей **Хост**, **Путь** и **Заголовки** совпадает с WebSocket (п. 6.4). Heartbeat для HTTPUpgrade не предусмотрен — это специфика WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (он же SplitHTTP) — современный мультиплексируемый HTTP-транспорт xray-core. Разделяет восходящий и нисходящий потоки на отдельные HTTP-запросы, что хорошо подходит для CDN и сред с ограничениями по длительности соединений. Поля видимы в редакторе не все сразу: часть из них появляется в зависимости от выбранного режима (`mode`) и включённых переключателей.

Базовые поля (видны всегда)

Поле	Ключ	По умолчанию	Описание
Хост	`host`	`""` (пусто)	Значение HTTP-заголовка `Host`
Путь	`path`	`/`	Базовый путь HTTP-запросов
Режим (`Mode`)	`mode`	`auto`	Режим передачи (см. ниже)
Server Max Header Bytes	`serverMaxHeaderBytes`	`0`	Лимит размера заголовков запроса на сервере (байт). `0` — значение по умолчанию xray-core
Padding Bytes	`xPaddingBytes`	`100-1000`	Диапазон случайного «набивочного» паддинга (в байтах, формат `мин-макс`) для затруднения анализа размеров
Заголовки	`headers`	`{}` (пусто)	Дополнительные HTTP-заголовки. «Плоская» карта `имя → значение`
HTTP-метод Uplink	`uplinkHTTPMethod`	`""` (Default = POST)	HTTP-метод восходящих запросов. Варианты: пусто (по умолчанию POST), `POST` , `PUT` , `GET` (последний доступен только в режиме `packet-up`)
Padding Obfs Mode	`xPaddingObfsMode`	`false` (выкл.)	Включает расширенную обфускацию паддинга и открывает дополнительные поля (см. ниже)
No SSE Header	`noSSEHeader`	`false` (выкл.)	Не отправлять заголовок `Content-Type: text/event-stream` (SSE). Включают, если он мешает прохождению через промежуточные узлы

Поле «Режим» (`mode`)

Выпадающий список со значениями:

Значение	Описание
`auto`	Авто-выбор режима (по умолчанию)
`packet-up`	Восходящий поток разбивается на отдельные HTTP-запросы (по пакету на запрос)
`stream-up`	Восходящий поток передаётся одним длительным потоковым запросом

Значение	Описание
`stream-one`	Один общий двунаправленный потоковый запрос

Выбор режима определяет, какие дополнительные поля становятся видимыми.

Поля, видимые только при `mode = packet-up`:

Поле	Ключ	По умолчанию	Описание
Макс. буферизованная загрузка	`scMaxBufferedPosts`	`30`	Максимум одновременно буферизуемых POST-запросов восходящего потока
Макс. размер загрузки (байт)	`scMaxEachPostBytes`	`1000000`	Максимальный размер одного восходящего POST-запроса (байт)
Uplink Data Placement	`uplinkDataPlacement`	`""` (Default = body)	Где размещать данные восходящего потока: `body`, `header`, `cookie`, `query`
Uplink Data Key	`uplinkDataKey`	`""`	Имя ключа/заголовка для данных uplink. Появляется, только если `uplinkDataPlacement` задан и не равен `body`

Поле, видимое только при `mode = stream-up`:

Поле	Ключ	По умолчанию	Описание
Stream-Up Server	`scStreamUpServerSecs`	`20-80`	Диапазон времени удержания серверного потокового соединения (в секундах, формат `мин-макс`)

Поля обфускации паддинга (видны при `xPaddingObfsMode = вкл.`)

Поле	Ключ	По умолчанию	Описание
Padding Key	`xPaddingKey`	`""` (placeholder `x_padding`)	Имя ключа для паддинга
Padding Header	`xPaddingHeader`	`""` (placeholder `X-Padding`)	Имя HTTP-заголовка, в котором передаётся паддинг
Padding Placement	`xPaddingPlacement`	`""` (Default = queryInHeader)	Где размещать паддинг: `queryInHeader`, `header`, `cookie`, `query`
Padding Method	`xPaddingMethod`	`""` (Default = repeat-x)	Метод генерации паддинга: `repeat-x` или `tokenish`

Размещение сессии и последовательности (видны всегда)

Поле	Ключ	По умолчанию	Описание
Session ID Placement	`sessionIDPlacement`	`""` (Default = path)	Где передавать идентификатор сессии: `path`, `header`, `cookie`, `query`
Session ID Key	`sessionIDKey`	`""` (placeholder `x_session`)	Имя ключа сессии. Появляется, только если `sessionIDPlacement` задан и не равен `path`
Session ID Table	`sessionIDTable`	`""` (placeholder `Base62`)	Набор символов для генерации идентификаторов сессии. Можно выбрать предопределённый из выпадающего списка-автодополнения (`ALPHABET`, `Alphabet`, `BASE36`, `Base62`, `HEX`, `alphabet`, `base36`, `hex`, `number`) или ввести произвольную ASCII-строку. Пусто — значение по умолчанию xray-core
Session ID Length	`sessionIDLength`	`""` (пусто)	Длина или диапазон (например `8–16`) генерируемых идентификаторов. Показывается, только когда задан `Session ID Table`; минимум должен быть больше 0
Sequence Placement	`seqPlacement`	`""` (Default = path)	Где передавать порядковый номер пакета: `path`, `header`, `cookie`, `query`
Sequence Key	`seqKey`	`""` (placeholder `x_seq`)	Имя ключа последовательности. Появляется, только если `seqPlacement` задан и не равен `path`

Поля сессии переименованы под xray-core v26.6.22: ранее они назывались **Session Placement** / **Session Key** (`sessionPlacement` / `sessionKey`) — теперь это **Session ID Placement** / **Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); старого имени ядро больше не понимает. Inbound, сохранённые до обновления, мигрируются на новые ключи автоматически — пересохранять их не требуется.

Рекомендации:

- Для большинства установок достаточно оставить **Режим =** `auto`, задать **Путь/Хост** и (при работе через CDN) согласовать их с клиентом.
- Поля размещения (`*Placement` / `*Key`) и обфускации паддинга нужны только для тонкой подстройки под конкретный анти-DPI/CDN-сценарий; при пустых значениях используются значения по умолчанию xray-core, указанные в скобках.
- Параметры, относящиеся к стороне клиента/outbound (например, интервалы повторных POST, размеры чанков), в форме inbound не выводятся — сервер-слушатель их игнорирует. Мультиплексор XMUX, напротив, в форме inbound доступен (см. ниже).
- **Служебные дефолты не проставляются.** Панель больше не записывает в конфиги XHTTP служебные значения по умолчанию `scMaxEachPostBytes` и `scMinPostsIntervalMs` — применяются внутренние значения xray-core. Это устраняет постоянную DPI-сигнатуру (литерал `scMinPostsIntervalMs=30`), по которой раньше мог блокироваться трафик. Для уже сохранённых inbound совпадающие с дефолтами xray-core значения не выводятся в ссылках и подписке (пересохранять inbound не требуется); заданные вручную значения сохраняются.

Пример `streamSettings` для XHTTP (режим `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Для большинства установок достаточно этих четырёх полей; поля размещения сессии/последовательности и обфускации паддинга оставляют пустыми — тогда используются значения по умолчанию xray-core.

XMUX (мультиплексирование соединений)

Переключатель **XMUX** (`enableXmux`) включает слой мультиплексирования, который распределяет параллельные запросы по небольшому пулу физических соединений. При включении раскрываются шесть настраиваемых полей (хранятся в `xhttpSettings.xmux`):

Поле	Ключ	По умолчанию	Описание
Max Concurrency	`maxConcurrency`	`16-32`	Максимум одновременных запросов на одно соединение (диапазон `мин-макс`)
Max Connections	`maxConnections`	`6`	Максимум физических соединений (`0` — без ограничения). С версии 3.4.2 новые inbound создаются со значением `6` ; ранее созданные сохраняют своё.
Max Reuse Times	`cMaxReuseTimes`	`""` (пусто)	Сколько раз переиспользовать соединение
Max Request Times	`hMaxRequestTimes`	`600-900`	Максимум запросов на соединение (диапазон)
Max Reusable Secs	`hMaxReusableSecs`	`1800-3000`	Время, в течение которого соединение пригодно для повторного использования (секунды, диапазон)
Keep Alive Period	`hKeepAlivePeriod`	`""` (пусто)	Период keep-alive для удержания соединения

> **Важно.** Задавать одновременно **Max Connections** и **Max Concurrency** нельзя — xray-core отклонит такой конфиг. По умолчанию при включении XMUX панель проставляет `Max Concurrency = 16-32` ; если вы зададите **Max Connections** (значение больше `0`), панель уберёт значение `Max Concurrency` по умолчанию, чтобы избежать конфликта.

6.8. Транспорт Hysteria (`hysteriaSettings`)

Транспорт **Hysteria** не выбирается в списке «Транспорт»: он автоматически активируется, когда inbound создаётся с протоколом Hysteria, и для других протоколов скрыт (при уходе с протокола Hysteria сеть принудительно возвращается к `tcp`). Параметры:

Поле	Ключ	По умолчанию	Описание
Версия	`version`	`2` (зафиксировано, поле заблокировано)	Версия Hysteria. Поддерживается только Hysteria 2
UDP idle timeout (с)	`udpIdleTimeout`	`60`	Таймаут простоя UDP-сессии (секунды). Допустимый диапазон — 2–600; xray-core отклоняет значения вне этого интервала при запуске
Masquerade	`masquerade`	выкл. (отсутствует)	Включает маскировку слушателя под HTTP/3-сервер при зондировании

При включённом **Masquerade** появляется выбор типа (`type`) и зависящие от него поля:

- `""` **— default (404 page)**: отдаётся стандартная страница 404 (доп. поля не требуются).
- `proxy` **(reverse proxy)**: обратное проксирование на внешний сайт.
  - `url` (**Upstream URL**, placeholder `https://www.example.com`) — целевой адрес;
  - `rewriteHost` (**Переписать Host**, по умолчанию `false`) — подменять заголовок `Host` ;
  - `xForwarded` (**Заголовки X-Forwarded**, по умолчанию выключено) — добавлять к проксируемому запросу `X-Forwarded-For` с IP клиента, а также `X-Forwarded-Host` и `X-Forwarded-Proto` , чтобы апстрим видел исходный запрос;
  - `insecure` (**Пропустить TLS verify**, по умолчанию `false`) — не проверять TLS-сертификат апстрима.
- `file` **(serve directory)**: раздача файлов из каталога.
  - `dir` (**Директория**, placeholder `/var/www/html`).
- `string` **(fixed body)**: фиксированный HTTP-ответ.
  - `statusCode` (**Код статуса**, по умолчанию `0` , диапазон 0–599);
  - `content` (**Body**) — тело ответа;
  - `headers` (**Заголовки**) — карта `имя → значение` .

Masquerade позволяет inbound на основе Hysteria выглядеть как обычный HTTP/3-сервер для активных проб, что повышает скрытность. По умолчанию маскировка выключена.

Пример `hysteriaSettings` с обратным проксированием (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Здесь при зондировании слушатель отдаёт ответ от `https://www.example.com`, маскируясь под обычный HTTP/3-сайт.

6.9. Сопутствующие параметры

Помимо выбора сети, на той же вкладке доступны два общих блока, не зависящих от конкретного транспорта (подробно — в соответствующих разделах):

- **External Proxy** (`externalProxy`) — список внешних адресов/портов, которые подставляются в ссылки-подписки вместо адреса самой панели.
- **Sockopt** (`sockopt`) — низкоуровневые опции сокета (TCP Fast Open, mark, домен-стратегия, прозрачное проксирование и т. п.).

Real client IP (определение реального IP за CDN/релеем)

Когда inbound стоит за посредником (CDN вроде Cloudflare, L4-туннель/релей или другая панель), Xray видит адрес посредника, а не реального посетителя. Этот адрес попадает в список онлайн-клиентов и по нему считается лимит IP на клиента, из-за чего оба становятся бесполезны за прокси. Чтобы восстановить реальный IP, в секции **Sockopt** формы inbound есть выбор пресета **Real client IP**, объединяющий настройки `acceptProxyProtocol` и `trustedXForwardedFor`:

Пресет	Что делает	Когда применять
Off / direct	Очищает оба поля.	Inbound доступен клиентам напрямую
Cloudflare CDN	Устанавливает `sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]`.	WebSocket / HTTPUpgrade / XHTTP / gRPC за CDN Cloudflare (оранжевое облако)
L4 relay / Spectrum (PROXY)	Включает `acceptProxyProtocol = true`.	L4-туннель/релей перед inbound или Cloudflare **Spectrum**

Пресеты взаимоисключающие: выбор одного очищает поле другого, поэтому устаревший `trustedXForwardedFor` не перебивает восстановленный по PROXY-протоколу IP. Ниже пресета остаются видны «сырые» переключатель **Proxy Protocol** и список **Trusted X-Forwarded-For** — пресет лишь заполняет их за вас, а при необходимости их правят вручную. Если выбранный пресет не поддерживается текущим

транспортом (например, PROXY-протокол на mKCP), форма показывает предупреждение. Эти поля относятся только к стороне сервера и **никогда не отправляются клиентам в подписках**.

> **Используйте что-то одно.** `acceptProxyProtocol` читает реальный IP из L4-заголовка PROXY-протокола, а `trustedXForwardedFor` — из HTTP-заголовка запроса; смешивать их вручную стоит, только если этого требует ваша цепочка посредников.

- **FinalMask** (`finalmask`) — общий механизм обфускации транспортного уровня (включая legacy-обфускацию mKCP), который заменил собой отдельные поля «seed»/«header type» внутри под-форм сетей. Подробнее — п. 6.12.

6.10. XDRIVE (`xdriveSettings`)

XDRIVE (с 3.10.0) передаёт поток файлами в папке общего хранилища, доступной обеим сторонам: сервер не принимает соединения на своём порту, а опрашивает эту папку, клиент пишет и читает в ней файлы сессий. Хранилищем может быть Google Drive, локальная папка или любой HTTP API хранилища, описанный шаблоном. Транспорт выбирается в списке **«Транспорт»** как **XDRIVE**; та же под-форма используется и в форме outbound.

Обе стороны должны использовать **одинаковые** сервис, папку и секреты.

Только VLESS с шифрованием (Xray-core#7182). xray-core (с 26.10.17) запускает XDRIVE только для VLESS внутри VLESS-шифрования и без flow, а в outbound — только через Mux; иначе ядро отказывается принимать всю конфигурацию. Поэтому панель:

- показывает **XDRIVE** в списке **«Транспорт»** только у inbound'а VLESS и отклоняет сохранение XDRIVE-inbound'а с другим протоколом или с пустым **«Расшифрование»** (`decryption` = `none`) — заполните пару ключей блоком «Генерация ключей» (5.3);
- не выдаёт flow клиента на XDRIVE: ни в серверный конфиг, ни в JSON-подписку, ни в формах (поле `flow` недоступно, даже при VLESS-шифровании);
- при выборе XDRIVE в форме outbound сама включает **Mux**;
- не включает в итоговый конфиг Xray сохранённые ранее XDRIVE-inbound'ы без VLESS-шифрования и outbound'ы XDRIVE из шаблона, которые ядро теперь отклонило бы (например, без Mux), а пишет в лог предупреждение. Остальные inbound'ы продолжают работать, а не останавливаются вместе с ядром.

Поле	Ключ	По умолчанию	Описание
Сервис хранилища	`service`	`google-drive`	**Google Drive** (`google-drive`), **Локальная папка** (`local`) или **Свой шаблон** (`template`). Прежнее имя `Google Drive` ядро читает как устаревший псевдоним; панель при сохранении записывает `google-drive` , а форма заменяет старое имя при открытии

Поле	Ключ	По умолчанию	Описание
Удалённая папка	`remoteFolder`	`""` (placeholder `xray-tunnel`, для локальной папки — `/var/lib/xdrive`)	Обязательное поле. Папка с файлами сессий: для Google Drive — идентификатор общей папки, для `local` — каталог на диске, общий для обеих сторон
Client ID / Client Secret / Refresh Token	`secrets[0]`, `secrets[1]`, `secrets[2]`	—	Только для Google Drive: три OAuth-секрета строго в этом порядке, все обязательны (Client Secret и Refresh Token вводятся скрытым полем)
Секреты	`secrets`	`[]`	Только для шаблона: значения, которые шаблон читает как `{secret0}`, `{secret1}`, … (ввод тегами через запятую)
Шаблон хранилища	`template`	—	Только для шаблона, обязательное: JSON-объект с операциями API хранилища (`auth`, `put`, `get`, `list`, `delete`). Редактор принимает только объект JSON, иначе показывает «Должен быть объект JSON»
Адрес фронтинга	`streamSettings.address`	пусто (placeholder `www.googleapis.com`)	Необязательный хост или IP, через который идёт подключение к API хранилища (domain fronting). Не показывается для `local`
Порт фронтинга	`streamSettings.port`	пусто (placeholder `443`)	Порт для адреса фронтинга (1–65535). Не показывается для `local`

Адрес и порт фронтинга хранятся на уровне `streamSettings`, а не внутри `xdriveSettings`; при уходе с XDRIVE на другую сеть панель их удаляет.

XMUX клиента. Необязательный объект `xmux` в `xdriveSettings` (как и в `masqueSettings`, см. 6.11) задаёт клиентское мультиплексирование. В форме его нет: задайте его в расширенном JSON (4.7). Панель не отбрасывает `xmux` при сохранении inbound'а, а JSON-подписка передаёт значение inbound'а клиентам. Пока он не задан, ядро подбирает значения само (для XDRIVE — по сервису).

Параметры тонкой настройки — необязательные целые числа ⩾ 0. Пустое поле означает значение по умолчанию xray-core (оно показано в поле как placeholder):

Поле	Ключ	Значение ядра
Размер сегмента (байт)	`segmentBytes`	`524288` (512 KiB; ядро ограничивает 16 MiB)
Интервал отправки (мс)	`flushIntervalMs`	`20`
Интервал опроса (мс)	`pollIntervalMs`	`50`

Поле	Ключ	Значение ядра
Макс. интервал опроса (мс)	`maxPollIntervalMs`	`500`
Окно упреждения (мс)	`eagerWindowMs`	`2000`
Тайм-аут пропущенного сегмента (мс)	`holeTimeoutMs`	`30000`
TTL сессии (с)	`sessionTtlSeconds`	`300`
Параллелизм	`concurrency`	`8` (ядро ограничивает 64)

Проверка хранилища при сохранении. xray-core открывает хранилище XDRIVE только в момент запуска слушателя, поэтому некорректный inbound раньше останавливал Xray вместе со всеми остальными inbound. Теперь панель применяет правила ядра при сохранении inbound (добавление и изменение), а также к inbound'ам и outbound'ам шаблона Xray:

- сервис должен быть `local`, `google-drive` (прежнее `Google Drive` принимается) или `template`;
- для `local` и `google-drive` нужна **Удалённая папка**;
- для Google Drive нужны ровно три непустых секрета — ClientID, ClientSecret, RefreshToken;
- шаблон должен содержать операции `put`, `get`, `list` и `delete`, каждую со своим `url`, а у `list` — регулярное выражение `namesRegex`, которое компилируется и содержит хотя бы одну группу захвата (имя объекта).

Безопасность. У XDRIVE нет собственного уровня TLS/REALITY: его трафик — это HTTPS самого API хранилища. При выборе XDRIVE в inbound панель сбрасывает безопасность в **Нет** (`none`) и удаляет `tlsSettings` / `realitySettings` — иначе ядро прочитало бы TLS inbound'а как клиентский TLS к API хранилища. На outbound режим **TLS** доступен и задаёт именно это HTTPS-подключение: API хранилища видит его SNI, uTLS-отпечаток и ALPN (подробнее о форме outbound — 11.4). Маски TCP из FinalMask к XDRIVE применимы (см. п. 6.12).

Ссылки и подписки. Формата ссылки у XDRIVE нет, а ссылка с секретами хранилища попала бы в каждый QR-код, поэтому для таких inbound ссылка, QR-код и запись Clash не создаются. JSON-подписка передаёт поток целиком — **вместе с папкой и секретами хранилища** — каждому клиенту.

Пример `streamSettings` для XDRIVE через Google Drive с фронтингом:

```
{
  "network": "xdrive",
  "address": "142.250.0.1",
  "port": 443,
  "xdriveSettings": {
    "service": "google-drive",
    "remoteFolder": "xray-tunnel",
    "secrets": [
      "000000000000-example.apps.googleusercontent.com",
      "GOCSPX-example-client-secret",
      "1//example-refresh-token"
    ]
  }
}
```

Параметры тонкой настройки здесь не указаны — действуют значения ядра из таблицы выше. Для inbound'а нужен VLESS с заданным `decryption` (см. выше).

6.11. Транспорт MASQUE (`masqueSettings`)

Транспорт **MASQUE** (с 3.10.0) — CONNECT-IP поверх HTTP/3 или HTTP/2 — работает только в паре с протоколом MASQUE (сам протокол, его клиенты, пул адресов и MTU описаны в 5.15). Как и Hysteria, он не выбирается в списке **«Транспорт»**: при выборе протокола MASQUE панель сама задаёт `network = "masque"` , безопасность **TLS** и путь по умолчанию, а при уходе с протокола MASQUE сеть возвращается к `tcp` с безопасностью **Нет**.

Поле	Ключ	По умолчанию	Описание
Путь	`path`	`/.well-known/masque/ip/*/*/`	Путь запроса, на который отвечает сервер. Это же значение — путь ядра по умолчанию, поэтому пустое поле работает так же

Серверу нужен только путь: хост, учётные данные Basic auth (`user` / `pass`), заголовки и блок WARP — клиентские поля, они задаются в форме outbound (см. 11.4), а клиентам панели приходят через JSON-подписку. Необязательный объект `xmux` (клиентское мультиплексирование) в форме не выводится: он задаётся в расширенном JSON, сохраняется вместе с inbound'ом и уходит клиентам в JSON-подписке; пока не задан, ядро держит одно соединение (подробнее об XMUX — 6.10).

TLS обязателен. На вкладке **«Безопасность»** у MASQUE есть только режим **TLS** (кнопок **Нет** и **Reality** нет). Новый MASQUE-inbound получает TLS как у Hysteria: ALPN = `["h3"]` , uTLS-отпечаток — **None**. **ALPN** определяет, какие слушатели поднимает ядро, и по нему же панель проверяет конфликт портов:

ALPN	Слушатель
`h3` (по умолчанию) или без `h2`	HTTP/3 по UDP
только `h2`	HTTP/2 по TCP
`h3` и `h2`	оба: HTTP/3 по UDP и HTTP/2 по TCP

Ссылки. Формата ссылки у MASQUE нет: для таких inbound ссылка, QR-код и запись Clash не создаются; клиенты получают готовый outbound `masque` из JSON-подписки.

Пример `streamSettings` для MASQUE с обоими слушателями:

```
{
  "network": "masque",
  "masqueSettings": {
    "path": "/.well-known/masque/ip/*/*/"
  },
  "security": "tls",
  "tlsSettings": {
    "serverName": "masque.example.com",
    "alpn": ["h3", "h2"],
    "certificates": [
      {
        "certificateFile": "/etc/ssl/certs/masque.crt",
        "keyFile": "/etc/ssl/private/masque.key"
      }
    ]
  }
}
```

6.12. FinalMask: обфускация транспортного уровня

FinalMask (`streamSettings.finalmask`) оборачивает трафик **после** транспорта и безопасности. Редактор масок расположен внизу вкладки транспорта inbound (для `tunnel` его нет); тот же редактор используется в форме outbound, в хостах и в настройке Final Mask JSON-подписки. Показываются только группы, применимые к выбранной сети:

Группа	Ключ	Где доступна
TCP Masks	`finalmask.tcp`	Сети `tcp` , `ws` , `grpc` , `httpupgrade` , `xhttp` , `xdrive` (кроме WireGuard)
UDP Masks	`finalmask.udp`	Hysteria, WireGuard, mKCP; в клиентских редакторах (outbound, хосты, JSON-подписка) — также XHTTP, чей HTTP/3-дозвон проходит через UDP-маски
QUIC Params	`finalmask.quicParams`	Hysteria и XHTTP

Типы масок: TCP — **Fragment**, **Header Custom**, **Sudoku**, **XMC (Minecraft)**; UDP — **mKCP Legacy**, **xDNS**, **xICMP**, **Realm**, **Header Custom**, **Noise**, **Sudoku**, у WireGuard также **Salamander**, а у Hysteria список ограничен маской **Salamander (Hysteria2)**. Клиентские редакторы дополнительно предлагают **UDP Hop**, а для TCP — готовые **«Пресеты»** (см. ниже); редактор inbound предлагает маску **Filter (source IPs)** для TCP и UDP (у Hysteria — наряду с Salamander). Особенности отдельных масок (режим Gecko у Salamander, Realm, XMC, списки Fragment) — в 11.4.

UDP Hop (`udphop`) — смена порта на стороне клиента

Начиная с xray-core 26.9.9 прыжки по портам выполняет только клиентская UDP-маска `udphop` : ключ `quicParams.udpHop` ядро пропускает, а на слушателе маска не запускается («udphop: client only»). Поэтому редактор различает стороны:

- **Inbound (сервер).** Маски **UDP Hop** в списке нет. В **QUIC Params** остаётся переключатель **UDP Hop** с полями **Hop Ports** и **Hop Interval (s)** — это только диапазон, объявляемый клиентам (`mport` в ссылке `hysteria2://` и маска `udphop` в JSON-подписке); сам сервер слушает лишь свой порт.
- **Клиентская сторона** (outbound, хосты, Final Mask JSON-подписки). Маска **UDP Hop** доступна в списке UDP-масок, в том числе для XHTTP; переключателя **UDP Hop** в QUIC Params нет, а сохранённый ранее `quicParams.udpHop` при открытии формы сам превращается в маску.

Поле	Ключ	По умолчанию	Описание
Mode	`mode`	`intervalRemote`	Обязательное, один или несколько режимов (на проводе — строка через запятую): **Rotate server port (interval)** (`intervalRemote`) — менять порт сервера по интервалу; **Random server port per connection** (`perConnRemote`) — случайный порт сервера для каждого соединения; **Rotate local port (interval)** (`intervalLocal`) — менять локальный порт по интервалу
Interval (s)	`interval`	`5–10`	Число или диапазон `мин–макс` в секундах; каждое значение — целое ⩾ 5
Remote Ports	`remotePorts`	`""` (placeholder `20000–50000`)	Диапазон портов сервера
Remote IPs	`remoteIPs`	пусто	Необязательный список адресов сервера, между которыми тоже можно переключаться (ввод тегами)

xray-core применяет UDP-маски в обратном порядке и отклоняет `udphop` где-либо, кроме конца списка, — причём только при подключении (ошибка «incorrect index»; `xray run -test` сообщает «Configuration OK»), так что клиент просто не соединяется. Поэтому везде, где панель формирует клиентский finalmask (JSON-подписка, параметр `fm=` ссылок, импорт ссылок, outbound'ы шаблона при сборке конфига), остаётся **одна** маска `udphop` — последняя из указанных — и она ставится в конец списка. Если диапазон задан в нескольких местах, JSON-подписка берёт самый конкретный: хост → сам inbound → глобальная Final Mask подписки.

Разовая миграция `UDPHopClientMaskFix` переводит устаревший `quicParams.udpHop` в маску `udphop` (режим `intervalRemote`) в хостах, Final Mask JSON-подписки, кэше подписочных outbound'ов и outbound'ах шаблона Xray; inbound'ы она не трогает.

Header Custom: переменные, группа Errors и режим UDP

В маске **Header Custom** (TCP и UDP) у каждого элемента появился выбор **Kind** — элемент задаёт ровно один вид содержимого, как требует ядро:

Kind	Ключ	Описание
Bytes	`type` + `packet` / `rand`	Обычные байты (тип Array / String / Hex / Base64), как раньше
Reuse variable	`reuse`	Записывает байты, ранее сохранённые под этим именем (поле **Reuse**, обязательно)
Transform	`transform`	Выражение-дерево в JSON, например `{"op": "concat", "args": [{"reuse": "nonce"}]}`. В конфиг попадает только корректный JSON-объект, иначе редактор показывает «Must be a JSON object»

Поле **Capture** (`capture` , у любого вида) сохраняет байты, которые элемент пишет или читает, под именем, доступным следующим элементам. Имена переменных — латинские буквы, цифры и `_` , не с цифры. При смене **Kind** поля прежнего вида очищаются.

У TCP-маски, помимо групп **Clients** и **Servers**, появилась группа **Errors** (`errors`) — ответ, отправляемый вместо обычного, когда соответствующая группа клиента не совпала (например, HTTP 400). У UDP-маски добавлен **Mode** (`mode`): **Prefix** (`prefix` , по умолчанию) — заголовок добавляется к каждому пакету; **Standalone** (`standalone`) — заголовок отдельными пакетами-рукопожатием перед данными.

Sudoku как UDP-маска, Reset у Noise

- **Sudoku** доступна теперь и в списке **UDP Masks** с теми же полями, что у TCP-варианта (**Password**, **ASCII**, **Custom Table**, **Custom Tables**, **Padding Min**, **Padding Max**); в параметре `fm=` ссылок она сохраняется.
- **Reset** у маски **Noise** (`reset`) — время в секундах до повторной отправки шума тому же адресу (`0` — однократно); теперь это число или диапазон `мин–макс` (например `60–120`), целые ⩾ 0, минимум не больше максимума.

Пароль Salamander — не короче 4 байт

xray-core требует для маски **Salamander** (и режима Gecko) пароль не короче **4 байт**, но проверяет это только при открытии сокета: `xray run -test` проходит, затем на inbound не поднимается слушатель и Xray не запускается вовсе (вместе со всеми остальными inbound), а на клиенте не проходит подключение. Теперь редактор подсвечивает короткий пароль сразу («Use at least 4 bytes; xray-core refuses a shorter password»), а сохранение с таким паролем отклоняется везде, где хранится finalmask: inbound (добавление и изменение), хосты, глобальная Final Mask подписки, inbound'ы и outbound'ы шаблона Xray. Длина считается в байтах, поэтому 2 кириллических символа (4 байта в UTF-8) проходят, а 3 латинских — нет.

xDNS под Xray-core 26.10.10

Формат маски **xDNS** изменился в ядре (Xray-core#7090): у домена вместо одного `name` — список **Names** (`names` , ввод тегами), а резолвер — список адресов **`addrs`** в виде URL `udp://…` или `tcp://…` (placeholder `udp://8.8.8.8:53, tcp://1.1.1.1`) вместо `{type, settings:{addr}}` . **Record Types** стали необязательными: пусто — значение ядра (TXT на клиенте, A/CNAME/TXT/AAAA на сервере). Старые ключи ядро молча игнорирует (туннель остался бы без домена и резолвера), поэтому разовая миграция `XdnsFinalmaskNamesAddrsFix` переписывает все сохранённые маски xDNS — и формата до 26.9.30, и объектного формата 26.9.30; то же преобразование выполняется при сохранении, сборке конфига, импорте

ссылок и открытии редактора. Формат на проводе не изменился, но клиенты на старом ядре новую форму конфига прочитать не могут.

xDNS: резолвер по DoH

Адрес резолвера xDNS может быть и `https://…` (DNS-over-HTTPS), а не только `udp://` / `tcp://`. Редактор показывает для каждого резолвера (блок **Resolver N** с кнопкой удаления) четыре поля: **Addresses** (`addrs`, ввод тегами; placeholder `udp://8.8.8.8:53, https://dns.google/dns-query`), **Workers** (`worker`, число ≥ 1; placeholder `32`), **DoH TLS** (`tlsConfig`, JSON-объект `tlsSettings` для адресов `https://`, например `{"serverName": "dns.google", "fingerprint": "chrome"}`; `serverName` позволяет прикрыть домен) и **DoH QUIC** (`quicConfig`, JSON-объект параметров QUIC для адреса, до которого идёт h3, например `{"congestion": "bbr"}`). Раньше редактор знал только `addrs`, и сохранение отбрасывало остальные поля. В поля JSON принимается только корректный объект.

Noise: скрипт Lua

У маски **Noise** есть поле **Script** (`script`) — имя Lua-файла (например, `noise.lua`), который xray-core выполняет, чтобы построить шум; файл ищется рядом с бинарником xray. Скрипт **заменяет** список шума: при непустом **Script** список скрывается, потому что ядро (с 26.10.17, Xray-core#7124) отказывается принимать скрипт вместе с элементами `noise`. Относительный путь панель разрешает так же, как дочерний процесс xray, — рядом с бинарником, — поэтому проверка при сохранении шаблона и горячее добавление не отклоняют файл, который ядро найдёт.

Filter: фильтр IP-источников

Маска **Filter (source IPs)** (`filter`, TCP и UDP; xray-core 26.10.21, Xray-core#7206) отсекает подключения к inbound'у по IP источника до работы остальных масок. Она доступна только в редакторе inbound (серверная сторона). Поля вводятся тегами:

Поле	Ключ	Описание
Allowed Source IPs	`allowedSourceIPs`	Подключаться могут только перечисленные источники; пусто — разрешены все. Пример: `10.0.0.0/8, geoip:ir`
Blocked Source IPs	`blockedSourceIPs`	Эти источники отбрасываются. Пример: `203.0.113.7, geoip:cn`

Совпадение с запрещённым списком сильнее совпадения с разрешённым. На стороне клиента маска ничего не делает, а клиентское ядро старой версии отклоняет неизвестный тип, поэтому ссылка (`fm=`) и JSON-подписка её не передают.

Пресеты tlshello

В клиентских редакторах (outbound, хосты, Final Mask JSON-подписки) рядом с кнопкой добавления в группе **TCP Masks** есть список **«Пресеты»** с двумя готовыми наборами PattN: `tlshello-0-len` и `tlshello-6-98`. Выбор пресета **заменяет весь список TCP-масок** двумя связанными масками Fragment. Первая режет TLS ClientHello (`packets` = `tlshello`) на три сегмента: длины `0,104,1` (в `tlshello-0-len` первый сегмент пустой) или `6,98,1` (`tlshello-6-98`), задержка `0`, `maxSplit` `0`.

Вторая режет следующий пакет: `packets` = `1-1`, длины `114,1`, задержка `1` мс, `maxSplit` `11`. В редакторе inbound пресетов нет. Проверка длин Fragment теперь отклоняет нулевой минимум только в последнем элементе `lengths` (так требует ядро); ноль в более раннем элементе означает пустой сегмент, на нём и построен `tlshello-0-len`.

---

7. Безопасность соединения: TLS, XTLS и REALITY

Каждый inbound, поддерживающий передачу через transport-поток (VMess, VLESS, Trojan, Shadowsocks, Hysteria, MASQUE), имеет вкладку **«Безопасность»** в редакторе. На ней настраивается, как именно шифруется и маскируется транспортный канал. Доступны три режима, переключаемые кнопками-радио:

Режим	Подпись в UI	Когда доступен
`none`	**Нет**	Всегда (кроме Hysteria и MASQUE, где TLS обязателен); для inbound на транспорте XDRIVE — единственный режим
`tls`	**TLS**	Для VMess/VLESS/Trojan/Shadowsocks на сетях `tcp`, `ws`, `http`, `grpc`, `httpupgrade`, `xhttp`; для Hysteria и MASQUE — всегда
`reality`	**Reality**	Только для VLESS/Trojan на сетях `tcp`, `http`, `grpc`, `xhttp`

Кнопка **Нет** не показывается, если протокол — Hysteria или MASQUE (для них TLS обязателен; о слушателях MASQUE по ALPN — п. 6.11). У inbound на транспорте XDRIVE, наоборот, доступен только режим **Нет**: при выборе XDRIVE панель сама сбрасывает безопасность в `none`, потому что ядро прочитало бы TLS inbound'а как клиентский TLS к API хранилища (п. 6.10). Кнопка **Reality** появляется только при допустимой комбинации протокола и сети (см. таблицу выше). У TUIC v5 вкладок **«Поток»** и **«Безопасность»** нет: сертификат и ключ сервера указываются прямо на вкладке **«Протокол»** (см. 5.13).

При смене режима панель полностью пересобирает блок `streamSettings`: удаляются `tlsSettings` и `realitySettings` от предыдущего режима и подставляются значения по умолчанию для выбранного. При выборе **TLS** создаётся одна пустая запись сертификата, а uTLS-отпечаток ставится в `chrome`; ALPN — `["h2", "http/1.1"]`, а на транспорте WebSocket — `["http/1.1"]` (#6787). При выборе **Reality** панель генерирует случайные `shortIds` и путь SpiderX и запрашивает у сервера свежую пару ключей X25519 (privateKey/publicKey); поля **Цель** (`target`) и **SNI** (`serverNames`) остаются пустыми — их заполняет сканер целей (см. 7.4).

7.1. В чём разница: TLS vs XTLS vs REALITY

- **TLS** — классическое шифрование транспорта по протоколу TLS 1.2/1.3. На сервере должен лежать действительный сертификат (свой домен + цепочка). Трафик выглядит как обычный HTTPS, но для активного цензора характерен распознаваемый TLS-handshake к вашему домену; при «отстреле» по SNI или при отсутствии доверенного сертификата соединение блокируется/выдаёт ошибку.
- **XTLS (Vision)** — это не отдельный режим в списке «Безопасность», а *flow*-механизм поверх TLS **или** Reality. Включается на стороне клиента inbound через поле **Flow** = `xtls-rprx-vision` (или `xtls-rprx-vision-udp443`). Vision доступен для VLESS на сети `tcp` при `security = tls` или `security = reality`, а также для VLESS на **любом** транспорте при включённом VLESS-шифровании (vlessenc / ML-KEM): xray-core выполняет Vision внутри VLESS-шифрования независимо от транспорта, поэтому с 3.10.0 это уже не только `xhttp`. В этом случае поле **Flow** также можно установить в `xtls-rprx-vision`, и значение корректно попадает в ссылку `vless://` (`flow=xtls-rprx-vision`). Если транспорт inbound'а Vision нести не может (например, inbound перевели с `tcp` + TLS на WebSocket без VLESS-шифрования), панель не передаёт flow клиентов в конфиг Xray — раньше сервер в такой ситуации требовал Vision и отклонял всех клиентов. То же правило действует для ссылок, подписок и

формы outbound. Vision снижает «двойное шифрование» (TLS-in-TLS), отдавая полезную нагрузку напрямую после рукопожатия, что ускоряет передачу и улучшает маскировку. Поэтому связка **VLESS + Reality + Flow** `xtls-rprx-vision` считается рекомендуемой современной конфигурацией.

> **Автоматическое восстановление flow Vision.** Если у VLESS-inbound (на любом транспорте) шифрование (ML-KEM, поля decryption/encryption) включается уже после того, как были добавлены клиенты, inbound становится flow-пригодным. В этой ситуации панель сама восстанавливает `flow = xtls-rprx-vision` у тех клиентов, которым он положен, но у кого поле **Flow** оказалось пустым. Раньше при таком сценарии Vision молча пропадал из конфигов, share-ссылок и подписок (особенно заметно на узловых inbound). Никаких ручных действий не требуется: исправление применяется автоматически при сохранении inbound и однократно при обновлении панели. Поведение консервативное — панель не выдумывает flow и не перезаписывает явно заданное клиентом значение.

- **REALITY** — механизм маскировки без собственного сертификата. Сервер «заимствует» TLS-рукопожатие реального стороннего сайта (`target` / `serverNames`), поэтому для наблюдателя соединение неотличимо от обращения к этому сайту, а сертификат не нужен вовсе. Аутентификация строится на паре ключей X25519 и наборе `shortIds` . REALITY устойчив к активным пробам (`active probing`) и блокировке по SNI, поскольку SNI указывает на настоящий внешний домен. Цена — более строгие требования к настройке (корректный `target` с портом, синхронизация ключей с клиентом).

Короткое правило выбора:

- есть свой домен и валидный сертификат, нужен простой HTTPS-вид → **TLS** (по возможности с Vision);
- нет домена/сертификата либо нужна максимальная скрытность от DPI → **REALITY** (с Vision для VLESS/TCP).

7.2. Режим «Нет» (`none`)

Транспорт передаётся без TLS-обёртки: блоки `tlsSettings` и `realitySettings` из `streamSettings` исключаются. Дополнительных полей у режима нет. Подходит, когда:

- inbound слушает только на `127.0.0.1` и служит fallback-целью (по правилу панели дочерний inbound для fallback должен слушать на `127.0.0.1` с `security=none`);
- шифрование/маскировку обеспечивает внешний слой (например, обратный прокси Nginx перед панелью);
- используется протокол с собственным шифрованием (Shadowsocks) во внутренней сети.

Для inbound, доступных извне, режим «Нет» не рекомендуется.

Пример: блок `streamSettings` для TLS на сети `tcp` (VLESS/Trojan/VMess). Так выглядит результат после выбора режима **TLS** и заполнения SNI и путей к сертификату:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. Режим TLS

Поля блока `tlsSettings`. Значения по умолчанию взяты из схемы панели.

Основные параметры

Поле (подпись)	Значение по умолчанию	Описание
SNI (`serverName`)	`""` (пусто)	Server Name Indication — имя домена, предъявляемое в TLS-рукопожатии. Должно совпадать с доменом сертификата. Англ. подсказка-плейсхолдер: «Server Name Indication».
Cipher Suites (`cipherSuites`)	`""` → **Авто**	Список разрешённых наборов шифров. По умолчанию пусто — выбор отдаётся на усмотрение Xray/Go (вариант **Авто**). Менять только при необходимости явно ограничить шифры. Поле — мультивыбор с вводом тегов; сверху выпадающего списка есть группа **«Пресеты»** (`semi-python` и `real-firefox`) — готовые упорядоченные списки клиентских наборов. Пресет **заменяет весь список целиком**, а не дополняет его: порядок наборов входит в ClientHello. В пресеты входят и наборы, которых нет в перечне ниже (например, `TLS_RSA_WITH_AES_128_GCM_SHA256`). Клиенты применяют наборы шифров **только с отпечатком uTLS `unsafe`** : для любого другого отпечатка Xray строит ClientHello через uTLS и список игнорируется. Если наборы заданы, а в поле **uTLS** выбран другой непустой отпечаток, под полем появляется предупреждение «Клиенты применяют эти наборы шифров только с отпечатком uTLS «unsafe»; при любом другом отпечатке они игнорируются.». Пустой отпечаток (**None**) Xray читает как `chrome` , поэтому ссылки и подписки сами ставят `fp=unsafe` , когда наборы заданы, а отпечаток пуст (см. 10.4). Список уходит клиентам параметром `cs` ссылки (тот же список через двоеточие).

Поле (подпись)	Значение по умолчанию	Описание
Мин/Макс версия (`minMaxVersion`)	min = `1.2`, max = `1.3`	Минимальная и максимальная версии TLS. Доступные значения: `1.0`, `1.1`, `1.2`, `1.3`. Рекомендуется оставить `1.2` – `1.3`; понижать минимум до 1.0/1.1 нежелательно (устаревшие, небезопасные версии).
uTLS (`settings.fingerprint`)	`chrome` при включении TLS; у нового Hysteria- и MASQUE-inbound — **None** (`""`)	Имитируемый TLS-отпечаток клиентского хелло (uTLS fingerprint), чтобы рукопожатие выглядело как у популярного браузера. См. список ниже. В TLS первый пункт списка — **None** (`""`), отключающий имитацию; Hysteria uTLS-отпечатки не принимает, поэтому для неё по умолчанию выбран None. Начиная с 3.8.0 выбранный **None** сохраняется — раньше при каждом сохранении он молча возвращался к `chrome`; отсутствующее в сохранённом inbound поле (например, у созданного через API) тоже читается как **None**.
ALPN (`alpn`)	`["h2", "http/1.1"]`; на WebSocket — `["http/1.1"]`; у Hysteria и MASQUE — `["h3"]`	Список протоколов прикладного уровня, согласуемых в TLS (множественный выбор). Допустимые значения: `h3`, `h2`, `http/1.1`. По умолчанию предлагаются `h2` и `http/1.1`. Для WebSocket панель ставит `http/1.1` — и при включении TLS, и при переключении TLS-inbound на WebSocket (#6787): рукопожатие WebSocket идёт по HTTP/1.1, и с `h2` в списке сервер согласовывал h2, после чего рукопожатие срывалось (#6782). У MASQUE ALPN определяет слушатели: `h3` — HTTP/3 по UDP, `h2` — HTTP/2 по TCP (п. 6.11).

Возможные значения **uTLS fingerprint** (одинаковы для TLS и REALITY): `chrome`, `firefox`, `safari`, `ios`, `android`, `edge`, `360`, `qq`, `random`, `randomized`, `randomizednoalpn`, `unsafe`. В TLS-форме дополнительно доступен пустой вариант **None** (имитация отпечатка не применяется).

Доступные значения **Cipher Suites** (TLS 1.3 и ECDHE-наборы): `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`.

Переключатели TLS

Переключатель	По умолчанию	Описание
Отклонить неизвестный SNI (`rejectUnknownSni`)	выкл. (`false`)	Если включено, сервер разрывает соединение, когда предъявленный клиентом SNI не совпадает с именем в сертификате. Повышает скрытность (сервер не отвечает на «чужие» запросы), но требует точного совпадения SNI у клиента.
Отключить System Root (`disableSystemRoot`)	выкл. (`false`)	Отключает использование системного хранилища доверенных корневых сертификатов.
Возобновление сессии (`enableSessionResumption`)	выкл. (`false`)	Включает возобновление TLS-сессии (session resumption / session tickets).

Дополнительные параметры TLS (vcn, кривые, лог ключей, ECH Sockopt)

Под основными настройками TLS доступны дополнительные поля.

Поле (подпись)	По умолчанию	Описание
Verify Peer Cert By Name (`settings.verifyPeerCertByName`)	`""`	Имена (через запятую), по которым клиент проверяет сертификат сервера вместо SNI. Это современная замена удалённого из Xray после 2026-06-01 поля `allowInsecure` . Значение только для панели: на сервер в конфиг xray не пишется, но передаётся в ссылки-приглашения и подписки (`vcn=...`), чтобы клиент применил его сам. Плейсхолдер: `example.com` .
Curve Preferences (`curvePreferences`)	`""`	Ограничение и порядок кривых обмена ключами TLS, в порядке предпочтения (например `X25519MLKEM768` , `X25519`). Пусто — берутся значения по умолчанию Xray-core.
Master Key Log (`masterKeyLog`)	`""`	Путь для записи TLS master keys в формате `SSLKEYLOGFILE` (для расшифровки трафика в Wireshark при отладке). Плейсхолдер: `/path/to/sslkeylog.txt` . В продакшене оставлять пустым — файл позволяет расшифровать весь трафик.
ECH Sockopt (`echSockopt`)	выкл.	Переключатель с параметрами сокета для соединения, через которое Xray запрашивает ECH config list. При включении доступны: **Dialer Proxy** (`dialerProxy` — пропустить запрос через указанный outbound по тегу), **Domain Strategy** (`domainStrategy`), **TCP Fast Open** (`tcpFastOpen`), **Multipath TCP** (`tcpMptcp`). Без необходимости оставлять выключенным.

Поля `verifyPeerCertByName` , `curvePreferences` , `masterKeyLog` и `echSockopt` располагаются на верхнем уровне `tlsSettings` и переживают «обрезку» панельных полей при сохранении конфигурации.

Сертификаты

Раздел **SSL-сертификат** (в UI заголовок «SSL-сертификат») задаётся списком: кнопкой **+** добавляется новая запись сертификата, кнопкой **– Удалить** — убирается (кнопка удаления доступна, только если записей больше одной). По умолчанию при включении TLS создаётся одна пустая запись.

Для каждой записи переключатель режима ввода (`useFile`):

- **Путь к сертификату** (значение `useFile = true` , по умолчанию) — указываются пути к файлам на сервере:
  - **Публичный ключ** (`certificateFile`) — путь к файлу сертификата (`.crt` / `.pem`);
  - **Приватный ключ** (`keyFile`) — путь к файлу приватного ключа (`.key`).
- **Содержимое сертификата** (значение `useFile = false`) — содержимое вставляется прямо в поля (многострочные текстовые области):
  - **Публичный ключ** (`certificate`) — PEM-содержимое сертификата;

- **Приватный ключ** (`key`) — PEM-содержимое ключа.

Под полями режима «Путь к сертификату» доступны две кнопки:

- **Установить сертификат панели** — подставляет в поля пути к собственному SSL-сертификату панели. Для inbound на центральной панели берётся её сертификат (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`); для inbound, назначенного на ноду, — сертификат самой ноды (`GET /panel/api/nodes/webCert/{nodeId}`), потому что пути центральной панели на ноде не существуют. Если сертификат не настроен, выводится предупреждение: «*Для панели не настроен сертификат. Сначала установите его в Настройках.*» (сам сертификат панели задаётся в разделе «Настройки → Безопасность»).
- **Очистить** — стирает оба пути.

Дополнительные поля каждой записи сертификата:

Поле	По умолчанию	Описание
OCSP Stapling (`ocspStapling`)	`0` (выкл.)	Интервал обновления OCSP-степлинга в секундах (минимум `0`). Для новых inbound по умолчанию выключен (`0`): это убирает ошибки в логах xray для сертификатов без OCSP-респондера (например, Let's Encrypt, отказавшегося от OCSP). Включайте только для сертификатов, которые поддерживают stapling.
Однократная загрузка (`oneTimeLoading`)	выкл. (`false`)	Если включено, сертификат читается с диска один раз при старте и не перечитывается при изменении файла.
Опция использования (`usage`)	`encipherment`	Назначение сертификата. Допустимо: `encipherment` (шифрование — обычный серверный сертификат), `verify` (проверка), `issue` (выпуск — сервер сам подписывает/выпускает сертификаты).
Build Chain (`buildChain`)	выкл. (`false`)	Показывается **только** при `usage = issue` . Достраивать цепочку сертификатов.

> Самоподписанный сертификат как отдельной кнопки в редакторе inbound нет: панель не генерирует самоподписанный сертификат на лету для inbound. Сертификат либо указывается путём/содержимым, либо подтягивается из настроек панели кнопкой «Установить сертификат панели». Выпуск/получение SSL-сертификата самой панели (включая загрузку файлов и привязку к домену) выполняется в разделе **Настройки** → **Безопасность**; ACME/Let's Encrypt-эндпоинтов для inbound здесь нет.

Проверка сертификатов при сохранении. Начиная с 3.8.0 панель не сохраняет TLS-inbound с неполной записью сертификата. Раньше такая запись доходила до Xray-core, сборка конфигурации на ней падала, и вместе с ней не поднимались все остальные inbound. Правила:

- в каждой записи должен быть сертификат — путь к файлу или содержимое, смотря по выбранному режиму ввода: «*Перед сохранением импортируйте сертификат TLS или укажите путь к его файлу*»;
- в каждой записи, кроме `usage = verify` , нужен и приватный ключ: «*Перед сохранением импортируйте закрытый ключ TLS или укажите путь к его файлу*»;
- хотя бы одна запись должна быть серверной (`encipherment` или `issue`) и с ключом — одних `verify` -записей недостаточно: «*Для TLS требуется хотя бы один сертификат сервера с закрытым ключом (encipherment или issue)*».

Ошибка выводится уведомлением «Сертификат TLS {номер}: {причина}», а редактор сам переключается на вкладку **«Безопасность»**. Проверяется только наличие значений: существуют ли файлы по указанным путям, панель не проверяет. В inbound сохраняется только пара полей выбранного режима — пути для «Путь к сертификату» или PEM-содержимое для «Содержимое сертификата»; значения другого режима отбрасываются. Та же проверка выполняется на сервере, в том числе для запросов API (ошибки вида `TLS certificate N is missing…`). Inbound, уже сохранённый с неполной записью, по-прежнему можно редактировать: отклоняется только сохранение, которое делает некорректным ранее корректный TLS-блок.

ECH и закрепление сертификата (расширенные поля TLS)

Поле	По умолчанию	Описание
ECH key (`echServerKeys`)	`""`	Серверные ключи Encrypted Client Hello. Только для прямых подключений: если inbound находится за CDN, оставьте поле пустым (плейсхолдер: «Только для прямых подключений — оставьте пустым, если inbound находится за CDN.»).
Список конфигураций ECH (`settings.echConfigList`)	`""`	Клиентская часть ECH, попадает в ссылки и подписки. Принимает либо локальный ECHConfigList (его заполняет кнопка **Получить новый ECH-сертификат**), либо динамическую форму `домен+URL DoH` (см. «Динамический ECH за CDN»). Плейсхолдер: `cloudflare-ech.com+https://dns.alidns.com/dns-query`.
SHA-256 сертификата пира (`settings.pinnedPeerCertSha256`)	`[]`	SHA-256-хеши сертификата пира (hex-строки, через запятую). Дословная подсказка: «*SHA-256-хеши сертификата пира в виде шестнадцатеричной строки (напр. e8e2d3…), через запятую. Только для панели — не записывается в конфиг xray сервера, но включается в ссылки-приглашения, чтобы клиенты могли закрепить сертификат.*»

Кнопки: Рядом с полем **SHA-256 сертификата пира** есть две кнопки автозаполнения:

- **Fill from this inbound's certificate** (иконка щита) — подставляет SHA-256-хеш сертификата самого этого inbound (берётся локально через эндпоинт `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (иконка загрузки) — получает хеш живого сертификата сервера, выполнив TLS-подключение по указанному SNI (на сервере вызывается `getRemoteCertHash`). Поле **SNI** (`serverName`) должно быть заполнено — иначе выводится подсказка «*Set the SNI (serverName) first to ping the remote certificate.*»

Полученные хеши добавляются в поле (через запятую) и попадают в ссылки-приглашения, чтобы клиент мог закрепить сертификат.

- **Получить новый ECH-сертификат** — запрашивает у сервера новую ECH-пару под текущий SNI (`POST /panel/api/server/getNewEchCert` , на сервере выполняется `xray tls ech --serverName <SNI>`); заполняет поля **ECH key** и **ECH config**.
- **Очистить** — обнуляет оба ECH-поля.

Динамический ECH за CDN

Локальная пара ключей, созданная кнопкой **Получить новый ECH-сертификат**, действует только при прямом подключении клиентов к этому серверу. Если inbound проксируется через Cloudflare CDN (VLESS + xhttp/WebSocket за оранжевым облаком), внешний TLS завершает Cloudflare со своими ключами ECH и регулярно их ротирует, поэтому локальный ключ всегда проваливает рукопожатие. Под полями ECH панель показывает об этом сообщение «Локально созданные ключи ECH работают только при прямом подключении». Для таких случаев **Список конфигураций ECH** принимает динамическую форму Xray `домен+URL DoH` (разделитель — первый `+`): клиент при каждом подключении запрашивает у DoH-резолвера HTTPS-запись целевого домена и всегда видит актуальные ключи, включая ротированные ключи Cloudflare (#6869).

Вручную значение набирать не нужно: под полем есть помощник **«Динамический ECH через Cloudflare (работает за оранжевым облаком)»** (есть и в форме inbound'а, и на вкладке **Security** хоста, см. 10.4).

- **Готовая комбинация** — готовые пары «домен + резолвер» (рекомендуемая помечена «рекомендуется»; по умолчанию `cloudflare-ech.com` и `https://dns.alidns.com/dns-query`) либо вариант **Своя комбинация…**, где отдельно выбираются домен и резолвер.
- **Целевой домен ECH** — домен, в HTTPS-записи которого есть `ech=` (пресеты вроде Cloudflare ECH; можно ввести свой). Вариант **Автоматически — разрешить запись ECH для SNI узла** подставляет SNI из формы.
- **DoH-резолвер** — публичные резолверы в группах «Материковый Китай (доступен из-за GFW)», «Весь мир», «Европа» и «Обычный DNS (udp://)» либо **Свой URL…**. Допустимы только схемы `https://`, `h2c://` и `udp://` (регистр схемы нормализуется); для остальных помощник сообщает: «Xray разрешает конфигурацию ECH только через https://, h2c:// или udp://. Этот адрес никогда не ответит.». Резолверам материкового Китая нужен целевой домен, в HTTPS-записи которого есть ECH — помощник напоминает проверить пару перед сохранением.
- **Применить к списку конфигураций ECH** — записывает собранное значение в поле. Если у inbound уже есть локально созданная пара ключей ECH, помощник спрашивает подтверждение («У этого inbound есть локально созданная пара ключей ECH»): кнопка **Заменить и удалить локальный ключ** подставляет динамическое значение и очищает **ECH key**. Если в поле лежит значение не в динамической форме, помощник предупреждает, что при заполнении заменит введённое вручную.

7.4. Режим REALITY

Поля блока `realitySettings`. REALITY не использует SSL-сертификат: вместо него — заимствованное TLS-рукопожатие внешнего домена и пара ключей X25519.

Параметры маскировки

Поле (подпись)	Значение по умолчанию	Описание
Показать (`show`)	выкл. (`false`)	Отладочный вывод REALITY в логи Xray. Обычно оставляют выключенным.
Xver (`xver`)	`0`	Версия PROXY-протокола, передаваемая на бэкенд (`0` — выключено). Минимум `0`.

Поле (подпись)	Значение по умолчанию	Описание
uTLS (`settings.fingerprint`)	`chrome`	Имитируемый TLS-отпечаток (тот же список, что в TLS, но без пустого варианта None).
Цель (`target`)	`""` (с 3.4.2 при включении REALITY остаётся пустым)	**Обязательное поле.** Реальный домен, чьё TLS-рукопожатие заимствует REALITY. Дословная подсказка: «*Обязательно. Должно содержать порт (например, example.com:443). Без порта Xray-core не запускается.*» Валидация в панели проверяет наличие и корректность порта; иначе выдаются ошибки «Цель REALITY обязательна» / «Цель REALITY должна содержать порт…» / «У цели REALITY указан недопустимый порт». Рядом с полем — кнопки **«Сканировать»** (проверить текущую цель «вживую») и **«Найти цели»** (открыть сканер целей REALITY); см. ниже.
SNI (`serverNames`)	`[]` (подставляется вместе с целью)	Список допустимых SNI (множественный ввод тегами). Должен соответствовать домену из **Цель**. При успешном сканировании цели SNI заполняется по её сертификату.
Макс. разница во времени (мс) (`maxTimediff`)	`0`	Максимально допустимое расхождение часов клиента и сервера в миллисекундах (`0` — без ограничения). Минимум `0`.
Мин. версия клиента (`minClientVer`)	`""`	Минимальная версия Xray-клиента: до трёх чисел через точку, каждое 0–255 (плейсхолдер — `x.y.z`); формат проверяется при сохранении. Пусто — нижней границы нет: ядро принимает клиента любой версии, включая сторонние ядра вроде Mihomo и sing-box (см. примечание ниже). Заданная версия отсекает клиентов, сообщающих более старую.
Макс. версия клиента (`maxClientVer`)	`""`	Максимальная версия Xray-клиента в том же формате. Пусто — верхнего предела нет. Значение ниже **Мин. версия клиента** не сохраняется («Макс. версия клиента не должна быть ниже минимальной версии клиента»): такой диапазон отклонял бы всех клиентов.
Short IDs (`shortIds`)	`[]` (генерируются при включении)	Список коротких идентификаторов (hex), различающих клиентов. Множественный ввод тегами; кнопка-обновление генерирует случайный набор.
SpiderX (`settings.spiderX`)	`/`	Путь «паука» (клиентская часть REALITY), используемый при имитации обращения к внешнему сайту. Попадает в ссылку-приглашение.

С версии 3.4.2 **Цель** (`target`) и **SNI** (`serverNames`) при включении REALITY больше **не** заполняются автоматически — оба поля остаются пустыми, а цель подбирается живым сканером (см. ниже). Выбирайте «жирный», стабильный сторонний HTTPS-сайт с поддержкой TLS 1.3 и HTTP/2, не находящийся за вашим же сервером.

> **Пустая «Мин. версия клиента» и версия ядра.** До 3.8.0 пустое поле не означало «без ограничения»: Xray-core подставлял встроенный минимум `26.3.27`, и сторонние ядра (Mihomo, sing-box) не проходили проверку REALITY даже с правильными ключами. В Xray-core v26.9.8 и новее (3.9.0 поставляется с v26.9.30) встроенного минимума нет; явно сохранённые значения по-прежнему действуют. На ядрах v26.7.11–v26.9.7 пустое поле означает встроенный минимум `26.3.27` — чтобы

принять такие клиенты, задайте `1.0.0`. Подсказка поля теперь учитывает версию ядра (#6568). Снижая или снимая ограничение, учитывайте, что вместе со сторонними клиентами пропускаются и устаревшие TLS-отпечатки.

Mihomo и ML-KEM. Xray-core v26.9.8+ принимает REALITY-рукопожатие только с ключом ML-KEM (`X25519MLKEM768`). Поэтому Clash/Mihomo-подписка для REALITY-узлов (включая внешние ссылки клиента) добавляет в `reality-opts` флаг `support-x25519mlkem768: true` и подставляет `client-fingerprint: chrome`, если отпечаток не задан; явно выбранный отпечаток сохраняется и должен поддерживать ML-KEM. Ссылка `vless://` несёт эквивалентную подсказку `support-x25519mlkem768=true`: клиенты, поддерживающие это расширение URI (в том числе актуальные сборки Mihomo), применяют её при импорте. Если серверу ML-KEM не нужен (Xray-core ниже 26.9.8) или сеть отбрасывает крупный ClientHello с ML-KEM, включите у хоста переключатель **Mihomo X25519**: Clash-прокси получит `support-x25519mlkem768: false`, а ссылка будет без подсказки. Подробнее о хостах и Clash-подписке — 10.4.

Поиск цели REALITY (живой сканер)

С версии 3.4.2 прежняя кнопка «случайная цель» (со статичным встроенным списком) заменена двумя действиями рядом с полем **Цель**:

- **«Сканировать»** — «вживую» проверяет текущую цель: устанавливает TLS-соединение и, если цель пригодна, подставляет SNI из её сертификата. Тосты: «Цель подходит — поля target и SNI заполнены.» / «Цель доступна, но не подходит для REALITY.» / «Не удалось просканировать цель REALITY.». Под полем появляется карточка результата — в том числе строка **«Цепочка сертификатов»** с размером цепочки в байтах (важно для ML-DSA-65, см. ниже).
- **«Найти цели»** — открывает окно **«Сканер целей REALITY»**: можно указать домен, **IP или диапазон CIDR** (тогда панель находит цели по сертификатам живых хостов) либо оставить поле пустым — тогда проверяется список из настройки **«Кандидаты сканирования Reality»** (`realityScanCandidates`: адреса `host:port` или CIDR через запятую; по умолчанию — прежний встроенный набор популярных доменов; задаётся в общих настройках панели, см. раздел 13). Результаты ранжируются; столбцы — «Цель», «Статус» («Подходит»/«Не подходит»), `TLS`, `ALPN`, «Обмен ключами», «Сертификат», «Цепочка сертификатов», «Задержка». Кнопкой **«Выбрать»** выбранная строка подставляется в поля inbound.

Цель считается **пригодной**, если сервер согласует **TLS 1.3**, **HTTP/2 (ALPN h2)**, обмен ключами **X25519/X25519MLKEM768** и предъявляет **доверенный** (не wildcard) сертификат. Сканирование выполняется на стороне сервера панели (`POST /panel/api/server/scanRealityTarget` и `…/scanRealityTargets`). Пустой параметр `targets` у `scanRealityTargets` означает список из `realityScanCandidates`; в ответе по каждой цели есть поле `certChainBytes` — суммарный DER-размер предъявленной цепочки сертификатов.

Пример: блок `streamSettings` для REALITY на сети `tcp` (VLESS). Сертификат не нужен — вместо него заимствованный домен и пара ключей X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Здесь поле **Цель** (`target`) панели соответствует `dest` в готовом конфиге Xray. Если REALITY-inbound был создан с destination в ключе `dest` (старыми версиями панели, через API или внешними инструментами), панель при разборе нормализует `dest` → `target` , когда `target` пуст, — поэтому такой inbound корректно загружается, поле **Цель** не оказывается пустым, и пересохранение не ломает рабочий REALITY.

Ключи REALITY (X25519)

Поле	По умолчанию	Описание
Публичный ключ (`settings.publicKey`)	`""`	Публичный ключ X25519 (его клиент кладёт в свою конфигурацию/ссылку).
Приватный ключ (`privateKey`)	`""`	Приватный ключ X25519 (хранится только на сервере).

Кнопки под ключами:

- **Получить новый сертификат** — запрашивает у сервера новую пару ключей (`GET /panel/api/server/getNewX25519Cert` ; на сервере выполняется `xray x25519`), заполняет **Приватный ключ** и **Публичный ключ**. Эта же пара автоматически генерируется при первом включении режима REALITY.

Пример: получить пару ключей X25519 через API (вне формы, например для скрипта). Запрос возвращает приватный и публичный ключи:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Ответ:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

`cookie.txt` — файл cookie сессии, полученный после входа через `POST /login` .

- **Очистить** — обнуляет оба ключа.

Пост-квантовая подпись ML-DSA-65 (mldsa65)

Дополнительный (необязательный) слой пост-квантовой аутентификации REALITY:

Поле	По умолчанию	Описание
mldsa65 Seed (`mldsa65Seed`)	`""`	Серверный seed ключа ML-DSA-65.
mldsa65 Verify (`settings.mldsa65Verify`)	`""`	Проверочное значение (клиентская часть, попадает в ссылку).

Кнопки:

- **Получить новый Seed** — запрашивает новую пару (`GET /panel/api/server/getNewmldsa65` ; на сервере выполняется `xray mldsa65`), заполняет **mldsa65 Seed** и **mldsa65 Verify**.
- **Очистить** — обнуляет оба поля.

Размер цепочки сертификатов цели. ML-DSA-65 в xray-core требует, чтобы цепочка сертификатов цели занимала не меньше 3500 байт; с более короткой цепочкой клиентские соединения без явной ошибки уходят в fallback и не работают. Если на inbound заполнено **mldsa65 Seed** или **mldsa65 Verify**, а сканер показывает цепочку меньше 3500 байт, результат **«Сканировать»** выводится как предупреждение, а в окне **«Сканер целей REALITY»** размер помечается предупреждающим тегом с советом выбрать другую цель или отключить ML-DSA-65.

Ограничение скорости fallback и лог ключей REALITY

В настройках REALITY доступно ограничение скорости fallback-трафика — оно не даёт активным зондам использовать сервер как бесплатный канал к заимствованному домену. Настройка задаётся отдельно для двух направлений — **Limit Fallback Upload** и **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), каждое с одинаковым набором полей:

Поле (подпись)	По умолчанию	Описание
After Bytes (`afterBytes`)	`0`	Сколько байт пропустить на полной скорости перед началом ограничения. `0` — ограничивать с первого байта.
Bytes Per Sec (`bytesPerSec`)	`0`	Потолок скорости fallback-трафика в байтах в секунду после порога. `0` — без ограничения (отключает это направление).
Burst Bytes Per Sec (`burstBytesPerSec`)	`0`	Запас для кратковременных всплесков сверх постоянной скорости (размер token-bucket). Если меньше **Bytes Per Sec**, поднимается до его значения.

Там же добавлено поле **Master Key Log** (`masterKeyLog`) — путь для записи TLS master keys в формате `SSLKEYLOGFILE` для отладки в Wireshark; в продакшене оставлять пустым.

Проверка таргета в локальной сети (3.7.0)

Проверка REALITY-таргета раньше упиралась в защиту панели от SSRF: адрес в приватной сети просто не проверялся. Теперь панель говорит прямо — «Target is reachable but sits on a private/local network» — и, если подтвердить («"{target}" resolves to a private or loopback address… Continue?»), выполняет проверку, **обходя защиту только для этого одного запроса**. В результат добавлены **срок действия сертификата** и **использованный SNI**, а к отчёту приписывается напоминание, что адрес проверялся по локальной сети и снаружи недоступен.

7.5. Практические рекомендации по настройке

1. **VLESS + Reality (рекомендуется):** создайте VLESS-inbound на сети `tcp`, на вкладке «Безопасность» выберите **Reality** — панель сама сгенерирует `shortIds` и ключи X25519, а цель и SNI подберите кнопкой **«Найти цели»** (или впишите цель и нажмите **«Сканировать»**). При необходимости нажмите «Получить новый сертификат» для своей пары ключей. Для клиентов VLESS включите **Flow** = `xtls-rprx-vision` (XTLS Vision) — это даст максимальную производительность и скрытность.

Пример: итоговая клиентская ссылка VLESS + Reality + Vision. Так выглядит ссылка-приглашение, которую панель выдаёт для такого inbound (значения ключей/ID — иллюстративные):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Здесь `pbk` — публичный ключ X25519, `sni` — заимствованный домен из **Цель**, `sid` — один из **Short IDs**, `flow=xtls-rprx-vision` — включённый XTLS Vision. 2. **TLS со своим доменом:** выберите **TLS**, заполните **SNI** именем домена, добавьте сертификат (путём к файлам или содержимым), либо нажмите «Установить сертификат панели», если домен и сертификат уже настроены в «Настройки → Безопасность». Оставьте **Мин/Макс версия** = `1.2` – `1.3` и **uTLS** = `chrome` для маскировки под обычный браузер. 3. Не оставляйте режим **Нет** для inbound, открытых наружу, — используйте его только для локальных fallback-целей (`127.0.0.1`) или когда TLS обеспечивает внешний прокси. 4. Совет из интерфейса: для большинства расширенных полей действует подсказка «*Рекомендуется оставить настройки по умолчанию*» — меняйте их только при понимании последствий.

---

8. Клиенты

Клиент — это учётная запись пользователя VPN: набор учётных данных (UUID или пароль), привязанный к одному или нескольким inbound, с собственной квотой трафика, сроком действия и лимитом одновременных подключений. В этом форке клиент является самостоятельной сущностью (таблица `clients`): один и тот же клиент может быть привязан к нескольким inbound сразу, сохраняя общий UUID/пароль и общий счётчик трафика. Раздел **Клиенты** показывает все учётные записи панели вне зависимости от inbound, с поиском, фильтрами, сортировкой и массовыми операциями.

8.1. Поля клиента

Ниже разобрано каждое поле редактора **Добавить клиента** / **Изменить клиента**.

Форма клиента разбита на две вкладки: **Основные** (email, привязка к inbound, лимиты, срок, группа, комментарий, обратный тег, переопределение правила хоста) и **Учётные данные** (UUID/пароль/auth, Flow, VMess Security). В подписях полей квота указана как **Лимит трафика (ГБ)**, а сроки — как **Длительность (дней)** и **Автопродление (дней)**; у полей **Лимит трафика (ГБ)** и **Лимит IP** есть подсказки, поясняющие, что `0` означает «без ограничений». При редактировании уже существующего клиента кнопка генерации случайного email скрыта, а кнопка журнала IP вынесена прямо к полю **Лимит IP** и показывает число зафиксированных адресов.

Поле	Ключ JSON	По умолчанию	Описание
Email	`email`	— (обязательно)	Уникальный идентификатор клиента
UUID	`id`	генерируется	Идентификатор для VMess/VLESS/TUIC
Пароль	`password`	генерируется	Пароль для Trojan/Shadowsocks/TUIC/MASQUE
Авторизация	`auth`	генерируется	Пароль для Hysteria
Flow	`flow`	пусто	Flow control (XTLS), только VLESS
VMess Security	`security`	`auto`	Метод шифрования VMess
Лимит IP	`limitIp`	`0` (без лимита)	Максимум одновременных IP
Всего отправлено/получено (ГБ)	`totalGB`	`0` (без лимита)	Квота трафика
Срок действия	`expiryTime`	`0` (бессрочно)	Дата истечения
Автопродление	`reset`	`0` (выкл.)	Период сброса трафика, дни
ID пользователя Telegram	`tgId`	`0` (нет)	Числовой Telegram ID
ID подписки	`subId`	генерируется	Идентификатор подписки
Группа	`group`	пусто	Логическая метка группировки
Комментарий	`comment`	пусто	Произвольная заметка
Включён	`enable`	`true`	Активна ли учётная запись

Поле	Ключ JSON	По умолчанию	Описание
Продление в день месяца	`resetDay`	`0` (выкл.)	День календарного месяца для продления вместо интервала (3.7.0)
Максимум продлений	`resetMax`	`0` (без лимита)	Сколько раз может сработать автопродление (3.7.0)
Цикл сброса трафика	`trafficReset`	`never`	Свой период обнуления счётчика у клиента (3.7.0)
День сброса трафика	`trafficResetDay`	`1`	День месяца для месячного цикла (3.7.0)
Лимит устройств	`limitHwid`	`0` (без лимита)	Максимум зарегистрированных устройств подписки (3.7.0)
Проброшенные порты	`forwardedPorts`	пусто	Порты/диапазоны DNAT на клиента AmneziaWG (3.7.0)
Keepalive (секунды)	`keepAlive`	`25` в форме нового клиента	Период PersistentKeepalive у клиента WireGuard/AmneziaWG, `0` — выкл.; у AmneziaWG допустим и диапазон вида `15–25`
День недели продления	`resetWeekday`	`0` (выкл.)	День недели для недельного календарного продления: 1=Пн … 7=Вс (3.9.0)
Переопределение правила хоста	`clientHostRuleIds`	пусто	id управляемых хостов, по которым строится подписка этого клиента; пусто — хосты его inbound'ов (см. Переопределение правила хоста)

Продление, циклы сброса и лимит устройств (3.7.0)

Продление в день месяца. Поле **«Renew on day»** (`resetDay` , 1–31) переводит клиента с режима «каждые N дней» на календарный: продление срабатывает в этот день каждого месяца, в полночь по часовому поясу панели. Если в месяце нет такого числа (31-е в апреле), продление происходит в последний день месяца. `0` сохраняет прежний режим интервала (`reset`). Срок, стоящий ровно на 23:59:59 накануне дня продления (запись «до конца месяца включительно»), панель переносит на полночь этого дня без траты продления — первое засчитанное продление получается полным месяцем, а лимит **«Max renewals»** не расходуется на выравнивание.

Режим автопродления (3.9.0). Вместо раздельных полей теперь единый селектор **«Автопродление»** с режимами: **«Отключено»**, **«Через фиксированный интервал (дни)»** (поле `reset` , дни), **«Календарное — по дням недели»** (новое) и **«Календарное — по дню месяца»** (поле `resetDay` , 1–31). Недельный режим задаётся полем **«День недели продления»** — поле клиента **`resetWeekday`** (1 = понедельник … 7 = воскресенье). Как и месячное, недельное продление срабатывает в полночь по часовому поясу панели и **обнуляет трафик** клиента. Рядом показывается **предпросмотр графика продлений**: дата ближайшего продления, срок действия до неё, срок после продления и число продлений, которое будет израсходовано; если срок ещё не задан, предпросмотр предлагает кнопку установки первого рубежа. Режим определяется по заполненному полю (`resetDay` → месячный, иначе `resetWeekday` → недельный, иначе `reset` → интервал). **Недельный режим требует, чтобы все узлы были обновлены:** старые узлы игнорируют `resetWeekday` (#6524).

Раскладка формы и предпросмотр (3.10.0). В форме одного клиента селектор **«Автопродление»**, поле выбранного режима (интервал в днях, **«День недели продления»** или **«Продлевать числа»**) и **«Лимит продлений»** стоят в один ряд (на телефоне — друг под другом), а пояснение и предпросмотр занимают всю ширину под ними; в форме массового добавления поля по-прежнему идут столбцом. Пояснение зависит от режима: для интервала — «После истечения срока добавляет указанное число 24-часовых дней к предыдущему сроку и сбрасывает трафик клиента…», для недельного — про полночь по часовому поясу панели и сброс трафика, а замечание о 1-м числе и конце месяца показывается только в месячном режиме. Из подсказок полей убраны упоминания «0 = отключено» — автопродление выключается режимом «Отключено» (#6672). Предпросмотр озаглавлен «Предпросмотр продления (часовой пояс панели: <пояс>)» — при часовом поясе `Local` вместо имени выводится «Местное время сервера» — и сопровождается подписью «Даты указаны в часовом поясе панели. Предпросмотр не продлевает клиента.». Даты выводятся в читаемом виде `ГГГГ-ММ-ДД ЧЧ:ММ:СС UTC±ЧЧ:ММ` : настенное время сервера с явным смещением, которое для каждой даты отдельно учитывает переход на летнее время. Счётчик «Будет использовано продлений: N» — это оценка того, сколько продлений из лимита потратится на ближайший рубеж, а не число уже выполненных продлений. Изменилось только отображение: расчёт продлений и сохраняемые значения прежние (#6662).

Потолок автопродлений. Поле **«Max renewals»** (`resetMax`) ограничивает, сколько раз автопродление может сработать, прежде чем клиента оставят истечь; `0` — без ограничения. Если панель догоняет несколько пропущенных периодов сразу, на каждый период тратится одно продление. Рядом отображается счётчик **«Renewals used»**.

Свой цикл сброса трафика. До 3.7.0 периодический сброс счётчика задавался только на уровне inbound'а; теперь у клиента есть собственные `trafficReset` (`never` / `hourly` / `daily` / `weekly` / `monthly`) и `trafficResetDay` — день месяца для месячного цикла.

Лимит устройств (HWID). Поле **«Лимит HWID»** (`limitHwid`) — максимум устройств, с которых принимаются запросы подписки; `0` — без ограничения. Рядом список **«HWID Devices»**: устройство, ОС и её версия, модель, время первого и последнего обращения и **«Отпечаток HWID»** — первые 12 символов SHA-256-хеша идентификатора устройства (поле `fingerprint` в API), по которым устройства легко различать; сам HWID и полный хеш панель не показывает. С 3.9.0 устройства фиксируются в списке **«HWID Devices»** и у клиентов **без лимита** (лимит 0): учёт ведётся, но подписку не ограничивает. Устройство удаляется по одному (`DELETE /panel/api/clients/hwids/:email/:id`) или списком целиком (`DELETE /panel/api/clients/hwids/:email`) — удалённое просто зарегистрируется заново при следующей выгрузке подписки, так что это способ «отвязать телефон», а не наказание.

Внешние ссылки клиента: что добавилось в 3.7.0

Сама вкладка **«Links»** (сторонние share-ссылки и внешние подписки в выдаче клиента) появилась раньше и разобрана в разделе 10.1. В 3.7.0 у каждой строки появилось управление: **переключатель** `enable` — временно выключить строку, не удаляя её; **срок действия** — после него строка выпадает из выдачи; **префикс имени**; **время последней выгрузки** и **текст последней ошибки**, если внешний источник не ответил. Значение по-прежнему проверяется при сохранении: share-ссылка должна разбираться, внешняя подписка — быть `http(s)` . Если у строки свой срок не задан, для неё действует срок самого клиента — он и показывается в поле даты, а свой срок строки нельзя поставить позже срока клиента. Строки выключенного или истёкшего клиента в выдачу подписки не попадают.

Email (идентификатор)

Поле **Email** — главный и обязательный идентификатор клиента. Несмотря на название, это не обязательно почтовый адрес: подойдёт любая текстовая метка (имя пользователя, номер). Значение должно быть **уникальным** в пределах панели; попытка создать второго клиента с занятым email отклоняется (`email already in use`), за исключением случая, когда совпадает и `subId` (это трактуется как привязка того же клиента).

Email **нельзя оставлять пустым** (`client email is required`) и он **не может содержать пробелы, символы `/`, `\` и управляющие символы** («Email не может содержать пробелы, '/', '' или управляющие символы»).

Двоеточие и MASQUE (3.10.0). Клиент MASQUE-inbound входит по Basic-auth, где имя пользователя — его email, поэтому email с символом `:` ядро Xray не принимает и отвергает **весь** inbound, а вместе с ним и запуск Xray. Панель отклоняет такой email при любой записи клиента в MASQUE-inbound — создании, привязке, массовых операциях, импорте и правке inbound'а — с ошибкой `MASQUE: client email "…" cannot contain ":"`. Для остальных протоколов `:` в email по-прежнему допустим. О самом протоколе — 5.15. Email участвует в учёте трафика, в журнале IP, в онлайн-списке и в именах операций — менять его задним числом не рекомендуется.

UUID / Пароль / Авторизация (учётные данные)

Конкретное поле учётных данных зависит от протокола inbound, к которому привязывается клиент. Значения подставляются автоматически, если оставить поле пустым:

- **UUID** (поле `id`) — для протоколов **VMess** и **VLESS**. Если не задан, генерируется случайный UUID v4.
- **Пароль** (поле `password`) — для **Trojan** и **Shadowsocks**. Для Trojan по умолчанию генерируется UUID без дефисов. Для Shadowsocks генерируется ключ нужной длины в Base64 в зависимости от метода шифрования inbound: 16 байт для `2022-blake3-aes-128-gcm`, 32 байта для `2022-blake3-aes-256-gcm` и `2022-blake3-chacha20-poly1305`; для прочих методов — UUID без дефисов. Если введённый вручную ключ не подходит под метод 2022-blake3, он будет заменён сгенерированным.
- **Авторизация** (поле `auth`) — пароль для **Hysteria**. По умолчанию UUID без дефисов.
- **Пароль** (поле `password`) вместе с email — для **MASQUE**: клиент входит по Basic-auth, где имя пользователя — email, а пароль — значение `password`; пустой пароль заполняется UUID без дефисов, как у Trojan. Ограничение на символ `:` в email — см. «Email» выше.
- **UUID** и **Пароль** вместе — для **TUIC** v5: клиент аутентифицируется парой UUID + пароль. Пустые поля заполняются автоматически (UUID v4 и UUID без дефисов). Подробнее о протоколе — 5.13.

Поскольку один клиент может быть привязан к inbound разных протоколов, у записи клиента могут одновременно присутствовать и UUID, и пароль, и auth — для каждого протокола используется своё поле.

Пример: как учётные данные клиента выглядят в `settings` разных inbound. Один и тот же клиент в VLESS-inbound идентифицируется по `id`, в Trojan — по `password`, в Shadowsocks — по `password` (ключ Base64):

```
// фрагмент settings.clients у VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-
vision" }

// тот же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// тот же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPyOaA3f7CO0az53eaQ8eqMfRDjmBlOh7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (поле `flow`) — управление потоком XTLS. Применимо **только к VLESS** и только когда inbound сконфигурирован для XTLS Vision: VLESS поверх транспорта **TCP** с security `tls` или `reality`. Допустимое значение — `xtls-rprx-vision` (а также исторический `xtls-rprx-vision-udp443`); пустое значение означает отсутствие flow.

Если inbound не поддерживает XTLS-flow, заданный flow **молча обнуляется** при сохранении клиента: для одного и того же клиента, привязанного к нескольким inbound, flow применяется только там, где это допустимо. Менять стоит только если вы целенаправленно используете VLESS-Vision.

VMess Security

VMess Security (поле `security`) — метод шифрования полезной нагрузки для VMess. Значение по умолчанию — `auto` (Xray сам выбирает шифр). Допустимые значения — стандартные для VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Для остальных протоколов поле не используется.

Параметры туннеля WireGuard / AmneziaWG

Если клиент привязан к WireGuard- или AmneziaWG-inbound, на вкладке **«Учетные данные»** появляются поля пира: приватный и публичный ключ, **«Общий ключ WireGuard»** / **«Общий ключ AmneziaWG»** (PresharedKey), адрес в туннеле и **«Keepalive (секунды)»**.

- **Общий ключ** — необязательный PresharedKey. Кнопка обновления рядом с полем генерирует случайные 32 байта в Base64, как `wg genpsk`.
- **Keepalive (секунды)** (`keepAlive`, 0–65535) — как часто клиент шлёт keepalive-пакет (`PersistentKeepalive` в конфиге клиента). В форме нового клиента стоит `25`: так NAT-маппинг остаётся открытым, а простаивающий пир сам восстанавливается после любого разрыва, включая рестарт панели. `0` отключает keepalive — молчащий клиент тогда остаётся отключённым, пока сам не отправит трафик. Существующие клиенты сохраняют своё значение (у созданных до 3.8.0 это обычно `0`), а запрос API без поля `keepAlive` сохранённое значение не меняет.

  У клиента **AmneziaWG** вместо числа можно задать упорядоченный диапазон `мин-макс`, например `15-25`: случайный разброс интервала труднее опознать по периодичности пакетов. Подсказка поля: *«Укажите секунды числом или диапазоном (например, 15-25). Значение должно быть от 0 до 65535; 0 отключает keepalive.»*; для неверного ввода — *«Введите целое число или диапазон от 0 до 65535.»* Родной `.conf` и профиль `vpn://` несут диапазон как есть, а вывод Clash/Mihomo, где keepalive

только числовой, получает нижнюю границу (`0–25` поэтому значит «выключено»). У обычного WireGuard допустимо только число. Узлы с диапазонами должны быть версии 3.11.0 или новее: старые читают keepalive как целое.

Клиент, привязанный к нескольким inbound одного туннельного протокола (например, WireGuard на разных нодах), — это несколько независимых пиров со своими ключами и адресами, а форма показывает один набор. Поэтому при сохранении такого клиента ключи, общий ключ и адрес на каждом inbound остаются прежними и не затираются значениями из формы; пира отдельного inbound правят запросом `POST /panel/api/clients/update/:email?inboundIds=<id>`. В окнах **«Информация о клиенте»** и **«QR-код»** конфиг выводится для каждого такого inbound — с подписью inbound и отдельным именем файла `<email>–<inbound>.conf`.

Лимит IP (одновременные подключения)

Лимит IP (поле `limitIp`) — максимальное число **разных IP-адресов**, с которых клиент может быть подключён одновременно. Значение по умолчанию — `0`, что означает **отсутствие ограничения**. При положительном значении панель отслеживает активные IP клиента и, при превышении лимита, отключает учётную запись фоновым заданием. (Начиная с **3.3.1** подсчёт IP идёт через online-stats API ядра Xray и **не требует** журнала доступа; на старых версиях ядра панель возвращается к чтению журнала доступа, который тогда должен быть включён.) Используйте, чтобы запретить раздачу одной подписки на много устройств: например, `2` — разрешить два устройства.

Лимит IP применяется средствами **Fail2ban**, поэтому поле **Лимит IP** активно только при установленном и работоспособном Fail2ban (панель проверяет его статус через `GET /panel/api/server/fail2banStatus`). Если Fail2ban не установлен, поле редактора клиента (и формы массового добавления) блокируется, а при наведении показывается подсказка с предложением установить Fail2ban из bash-меню `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); на Windows подсказка сообщает, что Fail2ban там недоступен («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), а если функция отключена на сервере — «The IP limit feature is disabled on this server.». При обновлении панели у клиентов на серверах без Fail2ban сохранённый лимит IP обнуляется разовой миграцией, так как он там всё равно не применялся.

Fail2ban нужен только для блокировки. Если он отключён переменной `XUI_ENABLE_FAIL2BAN=false`, панель по-прежнему записывает IP клиентов (окно 30 минут): **Журнал IP** и счётчик адресов в форме клиента остаются актуальными, а лимит не применяется (#6830).

Белый список для лимита IP (3.7.0). В общих настройках панели появилось поле **«IP limit allowlist»**: адреса и подсети через запятую (IP или CIDR), которые лимит IP **никогда не учитывает и никогда не банит**. Нужно там, где за одним адресом сидит много людей — общий офисный выход, кампус, оператор с CGNAT: без белого списка такой адрес съедал лимит клиента и приводил к блокировке.

Пример значений. `limitIp: 0` — без ограничения; `limitIp: 1` — строго одно устройство одновременно; `limitIp: 3` — до трёх разных IP. При четвёртом активном IP фоновое задание выключит клиента (`enable = false`), пока вы не выполните **Сбросить лимит IP**.

Связанные операции: **Журнал IP** показывает список зафиксированных IP клиента; каждая запись содержит, помимо самого IP, время последнего обращения и метку ноды (`@ имя_ноды`), через которую зафиксировано подключение, — в мультипанельной конфигурации видно, через какой узел подключался

клиент. **Сбросить лимит IP** очищает накопленный журнал IP, чтобы клиент снова смог подключиться, не дожидаясь естественного истечения записей.

Всего отправлено/получено (ГБ) — квота трафика

Всего отправлено/получено (ГБ) (поле `totalGB`) — суммарная квота трафика (отправка + приём). Значение по умолчанию — `0` — означает **безлимит**. По достижении квоты (`up + down >= total`) клиент считается **исчерпанным** (depleted) и отключается. В UI обычно вводится в гигабайтах; в базе хранится в байтах. С 3.9.0 персональная квота поддерживается и для клиентов **TUIC**: встроенный сервер учитывает их расход, и по достижении квоты клиент отключается, как у остальных протоколов.

В списке клиентов столбец **Трафик** показывает цветную полосу использования: объём израсходованного трафика, метку лимита (или знак ∞ при безлимите) и подсказку при наведении с разбивкой на отправлено/получено и остаток. Тот же компактный индикатор отображается в карточках клиентов на телефоне.

Срок действия (Expiry)

Срок действия (поле `expiryTime`) задаёт момент истечения учётной записи. Поле имеет три режима:

- **Бессрочно** — `0`. Клиент никогда не истекает по времени.
- **Конкретная дата** — положительный Unix-timestamp (в миллисекундах). По наступлении (`expiryTime <= сейчас`) клиент считается истёкшим (expired) и отключается. В UI обычно задаётся выбором даты или длительностью в днях (**Длительность**, единица — **Дни**).
- **Старт после первого использования** — **отрицательное** значение, кодирующее длительность. Пока клиент не передал ни одного байта, срок остаётся отрицательным («отложенный старт»). На первом же тике учёта трафика панель конвертирует его в абсолютную дату: `сейчас + |длительность|`. Это позволяет продать, например, «30 дней с момента первого подключения», не зная заранее, когда клиент активируется. Конвертация выполняется один раз на email, чтобы все привязанные inbound получили один и тот же срок.

Пример кодирования срока. Фиксированная дата 1 марта 2026, 00:00 UTC → `expiryTime: 1772323200000` (положительный timestamp в миллисекундах). «30 дней с первого подключения» → `expiryTime: −2592000000` (отрицательное значение, `30 × 24 × 60 × 60 × 1000`); при первом байте трафика панель заменит его на `сейчас + 2592000000`. Бессрочно → `expiryTime: 0`.

Автопродление (период сброса трафика клиента)

Поле **Автопродление** (поле `reset`) — это период автоматического продления/сброса в днях. Подсказка: «Автоматическое продление после окончания. (единица: день)» (до 3.10.0 в ней было ещё «0 = отключено»: в форме поле интервала принимает значения от 1, а выключается автопродление режимом «Отключено» селектора **«Автопродление»**; в API `reset: 0` по-прежнему означает «выкл.»).

- `0` — автопродление **отключено** (значение по умолчанию). По истечении срока клиент просто становится исчерпанным.
- `> 0` — фоновое задание при истечении срока **сбрасывает счётчики трафика в ноль** (`up = down = 0`), **сдвигает срок действия вперёд** на `reset` дней (если нужно — на несколько периодов, пока новый срок не окажется в будущем) и при необходимости снова **включает** клиента. Если клиент состоит в

нескольких inbound, новый срок и включение применяются ко всем этим inbound сразу, а продление засчитывается один раз. Это реализует периодическую подписку (например, помесячную). Автопродление **не применяется к inbound на узлах-нодах** (`node_id IS NOT NULL`).

Важное следствие: клиенты с `reset > 0` **исключаются** из понятия «исчерпанный» в операциях массового удаления — их трафик/срок ожидаемо обнуляются автопродлением, а не делают учётную запись кандидатом на удаление.

ID пользователя Telegram

ID пользователя Telegram (поле `tgId`) — числовой Telegram-идентификатор пользователя для привязки к встроенному Telegram-боту панели (уведомления, самостоятельный просмотр статистики). Подсказка: «Числовой ID пользователя Telegram (0 = нет)». Значение по умолчанию `0` — привязка отсутствует. По этому полю доступна фильтрация (**Есть** / **Нет**).

ID подписки (subId)

ID подписки (поле `subId`) — идентификатор, под которым клиент включается в **подписку** (subscription). Все клиенты с одинаковым `subId` отдаются по одной ссылке подписки. Если поле оставлено пустым при создании, панель **автоматически генерирует случайный** `subId` (UUID). Значение должно быть **уникальным** среди клиентов с другим email (`subId already in use`) и подчиняется тем же ограничениям символов, что и email («ID подписки не может содержать пробелы, '/', '' или управляющие символы»).

Без `subId` ссылка подписки для клиента недоступна («У этого клиента нет subId, ссылка для общего доступа недоступна.»).

Вкладка Links (внешние ссылки и подписки)

Кроме вкладок **Основные** и **Учётные данные**, в редакторе клиента есть третья вкладка **Links** (подсказка: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). На ней кнопкой **Add External Link** добавляют сторонние share-ссылки (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), а кнопкой **Add External Subscription** — URL удалённых подписок (например, `https://provider.example/sub/…`).

Всё перечисленное подмешивается в выдачу подписки данного клиента (форматы raw, JSON и Clash): ссылки добавляются как есть, а удалённые подписки панель периодически скачивает (с кэшем и коротким таймаутом) и объединяет их конфигурации с собственными. Так в одной ссылке подписки клиента можно отдавать вместе со своими серверами ещё и внешние конфигурации.

Группа

Группа (поле `group`) — логическая метка для объединения связанных клиентов. Подсказка: «Логическая метка для группировки связанных клиентов (например, команда, клиент, регион). Фильтруется из панели инструментов.», плейсхолдер — «например, customer-a». Поле необязательное (по умолчанию пустое). По группе можно фильтровать список и выполнять массовые операции; для очистки метки у клиента используется действие **Разгруппировать**.

Снять группу можно и прямо в редакторе одного клиента: если очистить поле **Группа** и сохранить, метка корректно снимается, и клиент перестаёт отображаться под прежней группой.

Переопределение правила хоста

Поле **Переопределение правила хоста** (`clientHostRuleIds` , список id; вкладка **Основные**, рядом с обратным тегом; плейсхолдер *«По умолчанию (правило входящего)»*) привязывает клиента к одному или нескольким управляемым хостам. Без привязки клиент получает в подписке хосты своих inbound'ов, как все. С привязкой его raw-, JSON- и Clash-подписки строятся **только** по выбранным хостам — при любом порядке выдачи хостов и inbound'ов в подписке (#6620).

- **Что предлагает список.** Хосты тех inbound'ов, которые выбраны в форме; подпись — `примечание (адрес) (#id)` . Привязанный, но отключённый хост остаётся в списке с суффиксом *«{label} (отключено)»*, чтобы открытие и сохранение формы не сбрасывали привязку молча. Снятие inbound'а в форме убирает привязки к его хостам.
- **Привязка к хостам другого inbound'а.** Привязка действует отдельно на каждый inbound: если среди привязанных хостов нет хостов данного inbound'а, он выдаётся клиенту обычным образом.
- **Закрытая привязка.** Если привязанные хосты удалены, отключены или исключены из формата подписки (вкладка **Advanced** хоста), в этом формате клиент не получает ни одной ссылки, но остаётся в учёте трафика: его не удаляют и не отключают.
- **Правки хоста.** Хост сохраняет свой id при редактировании, поэтому привязки переживают его правку.
- **API.** Поле `clientHostRuleIds` передаётся в теле создания и изменения клиента; без поля (`null`) сохранённая привязка не меняется, пустой список `[]` её снимает. Список хостов с числовыми id отдаёт `GET /panel/api/hosts/raw` .

Комментарий

Комментарий (поле `comment`) — произвольная текстовая заметка для администратора (по умолчанию пусто). Содержимое попадает в поиск и доступно для фильтрации (**Есть** / **Нет** комментарий).

Включён

Включён (поле `enable`) — флаг активности учётной записи. По умолчанию **включён** (`true`): если при создании флаг не передан (например, в запросе API), панель ставит `true` . Явно переданный `false` начиная с 3.8.0 сохраняется — клиент создаётся выключенным (раньше панель включала его принудительно). Выключенный клиент (`enable = false`) не может подключиться и в сводке относится к категории **неактивных** (deactive). Панель сама выключает клиентов, исчерпавших квоту, истёкших или превысивших лимит IP.

Поля только для чтения

В карточке клиента также отображаются служебные поля: **Создан** (`created_at`) и **Обновлён** (`updated_at`) — временные метки создания и последнего изменения, заполняются автоматически и не редактируются. Поле **Обратный тег** (`reverse`) — необязательный Reverse tag для простого обратного прокси VLESS («Необязательный Reverse tag»).

8.2. Привязка к inbound

Каждый клиент должен быть привязан хотя бы к одному inbound — при создании требуется минимум один (`at least one inbound is required`). В редакторе это поле **Привязанные входящие** с подсказкой **Выберите один или несколько входящих**.

- **Привязать** — добавить клиента к выбранным inbound (тот же UUID/пароль и общий трафик). Существующие привязки сохраняются.
- **Отвязать** — убрать клиента из выбранных inbound. Сама запись клиента сохраняется (для полного удаления используйте **Удалить**). Пары, где клиент не был привязан, тихо пропускаются.

При сохранении клиента, привязанного к нескольким inbound, поля, несовместимые с конкретным протоколом/транспортом (например, Flow вне VLESS-Vision), автоматически приводятся к допустимым значениям для каждого inbound.

Операции над клиентом с несколькими привязками — создание, привязка, изменение, отвязка и удаление — применяются к inbound параллельно и независимо: сбой на одном inbound не останавливает остальные, а все ошибки собираются в одно сообщение, по строке на inbound, с префиксом `inbound N:`. Поэтому ответ с ошибкой (`success: false`) может означать частичное применение: на успешных inbound изменение уже записано, и панель помечает Xray к перезапуску. Лимит устройств (`limitHwid`) при создании клиента применяется, только если все inbound прошли успешно.

Над списком выбора inbound (в форме клиента, при массовом добавлении клиентов и в окнах массового прикрепления/открепления) есть кнопки **Выбрать все** и **Очистить**. В этих списках каждый inbound подписывается своим примечанием (remark), если оно задано, иначе — тегом inbound.

Окна **«Привязать клиентов из «…»»** и **«Отвязать клиентов из «…»»**, которые открываются из меню inbound'а на странице Inbounds, с 3.10.0 показывают клиентов этого inbound'а уже при первом открытии (в окне привязки все они сразу отмечены). Раньше при первом открытии после загрузки страницы таблица оставалась пустой, пока окно не закрыть и не открыть снова (#6736).

8.3. Операции над клиентом

Для отдельного клиента (через карточку **Информация о клиенте** или контекстное меню **Действия**) доступны:

Просмотр информации, QR-кода и ссылки

- **Информация о клиенте** — карточка со всеми полями, использованным/оставшимся трафиком (**Остаток**), сроком действия и привязанными inbound. С 3.5.0 карточка также показывает **группу** клиента (строка «Группа» с меткой, над «Комментарий»), когда группа задана.

Запрос клиента через API (`GET /panel/api/clients/get/:email`) рядом с полями `client` и `inboundIds` дополнительно возвращает `usedTraffic` — фактически израсходованный трафик (отправлено + получено, с учётом данных узлов), что упрощает сопоставление расхода с квотой `totalGB`.

Найти клиента по ID подписки можно запросом `GET /panel/api/clients/get/subId/:subId`: он возвращает массив полных записей (настройки, привязанные inbound'ы, внешние ссылки, трафик, адреса

туннелей — в том же виде, что и `get/:email`) для точного совпадения `subId`. Это список, потому что у старых записей `subId` может повторяться; если совпадений нет, возвращается ошибка. Права те же, что у запроса по email: сессия администратора или токен с областью `admin` (#6908).

- **QR-код** и **Ссылка** — конфигурационная ссылка клиента для импорта в клиентское приложение. Формируется по всем привязанным inbound с поддерживаемым протоколом (`GET /links/:email`). Если подходящих ссылок нет: «Нет ссылок для общего доступа — сначала привяжите клиента к входящему с поддерживаемым протоколом.».
- **Ссылка подписки** — URL подписки по `subId` (`GET /subLinks/:subId`). Доступна, только если у клиента есть `subId` и сервис подписки включён в **Настройки панели** → **Подписка** (иначе «Сервис подписки отключён.»). Дополнительно отдаётся **URL JSON-подписки**.

Окно **«QR-код»** состоит из раскрывающихся панелей: **«Информация о подписке»** (ссылка подписки) и её JSON-вариант, если JSON-подписка включена; share-ссылки по inbound; **«Конфигурация WireGuard»** / **«Конфиг AmneziaWG»** — по панели на каждый туннельный inbound клиента; **«Конфиг TUIC (Clash)»**. У каждой панели есть кнопка **«Копировать»**.

Когда QR-код не показывается. QR не строится только для share-ссылок с постквантовым ключом, который слишком велик для сканирования: с непустым параметром `pqv` (проверочный ключ ML-DSA-65 у REALITY) или с VLESS-шифрованием, ключ аутентификации которого — ML-KEM-768; такую ссылку можно только скопировать. С 3.10.0 QR снова есть у ссылок REALITY с подсказкой `support=x25519mlkem768`, у VLESS-шифрования с ключом X25519 (его значение начинается с `mlkem768x25519plus`) и у ссылок, где `mlkem768` / `mldsa65` встречается лишь в имени или адресе: прежняя проверка искала эти подстроки во всей ссылке и убирала QR во всех местах — в окнах «QR-код» и «Информация о клиенте», в QR inbound'а и на странице подписки (#6730).

- **«Конфиг TUIC (Clash)»** — готовый YAML-фрагмент `proxies:` для Clash / Mihomo / Clash Verge (сервер, порт, `uuid`, `password`, ALPN, SNI, контроль перегрузок, режим ретрансляции UDP, `reduce-rtt`) с QR-кодом и скачиванием файла `<email>.yaml`. Строится по первому TUIC-inbound клиента, если у клиента есть и UUID, и пароль (см. 5.13).
- **Зашифрованная ссылка Happ.** В панели подписки есть переключатель **«Стандартный»** / **«Зашифрованная ссылка Happ»**. Второй вариант превращает ссылку подписки клиента в ссылку `happ://crypt5/…`, которую открывает только приложение Happ. Ссылка создаётся локально на сервере панели: адрес подписки шифруется встроенным открытым ключом Happ (RSA-4096 + ChaCha20-Poly1305), без обращения к внешним сервисам и без сохранения результата — при каждом выборе варианта и по кнопке **«Сгенерировать заново»** получается новая ссылка. Это маскировка адреса, а не защита: окно предупреждает, что «*любой, у кого есть эта ссылка, может восстановить URL подписки или поделиться им*». По умолчанию вариант заблокирован: пока в **Настройки панели** → **Подписка** → **Happ** → **Ссылки подписки** не включён переключатель **«Зашифрованные ссылки подписки»** (`happLinkEnable`), вместо QR-кода выводится «Создание зашифрованных ссылок Happ не включено» с кнопкой **«Перейти к настройкам»** (см. 10.2). Адрес подписки должен быть абсолютным `http(s)`-URL не длиннее 8192 байт UTF-8; готовая ссылка длиннее 2953 байт не помещается в QR-код — он скрывается, но **«Копировать»** отдаёт полную ссылку. При сбое окно предлагает **«Повторить»** или посмотреть **Дашборд** → **Логи** (строки `component=happ_link`). API: `POST /panel/api/clients/happLink/:id` (числовой id клиента) возвращает `{"encryptedLink": "happ://crypt5/…"}`.

Сбросить трафик

Сбросить трафик (`POST /resetTraffic/:email`) обнуляет счётчики отправки/приёма (`up` , `down`) конкретного клиента. Квота (`totalGB`) и срок действия **не затрагиваются** — обнуляется только использованный объём. Тост: «Трафик сброшен». Если клиент не привязан ни к одному inbound: «Сначала привяжите этого клиента к входящему.».

Кнопка **Сбросить трафик** доступна и из формы редактирования клиента — в нижней панели, рядом с **Отмена** / **Сохранить** (перед сбросом запрашивается подтверждение). Если клиент был отключён из-за исчерпания трафика, сброс (как одиночный, так и массовый) автоматически снова **включает** его (`enable = true`) и сразу рассылает это изменение на узлы-ноды — повторно включать клиента вручную на мастере и нодах больше не нужно. То же делают периодический сброс трафика inbound'а и кнопка **Сбросить трафик всех клиентов**: клиенты, отключённые по квоте, возвращаются в работу (#6833). Inbound, выключенный из-за собственного лимита трафика, периодический сброс тоже включает обратно, а выключенный вручную или по сроку — нет (#6834).

Сбросить лимит IP

Очищает накопленный журнал IP клиента (`POST /clearIps/:email`), чтобы снять временную блокировку по превышению лимита одновременных подключений. Тост: «Лог был очищен».

Удалить

Удалить (`POST /del/:email`) — полное удаление клиента. Диалог подтверждения: заголовок «Удалить клиента {email}?», текст «Клиент будет удалён из всех привязанных входящих, а его запись трафика будет уничтожена. Это действие нельзя отменить.». Удаление снимает клиента со **всех** inbound и уничтожает его запись трафика. Тост: «Клиент удалён».

8.4. Массовые операции

В списке клиентов можно отметить несколько записей (**Выбрать всё**, **Очистить всё**); счётчик — «{count} выбрано». Над выбранными доступны:

- **Удалить ({count})** (`POST /bulkDel`) — групповое удаление. Подтверждение: «Удалить {count} клиентов?», «Каждый выбранный клиент удаляется из всех привязанных входящих, его запись трафика уничтожается. Это действие нельзя отменить.». Тост: «Удалено клиентов: {count}», при частичной неудаче — «Удалено: {ok}, не удалось: {failed}».
- **Изменить ({count})** / **Корректировка** (`POST /bulkAdjust`) — массовое изменение срока, квоты, flow, лимита устройств и рекламного тега MTProto. Диалог «Изменить {count} клиентов» с подсказкой «Положительные значения добавляют, отрицательные — уменьшают. Клиенты с неограниченным сроком или трафиком пропускаются для соответствующего поля.». Поля: **Добавить дни**, **Добавить трафик (ГБ)**, **Set flow**, **Лимит HWID** и **Рекламный тег (спонсорский канал)**. Логика:
  - **Срок:** клиенты с бессрочным сроком (`expiryTime == 0`) пропускаются («unlimited expiry»); для клиентов с датой срок сдвигается на указанное число дней; для клиентов в режиме «после первого использования» (отрицательный срок) корректируется длительность ожидания. Уменьшение, выходящее за пределы остатка, пропускается («reduction exceeds remaining time/delay window»).

- **Трафик:** клиенты с безлимитом (`totalGB == 0`) пропускаются («unlimited traffic»); иначе квота меняется на указанный объём, не опускаясь ниже нуля.
- **Flow:** выпадающий список **Set flow** позволяет установить или сбросить XTLS flow сразу у всех выбранных клиентов. По умолчанию выбрано **No change** (без изменений). Вариант **Disable (clear flow)** сбрасывает flow, а значения `xtls-rprx-vision` и `xtls-rprx-vision-udp443` задают соответствующий vision-flow. Установка vision-flow применяется только к inbound, поддерживающим flow; неподходящие inbound остаются без изменений и помечаются как пропущенные, тогда как сброс flow допускается всегда.
- **Авто-включение (с 3.4.2):** клиент, отключённый **только из-за исчерпания** (истёк срок или превышена квота), при продлении автоматически снова включается — локально и на его узле, — если корректировка возвращает его в пределы лимитов. Отключённые вручную или всё ещё исчерпанные остаются выключенными (поле `enable` теперь записывается явно, поэтому состояние сохраняется и у клиентов без inbound).
- **Лимит HWID:** заданное значение `limitHwid` ставится всем выбранным клиентам (`0` — без ограничения; пустое поле — **Без изменений**). При снижении лимита лишние зарегистрированные устройства удаляются — остаются обращавшиеся последними.
- **Рекламный тег (спонсорский канал):** 32-значный hex-тег задаёт спонсорский канал, `none` очищает его. Применяется только к клиентам на MTProto-inbound, остальные попадают в пропущенные (`adTag not supported on inbound`); тег другого формата не принимается («Рекламный тег должен содержать ровно 32 шестнадцатеричных символа.»).
- Если не задано ни одно поле: «Укажите хотя бы одно поле перед применением.». Тост: «Изменено: {count}» / «Изменено: {ok}, пропущено: {skipped}».

Пример: продлить выбранных клиентов на 30 дней и добавить 50 ГБ. В диалоге **Изменить** укажите **Добавить дни** = `30` , **Добавить трафик (ГБ)** = `50` . Чтобы, наоборот, отнять неделю и урезать квоту на 10 ГБ, введите отрицательные значения: **Добавить дни** = `-7` , **Добавить трафик (ГБ)** = `-10` (клиенты с бессрочным сроком или безлимитом по соответствующему полю будут пропущены).

- **Привязать ({count})** / **Отвязать ({count})** (`POST /bulkAttach` / `bulkDetach`) — массовая привязка/отвязка выбранных клиентов к выбранным inbound. Цели — только многопользовательские inbound. Результат отвязки: «Отвязано {detached}, пропущено {skipped}.». С 3.5.0 селектор inbound в форме клиента **скрывает выключенные inbound** (кроме тех, к которым клиент уже привязан), а в окне «Отвязать» устранено накопление дублей записей одного клиента (данные чинятся автоматически при старте панели).

- **Sub-ссылки ({count})** — сводная таблица URL подписок и JSON-подписок выбранных клиентов с кнопкой **Копировать все**. Если ни у кого нет subId: «Ни у одного из выбранных клиентов нет ID подписки.».

- **Добавить в группу** и **Разгруппировать** — назначение и снятие метки группы.

- **Включить ({count})** / **Отключить ({count})** (`POST /bulkEnable` / `bulkDisable`) — массовое включение и отключение выбранных клиентов. **Включить** активирует каждого выбранного клиента на всех привязанных inbound; клиенты с исчерпанной квотой трафика или истёкшим сроком будут автоматически отключены снова. **Отключить** сразу лишает клиентов доступа, но их записи и накопленный трафик сохраняются. Перед выполнением панель запрашивает подтверждение, а после операции показывает уведомление с количеством обработанных клиентов и, при наличии, с количеством тех, для которых действие не удалось.

Сброс трафика и удаление по статусу

- **Сбросить трафик всех клиентов** (`POST /resetAllTraffics`) — обнуляет счётчики `up` / `down` у **всех** клиентов панели. Подтверждение: «Сбросить трафик всех клиентов?» и «Счётчики отправки/приёма всех клиентов сбрасываются в ноль. Квоты и срок действия не затрагиваются. Это действие нельзя отменить.». Тост: «Трафик всех клиентов сброшен».
- **Удалить исчерпанных** (`POST /delDepleted`) — удаляет всех клиентов, у которых **исчерпана квота** (`total > 0 and up + down >= total`) **или истёк срок** (`expiry_time > 0 and expiry_time <= сейчас`), при условии `reset = 0` (клиенты с автопродлением не трогаются). Подтверждение: «Удалить исчерпанных клиентов?», «Удаляются все клиенты, у которых исчерпана квота трафика или истёк срок. Это действие нельзя отменить.». Тост: «Удалено исчерпанных клиентов: {count}».

Экспорт, импорт и удаление непривязанных клиентов

Когда ничего не выделено, в меню **Ещё** на странице **Клиенты** доступны три операции.

Экспорт клиентов (`GET /clients/export`) открывает просмотрщик с JSON-списком всех клиентов в формате `{client, inboundIds}` с кнопками копирования и скачивания (файл `clients-export.json`). **Импорт клиентов** (`POST /clients/import`) открывает редактор, в который вставляют такой же JSON и жмут **Import**: клиенты с `inboundIds` создаются и привязываются к inbound, клиенты без привязок восстанавливаются как отдельные «голые» записи, а уже существующие email **никогда не перезаписываются** — они попадают в список пропущенных. Тосты: «{count} clients imported», «{ok} imported, {failed} skipped». Флаг `enable` переносится как есть: выключенный при экспорте клиент импортируется выключенным.

Удалить непривязанных клиентов (`POST /clients/delOrphans`) — опасная операция: удаляет всех клиентов, не привязанных ни к одному inbound, вместе с их записью трафика, журналом IP и внешними ссылками. Подтверждение: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.». Тост: «{count} unattached clients deleted». Действие необратимо.

Переносимый экспорт и импорт клиентов

Клиентов можно выгрузить и загрузить переносимым JSON (эндпоинты `POST /panel/api/clients/export` и `/panel/api/clients/import`). С 3.9.0 выгрузка несёт и **счётчики трафика** клиента (`up` , `down` , число сбросов, последний онлайн), а импорт восстанавливает их **только для вновь создаваемых** клиентов; существующие (совпавшие по email) попадают в `skipped` и свои счётчики сохраняют (#6469).

8.5. Поиск, фильтры и сортировка

Над списком есть строка поиска («Поиск email, комментария, sub ID, UUID, пароля, auth, Telegram ID…») — она ищет по email, комментарию, subId, UUID, паролю, auth и (снова с 3.5.0) по **Telegram ID**. Счётчик результатов: «Показано {shown} из {total}».

Список клиентов обновляется автоматически: панель раз в несколько секунд подтягивает актуальную текущую страницу, поэтому вновь подключившиеся клиенты и изменившийся порядок сортировки появляются без ручного обновления (индикатор загрузки при фоновом опросе не мигает).

С 3.5.0 в таблице есть колонка **«Скорость»** (между «Трафик» и «Остаток»): живая скорость клиента — синяя метка `↑ отдача / ↓ загрузка` (скользящее ~5-секундное среднее, обновляется каждые 5 секунд); при отсутствии трафика — серый прочерк. На мобильных скорость показывается строкой в карточке клиента.

Панель **Фильтр клиентов** позволяет отбирать по статусу (категории), протоколу, привязанному inbound, диапазону срока действия, диапазону использованного трафика, наличию автопродления (**Есть/Нет**), наличию Telegram ID и комментария, а также по группе. На панелях с узлами появляется мультиселект **Узлы**: можно ограничить список клиентами выбранных узлов; отдельный пункт **Локальная панель** отбирает клиентов inbound без привязки к узлу (фильтр виден только при наличии узлов). Сортировка: **Сначала старые/новые**, **Недавно обновлены**, **Недавно в сети**, **Email А→Я / Я→А**, **Больше трафика**, **Больше остатка**, **Скорее истекают**.

Поиск и фильтр по группе с не-ASCII (3.9.0). Поиск клиентов и фильтр по группе теперь сворачивают регистр и для не-латиницы (кириллица, фарси и др.): заглавная кириллическая буква в имени группы или поисковом запросе снова находится (прежде `LOWER()` в SQLite сворачивал только ASCII) (#6685).

Клик по карточке сводки (3.8.5). Карточки над списком (Клиенты, В сети, Исчерпано, Почти исчерпано, Отключено, Активна) кликабельны: клик по карточке ставит единственным фильтром её статусную категорию, повторный клик снимает фильтр, а клик по карточке «Клиенты» сбрасывает фильтр по статусу. Заодно выровнены сами категории «Активна»/«Отключено» — раньше они были шире, чем счётчик на карточке (в «Активна» попадали клиенты на грани исчерпания, в «Отключено» — уже исчерпанные отключённые), теперь фильтр и карточка всегда показывают одно и то же число.

8.6. Значки и статусы

Приоритет статусов: исчерпанный/истёкший → неактивный → скоро истекает → активный.

- **В сети** / **Не в сети** — клиент с активным подключением (есть в текущем онлайн-списке) и **включён**. Онлайн-список обновляется отдельными запросами (`/onlines` , `/onlinesByGuid`).
- **Исчерпан** (depleted) — квота выбрана (`up + down >= totalGB`) **или** срок истёк (`expiryTime <= сейчас`). Такой клиент автоматически отключается и попадает под действие **Удалить исчерпанных**.
- **Скоро истекает / заканчивается** (expiring) — включённый клиент, у которого до истечения срока осталось меньше порогового интервала **или** до исчерпания квоты осталось меньше порогового объёма (пороги задаются в настройках панели). Не считается, если клиент уже исчерпан/отключён.
- **Неактивный** (deactive) — клиент с `enable = false` (выключен вручную или фоновым заданием).
- **Активный** (active) — включён, не исчерпан, срок не истёк и до порогов ещё далеко.

Колонка «Last Subscription Fetch» (3.7.0). В таблице клиентов показывается время последней выгрузки подписки — видно, забрал ли клиент конфиг после ваших правок и живо ли вообще его приложение. Никогда не забирал — колонка пустая.

9. Группы клиентов

> Это возможность данного форка 3X-UI. В оригинальном проекте 3x-ui (MHSanaei) понятия «группа клиентов» нет — здесь добавлены отдельная таблица групп, страница **Группы** в меню панели и соответствующие методы API. Если вы переносите конфигурацию на оригинальный 3x-ui, метка группы просто не будет нигде учитываться.

9.1. Что такое группа клиентов и зачем она нужна

Группа — это именованная логическая метка (label), которую можно навесить на одного или нескольких клиентов. Она не создаёт нового способа подключения и не является ни inbound, ни нодой — это сугубо организационный ярлык, по которому клиентов удобно фильтровать и обрабатывать массово.

Ключевая идея модели клиентов в этом форке: **клиент — это сущность верхнего уровня, идентифицируемая по email** (поле `email` в таблице `clients` имеет уникальный индекс). Один и тот же клиент (один email с одними и теми же учётными данными) может одновременно числиться в нескольких inbound и даже на нескольких нодах, в том числе с разными протоколами. Метка группы хранится **один раз на клиента**, поэтому она автоматически распространяется на все его привязки к inbound сразу.

Метка группы — это логическая метка для группировки:

Слой	Где хранится	Поле
Запись клиента (БД)	таблица `clients`	`group_name` (по умолчанию пустая строка `''`)
Справочник групп (БД)	таблица `client_groups`	`name` (уникальный индекс, не пустой)
Настройки inbound (Xray)	JSON `settings.clients[].group`	дублируется в каждый клиентский объект каждого inbound, где состоит клиент

Зачем это нужно на практике:

- **Один клиент на нескольких inbound/нодах.** Если клиент «продаётся» как доступ сразу к нескольким inbound (например, разные протоколы или разные ноды), группа позволяет управлять им как единым целым: сбросить трафик, удалить, переименовать метку — одной операцией по всем его inbound.
- **Массовые операции и фильтрация.** На странице **Клиенты** список можно фильтровать по группе; на странице **Группы** доступны массовые действия над всеми участниками группы.
- **Организация большого парка клиентов.** Метки вида `vip`, `trial`, `team-A` помогают раскладывать тысячи клиентов по логическим корзинам, не плодя при этом отдельные inbound.

9.2. Связь группы с клиентами, inbound, нодами и протоколами

Это важнейший для понимания подраздел, поскольку синхронизация метки нетривиальна.

Группа описывает клиента, а не inbound. Метка живёт в записи клиента (`clients.group_name`). Когда клиент присоединён к нескольким inbound, при любой смене группы панель проходит по **всем** inbound, в которых этот клиент состоит, и проставляет/убирает поле `group` внутри их Xray-настроек

(`settings.clients[]`). Технически это делается так: по email клиента находятся все inbound, в которых он состоит, затем в JSON-настройках каждого такого inbound правится клиентский объект с этим email. Поэтому:

- Группа **не зависит от протокола.** Один email может быть VLESS-клиентом в одном inbound и Hysteria-клиентом в другом — метка группы у него всё равно одна и применится к обоим (учётные данные при этом у каждого протокола свои и сохраняются раздельно).
- Группа **охватывает ноды.** Inbound, принадлежащие нодам, отличаются от inbound главной панели полем `nodeId` (у inbound главной панели оно `null` / `0`). Метка группы распространяется на клиентские объекты в inbound вне зависимости от того, главный это inbound или нодовый — лишь бы клиент с таким email там присутствовал.

Метка группы устойчива к синхронизации с нод и к перестроению настроек. Это поведение специально заложено:

- Когда нода присылает снапшот трафика, её данные **не затирают** локальные `group_name` и `comment` клиента в БД панели. Группа и комментарий считаются «панель-локальными» полями — нода ими не управляет.
- При перестроении настроек inbound пустое значение `group` в приходящих данных **не сбрасывает** уже сохранённую метку. Группой управляют именно через страницу **Группы** (а не через редактирование Xray-настроек инбаунда), поэтому «пустая группа» при обычной перестройке настроек трактуется как «не трогать», а не как «очистить».

Практический вывод: единственные операции, которые **намеренно очищают** метку, — это удаление группы и явное удаление клиента из группы (см. ниже). Обычное редактирование inbound или фоновая синхронизация с нодой группу не потеряют.

9.3. Справочник групп и «пустые» группы

Список групп на странице формируется слиянием двух источников:

1. **Производные группы (derived)** — все непустые значения `group_name` , реально встречающиеся у клиентов, с подсчётом количества клиентов.
2. **Сохранённые группы (stored)** — записи из таблицы `client_groups` .

Это объединение даёт важный эффект: группа может существовать **без единого клиента**. Такая группа создаётся явной кнопкой «Добавить группу» (запись в `client_groups`) и в списке отображается со счётчиком `0` . Эти записи и считаются **пустыми группами**. Список всегда отсортирован по имени без учёта регистра.

Сводные счётчики на странице:

Поле (RU)	Что показывает
Всего групп	Общее число групп (сохранённых и производных вместе).
Клиенты с группой	Сколько клиентов имеют непустую метку группы.
Пустые группы	Сколько групп существует без клиентов (счётчик `0`).
Клиентов в группе	Число клиентов в конкретной группе (столбец таблицы).

9.4. Поля и столбцы группы

Запись группы в таблице `client_groups` содержит:

Поле	Тип	По умолчанию	Описание
`Id`	int	автоинкремент	Первичный ключ записи группы.
`Name`	string	— (обязательно)	Имя группы. Уникальный индекс, не может быть пустым. В UI — столбец **Имя**.
`CreatedAt`	int64 (мс)	время создания	Момент создания записи группы.
`UpdatedAt`	int64 (мс)	время изменения	Момент последнего изменения.

В таблице на странице отображаются как минимум столбцы **Имя** и **Клиентов в группе**, а также кнопки действий (см. ниже).

9.5. Создание группы

Кнопка **Добавить группу**.

Шаги:

1. Нажмите **Добавить группу**.
2. Введите имя группы.
3. Подтвердите.

Поведение бэкенда (`POST /panel/api/clients/groups/create` ,тело `{"name": "..."}`):

- Имя обрезается от пробелов по краям. Пустое имя отклоняется с ошибкой «group name is required».
- Если группа с таким именем уже есть — ошибка «group already exists».
- При успехе создаётся запись в `client_groups` (изначально без клиентов — это пустая группа).

Сообщение об успехе: **«Группа «{name}» создана.»**

Пример: создать пустую группу через API. Подготовьте набор меток заранее, ещё до наполнения клиентами:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"name": "vip"}'
```

Ответ при успехе:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Повторный вызов с тем же именем вернёт `"success": false` и сообщение `group already exists` .

> Создание пустой группы заранее удобно, когда вы хотите подготовить набор меток и затем наполнять их клиентами через «Добавить клиентов…».

9.6. Переименование группы

Кнопка **Переименовать**, заголовок диалога — **«Переименовать {name}»**.

Поведение (`POST /panel/api/clients/groups/rename` , тело `{"oldName": "...", "newName": "..."}`):

- Оба имени обрезаются от пробелов. Пустое старое имя — ошибка «old group name is required», пустое новое — «new group name is required».
- Если новое имя совпадает со старым — ничего не делается (затронуто `0` клиентов).
- Иначе переименование выполняется атомарно:
  - запись в `client_groups` переименовывается;
  - у всех клиентов с `group_name = oldName` поле обновляется на `newName` ;
  - во **всех inbound**, где состоят затронутые клиенты (включая нодовые), в Xray-настройках значение `group` правится со старого на новое.
- После переименования панель помечает Xray как требующий перезапуска и рассылает уведомление об изменении клиентов.

Сообщения:

- Успех: **«Группа переименована для {count} клиент(ов).»**
- Конфликт имён в UI: **«Группа с именем «{name}» уже существует.»**

9.7. Добавление клиентов в группу

Кнопка **Добавить клиентов…**, заголовок — **«Добавить клиентов в группу «{name}»»**.

Дословная подсказка в диалоге:

> «Выберите клиентов для добавления в эту группу. Существующие привязки к входящим сохраняются; меняется только метка группы. Клиенты, уже входящие в эту группу, не показываются.»

Если добавлять некого, показывается **«Нет других клиентов для добавления.»**

Поведение (`POST /panel/api/clients/groups/bulkAdd` , тело `{"emails": [...], "group": "..."}`):

- Имя группы обязательно (иначе ошибка «group name is required»); пустой список email — операция ничего не делает.
- Если такой группы ещё нет ни в `client_groups` , ни среди производных — она будет создана автоматически.
- Для выбранных email у клиентов проставляется `group_name = group` ; **привязки клиентов к inbound не меняются** — затрагивается только метка. Затем во всех inbound этих клиентов проставляется поле `group` .
- Возвращается число затронутых записей клиентов; Xray помечается на перезапуск.

Сообщение об успехе: **«Добавлено {count} клиент(ов) в {name}.»**

Пример: пометить нескольких клиентов группой одним запросом. Клиенты указываются по email, привязки к inbound при этом не меняются:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Если группы `vip` ещё нет, она будет создана автоматически. После запроса у этих клиентов в записи проставится `group_name = "vip"`, а в Xray-настройках каждого их inbound клиентский объект получит поле `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Удаление клиентов из группы (без удаления самих клиентов)

Кнопка **Удалить клиентов...**, заголовок — **«Удалить клиентов из группы «{name}»»**.

Дословная подсказка:

> «Выберите участников для удаления из этой группы. Сами клиенты сохраняются (используйте «Удалить клиентов группы» для полного удаления).»

Поведение (`POST /panel/api/clients/groups/bulkRemove`, тело `{"emails": [...]}`): технически это то же, что «Добавить в группу» с пустой группой. У выбранных клиентов очищается `group_name`, а в их inbound поле `group` удаляется из Xray-настроек. Сами клиенты и их привязки к inbound остаются.

Сообщение об успехе: **«Удалено {count} клиент(ов) из {name}.»**

9.9. Сброс трафика группы

Кнопка **Сбросить трафик**.

Диалог подтверждения:

- Заголовок: **«Сбросить трафик группы {name}?»**
- Текст: **«Сбрасывается только счётчик трафика группы. Счётчики отдельных клиентов не затрагиваются.»**

Поведение: обнуляется **только отображаемый счётчик самой группы** — панель запоминает текущую сумму `up/down` группы как базовую отметку и далее в списке групп показывает `max(0, сумма − базовая отметка)`. Счётчики `up` / `down`, квоты и поле `enable` отдельных клиентов **не изменяются**, перезапуск Xray не требуется. Это изменение поведения относительно прежних версий, где сброс обнулял трафик всех клиентов группы.

Запрос: `POST /panel/api/clients/groups/resetTraffic` с телом `{"name": "<имя группы>"}`.

Сообщение об успехе: **«Трафик группы {name} сброшен.»**

9.10. Удаление группы и удаление клиентов группы

На странице есть **две принципиально разные операции удаления** — их легко перепутать, поэтому различие критично.

9.10.1. Удалить группу (сохранить клиентов)

Кнопка **«Удалить группу (сохранить клиентов)»**.

Диалог:

- Заголовок: **«Удалить группу {name}?»**
- Текст: **«Это удаляет группу и очищает её метку у {count} клиент(ов). Сами клиенты не удаляются.»**

Поведение (`POST /panel/api/clients/groups/delete`, тело `{"name": "..."}`): запись группы удаляется из `client_groups`, у всех её клиентов `group_name` очищается, и из их inbound убирается поле `group`. **Клиенты, их подключения и трафик сохраняются.** Xray помечается на перезапуск.

Сообщение об успехе: **«Группа очищена у {count} клиент(ов).»**

9.10.2. Удалить клиентов группы (полное удаление)

Кнопка **«Удалить клиентов группы»**.

Диалог:

- Заголовок: **«Удалить всех клиентов в {name}?»**
- Текст: **«Это безвозвратно удаляет {count} клиент(ов) вместе с их записями трафика. Метка группы также очищается. Это нельзя отменить.»**

Это деструктивная операция: она удаляет самих клиентов (через массовое удаление по email, эндпоинт `POST /panel/api/clients/bulkDel`), включая их записи трафика, и тем самым убирает их из всех inbound.

Сообщения:

- Успех: **«Удалено {count} клиент(ов).»**
- Частичный результат: **«{ok} удалено, {failed} пропущено»**

> Если группа пуста, действия над её участниками недоступны — выводится **«В этой группе пока нет клиентов.»**

Массовый перенос: что изменилось в 3.7.0

Массовый перенос между группами теперь **сообщает, сколько клиентов действительно изменилось**, и **не перезапускает Xray**, если в конфиге менять нечего. Раньше перезапуск случался и тогда, когда перенос ничего фактически не поменял, — то есть рвал соединения без причины.

9.11. Связь со страницей «Клиенты»

Метка группы видна и используется и вне страницы **Группы**:

- В компактной записи клиента есть поле `group`, поэтому в списке клиентов отображается принадлежность к группе.
- Список клиентов (`/panel/api/clients/list/paged`) принимает параметр фильтра `group`: можно передать одно имя или несколько через запятую. Сопоставление выполняется по принципу «ИЛИ» внутри поля, без учёта регистра. Особый случай: пустой элемент в списке групп фильтра означает «клиенты без группы» (у которых `group` пустая).
- В ответе страницы клиентов отдаётся массив `groups` — полный перечень имён существующих групп, чтобы UI мог построить выпадающий список фильтра.

Пример: фильтрация клиентов по группам. Запрос отдаёт только клиентов с метками `vip` или `trial` (несколько имён — через запятую, «ИЛИ»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Чтобы получить клиентов **без** группы, передайте пустой элемент в списке — например, значение фильтра `group=` (пустая строка) или `group=vip,` (метка `vip` плюс клиенты без группы).

В меню действий группы есть и **«Изменить ({count})»** — тот же диалог массовой корректировки, что на странице **Клиенты** (см. 8.4), для всех участников группы; тост — «Скорректировано {count} клиент(ов) в {name}.». Начиная с 3.8.0 отсюда применяются все поля диалога: Flow, лимит HWID и рекламный тег MTProto. Раньше со страницы **Группы** учитывались только дни и трафик, а выбранный Flow молча отбрасывался.

9.12. Сводка эндпоинтов API

Все маршруты групп смонтированы под `/panel/api/clients`:

Метод и путь	Назначение	Тело запроса
`GET /panel/api/clients/groups`	Список групп со счётчиками клиентов	—
`GET /panel/api/clients/groups/:name/emails`	Email всех участников группы (сортировка по email)	—
`POST /panel/api/clients/groups/create`	Создать пустую группу	`{"name"}`
`POST /panel/api/clients/groups/rename`	Переименовать группу	`{"oldName","newName"}`

Метод и путь	Назначение	Тело запроса
`POST /panel/api/clients/groups/delete`	Удалить группу, сохранив клиентов (очистка метки)	`{"name"}`
`POST /panel/api/clients/groups/bulkAdd`	Добавить клиентов в группу (по email)	`{"emails":[...],"group"}`
`POST /panel/api/clients/groups/bulkRemove`	Убрать клиентов из группы (очистка метки)	`{"emails":[...]}`
`POST /panel/api/clients/bulkDel`	Полное удаление клиентов (используется «Удалить клиентов группы»)	`{"emails":[...],"keepTraffic"}`

Пример: типовой сценарий жизненного цикла группы через API.

```
# 1. Создать метку trial
curl -s .../panel/api/clients/groups/create   -d '{"name":"trial"}'

# 2. Навесить её на двух клиентов
curl -s .../panel/api/clients/groups/bulkAdd  -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Обнулить трафик всех участников (по email из /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Удалить группу, но сохранить клиентов (только очистка метки)
curl -s .../panel/api/clients/groups/delete   -d '{"name":"trial"}'
```

Шаг 4 удаляет запись группы и очищает `group_name` у её клиентов, но сами клиенты, их подключения и трафик остаются. Для безвозвратного удаления самих клиентов вместо этого используется `bulkDel`.

Операции, меняющие метку у клиентов (`rename` , `delete` , `bulkAdd` , `bulkRemove`), помечают Xray как требующий перезапуска и рассылают уведомление об изменении клиентов.

9.13. Трафик по группе

Новинка версии 3.3.0: в разделе **Группы** (страница «Клиенты», вкладка управления группами) таблица групп теперь показывает не только количество клиентов в каждой группе, но и суммарный израсходованный трафик группы. Колонка подписана **«Использованный трафик»**.

Что показывает колонка

Для каждой строки-группы выводится сумма трафика по всем клиентам, входящим в эту группу — то есть сложенные `up + down` (отданный + принятый трафик) всех её участников. Это даёт быстрый ответ на вопрос «сколько суммарно скачала/раздала вся группа», без необходимости открывать клиентов по одному и складывать вручную.

Рядом в таблице групп выводятся:

Колонка	Что означает
Имя	Имя группы
Клиенты	Сколько клиентов помечено этой группой (раньше столбец назывался «Клиентов в группе»)
Отправлено	Суммарный `up` (отданный трафик) по всем клиентам группы
Получено	Суммарный `down` (принятый трафик) по всем клиентам группы
Использованный трафик	Суммарный `up + down` по всем клиентам группы

Отданный и принятый трафик выводятся отдельными столбцами **Отправлено** и **Получено**, а столбец **Использованный трафик** показывает их сумму. Столбец числа клиентов называется просто **Клиенты**.

Сводка над таблицей дополнительно показывает агрегаты по всем группам — **«Всего групп»** и **«Клиенты с группой»**, а суммарный трафик разбит на две карточки: **«Всего отправлено / получено»** (со стрелками вверх/вниз — отдельно отданный и принятый трафик всех групп) и **«Общий трафик»** (с иконкой диаграммы — их суммарный итог).

Как считается

Расчёт выполняется одним SQL-запросом к таблице клиентов с присоединением (`LEFT JOIN`) таблицы учёта трафика:

- по полю-метке группы (`group_name`) клиенты группируются, считается их количество — это и есть «Клиентов в группе»;
- трафик берётся как сумма `up + down` из присоединённой таблицы `client_traffics` . То есть суммируются и отданные (`up`), и принятые (`down`) байты по каждому клиенту;
- поскольку email уникален и в таблице клиентов, и в таблице трафика, присоединение не задваивает трафик одного клиента.

Особенности значений:

- **Клиенты без записи трафика** учитываются в счётчике участников, но добавляют к сумме 0, поэтому свежесозданная группа показывает трафик `0` .
- **Пустые группы** (созданы, но без клиентов) также присутствуют в списке с нулевым счётчиком и нулевым трафиком: помимо групп, «выведенных» из меток клиентов, в результат подмешиваются явно сохранённые группы, после чего список сортируется по имени без учёта регистра.
- Клиенты без метки группы (`group_name` пустой) в расчёт не попадают.

Связанные действия

Из таблицы групп по-прежнему доступны действия над группой целиком, в том числе **«Сбросить трафик»** — обнуляет **только счётчик самой группы** (базовую отметку), не трогая трафик отдельных клиентов (см. 9.9). После такого сброса колонка «Использованный трафик» для этой группы показывает `0` .

---

10. Подписки (Subscription)

Подписка (subscription) — это механизм, который позволяет выдать клиенту одну постоянную ссылку (URL), по которой VPN-клиент сам скачивает и периодически обновляет полный набор конфигураций. Вместо того чтобы вручную пересылать пользователю отдельную ссылку на каждый inbound, ему передаётся единый адрес вида `https://домен:порт/<путь>/<subId>`, где `<путь>` — путь подписки этой панели (на новой установке он случайный, см. 10.2). По этому адресу панель на лету собирает все конфигурации, привязанные к данному клиенту, и отдаёт их в нужном клиенту формате. При изменении настроек на сервере (новый адрес, ротация Reality-ключей, добавление inbound) клиент получает актуальную конфигурацию при следующем автоматическом обновлении, не требуя никаких действий от пользователя.

Подписку обслуживает отдельный HTTP/HTTPS-сервер внутри панели, который запускается независимо от веб-панели и слушает на собственном порту. Это сделано из соображений безопасности: порт подписки можно открыть наружу, не открывая порт самой панели.

10.1. Что такое subId и как формируется ссылка

Каждый клиент в inbound имеет поле `subId` (в интерфейсе — «ID подписки»). Именно это значение является ключом подписки: панель ищет во всех inbound клиентов, у которых `subId` совпадает с запрошенным, и объединяет их конфигурации в один ответ.

- Если у нескольких клиентов (в одном или в разных inbound) задан одинаковый `subId`, их конфигурации попадут в одну подписку. Это штатный способ выдать одному пользователю сразу несколько серверов/протоколов по одной ссылке.

Пример: один пользователь — два сервера по одной ссылке. Допустим, есть два inbound (VLESS на сервере A и Trojan на сервере B). Чтобы выдать пользователю обе конфигурации одной ссылкой, задайте обоим его клиентам одинаковый `subId`:

```
Inbound 1 (VLESS):  email = ivan@vpn,  subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn,  subId = ivan2025
```

Тогда по адресу `https://sub.example.com:2096/sub/ivan2025` (в примерах раздела путь подписки — `/sub/`; у вашей панели он свой) панель отдаст обе конфигурации сразу. Добавите позже третий inbound с тем же `subId` — он появится у пользователя при следующем автообновлении подписки, без пересылки новой ссылки.

- Если у клиента поле `subId` пустое, ссылкой для общего доступа поделиться нельзя. В интерфейсе на это указывает подсказка: «У этого клиента нет subId, ссылка для общего доступа недоступна.»

Внешние ссылки и подписки клиента (вкладка «Links»)

В форме клиента есть вкладка **«Links»**, где для отдельного клиента можно прикрепить дополнительные источники конфигураций, которые подмешиваются именно в его подписку (форматы RAW, JSON и Clash):

- **Add External Link** — сторонняя share-ссылка (`vless://`, `trojan://`, `ss://` и т. д.). Добавляется в выдачу как есть, а для JSON/Clash дополнительно разбирается в конфигурацию.

- **Add External Subscription** — адрес внешней подписки. Панель сама скачивает её (с кэшированием и коротким таймаутом) и вливает полученные конфигурации в общий список клиента.

Это удобно, чтобы выдать клиенту дополнительные серверы поверх ваших inbound по той же единой ссылке. Если ответ удалённой подписки слишком велик, он больше не обрезается молча: панель возвращает ошибку и продолжает использовать последнее успешно закэшированное значение. Ссылки и подписки **выключенного или истёкшего** клиента не попадают ни в один формат выдачи, но сам клиент по-прежнему учитывается в заголовке `Subscription-Userinfo`. В Clash/Mihomo внешняя `ss://`-ссылка проходит те же проверки транспорта и TLS, что и Shadowsocks-inbound: узел, который Clash не может корректно описать, пропускается, а не выдаётся заведомо нерабочим.

- Значение `subId` нельзя задать произвольно: при сохранении проверяется, что в нём нет пробелов, символов `/`, `\` и управляющих символов. Соответствующая подсказка валидации: «ID подписки не может содержать пробелы, '/', '' или управляющие символы».

Итоговая ссылка строится как `<база>/<subPath>/<subId>` (см. раздел о настройках сервера подписок и поле «URI обратного прокси»). Если по `subId` не найдено ни одного клиента (клиент удалён, `subId` не существует), сервер возвращает HTTP 404 без тела. При внутренней ошибке — HTTP 500. VPN-клиенты ориентируются только на код ответа, поэтому тело ошибки намеренно пустое.

Лимит устройств и `hwid-status`. Если у клиента задан лимит устройств (HWID, см. 8.1), он проверяется на каждом адресе, который отдаёт конфигурации, — начиная с 3.8.0 и на `?view=raw` у JSON- и Clash-ссылок (см. 10.5); HTML-страница информации лимитом не ограничена. Узнать, сколько слотов осталось, можно запросом `GET <subPath><subId>/hwid-status` (например, `/sub/ivan2025/hwid-status`). Ответ — JSON вида `{"active":true,"limit":2,"registered":1,"remaining":1,"full":false}`: действует ли лимит, его значение, число зарегистрированных устройств, остаток слотов и признак заполнения; без лимита приходит `{"active":false,"limit":0,"registered":0,"remaining":0,"full":false}`. Запрос только читает счётчики: устройство не регистрируется, слот не тратится, сведений об устройствах (HWID, модель, IP) в ответе нет. Несуществующий и выключенный `subId` одинаково дают 404 без тела, поэтому перебором нельзя узнать, какие подписки существуют. Этот адрес, как и все адреса сервера подписок, отвечает и на `HEAD`.

Порядок ссылок inbound в подписке

У каждого inbound есть поле **«Порядок в подписке»** (`subSortIndex`) — целое число, задающее позицию ссылок этого inbound в выдаче подписки. Меньшие значения идут первыми; при равных значениях сохраняется исходный порядок создания (по id). По умолчанию `1`; `0` или пустое значение тоже считается `1`, а с 3.8.0 допускаются **отрицательные** числа — так основной inbound можно поставить впереди остальных, не перенумеровывая их. Порядок применяется ко всем форматам выдачи — сырой текст, страница подписки, JSON и Clash. На порядок inbounds в самой панели это поле не влияет. Так выдача работает по умолчанию (режим **Порядок входящих подключений (по умолчанию)** настройки **«Порядок узлов подписки»**, `subHostSortMode`): хосты упорядочены внутри своего inbound, и хост более позднего inbound не может встать перед хостом более раннего. Режим **Порядок хостов** выстраивает подписку по порядку страницы **Hosts** сквозь все inbound — см. 10.2 и 10.4.

Поле редактируется в форме inbound рядом с настройками адреса в ссылке (share address) и синхронизируется на узлы по обычным правилам. Если хотя бы у одного inbound порядок отличается от 1, в списке Inbounds появляется компактная колонка **«Порядок»**.

10.2. Настройки сервера подписок

Все параметры подписки находятся в разделе настроек панели на вкладке **«Подписка»**. Ниже разобран каждый параметр; в скобках указан внутренний ключ настройки и значение по умолчанию.

Сам раздел разбит на вкладки: **«Панель»**, **«Информация»**, **«Профиль»**, **«Сертификаты»**, **«Happ»**, **«Clash / Mihomo»** и **«Incy»**. Поля заголовка подписки, URL поддержки, страницы профиля, объявления и каталога темы находятся на вкладке «Профиль»; правила маршрутизации Clash/Mihomo и Incy — на одноимённых вкладках; кодирование, шаблон примечания, инфо-конфиг, шаблоны статусов и интервал обновления подписки — на вкладке «Информация». Вкладка «Happ» содержит переключатель «Автоопределение заголовков Happ» и собственные подвкладки — они разобраны в 10.7. Пути, обратные URI и параметры JSON- и Clash-подписки задаются на отдельной странице настроек **«Форматы подписки»** (см. 10.4), балансировщики — на странице **«Балансировщики подписки»** (см. 10.3).

Основные параметры

Поле (UI)	Ключ	Значение по умолчанию	Описание
Включить подписку	`subEnable`	`true` (включено)	Запускает отдельный сервер подписок. Подсказка: «Функция подписки с отдельной конфигурацией». Если выключено — сервер подписок не стартует, и ни одна из ссылок не работает.
Прослушивание IP	`subListen`	пусто	IP-адрес, на котором сервер подписок принимает соединения. Подсказка: «Оставьте пустым по умолчанию, чтобы отслеживать все IP-адреса».
Порт подписки	`subPort`	`2096`	TCP-порт сервера подписок. Подсказка: «Номер порта для обслуживания службы подписки не должен использоваться на сервере» — порт должен быть свободен и не конфликтовать с панелью или Xray.
URI-путь	`subPath`	случайный (на новой установке)	Путь, по которому отдаются обычные подписки. Подсказка: «Должен начинаться с '/' и заканчиваться на '/'». Свежая база панели получает случайный путь из 16 строчных латинских букв и цифр (вида `/k3v9x0q2m7a4t1zp/`); у установки, обновлённой с прежней версии, остаётся её путь (раньше по умолчанию был `/sub/`).
Домен прослушивания	`subDomain`	пусто	Домен, по которому разрешён доступ к подписке (валидация Host). Подсказка: «Оставьте пустым по умолчанию, чтобы слушать все домены и IP-адреса». Если задан — запросы с другим Host отклоняются.
Порядок узлов подписки	`subHostSortMode`	`inbound-first`	Выпадающий список на вкладке «Панель»: **Порядок входящих подключений (по умолчанию)** (`inbound-first`) или **Порядок хостов** (`hosts-first`). Подсказка: «Используется порядок со страницы хостов для всех входящих подключений. Настройка задаёт порядок выдачи сервером; клиентские приложения могут сортировать узлы самостоятельно.». Действует на raw-, JSON- и Clash-подписки; подробности — в 10.4.

Случайные пути и безопасность. Начиная с 3.8.0 при создании новой базы панель генерирует три разных случайных пути — `subPath`, `subJsonPath` и `subClashPath` — и сохраняет их в базе, так что после перезапуска выданные ссылки не меняются. При обновлении существующей установки пути не трогаются (часто это прежние `/sub/`, `/json/` и `/clash/`). Сброс настроек (`/usr/local/x-ui/x-ui setting -reset`) снова генерирует случайные пути — ранее выданные ссылки подписки после этого перестают работать. Пути `/sub/` и `/json/` широко известны и легко подбираются: пока они используются, панель показывает предупреждение «Путь подписки по умолчанию "/sub/" широко известен — измените его.» и аналогичное для JSON. Рекомендуется задать собственный неочевидный путь.

TLS / сертификат

Поле (UI)	Ключ	По умолчанию	Описание
Путь к файлу публичного ключа сертификата подписки	`subCertFile`	пусто	Полный путь к файлу сертификата (`.crt` / `fullchain`). Подсказка: «Введите полный путь, начинающийся с '/'».
Путь к файлу приватного ключа сертификата подписки	`subKeyFile`	пусто	Полный путь к файлу приватного ключа. Подсказка: «Введите полный путь, начинающийся с '/'».

Если заданы оба пути и сертификат успешно загружается, сервер подписок работает по **HTTPS**. Если поля пустые либо сертификат не удалось прочитать — сервер падает обратно на **HTTP** (ошибка пишется в лог). Наличие валидного TLS также влияет на формирование базового URL: при порте 443 с TLS и порте 80 без TLS номер порта в ссылке опускается.

User-Agent внешней подписки

Поле (UI)	Ключ	По умолчанию	Описание
User-Agent внешней подписки	`externalSubUserAgent`	`v2rayNG/1.8.5`	Заголовок `User-Agent`, с которым панель скачивает **внешние подписки клиента** (вкладка «Links» в карточке клиента). Пустое значение — стандартный `v2rayNG/1.8.5`. Нужен провайдерам, отклоняющим неизвестных клиентов (#6613).

Это глобальная настройка панели; она не связана с полем **User-Agent** у отдельной «Подписки исходящих» (см. 11.12), которое действует только для этого outbound-источника. При загрузке внешних подписок панель также отправляет стабильный идентификатор устройства `X-HWID` (и `X-Device-OS`), чтобы провайдеры с лимитом по устройствам не отклоняли запрос; настройки это не требует (#6567, #6579).

Интервал обновления

Поле (UI)	Ключ	По умолчанию	Описание
Интервалы обновления подписки	`subUpdates`	`12`	Как часто (в часах) клиентское приложение должно перезапрашивать подписку. Подсказка: «Интервал между обновлениями в клиентском приложении (в часах)». С 3.5.0 поле принимает **0–525600** и показывает диапазон (прежний предел интерфейса 168 не совпадал с сервером и после апгрейда с 2.x блокировал сохранение любых настроек).

Значение передаётся клиенту в HTTP-заголовке `Profile-Update-Interval`; современные клиенты используют его как период автообновления конфигурации.

Формат и информация в ответе

Поле (UI)	Ключ	По умолчанию	Описание
Кодировать	`subEncrypt`	`true`	Подсказка: «Шифровать возвращенные конфиги в подписке». Технически это не шифрование, а **Base64-кодирование** всего тела обычной подписки (формат, который ожидает большинство клиентов). При выключении ссылки отдаются открытым текстом, по одной на строку.
Показывать идентификатор на каждой ссылке	`subShowIdentityOnAllLinks`	`false`	«Если включено, {{EMAIL}} и {{USERNAME}} остаются в примечании каждой ссылки тела подписки. Токены использования по-прежнему только на первой ссылке.»
Отдельный инфо-конфиг / фиктивный узел	`subInfoNodeEnable`	`false`	Выносит примечание, остаток трафика и дней в отдельный SOCKS-конфиг в начале выдачи; при истёкшем сроке или исчерпанном трафике подписка отдаёт только статусный конфиг. Подробнее — «Отдельный инфо-конфиг и статусные конфиги» ниже.
Month-end subscription expiry display	`subCalendarExpireInclusive`	`false`	Подпись в интерфейсе не переведена. Для продлений 1-го числа показывает окончание подписки последней секундой предыдущего месяца — см. «Дата окончания в конце месяца» ниже.
Шаблон истекшей подписки	`subExpiredTemplate`	см. ниже	«Шаблон для фиктивного узла, когда срок действия подписки истек.»
Шаблон исчерпания трафика	`subTrafficDepletedTemplate`	см. ниже	«Шаблон для фиктивного узла, когда лимит трафика исчерпан.»

Шаблон ремарки (Remark Template)

Отображаемое название (remark) каждой конфигурации в подписке формируется по **шаблону ремарки** — поле **«Шаблон примечания»** (`remarkTemplate`) на вкладке **«Информация»** настроек подписки. Прежний конструктор модели примечания (отдельный выбор частей inbound/email/external proxy и символа-разделителя) из интерфейса убран; теперь вы пишете произвольный формат имени и вставляете в него переменные. Значение по умолчанию — `{{INBOUND}}-{{EMAIL}}|📊{{TRAFFIC_LEFT}}|⏳{{DAYS_LEFT}}D` (то есть по умолчанию имя профиля содержит email клиента). Если поле оставить пустым, применяется прежняя (не настраиваемая через интерфейс) модель ремарки.

Переменные сгруппированы по разделам **Client**, **Traffic** и **Time & status** и показываются рядом с полем как кликабельные чипы `{{VAR}}` с подсказкой при наведении; клик вставляет токен в шаблон, доступен живой предпросмотр. Каждая переменная подставляется индивидуально для конкретного клиента в момент генерации подписки. Допускается и упрощённая запись в одинарных скобках (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` и т. п.) — панель сама приводит её к внутреннему формату `{{...}}` .

Доступные переменные:

- **Идентификация клиента:** `{{EMAIL}}` (синоним — `{{USERNAME}}`), `{{INBOUND}}` (ремарка самого inbound), `{{HOST}}` (ремарка хоста), `{{ID}}` (UUID), `{{SHORT_ID}}` (первые 8 символов UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Трафик:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (и их `*_BYTES` -варианты в точных байтах), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Срок и статус:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (`ГГГГ-ММ-ДД`), `{{JALALI_EXPIRE_DATE}}` (дата по календарю джалали), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .
- **Подключение (Connection):** `{{PROTOCOL}}` — протокол (VLESS, VMess, Trojan и т. д.), `{{TRANSPORT}}` — транспортная сеть (tcp, ws, grpc и т. д.), `{{SECURITY}}` — безопасность транспорта (TLS, REALITY, NONE; выводится заглавными). Как и переменные расхода и срока, эти три переменные действуют только в теле подписки и автоматически убираются из ремарки на отображаемых ссылках в панели (QR/«Информация») и на странице информации о подписке.

Шаблон можно разбить на сегменты вертикальной чертой `|` . Сегмент, в котором переменная даёт «безлимитное» значение (`∞`) — например `{{TRAFFIC_LEFT}}` или `{{DAYS_LEFT}}` у клиента без ограничений, — автоматически скрывается. Кроме того, блок с расходом трафика и сроком показывается один раз, на первой ссылке клиента, чтобы не дублироваться в каждой конфигурации.

Пример. Шаблон `{{EMAIL}}|📊{{TRAFFIC_LEFT}}|⏳{{DAYS_LEFT}}D` даст у клиента с остатком 42 ГБ и 7 днями имя вида `ivan@vpn 📊42.00GB ⏳7D` , а у безлимитного клиента — просто `ivan@vpn` (сегменты с `∞` опущены).

С версии 3.4.2 переменная `{{EMAIL}}` (и синоним `{{USERNAME}}`) выводится только на **первой** ссылке клиента в теле подписки — как и блок остатка трафика/срока; на остальных ссылках того же клиента она опускается. Переключатель **«Показывать идентификатор на каждой ссылке»** (`subShowIdentityOnAllLinks`) оставляет `{{EMAIL}}` / `{{USERNAME}}` на всех ссылках; переменные расхода и срока при этом по-прежнему выводятся только на первой.

На отображаемых в панели ссылках (QR-код и окна «Информация» на странице «Клиенты») и на странице информации о подписке email клиента присутствует в имени профиля: вид «inbound-host-email» при заданном хосте либо «inbound-email» без хоста. Сведения о трафике и сроке (а также переменные группы «Подключение») в эти отображаемые имена не подставляются — они работают только в теле подписки, которое получает VPN-клиент.

Если строка статистики трафика клиента «осиротела» после удаления и повторного создания inbound, переменная `{{TRAFFIC_USED}}` (и другие показатели расхода) больше не показывает `0.00B`: панель дополнительно ищет статистику по email клиента и подставляет корректный использованный трафик.

Отдельный инфо-конфиг и статусные конфиги

Переключатель **«Отдельный инфо-конфиг / фиктивный узел»** (`subInfoNodeEnable`, по умолчанию выключен) на вкладке «Информация» выносит сведения о подписке из имён рабочих конфигураций в отдельную «пустышку» — SOCKS-конфиг на `127.0.0.1:1080`, который ставится **первым** в выдаче. Его имя строится по шаблону примечания с суммарным трафиком и сроком всей подписки (если в подписке несколько email, берётся первый по алфавиту). У настоящих конфигураций переменные расхода, срока и статуса убираются из имён на всех ссылках — вместе с `{{EMAIL}}` / `{{USERNAME}}`, если не включено «Показывать идентификатор на каждой ссылке».

Когда срок подписки истёк или трафик исчерпан, выдача состоит **только** из статусного конфига, без рабочих:

- истёк срок — имя по шаблону **«Шаблон истекшей подписки»** (`subExpiredTemplate`), по умолчанию ⛔ `{{EMAIL}} | Expired: {{EXPIRE_DATE}}`;
- исчерпан лимит трафика — по шаблону **«Шаблон исчерпания трафика»** (`subTrafficDepletedTemplate`), по умолчанию 🚫 `{{EMAIL}} | Traffic Depleted | {{TRAFFIC_USED}}/{{TRAFFIC_TOTAL}}`.

В этих шаблонах работают те же переменные, что и в шаблоне примечания. Режим действует во всех трёх форматах: в обычной подписке это строка `socks://127.0.0.1:1080#<имя>`, в JSON — отдельный конфиг с SOCKS-outbound на `127.0.0.1:1080`, в Clash/Mihomo — прокси `socks5` (в группу `PROXY` он включается, только если других прокси нет). Никуда этот конфиг не ведёт: он лишь показывает пользователю состояние подписки в списке серверов приложения. Страница информации о подписке и ссылки в панели не меняются.

Метаданные профиля (заголовки ответа)

Эти строки передаются клиенту в HTTP-заголовках ответа и отображаются в VPN-клиенте как метаданные профиля. Все они по умолчанию пустые.

Поле (UI)	Ключ	Заголовок	Описание
Заголовок подписки	`subTitle`	`Profile-Title` (в Base64)	«Название подписки, которое видит клиент в VPN-клиенте». Для Clash также используется как имя импортируемого профиля через `Content-Disposition`.
URL поддержки	`subSupportUrl`	`Support-Url`	«Ссылка на техническую поддержку, отображаемая в VPN-клиенте».

Поле (UI)	Ключ	Заголовок	Описание
Страница профиля	`subProfileMode`	управляет `Profile-Web-Page-Url`	Режим ссылки на встроенную страницу профиля: **Без ссылки** (`none` , по умолчанию) — заголовок не отправляется; **Встроенная** (`builtin`) — ссылка на встроенную страницу текущего запроса подписки (см. 10.5); **Свой сайт** (`custom`) — адрес из поля «URL профиля» ниже.
URL профиля	`subProfileUrl`	`Profile-Web-Page-Url`	«Ссылка на ваш сайт, отображаемая в VPN-клиенте». Показывается и применяется только в режиме «Свой сайт»; пустое значение заголовок не отправляет.
Объявление	`subAnnounce`	`Announce` (в Base64)	«Текст объявления, отображаемый в VPN-клиенте». С 3.5.0 текст также показывается **информационным баннером наверху инфо-страницы подписки** (когда непуст), а кастомным шаблонам доступна переменная `announce` .

Обновление до 3.8.5. Раньше при пустом `subProfileUrl` в заголовок `Profile-Web-Page-Url` автоматически подставлялся фактический URL запроса подписки — ссылка на встроенную страницу профиля уходила всегда, даже без явной настройки, и раскрывала сам адрес подписки. С 3.8.5 при отсутствии сохранённого `subProfileMode` пустой `subProfileUrl` переводится в режим «Без ссылки», а непустой прежний адрес — в режим «Свой сайт» без повторного ввода. Чтобы вернуть прежнее поведение, выберите «Встроенная»; учтите, что встроенная страница раскрывает адреса подписки и конфигураций.

Кроме того, в каждом ответе передаётся заголовок `Subscription-Userinfo` с агрегированными данными трафика клиента: `upload` , `download` , `total` и `expire` (момент истечения в секундах). По нему клиент показывает остаток трафика и срок действия.

Дата окончания в конце месяца. Переключатель **«Month-end subscription expiry display»** (`subCalendarExpireInclusive` , вкладка «Информация», по умолчанию выключен; подпись в русском интерфейсе не переведена) меняет только значение `expire` в этом заголовке. Если у всех клиентов подписки продление назначено на **1-е число** (поле «Renew on day» = `1` , см. 8.1) и срок истекает ровно в полночь 1-го числа по часовому поясу панели, вместо этой полуночи отдаётся последняя секунда предыдущего месяца — например, 31 октября 23:59:59 вместо 1 ноября 00:00:00, — и приложение показывает окончание в последний день месяца. Реальный момент отключения, продления и их счётчик не меняются, как и дата на странице подписки и в переменных шаблона. Приложения считают время в своём часовом поясе и могут счесть подписку истёкшей на секунду раньше.

Маршрутизация (только для клиента Happ)

Поле (UI)	Ключ	По умолчанию	Описание
Включить маршрутизацию	`subEnableRouting`	`false`	«Глобальная настройка для включения маршрутизации в VPN-клиенте. (Только для Happ)». Передаётся в заголовке `Routing-Enable` .

Поле (UI)	Ключ	По умолчанию	Описание
Правила маршрутизации	`subRoutingRules`	пусто	«Вставьте готовый happ:// deeplink либо одну постоянную HTTPS-ссылку на deeplink или JSON. Панель обновляет удалённые правила в фоне и хранит последнее рабочее значение, поэтому запрос подписки не ждёт источник. (Только для Happ)». Передаётся в заголовке `Routing`. Если поле пустое, в этот заголовок уходит «Профиль маршрутизации для JSON-подписки» (см. 10.4).
Скрыть настройки сервера	`subHideSettings`	`false`	«Скрыть возможность просмотра и редактирования конфигурации сервера в VPN-клиенте. (Только для Happ)». Передаётся в заголовке `Hide-Settings: 1`.

С 3.8.0 эти поля находятся на вкладке «Happ», подвкладка **«Маршрутизация и правила»**, рядом с пресетами и визуальным конструктором правил (см. 10.7). Заголовки `Routing-Enable`, `Routing` и `Hide-Settings` получает любой клиент, запросивший подписку. Если включено «Автоопределение заголовков Happ» и запрос пришёл от Happ, выключенные переключатели передаются явно (`Routing-Enable: 0`, `Hide-Settings: 0`), чтобы отменить значения, присланные раньше.

Маршрутизация Incy (только для клиента Incy)

Для VPN-клиента **Incy** в настройках подписки есть отдельная вкладка **«Incy»**. С 3.9.0 на ней — переключатель **«Автоопределение заголовков Incy»** (`subIncyAppAutoDetect`): заголовки управления приложением уходят только клиентам с `User-Agent` вида `INCY/<версия>/<платформа>`. Поля сгруппированы в разделы **«Приложение»** (описание профиля, порядок сортировки, e-mail поддержки, ссылки «О программе»/Premium), **«Баннеры»** (текст, кнопка, цвета), **«Приватность»** (скрытие URL/проверки, режим экономии памяти iOS), **«Сеть»** (прокси по приложениям Android — режимы `bypass` / `proxy`; фрагментация TCP; шум UDP; предразрешение адреса сервера через DoH) и **«Маршрутизация»** (#6650). Прежние поля маршрутизации: переключатель **«Включить маршрутизацию»** (`subIncyEnableRouting`, по умолчанию выключено) и текстовое поле **«Правила маршрутизации»** (`subIncyRoutingRules`) формата `incy://routing/onadd/<base64>`. Когда маршрутизация включена и поле заполнено, эта строка дописывается отдельной строкой в тело подписки (формат raw) — так профиль маршрутизации доставляется клиенту Incy, не конфликтуя с заголовком `Routing` клиента Happ. Настройки действуют только для клиента Incy. С 3.10.2 (#6810) клиенту Incy, которому уходит строка маршрутизации Incy (совпал `User-Agent`, маршрутизация включена и правила не пустые), заголовки маршрутизации Happ (`Routing-Enable`, `Routing` и JSON-заголовок маршрутизации) не отправляются — профили двух приложений больше не смешиваются.

URI обратного прокси

Поле (UI)	Ключ	По умолчанию	Описание
URI обратного прокси	`subURI`	пусто	«Изменить базовый URI URL-адреса подписки для использования за прокси-серверами».

Если поле пустое, базовый адрес ссылки панель строит сама из домена и порта подписки (с учётом TLS). Если же подписка раздаётся через внешний реверс-прокси/CDN на другом домене или пути, в это поле задают итоговый базовый URI, и все ссылки будут строиться от него. Аналогичные отдельные поля есть для JSON (`subJsonURI`) и Clash (`subClashURI`).

Если задан только общий `subURI` , а отдельные поля для JSON и Clash оставлены пустыми, ссылки этих форматов на странице подписки наследуют схему и хост из `subURI` (а не порт sub-сервера и `http`) — так они совпадают с адресом обратного прокси.

С 3.9.0 за обратным прокси, присылающим только `X-Real-IP` , адрес сервера в ссылках больше не берётся из этого заголовка (это публичный IP самого подписчика): хост определяется по доверенному `X-Forwarded-Host` или по адресу, на который пришёл запрос (#6608).

Пример: подписка за реверс-прокси. Сама подписка слушает на `2096` , но снаружи доступна через nginx/CDN на `https://cfg.example.com/u/` . Чтобы ссылки в ответе строились от внешнего адреса, а не от внутреннего `домен:2096` , в поле «Reverse proxy URI» задают итоговый базовый URI:

```
Reverse proxy URI: https://cfg.example.com/u
```

Тогда конечная ссылка примет вид `https://cfg.example.com/u/ivan2025` . Для JSON- и Clash-форматов при необходимости заполняют отдельные поля `subJsonURI` и `subClashURI` тем же способом.

10.3. Балансировщики подписки (3.7.0)

Страница настроек **«Балансировщики подписки»** добавляет в **JSON-подписку** отдельные профили, которые сами выбирают лучший из ваших серверов. Каждый включённый балансировщик — это один дополнительный профиль в выдаче; в конфиге клиента он разворачивается в `routing.balancers` + `burstObservatory` , то есть выбор делает клиентское приложение по своим замерам, а не панель.

Поле	Описание
Примечание	Название профиля в списке клиента (плейсхолдер — `Авто · самый быстрый`)
Стратегия	Как выбирается сервер: **Минимальная нагрузка**, **Минимальный пинг**, **Случайный**, **По очереди**
Инбаунды	Какие inbound'ы участвуют — их endpoint'ы и станут кандидатами
Веса участников	Только для стратегии «Минимальная нагрузка»: для каждого выбранного inbound — число больше 0 (в форме от 0,1 с шагом 0,1). «Чем меньше вес, тем чаще выбирается участник; без значения вес равен 1»
Порядок	Позиция в списке подписки; нумерация общая с порядком inbound'ов, при равных номерах балансировщик идёт **после** inbound'а
Включён	Выключенный балансировщик в подписку не попадает

Управление — через `GET/POST /panel/api/sub-balancers` , `POST /panel/api/sub-balancers/:id` (изменение) и `DELETE /panel/api/sub-balancers/:id` (есть POST-алиас `/del` для клиентов, не умеющих DELETE). Веса передаются полем `memberWeights` — JSON-объектом вида `{"3": 0.5}` (id inbound → вес). Сервер отклоняет веса при любой стратегии, кроме `leastLoad` , и значения ⩽ 0 («Веса должны быть больше 0»), а веса inbound'ов, не выбранных в балансировщике, отбрасывает при сохранении.

В конфиг клиента они попадают как `costs` стратегии балансировщика; если ни один вес не задан, `costs` не выдаются.

> Балансировщик существует только в **JSON**-формате подписки: в plain-списке ссылок и в Clash-выдаче его нет — там нечего описывать балансировщиком.

10.4. Форматы вывода

Подписка может отдаваться в трёх независимых форматах, каждый со своим эндпоинтом, который можно включать/выключать отдельно. Все ответы сервера подписок — raw, JSON, Clash, `/mihomo/` и `/clash-legacy/` — уходят с заголовками `Cache-Control: no-cache, no-store, must-revalidate`, `Pragma: no-cache` и `Expires: 0`: тело несёт живые данные о трафике, сроке и маршрутизации, и кэш перед сервером подписок (CDN, обратный прокси) не должен отдавать устаревшее, например прежние правила маршрутизации после правки администратором (#6862). Настройки inbound'ов без списка клиентов сервер подписок держит в коротком кэше (10 с, правки в панели его сбрасывают), поэтому на установках с десятками тысяч клиентов запрос подписки заметно быстрее (#6758).

Адрес сервера и узлы в выдаче

Адрес сервера в ссылках подписки подставляется по той же стратегии адреса в ссылке, что и обычные ссылки и QR-коды в панели: «listen» — маршрутизируемый адрес привязки, «custom» — заданный пользовательский адрес (`shareAddr`), «node» (по умолчанию) — адрес узла. Для inbound без явно заданной стратегии выдача подписки не меняется. Это позволяет узловому inbound, привязанному к конкретному публичному IP, отдавать клиентам достижимый адрес. Стратегия применяется к сырому, JSON и Clash форматам.

Имя узла (Node) не добавляется к названию (remark) профиля в подписке: в клиентском приложении показывается только ремарка inbound, заданная администратором, без внутреннего суффикса вида `@имя-узла`. Чтобы различать одноимённые записи в мультиузловой подписке, задавайте им разные ремарки вручную либо используйте управляемые хосты (Hosts) с собственными Remark.

Если из-за рассинхронизации между узлами один и тот же клиент попал в служебный JSON inbound дважды, выдача подписки автоматически устраняет такие дубликаты по email во всех трёх форматах, поэтому повторяющиеся профили в выдаче не появляются.

Управляемые хосты (Hosts)

Раздел **Hosts** (пункт бокового меню; сводная страница с количеством Total/Enabled/Disabled и списком) задаёт переопределения адреса для ссылок подписки. Для каждого inbound можно добавить один или несколько **хостов** — эндпоинтов, которые подставляются в выдаваемые клиенту subscription-ссылки **вместо адреса, порта и TLS-параметров самого inbound**. Это удобно, чтобы раздавать трафик через CDN или релей, не меняя сам inbound.

С версии 3.5.0 хост — это «группа»: одна запись охватывает **несколько inbound** и **несколько адресов** сразу. Действия списка (включение/выключение/удаление/перестановка) и API работают с группами (строковый

`groupId`); появился массовый эндпоинт `POST /panel/api/hosts/bulk/add` (`{"inboundIds": [...], "hosts": [...], ...}`), а `list` / `get` / `update` / `del` / `setEnable` оперируют объектами группы.

У каждого хоста (группы) задаются:

- **Remark** и **«Описание»** (до 64 символов; с 3.8.0 передаётся клиенту Happ как подпись сервера, см. 10.7), привязка к **Inbounds** (мульти-выбор с поиском; минимум один), переключатель **Enable** и назначение на узлы (**Nodes**).
- **Address** — с 3.5.0 это **список адресов** (ввод тегами; разделители — запятая, точка с запятой, пробел; плейсхолдер `cdn.example.com, cdn2.example.com:443`). У каждой записи может быть свой встроенный `:порт` (в том числе IPv6 в скобках `[::1]:443`); выпадающая подсказка предлагает адреса, уже использованные другими хостами. Запись может быть CIDR-диапазоном (см. «Диапазоны CIDR» ниже). Пустой список — наследуется адрес самого inbound (в списке это оранжевая метка **Inherits**). **Port** (`0` — наследуется порт inbound) служит портом по умолчанию для записей без встроенного порта; **Tags** (учитываются только в RAW-подписке).
- Вкладка **Security** — `same` / `tls` / `none` / `reality` с SNI (одним или пулом, см. «Пул SNI» ниже), отпечатком (fingerprint), ALPN, **наборами шифров** (`cipherSuites`, с 3.9.0 — мультивыбор с вводом тегами; для JSON-подписки заменяют наборы inbound'а, пустое поле наследует их), привязкой сертификата (pinned-cert), `allowInsecure` и ECH. Поля SNI и Fingerprint, а также ALPN, закреплённый SHA-256, «Проверять сертификат пира по имени», «Разрешить небезопасное» и ECH видны и при `same` — так ранее заданные переопределения можно очистить; в списке Fingerprint есть вариант «Нет» (без uTLS). Если хост переводит REALITY-inbound на `tls` или `none`, параметры REALITY (`pbk`, `sid`, `spx`, `pqv`, а также `sni` и `fp` цели REALITY) из ссылки убираются, а SNI и отпечаток самого хоста подставляются заново. С 3.10.0 в **JSON-подписке** заданные у хоста ECH (`echConfigList`), «Проверять сертификат пира по имени» (`verifyPeerCertByName`) и закреплённые SHA-256 (`pinnedPeerCertSha256`, строкой через запятую) выводятся на верхнем уровне `tlsSettings`, где их читает Xray, — раньше они оставались во вложенном служебном блоке `settings`, ядро их не видело, и подключения через такой хост не работали (#6743). `allowInsecure` в JSON, как и у самого inbound'а, не выдаётся (в Xray он удалён); выдача Clash/Mihomo не изменилась.
- Вкладка **Advanced** — Host header, Path, маршрут VLESS, Mux, Sockopt, Final Mask и исключение хоста из отдельных форматов подписки (raw / json / clash). С 3.5.0 **Final Mask хоста попадает и в raw-ссылки** (`fm=`; раньше — только в JSON/Clash): TCP/UDP-маски хоста добавляются к маскам inbound, QUIC-параметры хоста берутся, только если их нет у inbound. Переключатель **Allow insecure** теперь действует и для **Hysteria/Hysteria2** (`insecure=1` в ссылке, `skip-cert-verify: true` в Clash). С 3.9.0 конфиги **WireGuard, AmneziaWG и TUIC** во всех выдачах (raw, скачиваемый `.conf`, API ссылок, экспорт, QR, карточки клиента и inbound'а) объявляют адреса из привязанных включённых **хостов**, а не адрес самой панели; при нескольких хостах клиент получает по конфигу на каждый.
- Вкладка **Clash (mihomo)** — версия IP, Mihomo X25519, перемешивание хостов (Shuffle host).

Пул SNI (#6923). Поле SNI хоста принимает несколько значений (ввод тегами; разделители — запятая, точка с запятой, пробел; плейсхолдер `sni.example.com, sni2.example.com`), поэтому отдельный хост на каждый домен не нужен. При нескольких SNI каждая сгенерированная конфигурация получает случайный SNI из пула. Переключатель **Один случайный SNI для всех конфигураций** (`sniSharedRandom`; виден, когда в пуле больше одного SNI) выбирает один SNI на весь хост (группу) в каждом запросе подписки: все адреса и клиенты этой выдачи получают одно и то же значение. Пул работает в raw-, JSON- и Clash-подписках и в предпросмотре ссылок панели. Поле `sni` остаётся первым значением пула — API-клиенты и откатанный бинарник, знающие только одно поле, продолжают работать, а `sni` вне пула становится первым. Хост с

одним SNI выдаётся как раньше. Переключатели **Использовать адрес как SNI** и **Оставить SNI пустым** применяются уже после выбора из пула.

Диапазоны CIDR (#6784). Адрес хоста может быть CIDR-диапазоном (IPv4 или IPv6, например `203.0.113.0/24`) — для сервера за диапазоном CDN или anycast не нужно заводить хост на каждый адрес. Если в списке адресов есть CIDR, в форме появляется поле **Количество случайных IP** (`randomCount`, 1–100, по умолчанию 1). Каждый запрос подписки разворачивает диапазон в столько уникальных случайных адресов (сетевой и широковещательный адреса пропускаются; если в диапазоне меньше адресов, берётся сколько есть); при нескольких адресах к примечанию добавляется суффикс « - 1», « - 2» и т. д. Развёртка работает во всех форматах и в обоих порядках подписки, а также в предпросмотре и QR-кодах панели и в экспорте конфигов WireGuard, AmneziaWG, TUIC и MTProto.

Наборы шифров, отпечаток и ECH в ссылках (#6825). Наборы шифров хоста или TLS-inbound попадают в share-ссылки параметром `cs` (в `vmess://` — полем `cs` JSON-тела ссылки); раньше они доходили только до JSON- и Clash-подписок. Наборы действуют лишь при отпечатке `unsafe`, а Xray читает пустой отпечаток как `chrome`, поэтому если `cs` есть, а итоговый отпечаток пуст, raw-ссылки (включая `vmess://`), JSON-подписка и ссылки панели сами ставят `fp=unsafe`; явно заданный отпечаток не меняется. Оба разборщика ссылок (серверный и импорт outbound'а в панели) читают `cs` обратно в `tlsSettings.cipherSuites`. В поле наборов шифров хоста доступны пресеты, а в поле ECH — помощник динамического ECH (см. 7.3).

Mihomo X25519 (#6803). Переключатель убирает ML-KEM у REALITY-хоста (подсказка: «Только для Mihomo: REALITY-рукопожатие с обычным X25519, без ML-KEM (support-x25519mlkem768: false). Для серверов на Xray-core ниже 26.9.8, которым ML-KEM не нужен.»): Clash-прокси получает `reality-opts.support-x25519mlkem768: false`, а ссылка `vless://` выдаётся без подсказки `support-x25519mlkem768`. Он нужен, когда сервер на Xray-core ниже 26.9.8 принимает обычный X25519 или когда DPI сети отбрасывает вторую часть крупного ClientHello с ML-KEM и тем самым глушит сервер для клиента. Без переключателя все REALITY-прокси несут `true`. Значения переключателя, сохранённые тогда, когда он ни на что не влиял, при обновлении один раз сброшены, чтобы существующие подписки не изменились молча. Отпечаток REALITY-хоста попадает и в Clash-, и в JSON-подписку, а каждый адрес хоста получает собственную копию параметров потока: пустой SNI, отпечаток, заголовок Host или путь одного хоста больше не наследуются от другого.

Привязка клиентов к хостам (#6620). Правила хостов действуют на inbound целиком, и по умолчанию все его клиенты получают одни и те же адреса. Клиента можно привязать к одному или нескольким хостам (поле «Переопределение правила хоста» в форме клиента, см. 8.1): его raw-, JSON- и Clash-подписки содержат только эти хосты, в любом из двух порядков; клиент без привязки выдаётся как прежде. Привязка закрывается при ошибке (fail closed): если привязанные хосты удалены, выключены или исключены из формата, клиент в этом формате не получает ни одной ссылки, но остаётся в учёте трафика. Правка хоста сохраняет идентификаторы его строк, поэтому привязки переживают редактирование. Удаление хоста закрывает привязку только на том inbound, которому он служил: клиент на inbound'ах A и B, привязанный к хосту на A, после его удаления теряет ссылку на A, а ссылка на B остаётся; идентификатор, которому не соответствует ни один хост, по-прежнему закрывает выдачу везде. Через API привязка — поле клиента `clientHostRuleIds` (числовые `id` строк хостов, список даёт `GET /panel/api/hosts/raw`): отсутствие поля оставляет сохранённую привязку, пустой список снимает её.

В списке колонка **Endpoint** показывает адреса чипами (первый виден, остальные — в поповере «+N»), а колонка **Inbounds** — чипы inbound, раскрашенные по протоколу. С 3.5.0 хосты упорядочиваются **глобально**

(по порядку сортировки, затем по примечанию), а не в пределах inbound; порядок меняют кнопками **Вверх** и **Вниз** или перетаскиванием строк (#6895); массовые включение/выключение/удаление сохраняются. В подписках этот порядок по умолчанию действует только внутри каждого inbound (#6894). Чтобы raw-, JSON- и Clash-подписки следовали порядку страницы сквозь все inbound, выберите режим **Порядок хостов** в настройке **«Порядок узлов подписки»** (10.2): хосты идут по порядку страницы (при равенстве — по порядку inbound и по `id` хоста), а ссылки inbound'ов без подходящих хостов — следом за ними. Управляемые хосты приходят на смену прежнему массиву External Proxy. Хосты, импортированные из `streamSettings.externalProxy` inbound'а, остаются синхронными с ним: правка externalProxy при обновлении inbound'а переносится на импортированные хосты, которые остались ровно такими, как при импорте. Хост, который администратор изменил, выключил, удалил или сделал общим, заново не импортируется рядом с его версией, а поток без ключа `externalProxy` хосты не трогает (#6831). В списке **Inbounds** рядом с inbound выводятся примечания привязанных к нему включённых хостов (длинный перечень сокращается, полный — во всплывающей подсказке; по ним работает и поиск).

Маршрут VLESS (VLESS route). С версии 3.4.2 это одно число `0–65535` (а не список портов; подсказка — «одно значение VLESS route (0-65535), зашиваемое в UUID, например 443; пусто — без него», плейсхолдер `443`). Заданное значение реально «зашивается» в UUID каждой генерируемой подписки (raw / JSON / Clash): Xray читает байты 6-7 UUID и маскирует их перед аутентификацией, поэтому клиент по-прежнему совпадает. Пустое или некорректное значение UUID не меняет.

MTProto через хосты. С 3.8.0 публичные ссылки `tg://proxy` MTProto-inbound (в подписке, при копировании, в QR-коде и при экспорте) строятся по управляемым хостам — по ссылке на каждый адрес, — поэтому прокси за обратным прокси или NAT анонсируется с внешним адресом и портом. Прежний пользовательский адрес для ссылок MTProto-inbound при обновлении автоматически переносится в хост; адрес хоста без порта получает порт самого inbound.

Обычные ссылки (SUB) — Base64 / plain text

Базовый формат, эндпоинт `subPath` (на новой установке — случайный путь, см. 10.2). Включён всегда (при включённой подписке в целом). Возвращает список share-ссылок — по одной на строку: `vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`, для AmneziaWG — `vpn://`, для MTProto — `tg://proxy`, для TUIC (с 3.8.0) — `tuic://` (см. 5.13). Inbound'ы с транспортом XDRIVE и протокола MASQUE share-ссылок не имеют и в этот формат не попадают — см. «XDRIVE и MASQUE — только через JSON-подписку» ниже. При включённой опции «Кодировать» (`subEncrypt`) весь список кодируется в Base64; при выключенной — отдаётся открытым текстом. Этот формат понимают практически все клиенты (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ и др.).

Пример: тело ответа с выключенным «Кодировать». При `subEncrypt = false` эндпоинт `subPath` отдаёт открытый текст — по одной ссылке на строку:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

При `subEncrypt = true` (по умолчанию) тот же список целиком кодируется в Base64 и отдаётся одной строкой — именно этот вид ожидает большинство клиентов.

JSON-подписка (Xray JSON)

Эндпоинт `subJsonPath` (на новой установке — случайный путь; у обновлённой обычно `/json/`), включается отдельной галочкой **«JSON-подписка»** на вкладке «Панель». Путь, обратный URI и параметры формата задаются на странице **«Форматы подписки»**: в карточке «JSON-подписка» и на вкладках **Final Mask**, **Mux**, **«Прямое подключение»** и **«Блокировка соединений»**.

Поле (UI)	Ключ	По умолчанию	Описание
JSON-подписка	`subJsonEnable`	`false`	«Включить/отключить JSON-эндпоинт подписки независимо.».

Возвращает полную клиентскую конфигурацию Xray (Xray JSON) — отдельный документ на каждую конфигурацию подписки; такой формат понимают клиенты на ядре Xray. В каркас входят локальные inbound'ы SOCKS (порт `10808`) и HTTP (`10809`), outbound'ы `direct` и `block`, DNS и маршрутизация. С 3.8.0 локальные inbound'ы привязаны к `127.0.0.1` (раньше клиент, запускавший профиль как есть, открывал прокси без пароля на всех интерфейсах, а iOS-клиенты не получали через него трафик), а SOCKS-inbound ради совместимости снова объявлен протоколом `socks` (в 3.7.0 — `mixed`). С 3.5.0 в JSON-подписку попадают и клиенты **нативных WireGuard-inbound** (`secretKey`, `address`, `peers[]` с `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs`, `mtu`) — раньше они молча пропускались. Конфигурации **AmneziaWG** и **TUIC** в JSON не выдаются — для них используйте обычную подписку или Clash/Mihomo. С 3.10.0 VLESS-inbound, в настройках которого не сохранено поле `encryption` (создан старой версией панели или через API только с `"decryption": "none"`), выдаётся с `"encryption": "none"`: раньше в JSON уходила пустая строка, и Xray отказывался загружать такой конфиг; в ссылку `vless://` в этом случае тоже пишется `encryption=none` вместо пропуска параметра (#6781). У Hysteria-inbound с прыжками по портам (устаревший `quicParams.udpHop`, который выдаётся маской `udphop`) и несколькими хостами маска `udphop` с 3.10.0 попадает в конфиг **каждого** хоста — раньше её получал только первый, а остальные уходили без прыжков по портам. Дополнительные параметры формата:

- **Mux** (`subJsonMux`, по умолчанию пусто) — JSON-настройки мультиплексирования (Mux), которые внедряются в outbound каждого потока JSON-подписки. «Передача нескольких независимых потоков данных в одном соединении.». Для VLESS-клиентов с flow (`xtls-rprx-vision`) TCP-мультиплексирование несовместимо, поэтому в их outbound `concurrency` принудительно ставится в `-1`: mux.cool для TCP выключен, а параметры XUDP сохраняются.
- **Final Mask** (`subJsonFinalMask`, по умолчанию пусто) — «Маски finalmask xray (TCP/UDP) и настройки QUIC, добавляемые в каждый поток JSON-подписки. Требуется свежая версия xray на клиенте.». Настраивается через подполя: «Пакеты» (`packets`), «Длина» (`length`), «Интервал» (`interval`), «Макс. разбиение» (`maxSplit`), «Шумы» (`noises`: «Тип»/ `type`, «Пакет»/ `packet`, «Задержка (мс)»/ `delayMs`, «Применить к»/ `applyTo`, кнопка «+ Шум»), а также «Параллелизм» (`concurrency`), «Параллелизм xudp» (`xudpConcurrency`) и «xudp UDP 443» (`xudpUdp443`).
- **Прямое подключение** и **Блокировка соединений** (одноимённые вкладки; обе пишут простые правила в `subJsonRules`, по умолчанию пусто). «Прямое подключение» отправляет выбранные IP и домены в outbound `direct` (при включении подставляются `geoip:private`, `geoip:ir` и `geosite:category-ir`). «Блокировка соединений» — «Блокирует соединения с выбранными доменами или диапазонами IP через outbound blackhole»: поля «Заблокированные домены» (при включении — `geosite:category-ads-all`) и «Заблокированные IP-адреса»; правила блокировки ставятся перед правилами прямого подключения. Если задан профиль маршрутизации (следующий пункт), эти правила не применяются.

- **Профиль маршрутизации для JSON-подписки** (`subJsonRoutingRules` , по умолчанию пусто; карточка «JSON-подписка»). Принимает JSON профиля маршрутизации в формате Happ/INCY (`DirectSites` , `DirectIp` , `ProxySites` , `ProxyIp` , `BlockSites` , `BlockIp` , `DomainStrategy` , `RemoteDNSDomain` / `RemoteDNSIP` , `DomesticDNSDomain` / `DomesticDNSIP` , `DnsHosts` , `RouteOrder`), диплинк `happ://routing/onadd/…` или `incy://routing/onadd/…` либо `https://` -URL (метка «HTTPS URL»: панель обновляет источник в фоне и хранит последнее рабочее значение). Заданный профиль **заменяет** блоки `dns` и `routing` в каждом JSON-конфиге, включая балансировщики: домены из `DirectSites` резолвятся «домашним» DNS (по умолчанию `https://77.88.8.8/dns-query`), остальное — удалённым (`https://8.8.8.8/dns-query`); правила идут в порядке block → direct → proxy (меняется ключом `RouteOrder` , например `block-proxy-direct`), всё прочее уходит в `proxy` , `domainStrategy` по умолчанию `IPIfNonMatch` . Так маршрутизация доходит и до клиентов, игнорирующих заголовки (например, Happ и INCY с клиентскими балансировщиками). Если на вкладке «Happ» поле «Правила маршрутизации» пустое, этот же профиль отправляется в заголовке `Routing` (JSON и URL — в виде диплинка `happ://`), чтобы приложение скачало нужные geo-файлы. Неразбираемый профиль подписку не ломает: в лог пишется предупреждение, а конфиги выдаются с последним удачным профилем или, если его нет, с маршрутизацией каркаса.
- **DNS-серверы** (`subJsonDns` , по умолчанию пусто; карточка «JSON-подписка») — блок `dns` для всех JSON-конфигов, включая балансировщики и инфо-конфиг: полный объект `dns` Xray (например, `{"servers": ["https://dns.google/dns-query", "tls://1.1.1.1"]}`) или просто массив `servers` . Значение проверяется при сохранении по схеме Xray: нужен хотя бы один сервер с непустым адресом, `clientIp` — только IP-адрес; иначе сохранение отклоняется с ошибкой «JSON subscription DNS is invalid». Поле заменяет и DNS профиля маршрутизации, но только внутри конфигов: клиенты, применяющие заголовок `Routing` , используют резолверы профиля. Пусто — встроенный резолвер каркаса (`8.8.8.8`).

Clash / Mihomo-подписка (YAML)

Эндпоинт `subClashPath` (на новой установке — случайный путь; у обновлённой обычно `/clash/`), включается отдельной галочкой; путь и обратный URI задаются в карточке «Подписка Clash / Mihomo» на странице «Форматы подписки».

Поле (UI)	Ключ	По умолчанию	Описание
Подписка Clash / Mihomo	`subClashEnable`	`false`	Включает генерацию YAML-конфигурации для клиентов Clash и Mihomo.
Включить маршрутизацию	`subClashEnableRouting`	`false`	«Добавлять глобальные правила маршрутизации Clash/Mihomo в сгенерированные YAML-подписки.».
Глобальные правила маршрутизации	`subClashRules`	пусто	«Правила Clash/Mihomo, добавляемые в начало каждой YAML-подписки перед MATCH,PROXY.». Вместо YAML можно указать одну ссылку `https://` на шаблон Mihomo: панель скачивает её с кэшем (10 минут, до 2 МиБ) и берёт разделы `rules` , `rule-providers` и `proxy-groups` .

Поле (UI)	Ключ	По умолчанию	Описание
DNS из удалённых правил	`subClashRemoteDNS`	`false`	«Если в глобальных правилах маршрутизации указана ссылка, брать из неё также разделы dns и sniffer. Включайте только для шаблона, которому доверяете.» Публичные шаблоны Mihomo приходят вместе с блоком `dns` (диапазоны fake-ip, политика nameserver), от которого зависят их правила; вставленный в поле YAML сохраняет свой `dns`, а у ссылки он без этого переключателя отбрасывался. Включённый переключатель переносит в профиль `dns` и `sniffer` шаблона, то есть шаблон решает, куда клиент отправляет каждый DNS-запрос, — поэтому он выключен по умолчанию. Если значение `dns` или `sniffer` не является словарём, пропускается только оно, а не весь шаблон.

Ответ отдаётся с типом `application/yaml; charset=utf-8`. Если задан «Заголовок подписки» (`subTitle`), он передаётся также в заголовке `Content-Disposition` (`attachment; filename*=UTF-8''<title>`), чтобы Clash-клиент назвал импортируемый профиль этим именем.

При включённой Clash-подписке сервер отвечает ещё по двум фиксированным адресам:

- `/mihomo/<subId>` — тот же полный профиль Mihomo, что и по `subClashPath`;
- `/clash-legacy/<subId>` — упрощённый профиль для старого Clash (Clash for Windows). В нём остаются только VMess (транспорт TCP, WebSocket или gRPC; шифр `auto`, `aes-128-gcm`, `chacha20-poly1305` или `none`), Trojan с TLS (TCP, WebSocket или gRPC) и Shadowsocks без транспорта и TLS с поддерживаемым шифром (`chacha20-poly1305` переименовывается в `chacha20-ietf-poly1305`); Shadowsocks с плагином `obfs` отбрасывается. Узлы REALITY и все прочие протоколы отбрасываются, незнакомые старому Clash поля удаляются, глобальные правила маршрутизации Clash не подмешиваются. Если совместимых узлов не осталось, сервер отвечает `422` с текстом «no Clash for Windows-compatible proxies found; use the Mihomo subscription for modern proxy types».

Алиас не регистрируется, если его путь совпадает с одним из настроенных путей подписки (в лог пишется предупреждение); если `subClashPath` сам равен `/mihomo/`, полный профиль отдаётся по нему.

Формат генерируемых ссылок и YAML поддерживается в актуальном состоянии для современных клиентов: Shadowsocks-2022 (SS2022) больше не кодирует userinfo в Base64; ссылки Shadowsocks с http-обфускацией выдаются в формате SIP002 с плагином `obfs-local`; в Clash/Mihomo такой узел (Shadowsocks-inbound с HTTP-заголовком на TCP) выдаётся с `plugin: obfs`, `plugin-opts` с `mode: http` и заголовком Host запроса (`host`), если он задан (#6838). Mihomo всегда запрашивает `GET /`, а xray-core отвергает запрос с путём вне списка путей заголовка, поэтому узел выдаётся, только если заголовок принимает `/` (пустой список путей тоже подходит); иначе, как и узлы других протоколов с таким заголовком, он пропускается; в legacy-профиль он не попадает. Для подписок Clash/Mihomo реализован полный набор полей XHTTP, а числовые параметры XMUX inbound'а (например, `maxConnections`; по умолчанию в панели `3`) попадают в `reuse-settings` целыми числами, нулевые не выдаются (#6843). С 3.5.0 в Clash/Mihomo-подписку попадают и клиенты **нативных WireGuard-inbound** (поля `private-key`, `public-key`, `pre-shared-key`, `persistent-keepalive`, `ip` / `ipv6`, `mtu`, `dns`) — раньше они молча пропускались. Отдельных настроек это не требует — ссылки просто корректнее распознаются клиентами. С

3.8.0 в Clash/Mihomo-подписку попадают **AmneziaWG** (прокси `type: wireguard` с блоком `amnezia-wg-option`: `jc` / `jmin` / `jmax`, `s1` – `s4`, `h1` – `h4`, `i1` – `i5`, а при заданных параметрах третьей версии, например `header-protection-key`, — ещё `version: 3`; `mtu` выдаётся всегда, с учётом `S4`) и **TUIC** (`type: tuic` с `uuid`, `password`, `congestion-controller`, `udp-relay-mode`, `reduce-rtt`, `alpn`, `sni`; см. 5.13). У узлов REALITY в `reality-opts` выставляется `support-x25519mlkem768: true` (Xray 26.9.8+ отклоняет REALITY-рукопожатие без ключа ML-KEM), (у хоста с включённым **Mihomo X25519** — `false`), а `client-fingerprint` равен `chrome`, если отпечаток не задан ни у хоста, ни у inbound'а. Адрес сервера в прокси выдаётся «голым» хостом — IPv6 без квадратных скобок, которые Mihomo не разбирал. С 3.9.0 у клиента на нескольких туннельных inbound'ах (например, WireGuard/AmneziaWG на разных узлах) каждая подписка несёт ключи, `AllowedIPs` и keepalive именно своего inbound'а, а не общую запись последнего синхронизированного (#6653). С 3.10.2 Hysteria2 с маской Salamander в режиме **Gecko** (задан диапазон размеров пакетов) выдаётся как `obfs: gecko` с целыми `obfs-min-packet-size` / `obfs-max-packet-size` (границы те же, что у ссылки `hysteria2://`). Раньше в YAML попадал только пароль — обычный salamander, и клиент Clash/Mihomo не проходил рукопожатие с Gecko-сервером.

> Примечание: в этой сборке поддерживаются именно три формата — обычные ссылки (Base64/текст), JSON (Xray JSON) и Clash/Mihomo (YAML, включая алиасы `/mihomo/` и `/clash-legacy/`). Отдельного формата Outline в сервере подписок нет.

XDRIVE и MASQUE — только через JSON-подписку (3.10.0)

У inbound'а с транспортом **XDRIVE** (`network: "xdrive"` при любом протоколе, см. 6.10) и у inbound'а протокола **MASQUE** (см. 5.15 и 6.11) нет формата share-ссылки: генератор ссылок сервера подписок (`GetLink`) и панели (`genLink`) возвращает для них пустое значение. Поэтому такие inbound'ы не попадают в обычную подписку (Base64/текст), в окна «QR-код» и «Информация о клиенте» и в список конфигураций на странице подписки; выдача Clash/Mihomo их тоже пропускает. Клиенты получают конфиг **только из JSON-подписки**: для MASQUE — outbound `masque` с `security: "tls"` и `masqueSettings`, где `user` — email клиента, `pass` — его пароль, а `path` берётся из inbound'а; для XDRIVE — `streamSettings` inbound'а вместе с блоком `xdriveSettings`. Без включённой JSON-подписки (`subJsonEnable`) выдать таким клиентам конфиг через подписку нечем.

> ⚠️ **JSON-подписка раздаёт секреты хранилища XDRIVE.** Оба конца XDRIVE работают с одним и тем же хранилищем и одними учётными данными, поэтому блок `xdriveSettings` уходит в JSON-подписку целиком — с `secrets` (для Google Drive это Client ID, Client Secret и Refresh Token), `remoteFolder` и шаблоном хранилища. **Каждый клиент такого inbound'а получает эти учётные данные** и с ними доступ к хранилищу в пределах их прав. Заводите для XDRIVE отдельный аккаунт/проект с минимальными правами, выдавайте подписку только доверенным пользователям, а при утечке отзывайте и перевыпускайте токен.

10.5. Страница информации о подписке и QR-коды

Если открыть **любую из трёх ссылок подписки** — raw, JSON или Clash — в браузере (или явно добавить к URL параметр `?html=1` либо `?view=html`, либо отправить заголовок `Accept: text/html`), сервер вместо «сырого» ответа отдаёт визуальную **страницу информации о подписке** («Информация о подписке»). До

3.5.0 так работала только основная ссылка `/sub/`, а `/json/` и `/clash/` отдавали в браузер сырой JSON/YAML. VPN-клиенты по-прежнему получают машинный ответ, так как они не запрашивают HTML.

Чтобы скачать из браузера сам JSON или YAML, добавьте к адресу JSON- или Clash-подписки `?view=raw`: ответ придёт вложением `subscription.json` / `subscription.yaml`. Начиная с 3.8.0 такой запрос проходит проверку лимита устройств (HWID) так же, как запрос VPN-клиента.

С 3.8.5 страница подписки перестроена: вверху — кольцевой индикатор использования с процентом (или «∞» для безлимитных подписок) и остатком либо использованным трафиком, рядом — сетка краткой статистики (дней до окончания, срок действия, статус, загружено/отправлено/лимит, последний онлайн). Остальное содержимое разбито на вкладки **«Подписка»**, **«Приложения»** и **«Конфиги»**. В подвале вкладки «Подписка» — ссылка техподдержки (`subSupportUrl`, если задана) и интервал автообновления клиента (`subUpdates`, см. 10.2). Для языков `fa–IR` и `ar–EG` страница отображается справа налево; адреса, идентификаторы и объёмы данных остаются слева направо. С 3.10.0 страница оформлена в теме самой панели: те же цвета фона и карточек для светлой, тёмной и ультратёмной темы и синий акцент панели вместо собственной фиолетово-бирюзовой палитры с анимированным фоном, сеткой и «стеклянной» карточкой; содержимое и вкладки не изменились.

Страница (одностраничное приложение, собранное Vite) показывает:

- **Информацию о подписке** (блок Descriptions):
  - «ID подписки» — значение `subId`;
  - «Статус» — «Активна», «Неограниченно», «Истёк», «Трафик исчерпан» или «Неактивна» (для выключенного клиента). С 3.8.5 три причины неактивности (истёк срок, исчерпан трафик, клиент выключен) различаются, а не сводятся к одному значению «Неактивна»;
  - «Загружено» и «Отправлено» — объёмы трафика;
  - «Общий лимит» — лимит трафика или `∞`, если не ограничен;
  - «Срок действия» — дата окончания или «Бессрочно»;
  - остаток трафика и время последнего онлайна.
  - Даты отображаются по григорианскому или джалали-календарю в зависимости от настройки «Calendar Type» панели (`datepicker`, по умолчанию `gregorian`).
- **Ссылки на подписку**: для каждого включённого формата — отдельная строка с цветным тегом (зелёный **SUB**, фиолетовый **JSON**, золотой **CLASH**), кнопкой копирования и кнопкой **QR-кода** (всплывающее окно, размер 240 px). Строка с JSON и CLASH появляется только если соответствующий формат включён в настройках.
- **Индивидуальные ссылки** («Скопировать ссылку»): полный список отдельных конфигураций, входящих в подписку, каждая со своим тегом протокола, кнопкой копирования и QR-кодом (QR не строится только для ссылок с постквантовым ключом — `pqv` или ключом VLESS-шифрования ML-KEM-768, см. 8.3).
- **Кнопка «Копировать все конфигурации»** (над списком индивидуальных ссылок): одним нажатием копирует в буфер обмена сразу все ссылки конфигураций (каждую с новой строки), не требуя копировать их по одной; по завершении показывается уведомление «Все конфигурации скопированы».
- **Кнопки быстрого импорта в приложения** (вкладка «Приложения», переключатель платформы Android/iOS — по умолчанию платформа посетителя по `User–Agent`): у каждого приложения одна кнопка

«Добавить». Для Android она открывает deep-link v2box, v2rayNG (`v2rayng://install-config?url=…`), Sing-box (`sing-box://import-remote-profile?url=…`), V2RayTun (`v2raytun://import/…`), Happ (`happ://add/…`) или Incy (`incy://add/…`); для iOS — Shadowrocket (параметр `flag=shadowrocket`), v2box (`v2box://install-sub?url=…&name=…`), Streisand (`streisand://import/…`), V2RayTun, Happ или Incy. С 3.8.5 Sing-box и V2RayTun на Android и V2RayTun на iOS тоже открывают приложение (раньше только копировали ссылку), а NPV Tunnel убран из списка обеих платформ.

Страница информации отдаётся с заголовками запрета кэширования (`Cache-Control: no-cache`), чтобы клиент всегда видел актуальные данные о трафике и сроке действия.

Язык страницы подписки хранится в браузере отдельно от языка панели (своя cookie): смена языка на странице подписки не переключает панель, и наоборот. При первом открытии страница берёт язык панели, если он уже выбран в этом браузере, иначе — язык браузера; даты на странице форматируются на выбранном языке.

10.6. Кастомные шаблоны страницы подписки

Начиная с 3.3.0 можно заменить стандартную страницу-лендинг подписки собственным HTML-шаблоном. По умолчанию по адресу подписки отдаётся встроенная страница, но если указать каталог со своим шаблоном, панель будет рендерить его и подставлять в него актуальные данные клиента (трафик, срок действия, ссылки и т. д.).

Важно: панель **не поставляет** готовых шаблонов — свою тему нужно создать самостоятельно. Инструкция по созданию и список доступных переменных — в `docs/custom-subscription-templates.md`.

Где включается

Каталог темы задаётся в настройках панели:

Настройки → **Подписка** → **раздел «Информация о подписке»**, поле **«Каталог темы подписки»** (`subThemeDir`).

Описание поля в интерфейсе: «Абсолютный путь к папке с пользовательским шаблоном (index.html/sub.html) для страницы подписки (например, /etc/3x-ui/sub_templates/my-theme/). Оставьте пустым, чтобы использовать страницу по умолчанию.»

Рядом в том же разделе расположены смежные настройки, значения которых доступны в шаблоне:

В описании поля «Каталог темы подписки» есть ссылка **«Руководство по шаблонам ↗»** на документацию по созданию собственных шаблонов оформления страницы подписки.

- **«Заголовок подписки»** (`subTitle`) — название, видимое клиенту;
- **«URL поддержки»** (`subSupportUrl`) — ссылка на техподдержку.

Параметр настройки

Параметр	Значение по умолчанию	Назначение
`subThemeDir`	`""` (пусто)	Абсолютный путь к каталогу с вашим HTML-шаблоном. Пусто = встроенная страница по умолчанию.

Как подставить свой шаблон

1. Создайте на сервере папку для темы (где угодно), например `/etc/3x-ui/sub_templates/my-theme/`.
2. Положите внутрь HTML-файл с именем `index.html` или `sub.html`.

Пример: путь к теме. Финальная раскладка на сервере и значение поля в настройках:

```
/etc/3x-ui/sub_templates/my-theme/
└── index.html        (или sub.html — он имеет приоритет)
```

```
Настройки → Подписка → «Каталог темы подписки»:
/etc/3x-ui/sub_templates/my-theme/
```

Путь должен быть **абсолютным** (начинаться с `/`). Если в папке нет ни `index.html`, ни `sub.html`, панель отдаст встроенную страницу. 3. В панели откройте **Настройки** → **Подписка** и впишите **абсолютный** путь к этой папке в поле «Каталог темы подписки». 4. Сохраните настройки.

Поведение выбора файла и рендеринга:

- Если в каталоге есть `sub.html`, используется именно он; иначе берётся `index.html`. То есть `sub.html` имеет приоритет над `index.html`.
- Шаблон рендерится стандартным движком Go `html/template`.
- Разобранный шаблон **кэшируется** и перечитывается с диска только при изменении времени модификации файла. Поэтому правки шаблона подхватываются без перезапуска панели, но без накладных расходов на чтение/парсинг при каждом запросе.
- Ответ формируется в буфер целиком и только потом отдаётся клиенту: если шаблон упадёт во время выполнения, частично сгенерированная (битая) страница не уйдёт пользователю.

Поведение по умолчанию и откат (fallback)

- Поле пустое → отдаётся встроенная SPA-страница (данные внедряются в `window.__SUB_PAGE_DATA__`).
- Путь не существует или это не каталог → используется страница по умолчанию.
- В каталоге нет ни `index.html`, ни `sub.html` → в лог пишется предупреждение «subThemeDir set but no usable template found», отдаётся страница по умолчанию.
- Файл шаблона есть, но не парсится → в лог пишется ошибка «custom template parse failed», отдаётся страница по умолчанию.
- Ошибка при выполнении шаблона → в лог пишется «custom template execution failed», отдаётся страница по умолчанию.

То есть любая проблема с кастомным шаблоном не «ломает» подписку — панель всегда деградирует к встроенной странице. Все страницы подписки (и кастомная, и стандартная) отдаются с заголовками запрета кэширования (`Cache-Control: no-cache, no-store, must-revalidate`), чтобы клиенты всегда получали свежие данные о трафике и сроке.

Доступные переменные шаблона

В контекст шаблона передаётся набор данных клиента подписки. Обращение — через `{{ .имя }}` :

Переменная	Тип	Описание
`{{ .sId }}`	string	ID подписки (UUID).
`{{ .enabled }}`	bool	Включён ли клиент/подписка.
`{{ .download }}`	string	Форматированный объём загрузки (напр. «2.5 GB»).
`{{ .upload }}`	string	Форматированный объём отправки.
`{{ .total }}`	string	Форматированный общий лимит трафика.
`{{ .used }}`	string	Форматированный использованный трафик (download + upload).
`{{ .remained }}`	string	Форматированный остаток трафика.
`{{ .expire }}`	int64	Срок действия — Unix-время в **секундах** (`0` = бессрочно). Для JS `Date` умножьте на 1000.
`{{ .lastOnline }}`	int64	Время последнего онлайна — Unix-время в **миллисекундах** (`0` = ни разу).
`{{ .downloadByte }}`	int64	Загрузка в точных байтах.
`{{ .uploadByte }}`	int64	Отправка в точных байтах.
`{{ .totalByte }}`	int64	Общий лимит в точных байтах.
`{{ .subUrl }}`	string	URL страницы подписки.
`{{ .subJsonUrl }}`	string	URL JSON-конфигурации подписки.
`{{ .subClashUrl }}`	string	URL конфигурации Clash/Mihomo.
`{{ .subTitle }}`	string	Заголовок подписки из настроек (может быть пустым).
`{{ .subSupportUrl }}`	string	URL поддержки из настроек (может быть пустым).
`{{ .links }}`	[]string	Список строк-конфигов (VMess, VLESS, и т. д.). Перебор: `{{ range .links }} … {{ end }}` .
`{{ .emails }}`	[]string	Список email, относящихся к подписке.
`{{ .datepicker }}`	string	Текущий формат календаря панели: `gregorian` или `jalali` (берётся из настройки «Тип календаря»; если пусто — `gregorian`).

Минимальный пример тела шаблона, использующего часть переменных:

```
<h1>{{ .subTitle }}</h1>
<p>Использовано: {{ .used }} из {{ .total }} (осталось {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Пример: дата окончания из `expire`. Поле `{{ .expire }}` — это Unix-время в **секундах**, поэтому для JavaScript его умножают на 1000; значение `0` означает «без срока»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Без срока'
    : 'До ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Обратите внимание: `{{ .lastOnline }}` задаётся уже в **миллисекундах** — его умножать на 1000 не нужно.

10.7. Интеграция с клиентом Happ

Вкладка **«Happ»** настроек подписки собирает то, что панель передаёт приложению Happ в HTTP-заголовках ответа подписки (во всех трёх форматах). Вверху вкладки — переключатель **«Автоопределение заголовков Happ»** (`subHappAutoDetect`, по умолчанию выключен): «Автоматически добавлять заголовки и маршрутизацию для клиентов с User-Agent Happ». Клиентом Happ считается запрос, в `User-Agent` которого есть слово `happ` (регистр не важен). Пока переключатель выключен, заголовки управления приложением (все поля подвкладок ниже, кроме трёх полей маршрутизации из 10.2) **не отправляются никому**; когда включён — только клиентам Happ. Пустое поле, выключенный переключатель и список, в котором ничего не выбирали (даже если он показывает значение по умолчанию), заголовка не дают. Заголовки берутся из настроек при запуске сервера подписок, поэтому после сохранения нужен перезапуск панели.

«Маршрутизация и правила». Кроме полей «Включить маршрутизацию», «Правила маршрутизации» и «Скрыть настройки сервера» (см. 10.2) здесь есть:

Поле (UI)	Ключ	Заголовок	Описание
Пресеты маршрутизации	—	—	Список пресетов и кнопка **«Применить шаблон»**: в «Правила маршрутизации» записывается готовый диплинк, прежнее содержимое заменяется. С 3.9.0: **«Обход сайтов Ирана (Iran Bypass)»**, **«Китай напрямую (Bypass-CN)»** (DoH-DNS, `geosite:geolocation-cn`), **«Полный прокси (Global)»**, **«Всё через прокси, локальная сеть напрямую»** (новый LAN-пресет: `GlobalProxy`, `DirectSites: geosite:private`, `DirectIp: geoip:private`) и **«Отключить маршрутизацию»** (`happ://routing/off`). Рядом — отдельный переключатель **«Добавить AdBlock»** (`subHappIncludeAdblock`): подмешивает `geosite:category-ads-all` в блок-список выбранного пресета (прежний отдельный пресет AdBlock убран).

Поле (UI)	Ключ	Заголовок	Описание
Редактор правил маршрутизации	—	—	Кнопка открывает окно редактора, которое **загружает текущий профиль маршрутизации и даёт его править**: вкладка **«Основные правила»** (по правилу на строку) и **«Расширенный редактор»** (полный JSON профиля). Кнопка генерации записывает результат в «Правила маршрутизации» как `happ://routing/onadd/<base64>`. Пришёл на смену прежнему односторонному конструктору по спискам доменов/IP (#6545).
Режим No-Limit	`subHappNoLimit`	`No-Limit-Enabled: 1`	«Повышает лимит оперативной памяти xray-core в Happ для стабильности и производительности (бета).»

«Ссылки подписки». Переключатель **«Зашифрованные ссылки подписки»** (`happLinkEnable`, по умолчанию выключен) — «Разрешает создавать зашифрованные ссылки Happ в окне QR-кода клиента. Адреса подписок обрабатываются локально.» Заголовков он не добавляет и от автоопределения не зависит; сама зашифрованная ссылка описана в 8.3.

«Баннеры и уведомления».

Поле (UI)	Ключ	Заголовок	Описание
Текст информационного баннера	`subHappSubInfoText`	`Sub-Info-Text`	Баннер в верхней части приложения, до 200 символов. Цвет и кнопка баннера отправляются только вместе с текстом.
Цвет баннера	`subHappSubInfoColor`	`Sub-Info-Color`	`blue` (по умолчанию), `green` или `red`.
Текст кнопки баннера	`subHappSubInfoButtonText`	`Sub-Info-Button-Text`	Подпись кнопки, до 25 символов.
Ссылка кнопки баннера	`subHappSubInfoButtonLink`	`Sub-Info-Button-Link`	URL, открывающийся по кнопке.
Баннер об окончании подписки	`subHappSubExpire`	`Sub-Expire: 1`	Баннер об истёкшей подписке, когда у пользователя закончился трафик или срок действия.
Ссылка для продления подписки	`subHappSubExpireButtonLink`	`Sub-Expire-Button-Link`	URL кнопки «Продлить»; отправляется только при включённом баннере об окончании.
Уведомление об окончании подписки	`subHappNotificationExpire`	`Notification-Subs-Expire: 1`	Напоминание пользователю за 3 дня до окончания подписки.

«Сетевые настройки и TUN».

Поле (UI)	Ключ	Заголовок	Описание
Режим TUN	`subHappTunMode`	`Tun-Mode`	Сетевой стек TUN на десктопе: «По умолчанию» (заголовок не отправляется), «Системный (стандартный стек ОС)» — `system`, «gVisor (пользовательский стек)» — `gvisor`.
Движок TUN	`subHappTunType`	`Tun-Type`	Ядро TUN-подключения на десктопе: `singbox`, `tun2proxy`, `default` («По умолчанию (Happ TUN)») или `xray` («Xray TUN»).
Исключения маршрутов (CIDR)	`subHappExcludeRoutes`	`Exclude-Routes`	Подсети через запятую, трафик которых идёт мимо туннеля, например `192.168.0.0/16, 10.0.0.0/8`.
Исключить push-уведомления Apple (APNS)	`subHappExcludeApns`	`Exclude-Apns-Enable: true`	Трафик уведомлений Apple идёт напрямую — для надёжной доставки на iOS.
Метод проверки пинга	`subHappPingType`	`Ping-Type`	`proxy` (через прокси, задержка GET), `proxy-head` (через прокси, HEAD), `tcp` или `icmp`.
Автоподключение при запуске	`subHappAutoConnect`	`Subscription-Autoconnect: 1`	Подключаться к VPN при запуске приложения.
Критерий автоподключения	`subHappAutoConnectType`	`Subscription-Autoconnect-Type`	`lowestdelay` (по умолчанию), `lastused` или `random`; отправляется только при включённом автоподключении.
Авторизация локального прокси	`subHappLocalProxyAuth`	`Socks-Auth-Mode` / `Http-Auth-Mode`	(3.9.0) Авторизация локальных SOCKS/HTTP-прокси Happ на устройстве: **Авто (случайные учётные данные)** (`auto`), **Отключена** (`disable`) или **Не отправлять** (пусто — оставить настройку клиента). Прежде прокси шли без авторизации, и любое приложение на устройстве могло достучаться до туннеля и узнать адрес сервера (#6628).

«Внешний вид и темы».

Поле (UI)	Ключ	Заголовок	Описание
Цветовая тема клиента	`subHappColorProfile`	`Color-Profile`	Своя цветовая тема для iOS в виде JSON-строки (например, `{"serverRowBackgroundColor":"#21003D67"}`) или `resetcolors` для возврата стандартных цветов. Кнопки «Сброс», «Violet» и «Turquoise» подставляют готовые значения.

«Миграция и управление».

Поле (UI)	Ключ	Заголовок	Описание
Идентификатор провайдера (Provider ID)	`subHappProviderId`	`ProviderID`	Идентификатор провайдера для управления приложением, удалённой конфигурации и миграции.
Новый URL подписки (Миграция)	`subHappNewUrl`	`New-Url`	Получив его, Happ автоматически переключается на новый адрес подписки.
Резервный URL подписки (Fallback)	`subHappFallbackUrl`	`Fallback-Url`	Адрес на случай недоступности основного.
Обязательный HWID	`subHappAlwaysHwid`	`Subscription-Always-Hwid-Enable: 1`	Запрещает пользователю отключать отправку HWID в настройках Happ — пригодится вместе с лимитом устройств (см. 8.1).

«Прокси приложений Android».

Поле (UI)	Ключ	Заголовок	Описание
Прокси для приложений (Android)	`subHappPerAppMode`	`Per-App-Proxy-Mode`	«Выкл» (`off`, по умолчанию; заголовок не отправляется), «Вкл (только выбранные приложения)» — `on`, «Обход (исключить выбранные приложения)» — `bypass`.
Пакеты приложений Android	`subHappPerAppList`	`Per-App-Proxy-List`	Имена пакетов через запятую, например `org.telegram.messenger`; отправляется только при режиме, отличном от «Выкл».

Подпись сервера в списке Happ задаётся полем **«Описание»** управляемого хоста (см. 10.4): в ссылках VLESS, Trojan, Shadowsocks и (с 3.10.0) Hysteria/Hysteria2 оно добавляется к имени как `#Имя?serverDescription=<base64>`, в VMess — полем `serverDescription`. До 3.10.0 Hysteria-серверы такого хоста показывались в Happ со стандартной подписью вида «Hysteria | hysteria | TLS» (#6740); в ссылки `tuic://` описание пока не добавляется. Эта подпись от автоопределения не зависит.

11. Xray: маршрутизация, outbounds, DNS и расширения

Раздел **«Настройки Xray»** — это редактор шаблона конфигурации Xray-core, на основе которого панель генерирует итоговый `config.json` для запуска ядра. Подсказка к разделу шаблона: *«На основе шаблона создаётся конфигурационный файл Xray.»* В отличие от inbounds (которые хранятся отдельно в БД и подставляются в шаблон при сборке конфигурации), всё остальное — логи, маршрутизация, outbounds, DNS, политика, статистика — задаётся именно здесь.

> Важно: значение шаблона хранится в БД под ключом `xrayTemplateConfig`. При сохранении панель прогоняет его через ряд автоматических преобразований (см. 11.11). Любой синтаксически некорректный JSON будет отклонён с ошибкой *«xray template config invalid»*.

Расположение в меню: «Исходящие» и «Маршрутизация»

«Исходящие» (Outbounds) и **«Маршрутизация» (Routing)** — это отдельные пункты бокового меню (сразу под «Хосты», над «Настройки панели»), у каждого свой адрес: `/outbound` и `/routing`. Прямые ссылки на эти страницы и перезагрузка страницы работают как ожидается. В подменю **«Конфигурации Xray»** при этом остаются только: Основное, Балансировщик, DNS и Расширенный шаблон. В описании ниже разделы 11.3 и 11.4 соответствуют страницам «Маршрутизация» и «Исходящие».

11.1. Структура редактора: вкладки/режимы

Редактор предлагает несколько режимов отображения шаблона (фильтры по разделам JSON):

Режим	Что показывает
Основное	Базовые секции (Лог, базовая маршрутизация, основные настройки)
Расширенный шаблон	Полный JSON-шаблон Xray
Все	Все секции одновременно

Логические группы настроек внутри редактора:

- **Основные настройки** (подсказка: *«Эти параметры описывают общие настройки»*).
- **Лог** (см. 11.10).
- **Базовые соединения**: блокировки и прямые маршруты.
- **Входящие** (подсказка: *«Изменение шаблона конфигурации для подключения определенных клиентов»*).
- **Исходящие** (см. 11.4).
- **Балансировщик** (см. 11.5).
- **Маршрутизация** (подсказка: *«Важен приоритет каждого правила!»*, см. 11.3).
- **DNS / Fake DNS** (см. 11.6).

11.2. Основные настройки (General)

Freedom Protocol Strategy

Поле	Подпись	Описание	По умолчанию
`FreedomStrategy`	**Настройка стратегии протокола Freedom**	Стратегия вывода сети для прямого (freedom) outbound. Подсказка: *«Установка стратегии вывода сети в протоколе Freedom»*. Управляет полем `streamSettings.sockopt.domainStrategy` outbound-а `direct` с протоколом `freedom` — там, где его читает ядро; при выборе значения прежние ключи `settings.domainStrategy` и `settings.targetStrategy` удаляются.	**`AsIs`** (адрес не резолвится, передаётся в исходном виде): в эталонном шаблоне ключа нет, а выбор `AsIs` удаляет его из `sockopt`.

`domainStrategy` для freedom (значения Xray-core): `AsIs` (не резолвить домен на стороне сервера), а также семейство `UseIP` / `UseIPv4` / `UseIPv6` и их «принудительные» варианты `ForceIP*`, заставляющие выходной сервер резолвить домен и подключаться по полученному IP. Меняйте на `UseIPv4`, если на выходном сервере нет IPv6 или нужно принудительно ходить только по IPv4. Протокол сравнивается без учёта регистра (`"Freedom"` — тоже freedom). Если тег `direct` занят outbound-ом другого протокола, этот селектор и переключатель **Freedom Happy Eyeballs** неактивны: второй outbound с тегом `direct` ядро не загрузит (*existing tag found*), поэтому панель его не создаёт.

Freedom Happy Eyeballs (IPv4/IPv6)

Поле	Подпись	Описание
`FreedomHappyEyeballs`	**Freedom Happy Eyeballs (IPv4/IPv6)**	Подсказка: *«Двухстековый набор для прямого (freedom) исходящего — полезно на выходных серверах с IPv4 и IPv6.»* Включает алгоритм Happy Eyeballs (одновременная попытка по обоим семействам адресов) для freedom-outbound.

Отдельный Happy Eyeballs есть и в **настройках dial (sockopt) конкретного outbound**: с 3.5.0 он действительно включается (раньше конфиг сериализовался как выключенный и переключатель «слетал»). «Try delay (ms)» по умолчанию теперь **250**; явно заданный `0` (= выключено) сохраняется; также доступны Prioritize IPv6, interleave (1) и maxConcurrentTry (4). | try delay | (подсказка) | *«Миллисекунды перед попыткой другого семейства адресов. 150–250 мс — хорошая отправная точка.»* Задержка перед переключением на альтернативное семейство адресов. Рекомендуемый диапазон — 150–250 мс. |

Overall Routing Strategy

Поле	Подпись	Описание	По умолчанию
`RoutingStrategy`	**Настройка маршрутизации доменов**	Общая стратегия разрешения DNS для маршрутизации. Подсказка: *«Установка общей стратегии маршрутизации разрешения DNS»*. Управляет полем `routing.domainStrategy`.	В эталонном шаблоне `routing.domainStrategy` = **`AsIs`**.

`routing.domainStrategy` определяет, как сопоставляются IP-правила маршрутизации с доменными запросами: `AsIs` (только доменные правила, без резолва), `IPIfNonMatch` (если домен не совпал с правилами — резолвить и проверить IP-правила), `IPOnDemand` (резолвить сразу при встрече IP-правила). Для работы IP-правил (например, `geoip:*`) при доменном запросе обычно требуется `IPIfNonMatch`.

Скрипт маршрутизации (с 3.10.0)

Сразу под «Настройка маршрутизации доменов» — поле **Скрипт маршрутизации** (`routing.script`, плейсхолдер `routing.lua`): путь к Lua-файлу с функцией `HandleRoute`; пустое поле удаляет ключ из шаблона. Ядро не принимает скрипт маршрутизации рядом с правилами и `domainStrategy`, а служебное правило `api → api` панель держит всегда, поэтому **сохранение шаблона с непустым `routing.script` отклоняется**. Подробно — в 11.14.

Outbound Test URL

Поле	Подпись	Описание	По умолчанию
`outboundTestUrl`	**URL для теста исходящего**	URL для проверки связности при тестировании outbound. Подсказка: *«URL для проверки подключения исходящего»*. Хранится отдельно от шаблона, под ключом `xrayOutboundTestUrl`.	**`https://www.google.com/generate_204`**

Значение проходит санитизацию. При самом тесте outbound оно дополнительно проверяется как публичный URL — это защита от SSRF: пользователь не может подсунуть произвольный (в т. ч. внутренний) URL через клиента, тестовый URL всегда берётся из серверной настройки. Пустое значение при сохранении/тесте заменяется на дефолтный `generate_204`.

Default Outbound (с 3.5.0)

Первый пункт вкладки **«Базовая маршрутизация»** — селектор **Default Outbound**: «Трафик, не совпавший ни с одним правилом маршрутизации, использует этот outbound (Xray берёт первый outbound в списке)». В списке — `direct`, `blocked` и все существующие теги; по умолчанию `direct`. Выбор **перемещает указанный outbound на первую позицию** шаблона; отсутствующие `direct` / `blocked` создаются на лету, а при переключении не удаляются.

Block BitTorrent

Поле	Подпись	Описание
`Torrent`	**Заблокировать BitTorrent**	Добавляет в `routing.rules` правило, отправляющее трафик с `protocol: ["bittorrent"]` на outbound `blocked`. В эталонном шаблоне это правило присутствует по умолчанию.

Ограничения соединения (Connection Limits)

Подсказка: *«Политики уровня соединения для пользователей уровня 0. Оставьте поле пустым, чтобы использовать значение Xray по умолчанию.»* Эти параметры пишутся в `policy.levels.0`.

Поле	Подпись	Описание	По умолчанию
`connIdle`	**Тайм-аут простоя** (секунд)	*«Закрывает соединение после простоя в течение указанного числа секунд. Уменьшение значения быстрее освобождает память и файловые дескрипторы на нагруженных серверах (по умолчанию в Xray: 300).»*	пусто → дефолт Xray **300**
`bufferSize`	**Размер буфера** (КБ)	*«Размер внутреннего буфера на соединение в КБ. Установите 0, чтобы минимизировать использование памяти на серверах с малым объёмом ОЗУ (значение Xray по умолчанию зависит от платформы).»* Плейсхолдер: **«авто»**.	пусто → зависит от платформы; `0` — минимизировать

Пример (`policy.levels.0`). Поля из этой группы пишутся в политику уровня 0. На нагруженном сервере с малым ОЗУ можно ускорить освобождение ресурсов так:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Здесь соединение закрывается уже после 120 с простоя (вместо дефолтных 300), а `bufferSize: 0` минимизирует расход памяти на буферы. Оставленное пустым в форме поле просто не попадёт в JSON — и Xray применит своё значение по умолчанию.

11.3. Правила маршрутизации (routing)

Список правил `routing.rules`. **Порядок критичен** (*«Важен приоритет каждого правила!»*): правила оцениваются сверху вниз, срабатывает первое совпавшее. Подсказка: *«Перетащите для изменения порядка»*. Кнопки управления порядком: **Первый**, **Последний**, **Поднять вверх**, **Опустить вниз**.

> Если в разделе «Основное» задан **Скрипт маршрутизации** (с 3.10.0, см. 11.14), над правилами показывается предупреждение *«В разделе «Основное» задан скрипт маршрутизации, поэтому Xray*

> *игнорирует эти правила.»* Правила остаются в шаблоне, но ядро их не применяет; сохранить шаблон с таким скриптом панель не даст (см. 11.14).

Каждое правило имеет `type: "field"`. Кнопки: **Создать правило**, **Редактировать правило**. Подсказка к полям-спискам: *«Элементы, разделённые запятыми»*.

На странице «Маршрутизация» кнопки **«Импорт правил»** и **«Экспорт правил»** собраны в выпадающее меню **«ещё»** (more) — как и на странице «Исходящие». Кнопка **«Экспорт правил»** не скачивает файл сразу, а открывает модальное окно с предпросмотром JSON и кнопками **«Копировать»** и **«Скачать»**: содержимое можно просмотреть перед сохранением. Аналогично работает экспорт исходящих на странице «Исходящие».

Route Tester (тестировщик маршрута)

На вкладке Routing есть подвкладка **Route Tester** — она спрашивает у работающего Xray, какой outbound обработал бы конкретное соединение, не отправляя реального трафика. Укажите домен или IP, порт, сеть (TCP/UDP) и, при необходимости, inbound и перехваченный протокол (`http` / `tls` / `quic` / `bittorrent`), затем нажмите **Test Route**. Решение берётся напрямую из живого движка маршрутизации.

В ответе показывается подобранный outbound, а при использовании балансировщика — ещё и тег балансировщика. Если ни одно правило не совпало, тестировщик сообщает, что трафик идёт в outbound по умолчанию (первый в списке `outbounds`). Это удобно для проверки порядка правил перед тем, как полагаться на них.

Включение и отключение отдельного правила

Отдельное правило маршрутизации можно временно **отключить** переключателем, не удаляя его. В таблице правил есть столбец **«Включить»** с переключателем (Switch), а в форме правила поле **«Включить»** — тоже переключатель. Отключённое правило не попадает в итоговую конфигурацию Xray, но сохраняется в шаблоне и его можно снова включить в любой момент.

Служебное правило статистики (`inboundTag: ["api"] → outboundTag: "api"`) отключить нельзя — его переключатель заблокирован, чтобы не сломать учёт трафика панели (см. 11.11).

Поля формы правила

Поле формы	Подпись (RU)	JSON-поле	Описание
Комментарий	**Комментарий**	`comment`	Заметка только для панели, до 200 символов (со счётчиком). Показывается в столбце **«Комментарий»** таблицы правил (полный текст — во всплывающей подсказке), на мобильных — плашкой в карточке правила. В шаблоне сохраняется, но при сборке конфигурации удаляется, как и флаг `enabled` , — в Xray-core не попадает.
Источник	**Источник**	`source`	IP-адреса/подсети источника. Список через запятую.
Порт источника	**Порт источника**	`sourcePort`	Порт(ы) источника.

Поле формы	Подпись (RU)	JSON-поле	Описание
Пункт назначения	**Пункт назначения**	`domain` + `ip` + `port`	Целевые домены, IP и порты. Домены поддерживают префиксы `domain:`, `full:`, `regexp:`, `keyword:`, а также `geosite:*`; IP — `geoip:*` и CIDR.
Сеть	—	`network`	`tcp`, `udp` или `tcp,udp`. С 3.4.2 в конфиг пишется строчными (`tcp` / `udp`), а в таблице маршрутов отображается прописными (`TCP` / `UDP`); пусто — любая сеть.
Протокол	—	`protocol`	`http`, `tls`, `bittorrent` (определяется по sniffing).
Пользователь	**Пользователь**	`user`	Фильтр по e-mail/идентификатору пользователя.
Атрибуты / Значение	**Атрибуты** / **Значение**	`attrs`	Атрибуты HTTP-заголовков для сопоставления.
VLESS route	**VLESS route**	—	Маршрутизация по полю route для VLESS.
Теги входящих	**Теги входящих**	`inboundTag`	Один или несколько тегов inbound, к которым применяется правило (включая встроенные `api`, и DNS-тег из настроек DNS). В списках inbound отображается как «tag (remark)», если у inbound задано отдельное примечание, иначе — просто тег; в сохранённых правилах по-прежнему хранятся только теги.
Тег исходящего	**Тег исходящего** / **Исходящее подключение**	`outboundTag`	Куда направить совпавший трафик.
Тег балансировщика	**Тег балансировщика** / **Балансировщик**	`balancerTag`	Подсказка: *«Направляет трафик через один из настроенных балансировщиков нагрузки».*

> Взаимоисключение `outboundTag` и `balancerTag`: *«Невозможно одновременно использовать balancerTag и outboundTag. При одновременном использовании будет работать только outboundTag.»* В одном правиле задавайте либо тег исходящего, либо тег балансировщика.

Встроенные правила эталонного шаблона

В стандартном `config.json` секция `routing` содержит три правила (в этом порядке):

1. `inboundTag: ["api"] → outboundTag: "api"` — служебное правило для gRPC-API статистики панели.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` — блокировка приватных диапазонов.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` — блокировка BitTorrent.

> Правило `api → api` всегда автоматически поднимается на позицию 0 при сохранении (см. 11.11), чтобы запрос статистики не «съело» вышестоящее catch-all-правило.

Пример правила. Отправить весь трафик к российским сайтам и приватным сетям напрямую (мимо прокси), а остальное — на балансировщик. Порядок важен: правило-«направить напрямую» должно стоять выше catch-all. В `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Чтобы IP-правила (`geoip:ru`) срабатывали и для доменных запросов, на верхнем уровне маршрутизации обычно нужно `routing.domainStrategy: "IPIfNonMatch"` (см. 11.2).

Преднастроенные группы маршрутизации (Базовые соединения)

В режиме «Базовые соединения» панель помогает собрать типовые правила из готовых списков:

Группа	Поля	Подсказка
Блокировка по протоколам/сайтам	—	*«Настройте, чтобы клиенты не имели доступа к определенным протоколам»*
Блокировка по странам	**Заблокированные IP-адреса**, **Заблокированные домены**	*«Эти параметры будут блокировать трафик в зависимости от страны назначения.»*
Прямые соединения	**Прямые IP-адреса**, **Прямые домены**	*«Прямое соединение означает, что определенный трафик не будет перенаправлен через другой сервер.»*
Правила IPv4	—	*«Эти параметры позволят клиентам маршрутизироваться к целевым доменам только через IPv4»*
Правила WARP	—	*«Эти опции будут направлять трафик в зависимости от конкретного пункта назначения через WARP.»*
Маршрутизация NordVPN	—	*«Эти опции будут направлять трафик в зависимости от конкретного пункта назначения через NordVPN.»*

MTProto-inbound: маршрутизация Telegram-трафика через Xray

У MTProto-inbound есть переключатель **«Route through Xray»** (по умолчанию выключен) и необязательный выбор **Outbound**. При включении панель добавляет в конфиг Xray петлевой SOCKS-мост с тегом самого inbound, и mtg направляет Telegram-трафик через него. После этого исходящим Telegram-трафиком управляет маршрутизатор: его можно сопоставлять обычными правилами на вкладке Routing по тегу inbound либо принудительно направить в выбранный outbound или балансировщик через поле **Outbound**. Оставьте **Outbound** пустым, чтобы решение принимали правила маршрутизации. В списке **Outbound** нет

blackhole-outbound-ов (например, `blocked`; протокол сравнивается без учёта регистра): такой выбор выглядел бы рабочим, но молча отбрасывал бы весь Telegram-трафик inbound-а.

Что добавилось в правилах в 3.7.0

- **Выбор клиента вместо ручного ввода.** Поле `user` в пользовательском правиле заполняется из списка клиентов панели («Select users»); опечатка в email больше не превращается в правило, которое молча ни на что не срабатывает.
- **Просмотр гео-категорий прямо из правила.** Рядом с полем домена/IP появилась кнопка просмотра баз: слева список категорий, справа — их содержимое с постраничной прокруткой (`{count} entries`, `{count} subnets`) и поиском. Значения из правила проверяются против базы, и панель отличает четыре разные беды: **нет такой категории** («Not in the database»), **нет такого атрибута** («Attribute not found, the rule would match nothing»), **не тот тип базы** для этого поля и **файла базы нет — добавьте его в разделе гео-баз**. Подробнее — в разделе 15.
- **Удалённый набор правил по URL.** Правила можно держать снаружи: панель периодически подтягивает их и хранит последнее валидное значение (см. также раздел 10.2 — там же по URL обновляются правила для Happ, Incy и Clash).

11.4. Outbounds (исходящие подключения)

Список `outbounds`. Кнопки: **Создать исходящее подключение**, **Изменить исходящее подключение**. Подсказка: *«Изменение шаблона конфигурации, чтобы определить исходящие подключения для этого сервера»*.

В эталонном шаблоне два обязательных outbound-а:

- `protocol: "freedom"`, `tag: "direct"` — прямой выход в интернет (без `domainStrategy`, что равно `AsIs`, и с `finalRules: [{action: "block", ip: ["geoip:private"]}, {action: "allow"}]`);
- `protocol: "blackhole"`, `tag: "blocked"` — «чёрная дыра» для заблокированного трафика.

Общие поля формы outbound

Поле	Подпись (RU)	Описание
Тег	**Тег** (подсказка: *«Уникальный тег»*)	Уникальный идентификатор outbound. Плейсхолдер: *«уникальный-тег»*. Валидация: *«Тег обязателен»*, *«Тег уже используется другим исходящим»*.
Протокол	—	Тип исходящего (см. ниже).
Адрес / Порт	**Адрес** / Порт	Цель подключения. Адрес и порт обязательны.
Отправить через	**Отправить через**	Локальный IP-адрес исходящего интерфейса (`sendThrough`). Плейсхолдер: *«локальный IP»*.

Поле	Подпись (RU)	Описание
Target Strategy	**Target Strategy**	**Новое в 3.5.0.** Как резолвится домен назначения перед подключением. 11 значений: `AsIs` (по умолчанию, не резолвить), `UseIP`, `UseIPv4`, `UseIPv6`, `UseIPv6v4`, `UseIPv4v6` (резолв с fallback), `ForceIP`, `ForceIPv6v4`, `ForceIPv6`, `ForceIPv4v6`, `ForceIPv4` (требуют успешного резолва). Пусто = `AsIs` (в конфиг не пишется). Для `freedom` поле скрыто: ядро само переносит его корневой `targetStrategy` в `sockopt.domainStrategy`, поэтому стратегию freedom задаёт поле **Стратегия** в карточке протокола (см. ниже).
Dialer proxy (цепочка)	—	Подсказка: *«Подключайте это исходящее через другое исходящее (по тегу), чтобы построить цепочку прокси. Оставьте пустым для прямого подключения.»* Плейсхолдер: *«Выберите исходящее для цепочки»*. Реализуется через `streamSettings.sockopt.dialerProxy`.

Выпадающий список **Dialer Proxy** показывает не только локальные outbounds, но и теги outbounds из подписок — так цепочку можно построить и через выход, полученный по подписке. Из списка по-прежнему исключены blackhole-outbound и сам редактируемый outbound. Оставьте поле пустым для прямого подключения.

Поддерживаемые протоколы outbound

Поддерживаемые формой протоколы:

- **`freedom`** — прямой выход. Поля **Стратегия** (записывается в `streamSettings.sockopt.domainStrategy` — туда, где её читает ядро; старые `settings.domainStrategy` и корневой `targetStrategy` форма учитывает при открытии, но при сохранении удаляет), `finalRules` (см. ниже), Happy Eyeballs. Тестированию не подлежит (*«Outbound has no testable endpoint»*).
- **`blackhole`** — отбрасывает трафик. Поле **Тип ответа** (`settings.response.type`): пусто или `none` — просто закрыть соединение, `http` — перед закрытием отправить стандартный ответ HTTP 403, `custom` — отправить свои данные из поля **Свой ответ (base64)** (`customResponseData`, стандартный base64; плейсхолдер `SFRUUC8xLjEgNDAzIEZvcmJpZGRlbg0KDQo=` — это `HTTP/1.1 403 Forbidden`). Некорректный base64 ядро отвергает при сохранении шаблона (*failed to decode custom response data*). Не тестируется.
- **`socks`**, **`http`** — список `settings.servers[]` с `address` / `port`; поле **Пароль авторизации**. Для протокола **`http`** ниже полей **Username/Password** есть редактор **Headers** (Заголовки) — пары ключ/значение для CONNECT-заголовков, отправляемых вышестоящему HTTP-прокси. Эти заголовки сохраняются при повторном открытии и сохранении outbound (ранее терялись). Учтите: применяются только заголовки уровня настроек (`settings.headers`); заголовки на уровне отдельного сервера xray-core игнорирует.
- **`vmess`** — `settings.vnext[]` (`address` / `port`).
- **`vless`** — `settings.address` / `settings.port`; если серверы заданы массивом `settings.vnext[]`, тест проверяет каждый из них, а столбец адреса показывает первый.
- **`trojan`**, **`shadowsocks`** — `settings.servers[]`.
- **`wireguard`** — `settings.peers[]` с `endpoint`, плюс ключи и **Удалённый DNS** (см. 11.8).
- **`amneziawg`** — клиент AmneziaWG: туннель поднимает сама панель, а в Xray outbound попадает как локальный SOCKS (см. Outbound AmneziaWG ниже).
- **`hysteria`** — `settings.address` / `settings.port` (UDP-транспорт).

- `masque` (с 3.10.0) — клиент MASQUE (см. 5.15): `settings.address` / `settings.port` и поле **Удалённый DNS** (`settings.remoteDNS`, список через запятую или пробел, плейсхолдер `1.1.1.1, 2606:4700:4700::1111`). Транспорт всегда `masque` с безопасностью TLS — форма выставляет их сама при выборе протокола; параметры транспорта и учётные данные — в 6.11. Готовый outbound к Cloudflare WARP создаёт кнопка в окне WARP (см. 11.8). При тесте такой outbound считается UDP и всегда проверяется в HTTP-режиме, если ALPN в TLS содержит `h3` или не содержит `h2`; при ALPN только с `h2` (HTTP/2 поверх TCP) — как обычный TCP-outbound.
- `dns` — перехват DNS-запросов: **Переписать сеть**, **Переписать адрес**, **Переписать порт**, **Уровень пользователя** и список **Правила** (у каждого правила — **Действие** `direct` / `drop` / `return` / `hijack`, **QType**, **Домен**, **RCode**). Прежние ключи `nonIPQuery` / `blockTypes` при обновлении до 3.8.0 переводятся в правила (см. 11.11).

3.8.5: сохранённый `qType: 0` у правила DNS-outbound ядро читает как «тип не задан», поэтому такое правило совпадало с любым DNS-запросом через outbound — блокируя, отклоняя или хайджекая весь DNS вместо одного типа запроса. Исправлено: `0` теперь пишется как строка `"0"`. Одноразовая миграция `DNSOutboundQTypeZeroFix` чинит уже сохранённые шаблоны — актуально, если апгрейд до 3.8.0 превратил старый `blockTypes: [0]` в такой `qType: 0`; шаблон с нечитаемым JSON миграция пропускает, лишь пишет предупреждение в лог, не прерывая запуск панели.

Идентификатор протокола (`protocol`) и имя транспорта (`streamSettings.network`) панель читает так же, как ядро, — без учёта регистра, а `mkcp` считает синонимом `kcp`. Outbound, записанный в шаблоне как `"Freedom"`, `"VMess"` или `"WireGuard"`, открывается в форме своего протокола (раньше `"Freedom"` открывался пустой формой VLESS), показывает адрес в таблице и тестируется в правильном режиме, а `"Blackhole"` не предлагается в списках, из которых blackhole-outbounds исключены (Dialer proxy, **Outbound** у MTProto-inbound, загрузка geodata).

Для outbound типа **loopback** доступен блок **Sniffing** с теми же параметрами, что и у inbound: включение, **destOverride**, **Metadata Only**, **Route Only** и список **исключённых доменов**.

В маске **UDP** (FinalMask) для **Hysteria2** доступны дополнительные режимы. У маски **Salamander** есть селектор **Mode** со значениями **Salamander** и **Gecko**: режим Gecko добавляет случайное дополнение пакетов с полями **Min**/**Max** размера (`packetSize`, диапазон 1–2048, по умолчанию 512–1200) — это защищает от фингерпринтинга по длине пакетов. У маски **Realm** (UDP hole-punching) появился необязательный блок **TLS Config** с полями **Server Name** (SNI), **ALPN** (`h3` / `h2` / `http/1.1`), **Fingerprint** (uTLS) и переключателем **Allow Insecure**. У **Realm** также есть **IP Mode** (`ipMode`: Dual, IPv4 или IPv6; пусто — dual) и переключатель **Port Mapping (UPnP / NAT-PMP)** (`portMapping.enabled`) с полями **Mapping Timeout (s)** (по умолчанию 10) и **Mapping Lifetime (s)** (по умолчанию 600). В QUIC-параметрах есть переключатели **Disable Chrome Parrot**, **Disable GSO** и **Disable Stateless Reset**, а при congestion `brutal` / `force-brutal` — **Brutal Disable Loss Comp**. С xray-core 26.9.9 клиент прыгает по портам только через UDP-маску `udphop`, а ключ `quicParams.udpHop` ядро пропускает. Поэтому с 3.10.0 переключателя **UDP Hop** в QUIC-параметрах outbound'а нет: вместо него в списке UDP-масок outbound'а (у Hysteria2, а также у XHTTP, чей HTTP/3-дозвон тоже проходит через UDP-маски) есть тип **UDP Hop** (`udphop`) с полями **Mode** (мультивыбор, нужен хотя бы один: *Rotate server port (interval)* — `intervalRemote`, *Random server port per connection* — `perConnRemote`, *Rotate local port (interval)* — `intervalLocal`), **Interval (s)** (секунды, не меньше 5: `30` или `5-10`; по умолчанию `5-10`), **Remote Ports** (например `20000-50000`) и **Remote IPs**. Если у outbound'а остался старый `quicParams.udpHop`, форма при открытии сама переводит его в маску `udphop` режима `intervalRemote`. У inbound переключатель **UDP Hop** остаётся: он только объявляет клиентам диапазон (`mport` в ссылке, маска `udphop` в JSON-подписке), а

сам сервер слушает свой порт. Маска `udphop` должна быть **единственной и последней** в списке UDP-масок: ядро разворачивает список и при дозвоне отвергает маску, открывающую собственный сокет, не на своём месте (`incorrect index`), — причём `xray run -test` пишет «Configuration OK», и клиент просто не подключается. Поэтому при сборке конфигурации (outbound'ы шаблона), импорте ссылок с `fm=` и выдаче подписок панель оставляет только последнюю указанную маску `udphop` и переносит её в конец списка; в JSON-подписке при нескольких источниках диапазона побеждает самый частный: хост, затем сам inbound, затем глобальная маска подписки.

С 3.5.0 добавлен новый тип **TCP-маски** — **XMC (Minecraft)** (`xmc`): поток маскируется под протокол Minecraft. Поля: **Hostname** (необязательно; «адрес сервера, имитируемый в рукопожатии»), **Usernames** (необязательный список; «имена игроков, предлагаемые проберам; по умолчанию ядро использует Dream») и обязательный **Password** (16 символов, автогенерация с кнопкой ↻; «пароль обфускации»). Важно: **сочетание TCP-Finalmask с REALITY отклоняется при сохранении** — оно роняло Xray-core при первом подключении (ошибка: «Finalmask is not supported with REALITY security…»; см. Xray-core#6453). UDP-маска с REALITY допустима; сохранённые ранее некорректные комбинации автоматически «лечатся» (маска отбрасывается) при сборке конфига.

Пример: цепочка через вышестоящий SOCKS. Outbound `upstream` ходит на внешний SOCKS5-прокси, а `chained` отправляет свой трафик через него (`dialerProxy`), образуя цепочку. В `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Теперь правило маршрутизации с `outboundTag: "chained"` будет выводить трафик в интернет через `upstream`.

Импорт outbound из share-ссылки

Outbound можно импортировать из share-ссылки (`vless://`, `vmess://` и т. п.). При импорте сохраняются и настройки мультиплексора **xmux** (XHTTP), переданные в блоке `extra=` ссылки: после импорта их значения подставляются в подформу **XMUX** созданного outbound. Ссылка `hysteria2://` с диапазоном портов `mport` превращается в UDP-маску `udphop` (`mode: "intervalremote"`, `interval: "5–10"`, `remotePorts` — диапазон из ссылки); маска, переданная в `fm=`, имеет приоритет. Так же разбираются ссылки подписок исходящих (см. 11.12). С 3.10.0 маску `udphop` можно и добавить вручную в форме outbound'а (см. выше), а старый `quicParams.udpHop` в параметре `fm=` ссылки при импорте тоже переводится в маску. Outbounds, сохранённые раньше с `quicParams.udpHop` (в шаблоне и в кэше

подписок исходящих), переводятся в маску однократной миграцией `UDPHopClientMaskFix` и дополнительно исправляются при сборке конфигурации (см. 11.11) — импортировать их заново больше не нужно.

С 3.10.2 ссылки `vmess://` разбираются полностью — при импорте, в подписках исходящих и во внешних ссылках клиента. VMess-ссылка не имеет блока `extra=`: поля XHTTP (размещение session/seq и uplink, обфускация паддинга, xmux и т. п.) лежат прямо в её JSON, и теперь они читаются все, а не только `xPaddingBytes`, `scMaxEachPostBytes` и `scMinPostsIntervalMs` (#6808). Также читаются маска Finalmask (`fm`), заголовок mKCP (`type`), seed (`path`), `mtu` и `tti` (#6809). Раньше outbound из такой ссылки (например, XHTTP с `sessionIDPlacement=header` или mKCP с seed) не мог подключиться.

Поля Mux (мультиплексирование)

Макс. параллелизм, **Макс. соединений**, **Макс. повторных использований**, **Макс. запросов**, **Макс. секунд повторного использования**, **Период keep alive**. Эти параметры настраивают mux/XUDP-поведение исходящего.

Sockopts (настройки сокета)

Группа **Sockopts**: **Интервал keep alive**, **Mark (fwmark)**, **Интерфейс**, **Только IPv6**, **Принимать proxy protocol**, **Proxy protocol**, **TCP user timeout (мс)**, **TCP keep-alive idle (с)**. Здесь же задаётся dialer-proxy цепочки. У `freedom` поля **Стратегия домена** в этой группе нет: тот же ключ `sockopt.domainStrategy` задаёт поле **Стратегия** в карточке протокола.

Freedom finalRules (переопределение блокировки приватных IP)

Для freedom-outbound доступна группа **Финальные правила**:

Поле	Подпись	Описание
`overrideXrayPrivateIp`	**Переопределить дефолтный блок частных IP в Xray**	Снимает встроенный в Xray запрет на исходящие к приватным IP.
`action`	**Действие**	`allow`, `block`, `redirect` (**Redirect**) или прочие; в эталонном шаблоне — `finalRules: [{action: "block", ip: ["geoip:private"]}, {action: "allow"}]`.
`blockDelay`	**Задержка блока (мс)**	Задержка перед отбрасыванием соединения.
`redirect` / `fragment`	**Redirect** / **Fragment**	Действия перенаправления и фрагментации трафика.

Маска fragment: пофрагментные Lengths и Delays

В маске **fragment** (тип fragment в FinalMask, для TCP) одиночные поля Length и Delay заменены списками **Lengths** и **Delays**: для каждого сегмента можно задать отдельный диапазон длины (например `100–200`) и задержки в миллисекундах (например `10–20` или `0`). Строки списков можно добавлять и удалять; ранее сохранённые одиночные значения переносятся в массив из одного элемента автоматически.

Прочие поля формы

- **UDP over TCP** и **Версия UoT** — для shadowsocks-подобных протоколов.
- **Без gRPC-заголовка**, **Размер chunk Uplink** — параметры gRPC-транспорта.
- TLS/uTLS-поля: **Проверять имя peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, плейсхолдер «имя сервера».
- **Использовать системные CA** (`tlsSettings.useSystemCA` , с 3.10.0). Начиная с Xray-core 26.10.10 ядро проверяет сертификат сервера по встроенному набору корневых сертификатов Mozilla, а не по системному хранилищу ОС. Включите переключатель у outbound'а к серверу, сертификат которого подписан CA, установленным в систему вручную (частный или корпоративный CA).
- **XDRIVE в outbound.** Транспорт XDRIVE (см. 6.10) предлагается только у протокола `vless` : ядро пропускает через XDRIVE лишь VLESS с шифрованием, без `flow` и только поверх Mux. Поэтому при выборе XDRIVE переключатель Mux включается сам, а `flow` в XDRIVE-потоке не пишется ни в конфиг, ни в формы. Outbound на XDRIVE, который ядро отвергло бы (без VLESS-шифрования или без Mux, например сохранённый раньше), панель не включает в собираемый конфиг и пишет в лог предупреждение *«Outbound "mez": not started, xray-core refuses it: …»*; остальные outbound'ы запускаются. Сервис хранилища записывается как `google-drive` (в форме — «Google Drive»); прежнее `Google Drive` читается как устаревший псевдоним.
- **TLS для XDRIVE** (с 3.10.0). У outbound'ов `vless` на транспорте XDRIVE (см. 6.10) теперь можно выбрать безопасность **TLS**: её SNI, отпечаток uTLS и ALPN задают HTTPS-подключение к API хранилища — именно этот TLS-хендшейк видит API. Раньше TLS для XDRIVE не предлагался, а при переключении TLS-outbound'а на XDRIVE безопасность молча сбрасывалась в `none` . У inbound'ов на XDRIVE TLS по-прежнему недоступен.

Тестирование исходящих

Кнопки: **Тест**, **Тестировать все**. Состояния: **Тестирование соединения…**, **Тест успешен**, **Тест не пройден**, **Не удалось протестировать исходящее подключение**. Результат: **Результат теста**, задержка в миллисекундах.

Два режима (подсказка: *«TCP: быстрый dial-only probe. HTTP: полный запрос через xray.»*):

- **TCP** (`mode=tcp`) — простой dial до `host:port` , выполняется параллельно по всем эндпоинтам, ~таймаут 5 с. Проверяет только TCP-достижимость, не валидирует прокси-протокол. IPv6-литерал в адресе сервера принимается и в скобках, и без них (#6850). Для `freedom` / `blackhole` /тега `blocked` вернёт *«Outbound has no testable endpoint»*.

- **HTTP** (`mode=http` или пусто) — поднимает временный экземпляр Xray, прогоняет реальный HTTP-запрос (probe URL = серверный `outboundTestUrl`), измеряет реальную задержку. Авторитетный, но дорогой режим: сериализуется глобальной блокировкой (*«Another outbound test is already running, please wait»*). Тайм-аут одной попытки — 10 с, окно ожидания результата — 15 с (увеличены, чтобы здоровые outbounds на медленных или туннелированных каналах не помечались как «Failed»). При неудаче реальная причина (ошибка DNS, connection refused, истечение дедлайна, ошибка TLS и т. п.) пишется в лог панели/Xray, на который указывают общие сообщения о тайм-ауте.

- **Real delay** (**новое в 3.5.0**) — третий режим переключателя. Использует тот же временный экземпляр Xray, но сообщает **полное время «холодного» запроса, включая установление туннеля** — цифру, близкую к той, что показывают клиентские приложения. Подсказка переключателя: «TCP: быстрый dial-

пробник. HTTP: полный запрос через xray. Real delay: полное время, включая установление соединения». Обычный **HTTP**-режим с 3.5.0 измеряется по «тёплому» (keep-alive) соединению, поэтому его цифры ниже и ближе к задержке одного запроса.

> UDP-протоколы (`wireguard` , `amneziawg` , `hysteria`) и UDP-транспорты (`kcp` / `mkcp` , `quic` , `hysteria`) **всегда** тестируются в HTTP-режиме, даже если запрошен TCP — голый UDP-dial не отличает «живой» эндпоинт от «мёртвого». Регистр не важен: `"WireGuard"` или `"network": "KCP"` тоже уходят в HTTP-режим. Для wireguard в тестовой конфигурации принудительно ставится `noKernelTun: true` .

Колонки Egress и Country (с 3.5.0)

После успешного **HTTP** или **Real delay** теста панель через тот же временный маршрут запрашивает метаданные Cloudflare trace и показывает в списке outbound две колонки: **Egress** — IP выхода (IPv4/IPv6, каждый скрыт за «глазом»; пока теста не было — прочерк с подсказкой «Выполните HTTP-тест, чтобы показать IP выхода и страну») и **Country** — флаг и название страны выхода; если выход идёт через Cloudflare WARP, добавляется оранжевая метка WARP. TCP-тест эти колонки не заполняет; на мобильных данные показываются в карточке outbound.

Батчевая проверка и разбивка по этапам

Тест и **Тестировать все** в HTTP-режиме поднимают один общий временный экземпляр Xray на пакет outbounds, для каждого создают петлевой SOCKS-inbound с правилом и параллельно отправляют через него реальный HTTP-запрос; **Тестировать все** проверяет outbounds пачками. **Тестировать все** проверяет и outbounds, полученные из подписок (таблица «из подписок», только для чтения) — их строки также подсвечиваются результатом теста. При этом outbounds `freedom` («direct») и `dns` не тестируются ни в одном режиме (это не прокси): кнопка теста у них недоступна, **Тестировать все** их пропускает, а серверная защита запрещает их HTTP-тест даже при прямом вызове API. Помимо успеха/ошибки попап-результат показывает HTTP-статус ответа и разбивку времени по этапам: **Proxy connect** (подключение к прокси), **TLS via outbound** (TLS через outbound) и **First byte** (время до первого байта) — это помогает понять, на каком шаге возникла задержка или сбой.

Статистика трафика outbounds

Панель ведёт счётчики трафика по тегам (`up` / `down` / `total`). Кнопка сброса сбрасывает счётчики для конкретного тега или для всех (`tag = "–alltags–"`). Поля **Информация об учетной записи** и **Статус исходящего подключения** показывают сводку.

Outbounds PIA по логину (3.7.0)

Раздел **«PIA»** добавляет WireGuard-outbound к Private Internet Access, не заставляя вручную добывать ключи. Вводите **логин и пароль PIA**, панель показывает аккаунт и список регионов; выбираете страну (или «All regions») и сервер — панель сама получает ключ и собирает готовый outbound. Если такой сервер уже добавлен, панель предложит **обновить ключ** вместо создания дубля. Ошибка провижининга сообщается явно («Could not build the PIA outbound»), а не оставляет полупустой outbound.

Outbound AmneziaWG

Протокол `amneziawg` делает сервер **клиентом AmneziaWG**: трафик, направленный в такой outbound, уходит через туннель на внешний сервер AmneziaWG — например, на AmneziaWG-inbound другой панели (см. 5.10). В Xray-core этого протокола нет, поэтому туннель поднимает сама панель тем же встроенным движком `amneziawg-go` поверх gVisor, что и у inbound. Раз в 10 секунд она сверяет запущенные туннели с **сохранённым** шаблоном: поднимает новые, перенастраивает изменённые, останавливает удалённые. В итоговую конфигурацию Xray такой outbound попадает как `socks` с тем же тегом, направленный на локальный SOCKS5-шлюз панели `127.0.0.1:64900` (вход по паролю, имя пользователя — тег), поэтому правила маршрутизации и балансировщики ссылаются на тег как обычно. Сами UDP-пакеты туннеля движок отправляет с хоста напрямую, а не через Xray.

Поля формы (параметры обфускации те же, что у inbound, их смысл описан в 5.10):

Поле	JSON (`settings`)	По умолчанию	Описание
MTU	`mtu`	`0`	`0` — вычислить автоматически: 1420 минус `S4`, но не меньше 1280.
Listen Port (optional)	`listenPort`	`0`	Фиксированный локальный UDP-порт источника; `0` — выбрать автоматически (подсказка в интерфейсе не переведена).
Приватный ключ	`secretKey`	—	Обязателен. Кнопка рядом генерирует новый ключ.
Адрес	`address`	—	Обязателен: адрес этого клиента в туннеле в формате CIDR, например `10.8.0.2/32`.
DNS	`dns`	пусто	Резолвер **внутри туннеля** для доменных адресов назначения: IP или `IP:порт` (без порта — 53). Пусто — `1.1.1.1:53`, а если у туннеля только IPv6-адреса — `[2606:4700:4700::1111]:53`.
HeaderProtectionKey (защита заголовков)	`headerProtectionKey`	пусто	Base64-ключ длиной 32 байта; пусто — защита заголовков выключена.
Параметры обфускации: Jc, Jmin, Jmax, S1–S4, H1–H4, I1, ContentPaddingAddition	`jc` … `contentPaddingAddition`	заготовка формы	Подсказка *«Must exactly match the server side parameters.»*: значения должны **в точности** совпадать с параметрами сервера, иначе рукопожатие не состоится. Кнопки случайной генерации, как у inbound, здесь нет.

Поле	JSON (`settings`)	По умолчанию	Описание
Peers → Peer N	`peers[]`	—	Нужен хотя бы один: **Конечная точка** (`endpoint` , `host:port` , имя хоста допустимо), **Публичный ключ** (`publicKey` , обязателен), **PSK** (`presharedKey` , необязателен), **Разрешенные IP-адреса** (`allowedIPs` ; для нового peer — `0.0.0.0/0` и `::/0`), **Keep alive** (`keepAlive` ; для нового peer — 25).
RandomTrailers, **DisableCookies**	`randomTrailers` , `disableCookies`	выкл. / вкл.	Как у inbound; RandomTrailers требует AmneziaWG 3.1+ с обеих сторон.

Ключи `i2` – `i5` , `rekeyAfterTime` , `rekeyTimeout` , `rejectAfterTime` , `keepaliveTimeout` и `maxHandshakeAttempts` в форме не показываются: их задают на вкладке JSON, и при сохранении формы они не теряются. Импорта из share-ссылки для этого протокола нет.

При сохранении шаблона панель проверяет AmneziaWG-outbound сама (загрузчик ядра такой протокол отверг бы) и отклоняет сохранение с ошибкой вида `amneziawg outbound "<тег>": …` , если нет адреса туннеля или он не в формате CIDR; DNS — не IP и не `IP:порт` ; приватный ключ пуст или некорректен; параметры обфускации не проходят те же проверки, что у inbound; нет ни одного peer; у peer пуст или некорректен публичный ключ, некорректен PSK, `endpoint` не вида `host:порт` или нет ни одной корректной записи `allowedIPs` .

- **Порт 64900 зарезервирован.** SOCKS5-шлюз слушает `127.0.0.1:64900` , пока в шаблоне есть хотя бы один рабочий AmneziaWG-outbound, и освобождает порт, когда их не остаётся. Но локальный inbound с TCP на порту 64900, слушающий `127.0.0.1` или все адреса, панель не сохранит в любом случае: `port 64900 (tcp) already used by inbound 'amneziawg-egress' on 127.0.0.1` .
- **Тест.** В режиме TCP такой outbound всё равно проверяется HTTP-запросом, как WireGuard (см. «Тестирование исходящих» выше). Запрос идёт через уже запущенный туннель, то есть проверяет **сохранённую** версию outbound: после сохранения дайте панели до 10 секунд поднять туннель.
- **Диагностика.** Запуск туннелей и ошибки их настройки пишутся в лог панели: `amneziawgnet: started embedded outbound <тег>` , `amneziawgnet: reconcile failed for outbound "<тег>": …` .

11.5. Балансировщики (Balancers)

Список `routing.balancers` . Кнопки: **Создать балансировщик**, **Редактировать балансировщик**.

На вкладке Balancers есть столбцы живого состояния: **Текущая цель** (*Live Target*) показывает текущую активную цель балансировщика в работающем Xray, а **Переопределить** (*Override*) позволяет вручную закрепить выбор цели (значение **Авто (стратегия)** возвращает выбор по стратегии). Состояние обновляется отдельной кнопкой (подсказка «Обновить состояние балансировщика»). Если балансировщик ещё не активен в работающем Xray, панель предложит сначала сохранить изменения или запустить Xray.

С 3.10.0 переопределение **переживает горячее применение маршрутизации** (#6779). Раньше оно жило только внутри работающего ядра и молча пропадало при любом сохранении настроек Xray, затрагивающем правила или балансировщики, а также при проверке узла через **Connection outbound** (см. 12.2). Теперь

панель запоминает заданные переопределения и восстанавливает их после своих перезагрузок маршрутизации. Целью можно выбрать только outbound, который есть в работающей конфигурации, иначе запрос отклоняется (`override target is not an outbound in the running config`): ядро приняло бы любой тег, но обрывало бы все соединения, направленные в несуществующий. Если целевой outbound удалён, переопределение снимается, а балансировщик возвращается к выбору по стратегии и не превращается в «чёрную дыру». Перезапуск Xray, как и прежде, сбрасывает все переопределения.

Поле	Подпись (RU)	Описание
Тег	**Тег** (подсказка: *«Уникальный тег»*)	Уникальный идентификатор. Плейсхолдер: *«уникальный тег балансировщика»*. Валидация: *«Тег обязателен»*, *«Тег уже используется другим балансировщиком»*.
Селекторы	**Селекторы**	Список тегов outbound (по подстроке), среди которых балансировщик выбирает выход. Должен быть выбран хотя бы один: *«Выберите хотя бы одно исходящее»*.
Fallback	**Fallback**	Резервный тег, если ни один селектор не подошёл. **С 3.5.0 можно выбрать и другой балансировщик** (в списке такие варианты помечены синей меткой «Balancer», подсказка «Выберите другой балансировщик как fallback»). Xray нативно этого не умеет, поэтому панель сама строит скрытый loopback-outbound `_bl_<цель>` и правило маршрутизации (путь: балансировщик → loopback → сервер → целевой балансировщик → outbound; предупреждение о лишнем «хопе» показывается в форме). Есть защита от циклов (ошибка «нельзя выбрать — возникла бы циклическая зависимость», вариант в списке блокируется с подсказкой пути цикла) и от удаления используемого балансировщика («Нельзя удалить — используется как fallback у: …»). Префикс тега `_bl_` зарезервирован (в тегах запрещён); служебные `_bl_` -объекты скрыты из списков Outbounds/Routing, а колонки Fallback/Live Target/Override показывают имя целевого балансировщика, а не loopback-тег.
Стратегия	**Стратегия**	Алгоритм выбора (см. ниже).

Стратегия и параметры наблюдения

Стратегия (`strategy.type`) определяет, как балансировщик выбирает outbound из селекторов. Значения Xray-core: `random` (случайно), `roundRobin` (по кругу), `leastPing` (минимальная задержка по результатам observatory), `leastLoad` (минимальная нагрузка). Для `leastLoad` / `leastPing` используются параметры из `strategy.settings` :

Поле	Подпись	Описание
`expected`	**Ожидаемое**	Плейсхолдер: *«оптимальное число узлов»* — целевое число живых узлов.
`maxRtt`	**Макс. RTT**	Верхняя граница допустимого RTT при отборе кандидатов.
`tolerance`	**Допуск**	Допуск (tolerance) при сравнении задержек/нагрузки.
`baselines`	**Baselines**	Пороговые значения задержки для группировки узлов.
`costs`	**Costs**	Весовые коэффициенты (cost) для отдельных тегов.

Примеры стратегий. Блок `strategy` живёт внутри балансировщика (в JSON — рядом с `tag` и `selector`):

```
"strategy": { "type": "random" }      // случайный выбор из селекторов
"strategy": { "type": "roundRobin" }  // по кругу, поочерёдно
"strategy": { "type": "leastPing" }   // минимальная задержка (нужен наблюдатель)
```

Для `leastLoad` параметры задаются в `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regexp": false, "match": "proxy-premium",   "value": 0.1 },
      { "regexp": true,  "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

Как это отрабатывает (на примере). Пусть наблюдатель намерил задержки по выходам: `A = 250 мс`, `B = 280 мс`, `C = 700 мс`, `D = 1500 мс`. С настройками выше выбор идёт так:

1. **`maxRTT: "1s"`** — выходы с задержкой выше 1 с отбрасываются: `D` (1500 мс) выбывает. Остаются `A`, `B`, `C`.
2. **`baselines` + `expected`** — выходы группируются по порогам задержки, и берётся **наименьший** порог, в который попадает не менее `expected` выходов. Порог `500ms` уже содержит `A` и `B` — это 2 (= `expected`), поэтому выбирается группа { `A`, `B` }. `C` (700 мс) в выбор не попадает, пока быстрых хватает (он — «горячий резерв»).
3. **`tolerance: 0.05`** — внутри выбранной группы выходы, чьи задержки отличаются не более чем на 5%, считаются равноценными, и нагрузка делится между ними поровну. `A` (250) и `B` (280) отличаются на ~12% (> 5%), поэтому при прочих равных предпочтение у более быстрого `A`; будь разница в пределах 5% — трафик шёл бы и через `A`, и через `B`.
4. **`costs`** — до сравнения корректируют «стоимость» отдельных выходов: меньшее `value` делает выход привлекательнее, большее — наоборот. В примере `proxy-premium` получает `0.1` (становится «дешевле» и выбирается охотнее), а все `proxy-cheap-*` (по регулярному выражению, `regexp: true`) — `5` (становятся «дороже» и используются в последнюю очередь). Так можно мягко приоритизировать выходы, не исключая их жёстко.

Итог: трафик пойдёт в основном через `A` (при близких задержках — поровну с `B`), `C` останется резервом, `D` исключён, пока его RTT не опустится ниже `maxRTT`.

Наблюдатель: `observatory` и `burstObservatory` (замеры для `leastPing` / `leastLoad`)

Стратегии `leastPing` и `leastLoad` сами ничего не измеряют — им нужны данные о задержке и доступности каждого outbound. Их собирает **наблюдатель** (observatory): он периодически «пингует» каждый отслеживаемый outbound и запоминает время отклика и доступность. Те же данные показываются на вкладке **«Обсерватория»** (статусы **Активен / Недоступен**, **«Последняя активность»**, **«Последняя попытка»**).

С версии 3.4.2 страница «Балансировщики» разделена на вкладки **«Настройки балансировщика»** (таблица и добавление) и **«Observatory»**, а наблюдатель редактируется **структурной формой** (прежний сырой JSON-редактор убран). Панель сама создаёт и удаляет наблюдателей по необходимости: обычный `observatory` — для стратегии **Least Ping**, расширенный `burstObservatory` — для **Least Load**, а также для **Random/Round-robin с заданным** `fallbackTag` (этого требует xray-core; при очистке последнего `fallbackTag` наблюдатель убирается). Создание или удаление наблюдателя требует полного перезапуска Xray.

Доступны два варианта:

- **`observatory`** — простой: `subjectSelector` + `probeURL` + `probeInterval`.
- **`burstObservatory`** — расширенный, с тонкой настройкой пинга через `pingConfig`; удобен для нескольких выходов.

Пример блока `burstObservatory`:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Назначение полей:

Поле	Что задаёт
`subjectSelector`	Список **префиксов тегов** outbound для наблюдения. Xray берёт все outbound, чьи теги начинаются с указанных строк. В примере наблюдаются выходы `WS-SE…`, `WS-FR…`, `WS-PL…`. Эти теги должны совпадать с тем, что выбрано в **Селекторах** балансировщика.
`pingConfig.destination`	URL, запрашиваемый **через каждый outbound** для замера задержки. Берут «лёгкую» страницу с ответом `204` без тела — например `https://www.google.com/generate_204`. Время до ответа и есть измеренная задержка.
`pingConfig.interval`	Как часто пинговать каждый outbound. Строка длительности: `"1m"` — раз в минуту, также `"30s"`, `"5m"` и т.п. Чаще — свежее данные, но больше фонового трафика.
`pingConfig.connectivity`	(необязательно) URL проверки **базовой связности** самого сервера. Если он недоступен — значит проблема в сети сервера, и наблюдатель **не** помечает outbound как недоступный (защита от ложных срабатываний при локальном сбое). Обычно тоже endpoint с ответом `204`.
`pingConfig.timeout`	Сколько ждать ответ на один пинг, прежде чем считать попытку неудачной (например `"5s"`).
`pingConfig.sampling`	Сколько последних замеров хранить и усреднять по каждому outbound. `2` — учитывать два последних пинга (сглаживает случайные скачки).
`pingConfig.httpMethod`	**Новое в 3.4.2.** HTTP-метод проб: `HEAD` (по умолчанию) или `GET`.

На вкладке **«Observatory»** те же параметры задаются формой (а не JSON). Для обычного `observatory` это **Watched Outbounds** (`subjectSelector` , только чтение), **Probe URL** (`probeURL` , по умолчанию `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval` , `1m`) и **Enable Concurrency** (`enableConcurrency` , по умолчанию включено). Для `burstObservatory` — перечисленные выше поля плюс новое **«HTTP-метод»**. Если у балансировщика есть оба наблюдателя, между формами Observatory и Burst переключает сегментный переключатель.

Как всё связать:

1. На вкладке **«Observatory»** настройте параметры наблюдателя (он создаётся автоматически под выбранную стратегию; `subjectSelector` следует за селекторами балансировщика).
2. Создайте балансировщик: **Стратегия** = `leastPing` , в **Селекторах** укажите те же теги outbound (`WS-SE` , `WS-FR` , `WS-PL`).
3. Направьте на него трафик правилом маршрутизации (поле **Тег балансировщика**, см. 11.3).
4. Перезапустите Xray. На вкладке **«Обсерватория»** появятся статусы выходов, а балансировщик начнёт выбирать самый быстрый из живых.

> В одном правиле нельзя одновременно задать `balancerTag` и `outboundTag` — сработает только `outboundTag` .

Что происходит при удалении outbound или балансировщика (с 3.4.2)

При удалении outbound или балансировщика панель в том же действии **вычищает ссылки на него в маршрутизации** и показывает **предпросмотр последствий** в диалоге подтверждения (заголовок «При удалении также обновится ваша маршрутизация:», далее по каждому правилу — «удалено (не осталось назначения)» или «оставлено (теперь использует …)», и «Балансировщик … удалён (не осталось целей)»). Так предотвращается «висячая» ссылка (`balancerTag` / `outboundTag` / `dialerProxy`), из-за которой раньше xray-core мог не запуститься при следующем перезапуске. Удаление балансировщика: правила сохраняют свой `outboundTag` , иначе удаляются. Удаление outbound: его тег убирается из селекторов и `fallbackTag` балансировщиков, очищается `dialerProxy` у других outbound, опустевшие балансировщики удаляются вместе с теперь-беспредметными правилами, наблюдатели пересобираются.

11.6. DNS

Раздел `dns` . Включение: **Включить DNS** (подсказка: *«Включить встроенный DNS-сервер»*).

> С 3.5.0 DNS-серверы на **приватных IP** (например, свой AdGuard/Pi-hole в той же docker-сети) работают «из коробки»: панель автоматически вставляет и поддерживает управляемое allow-правило (direct, ограниченное портом DNS-сервера: у удалённого DoH — `https://` и `h2c://` — это порт из URL, по умолчанию 443, а поле `port` объектной записи для него игнорируется, как и самим Xray, #6918) **перед** блокирующим правилом `geoip:private` и синхронизирует его при изменении `dns.servers` . Раньше собственные DNS-запросы Xray молча блокировались этим правилом, добавляя ~4 с задержки на каждый новый домен; ручные правила больше не нужны (созданные вручную не затрагиваются).

Общие параметры DNS

Поле	Подпись (RU)	JSON	Описание / подсказка
`tag`	**Название тега DNS**	`dns.tag`	*«Этот тег будет доступен как входящий тег в правилах маршрутизации.»* Позволяет маршрутизировать сами DNS-запросы через `inboundTag`.
`clientIp`	**IP клиента**	`dns.clientIp`	*«Используется для уведомления сервера о указанном местоположении IP во время DNS-запросов»* (EDNS Client Subnet).
`strategy`	**Стратегия запроса**	`dns.queryStrategy`	*«Общая стратегия разрешения доменных имен».* Значения: `UseIP`, `UseIPv4`, `UseIPv6`.
`disableCache`	**Отключить кэш**	`dns.disableCache`	*«Отключает кэширование DNS».*
`disableFallback`	**Отключить резервный DNS**	`dns.disableFallback`	*«Отключает резервные DNS-запросы».*
`disableFallbackIfMatch`	**Отключить резервный DNS при совпадении**	`dns.disableFallbackIfMatch`	*«Отключает резервные DNS-запросы при совпадении списка доменов DNS-сервера».*
`enableParallelQuery`	**Включить параллельные запросы**	—	*«Включить параллельные DNS-запросы к нескольким серверам для более быстрого разрешения».*
`useSystemHosts`	**Использовать системные Hosts**	`dns.useSystemHosts`	*«Использовать файл hosts из установленной системы».*
`script`	**Lua-скрипт**	`dns.script`	С 3.10.0. *«Путь к файлу .lua с функцией HandleDNSQuery — абсолютный или относительно папки bin Xray…»* Плейсхолдер `dns.lua`; пустое поле удаляет ключ. См. 11.14.

Пример блока `dns`. Запросы к доменам Google резолвятся через DoH-сервер Cloudflare, всё остальное — через `1.1.1.1`; на запросы Google ожидаются только не-приватные IP. На верхнем уровне конфига:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Строка-сервер (`"1.1.1.1"`) без полей — это сервер по умолчанию для всех прочих доменов. Тег `dns-inbound` затем можно использовать как `inboundTag` в правилах маршрутизации, чтобы направлять сами DNS-запросы через нужный outbound.

Кэш устаревших записей

Поле	Подпись	Описание
`serveStale`	**Использовать устаревшие**	*«Возвращать устаревшие результаты из кэша во время обновления в фоне».*
`serveExpiredTTL`	**TTL устаревших**	*«Срок действия (секунды) устаревших записей кэша; 0 = бессрочно».*

DNS-серверы (список `dns.servers`)

Кнопки: **Создать DNS**, **Редактировать DNS**, **Удалить все** (подтверждение: *«Все DNS-серверы будут удалены из списка. Это действие нельзя отменить.»*). Шаблоны: **Использовать шаблон**, окно **Шаблоны DNS**, включая пресет **Семейный**.

При нажатии **Редактировать DNS** на записи DNS-сервера (как и на записи Fake DNS) окно редактирования подставляет сохранённые значения сервера, а не значения по умолчанию.

Поля DNS-сервера:

Поле	Подпись	Описание
address	—	Адрес DNS (IP, DoH-URL, `localhost` , `fakedns` и т. п.).
port	**Порт**	Порт DNS-сервера. Для адресов `https://` , `https+local://` , `h2c://` , `h2c+local://` и `quic+local://` поле не показывается: порт таких серверов Xray берёт только из URL, поэтому нестандартный порт указывают в самом адресе (например `https://dns.example.com:8443/dns-query`). Для DoT (`tls://`) и обычных адресов поле доступно.
`domains`	**Домены**	Список доменов, для которых используется этот сервер.
`expectIPs`	**Ожидаемые IP**	Принимать ответ только если IP попадает в список.
`unexpectIPs`	**Неожидаемые IP**	Отбрасывать ответы с указанными IP.

Поле	Подпись	Описание
`skipFallback`	**Пропустить Fallback**	Не использовать этот сервер как fallback.
`finalQuery`	**Финальный запрос**	Помечает сервер как финальный в цепочке.
`timeoutMs`	**Тайм-аут (мс)**	Таймаут запроса к серверу.
`id`	**ID**	С 3.10.0. *«Имя, по которому DNS-скрипт на Lua выбирает этот сервер»* (см. 11.14). Пустое поле в конфиг не пишется.

Hosts (статические записи)

Группа **Hosts** (`dns.hosts`). Кнопка **Добавить Host**; пустое состояние **Host не определены**. Поля: домен (плейсхолдер: *«Домен (напр. domain:example.com)»*) и значения (плейсхолдер: *«IP или домен — введите и нажмите Enter»*).

Логи DNS

См. 11.10: флаг **Логи DNS** (`dnsLog`) в секции логирования.

11.7. Fake DNS

Раздел `fakedns`. Кнопки: **Создать Fake DNS**, **Редактировать Fake DNS**.

Поле	Подпись	Описание
`ipPool`	**Подсеть пула IP**	CIDR-диапазон, из которого выдаются фиктивные IP (например `198.18.0.0/15`).
`poolSize`	**Размер пула**	Сколько адресов держать в кольцевом пуле.

Fake DNS используется в связке с sniffing на inbound: ядро выдаёт клиенту фиктивный IP, запоминает соответствие домен⍰IP и восстанавливает домен при маршрутизации. Чтобы Fake DNS работал, DNS-сервер с адресом `fakedns` должен быть добавлен в список DNS-серверов.

Пример: связка Fake DNS + DNS-сервер. Сначала задаём пул фиктивных адресов, затем добавляем DNS-сервер `fakedns`, чтобы доменные запросы получали IP из этого пула:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Дополнительно на inbound нужно включить sniffing с `destOverride: ["fakedns"]`, иначе ядру неоткуда взять реальный домен для восстановления.

11.8. WireGuard / WARP / NordVPN

Поля WireGuard (`wireguard`)

Поле	Подпись	Описание
`secretKey`	**Секретный ключ**	Приватный ключ локального интерфейса.
`publicKey`	**Публичный ключ**	Публичный ключ peer.
`psk`	**Общий ключ**	PreShared Key (опционально).
`allowedIPs`	**Разрешенные IP-адреса**	Диапазоны, маршрутизируемые в туннель.
`endpoint`	**Конечная точка**	`host:port` peer.
`domainStrategy`	**Стратегия домена**	Стратегия резолва для WireGuard-outbound.
`remoteDNS`	**Удалённый DNS**	DNS-серверы для разрешения доменных адресов назначения **через туннель**, через запятую (например `1.1.1.1,2606:4700:4700::1111`). Пусто — `1.1.1.1` , `1.0.0.1` , `2606:4700:4700::1111` , `2606:4700:4700::1001` ; единственное значение `local` — резолвить встроенным DNS самого Xray, а не через туннель.

Cloudflare WARP (`warp`)

Интеграция использует API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Действия контроллера (`/xray/warp/:action`): `config` , `reg` , `license` , `data` , `del` , а с 3.10.0 — `regMasque` (см. «WARP через MASQUE» ниже).

Пошагово:

1. **Создать аккаунт WARP** → `reg` : панель генерирует/принимает приватный (`privateKey`) и публичный (`publicKey`) ключи, регистрирует устройство в Cloudflare и сохраняет `access_token` , `device_id` , `license_key` , `private_key` (а также `client_id`) в настройке `warp` .
2. **Лицензионный ключ WARP / WARP+** → `license` : установка 26-символьного ключа WARP+ (плейсхолдер: *«26-символьный ключ WARP+»*). При ошибке: *«Не удалось установить лицензию WARP.»* Если конфиг ещё не получен: *«Сначала получите WARP-конфиг.»*
3. **Информация об аккаунте**: **Имя устройства**, **Модель устройства**, **Устройство включено**, **Тип аккаунта**, **Роль**, **WARP+ data**, **Квота**, **Использование**.
4. **Добавить исходящий** — создаёт WireGuard-outbound с полученными ключами и эндпоинтом Cloudflare.
5. **Удалить аккаунт** → `del` : очищает сохранённые данные WARP.

WARP через MASQUE (с 3.10.0)

Внизу окна **Cloudflare WARP**, под разделителем **MASQUE**, есть кнопка **«Добавить аутбаунд WARP через MASQUE»** с пояснением *«Регистрирует отдельное устройство WARP для MASQUE; регистрация WireGuard выше не затрагивается.»* Кнопка доступна всегда — WireGuard-аккаунт WARP для неё не нужен.

По нажатию (`POST /panel/api/xray/warp/regMasque`) панель регистрирует в Cloudflare **отдельное** устройство WARP, генерирует для него ключ ECDSA P-256 (secp256r1) и привязывает его к туннелю MASQUE. Привязка MASQUE-ключа заменяет WireGuard-ключ устройства, поэтому для MASQUE всегда заводится своё устройство, а сохранённая WireGuard-регистрация (настройка `warp`) не меняется. Из ответа Cloudflare собирается outbound:

```
{
  "tag": "warp-masque",
  "protocol": "masque",
  "settings": { "address": "<эндпоинт Cloudflare>", "port": 443 },
  "streamSettings": {
    "network": "masque",
    "security": "tls",
    "tlsSettings": { "serverName": "consumer-masque.cloudflareclient.com", "alpn":
["h3"] },
    "masqueSettings": {
      "warp": { "privateKey": "<PEM-ключ>", "publicKey": "<ключ эндпоинта>", "address":
["<IPv4>", "<IPv6>"] }
    }
  }
}
```

Если outbound с тегом `warp-masque` уже есть, он обновляется на месте (новое устройство и ключи), иначе добавляется новый. Как и после «Добавить исходящий», изменение попадает в шаблон и вступает в силу после **Сохранить**. Данные этой регистрации панель отдельно не хранит — они есть только в самом outbound'е, поэтому «Удалить аккаунт» и ротация IP (см. 11.13) его не касаются. Сам протокол MASQUE описан в 5.15, его транспорт — в 6.11.

NordVPN (`nord` / `nordvpn`)

Интеграция использует NordLynx (= WireGuard). Действия контроллера (`/xray/nord/:action`): `countries` , `servers` , `reg` , `setKey` , `data` , `del` .

Пошагово:

1. **Токен доступа** → `reg` : панель запрашивает у `api.nordvpn.com` учётные данные NordLynx и извлекает `nordlynx_private_key` . Сохраняет `private_key` и `token` в настройке `nord` . Альтернатива — `setKey` : ввести **Приватный ключ** напрямую (не может быть пустым).
2. **Страна** → `countries` загружает список стран, где есть серверы NordLynx (флаг и название на языке интерфейса, с поиском); **Город** (либо **Все города**).
3. **Сервер** → `servers` загружает серверы NordLynx выбранной страны (`countryId` валидируется как число — защита от инъекций). Список отсортирован по нагрузке, и первым сразу выбран наименее загруженный сервер; у каждого показаны название, город, hostname, адрес `station:51820` и **Нагрузка сервера** в процентах с цветовой меткой, поиск идёт по городу, названию, hostname и адресу.

Серверы с низкой нагрузкой больше не скрываются (прежний фильтр «нагрузка > 7%» убран). Если серверов нет: *«Серверов для выбранной страны не найдено»*. Если у сервера нет публичного ключа NordLynx: *«Выбранный сервер не сообщает публичный ключ NordLynx.»*

4. **Добавить исходящий** создаёт WireGuard-outbound с тегом `nord-<hostname>` : адрес `10.5.0.2/32` , peer с публичным ключом сервера и конечной точкой `station:51820` , приватный ключ — из сохранённых учётных данных. Можно добавить **несколько серверов** — по одному outbound на hostname. Уже добавленный сервер повторно не добавляется: кнопка неактивна, рядом — *«Этот сервер уже есть в списке исходящих подключений. Используйте Сброс, чтобы обновить его ключ.»* Тост при добавлении: *«Исходящий NordVPN добавлен»*.

В блоке **«Добавленные серверы»** перечислены все outbounds с тегом `nord-…` и их конечные точки. Кнопка **Сброс** в строке оставляет сервер, тег, peer и ссылки из маршрутизации как есть, но подставляет приватный ключ из текущих сохранённых учётных данных NordVPN (тост *«Исходящий NordVPN обновлён»*); у outbound без адреса или peer она неактивна. **Выход** (`del`) очищает только сохранённые учётные данные: добавленные outbounds и правила маршрутизации остаются и продолжают работать со своими ключами (до 3.8.0 выход удалял и outbound, и его правила). Ненужные NordVPN-outbounds удаляйте в общем списке исходящих.

Приоритет IPv4 и userspace TUN

Генерируемый мастером WARP WireGuard-outbound использует `domainStrategy: "ForceIPv4v6"` (приоритет IPv4 с откатом на IPv6 на v6-only хостах) вместо `ForceIP` — это устраняет «зависание» рукопожатия на хостах с наполовину настроенным IPv6, когда выбирается AAAA-запись эндпоинта Cloudflare. Кроме того, у outbounds WARP и NordVPN включён userspace TUN (`noKernelTun: true`) вместо kernel TUN: последний требует прав и fwmark-маршрутизации и тихо падает на многих VPS, тогда как встроенная проверка соединения панели всегда тестирует через userspace TUN — теперь реальный трафик и проверка идут одним путём. Изменение действует только для вновь добавляемых или сброшенных outbounds; уже сохранённые шаблоны сохраняют свои настройки.

11.9. Reverse-прокси и TUN

Reverse (реверс-прокси)

Раздел `reverse` конфигурации Xray. В форме outbound есть переключатель на тип **Реверс-прокси**. Кнопки: **Создать реверс-прокси**, **Редактировать реверс-прокси**.

Поле	Подпись	Описание
Тип	**Тип**	**Bridge** или **Portal** — две роли реверс-прокси Xray.
Домен	**Домен**	Служебный домен-метка для пары bridge⍰portal.
Тег / Соединение	**Тег** / **Соединение**	Теги для связки bridge и portal.
Reverse Tag	**Тег реверс-прокси**	Подсказка: *«Тег исходящего подключения для простого реверс-прокси VLESS. Оставьте пустым для отключения.»* Плейсхолдер: *«тег исходящего (пусто = отключено)»*. Реализует упрощённый VLESS reverse.

В форме outbound присутствуют также поля обратного потока: **Обратный sniffing**, **Воркеры**, **Зарезервировано**, **Мин. интервал загрузки (мс)**, **Макс. размер загрузки (байт)**.

TUN (`tun`)

Поле	Подпись	Описание	По умолчанию
name	—	*«Имя интерфейса TUN.»*	`xray0`
mtu	—	*«Максимальная единица передачи. Максимальный размер пакетов данных.»*	**1500**
`userLevel`	**Уровень пользователя**	*«Все соединения, установленные через этот входящий поток, будут использовать этот уровень пользователя.»*	**0**

11.10. Логи и статистика (Stats, metrics)

Лог (`log`)

Подсказка: *«Логи могут замедлять работу сервера. Включайте только нужные вам виды логов при необходимости!»* Секция `log` эталонного шаблона: `access: "none"`, `error: ""`, `loglevel: "warning"`, `dnsLog: false`, `maskAddress: ""`.

Поле	Подпись	JSON	Описание	По умолчанию
`logLevel`	**Уровень логов**	`loglevel`	*«Уровень журнала для журналов ошибок...»* Значения: `debug`, `info`, `warning`, `error`, `none`.	`warning`
`accessLog`	**Логи доступа**	`access`	*«Путь к файлу журнала доступа. Специальное значение «none» отключает логи доступа.»*	`none`
`errorLog`	**Логи ошибок**	`error`	*«Путь к файлу логов ошибок. Специальное значение «none» отключает логи ошибок.»*	`""` (по умолчанию)
`dnsLog`	**Логи DNS**	`dnsLog`	*«Включить логи запросов DNS»*	**false**
`maskAddress`	**Маскировка адреса**	`maskAddress`	*«При активации реальный IP-адрес заменяется на маскировочный в логах.»*	`""` (выкл.)

> Файл лога не может оказаться за пределами папки логов панели: при сборке конфигурации от значений `access` и `error` остаётся только имя файла, а сам файл создаётся в папке логов (`XUI_LOG_FOLDER`, в Linux по умолчанию `/var/log/x-ui`). Пустое значение и `none` не меняются. Ключи в другом регистре (`Access`, `ERROR` — ядро принимает и их) приводятся к `access` / `error`; если заданы оба варианта, берётся ключ в нижнем регистре.

Статистика (`stats` / `policy`)

Группа **Статистика**. Включает счётчики в `policy.system` и `policy.levels`. В эталонном шаблоне: `statsInboundUplink: true`, `statsInboundDownlink: true`, `statsOutboundUplink: false`, `statsOutboundDownlink: false`; для уровня `0` — `statsUserUplink: true`, `statsUserDownlink: true`.

Поле	Подпись	Описание	По умолчанию
`statsInboundUplink`	**Статистика входящего аплинка**	*«Включает сбор статистики для исходящего трафика всех входящих прокси.»*	**true**
`statsInboundDownlink`	**Статистика входящего даунлинка**	*«Включает сбор статистики для входящего трафика всех входящих прокси.»*	**true**
`statsOutboundUplink`	**Статистика исходящего аплинка**	*«Включает сбор статистики для исходящего трафика всех исходящих прокси.»*	**false**
`statsOutboundDownlink`	**Статистика исходящего даунлинка**	*«Включает сбор статистики для входящего трафика всех исходящих прокси.»*	**false**

> Статистика по клиентам и inbounds (uplink/downlink) — основа отображения трафика в дашборде и у клиентов; отключать её не рекомендуется. Outbound-статистика по умолчанию выключена и нужна, только если вы отслеживаете трафик по тегам исходящих.

Metrics

В эталонном шаблоне присутствует секция `metrics` (`listen: "127.0.0.1:11111"`, `tag: "metrics_out"`) и соответствующий API `metrics_out`. Панель использует этот listener для сбора метрик и снимков observatory: она парсит `metrics.listen` из шаблона, опрашивает `/debug/vars` и агрегирует историю задержек по тегам. Если вы меняете адрес/порт `metrics.listen`, панель будет обращаться к новому адресу; удаление секции `metrics` отключит сбор графиков observatory.

> Тестирование outbound в HTTP-режиме поднимает **отдельный временный** экземпляр Xray со своим `metrics`-listener на случайном порту — это не тот же listener, что в основном конфиге.

11.11. Сохранение, перезапуск и автоматические преобразования

Кнопки

Кнопка	Действие
Сохранить	`POST /xray/update`: валидирует и сохраняет шаблон + `outboundTestUrl`.
Перезапуск Xray	Перезагружает сервис с сохранённой конфигурацией. Подтверждение: *«Перезапустить xray?»* / *«Перезагружает сервис xray с сохранённой конфигурацией.»*

Тосты: успех — *«Xray успешно перезапущен»*, *«Xray успешно остановлен»*; ошибки — *«Произошла ошибка при перезапуске Xray.»*, *«Произошла ошибка при остановке Xray.»* Окно **Вывод перезапуска Xray** показывает диагностический вывод ядра.

Горячее применение изменений (без полного перезапуска)

Изменения inbounds, outbounds и правил маршрутизации применяются «вживую»: при нажатии **Сохранить** панель вычисляет разницу между старым и новым конфигом и применяет только изменившиеся части через gRPC-API Xray (HandlerService/RoutingService), не перезапуская процесс. Полный перезапуск выполняется автоматически лишь тогда, когда меняются секции без API горячей перезагрузки (`log` , `dns` , `policy` , `observatory` и т. п.). Поэтому на странице Xray отдельная кнопка «Перезапустить» не нужна — **Сохранить** само применяет изменения. Перезапуск ядра при необходимости по-прежнему выполняется автоматически (см. также авто-перезагрузку при обновлении подписок и ротации WARP).

Клиенты меняются поштучно — добавлением и удалением пользователя в работающем inbound'е, без его пересоздания — у VLESS, VMess, Trojan, Hysteria, MASQUE, а с 3.10.2 и у Shadowsocks. Раньше любая правка клиента Shadowsocks пересоздавала inbound и рвала все соединения на его порту.

Восстановление шаблона по умолчанию

Эндпоинт `GET /xray/getDefaultJsonConfig` возвращает эталонный шаблон (`config.json` , встроенный в бинарник). Им можно сбросить конфигурацию к заводской.

Автоматические преобразования при сохранении

При сохранении настроек Xray панель выполняет (в этом порядке):

1. **Снятие обёрток** — снимает обёртки вида `{ "xraySetting": <конфиг>, "inboundTags": …, "outboundTestUrl": … }` , если они случайно попали в значение (иначе слои накапливались бы при каждом сохранении). Снимается до 8 слоёв.
2. **Проверка конфигурации** — JSON разбирается в структуру конфигурации Xray; при ошибке — отказ с *«xray template config invalid»*.
3. **Гарантия правила статистики** — правило `inboundTag: ["api"] → outboundTag: "api"` принудительно поднимается на позицию 0 в `routing.rules` (или добавляется, если отсутствует). Это гарантирует, что запрос gRPC-статистики панели не будет перехвачен вышестоящим catch-all-правилом (иначе клиенты могут отображаться офлайн с нулевым трафиком при работающем прокси).

> Из-за п. 3 не пытайтесь убрать или передвинуть правило `api → api` — панель всё равно вернёт его на место при следующем сохранении. Это служебная инфраструктура статистики, а не пользовательский маршрут.

Одноразовые преобразования шаблона при обновлении до 3.10.0

В комплекте 3.10.0 — Xray-core 26.10.10, который снова изменил формат части масок. При первом запуске 3.10.0 панель один раз выполняет два преобразования (каждое отмечается в `history_of_seeders` и не повторяется):

1. **Маски XDNS → формат `names` / `addrs`** (`XdnsFinalmaskNamesAddrsFix`). Домен маски теперь задаётся списком `names` вместо одиночного `name`, а резолвер — списком `addrs` из URL вида `udp://…` или `tcp://…` вместо объекта `{type, settings: {addr}}`. Старые ключи новое ядро не отвергает, а молча пропускает: маска загрузилась бы без домена и без резолвера, и туннель ничего бы не передавал. Переписываются маски в inbound'ах, хостах, кэше подписочных outbound'ов, шаблоне (`xrayTemplateConfig`) и маске JSON-подписки (`subJsonFinalMask`); принимаются и списки строк до 3.9.0, и объекты 3.9.0, а записи, которые старый загрузчик отвергал, отбрасываются. Это преобразование заменило `XdnsFinalmaskObjectsFix` из 3.9.0, поэтому при обновлении сразу с 3.8.x маски переводятся прямо в новый формат. Формат на проводе не изменился, но клиенты на ядре старше 26.10.10 новую форму конфигурации не прочитают.
2. **`quicParams.udpHop` → маска `udphop`** (`UDPHopClientMaskFix`). С xray-core 26.9.9 клиент прыгает по портам только через маску `udphop`, а панель продолжала писать старый ключ — outbounds Hysteria/XHTTP и клиенты JSON-подписки молча перестали менять порт. Диапазон переносится в маску `udphop` режима `intervalRemote` в outbound'ах шаблона, кэше подписочных outbound'ов, хостах и маске JSON-подписки. Inbound'ы не затрагиваются: у них `udpHop` — диапазон, объявляемый в ссылках, а маска `udphop` на слушателе не работает.

Те же исправления панель применяет к outbound'ам шаблона и при каждой сборке конфигурации Xray, поэтому шаблон в старом формате, вставленный или восстановленный позже, тоже заработает. В редакторе маски XDNS домены (`names`) и резолверы (`addrs`) вводятся списками тегов, а типы записей больше не обязательны: ядро по умолчанию берёт TXT на клиенте и A/CNAME/TXT/AAAA на сервере. **Перед обновлением сделайте бэкап базы.**

Одноразовые преобразования шаблона при обновлении до 3.9.0

Xray-core 26.9.30 изменил ещё часть ключей, поэтому при первом запуске 3.9.0 панель один раз переписывает сохранённый шаблон (`xrayTemplateConfig`); каждое преобразование отмечается в `history_of_seeders` и не повторяется:

1. **Маски XDNS (Final Mask) → объектный формат** (`XdnsFinalmaskObjectsFix`). Списки строк заменяются объектами `{name, types, edns0, lenLimit, labelLimit}` и объектами резолверов; преобразуются inbound'ы, хосты, шаблон, маска JSON-подписки и кэш подписочных outbound'ов. Формат масок в протоколе изменился — старым клиентам нужен новый клиентский Xray, а несмигрированная маска не даёт ядру стартовать.
2. **WireGuard-outbound: стратегия доменов** (`WireguardDomainStrategyFix`). `settings.domainStrategy` и режим remoteDNS `"local"` переносятся в `sockopt.domainStrategy` и `targetStrategy`; оставшийся `"local"` ядро больше не запускает. В форме WireGuard/WARP-outbound селектор `domainStrategy` убран.

Кроме того, `remoteDNS`, не являющийся IP, теперь отклоняется при сохранении шаблона и у подписочных outbound'ов.

Одноразовые преобразования шаблона при обновлении до 3.8.0

xray-core 26.9.8–26.9.9 перестал принимать часть ключей и объявил устаревшими другие, поэтому при первом запуске 3.8.0 панель один раз переписывает сохранённый шаблон (`xrayTemplateConfig`). Каждое преобразование отмечается в таблице `history_of_seeders` и больше не повторяется; шаблон с некорректным JSON пропускается с записью в лог.

1. **`proxySettings` → `dialerProxy`.** Outbound с `proxySettings` ядро больше не загружает (ошибка о снятой функции с отсылкой к `streamSettings.sockopt.dialerProxy`). Тег из `proxySettings.tag` переносится в `streamSettings.sockopt.dialerProxy` (уже заданный `dialerProxy` не перезаписывается), сам ключ удаляется. Заодно у outbounds `freedom` из `sockopt` удаляется `addressPortStrategy` — с ним ядро не запускается (*freedom outbound does not support "sockopt.addressPortStrategy"*).
2. **Стратегия freedom → `sockopt.domainStrategy`.** Значение из корневого `targetStrategy` или из `settings.targetStrategy` / `settings.domainStrategy` (в порядке, в котором их читает ядро) переносится в `streamSettings.sockopt.domainStrategy` , старые ключи удаляются; `AsIs` и недопустимые значения просто убираются. Иначе ядро при каждом запуске пишет предупреждение об устаревшем ключе.
3. **DNS-outbound: `nonIPQuery` / `blockTypes` → `rules`.** Ключи заменяются тремя правилами, которые ядро раньше строило из них само (режим — значение `nonIPQuery` , по умолчанию `reject`): для перечисленных в `blockTypes` `qType` — `drop` (в режиме `reject` — `return` с `rCode: 5`), затем `hijack` для `qType` `1,28` (A и AAAA), затем правило для остальных запросов: `direct` при `skip` , `drop` при `drop` , `return` с `rCode: 5` при `reject` и любом другом значении. Если у outbound уже есть `rules` , старые ключи просто удаляются.
4. **Freedom с протоколом не в нижнем регистре.** Если протокол записан, например, как `"Freedom"` , к нему повторно применяются прежние исправления `finalRules` (включая запрет выхода в приватные сети), которые раньше такой outbound пропускали.

Вручную добавленный `proxySettings` теперь не пройдёт проверку при сохранении шаблона — сообщение придёт от ядра. Прыжки по портам Hysteria2 (`quicParams.udpHop`) в 3.8.0 не мигрировались — их переносит преобразование `UDPHopClientMaskFix` при обновлении до 3.10.0 (см. выше).

11.12. Outbound из подписки (с авто-обновлением)

Начиная с версии 3.3.0 панель умеет импортировать `outbound` 'ы напрямую из URL подписки — того же формата, что отдают провайдеры VPN для клиентских приложений. Подписки регулярно перечитываются в фоне, поэтому набор `outbound` 'ов на сервере поддерживается в актуальном состоянии без ручного редактирования шаблона конфигурации. С 3.5.0 у `ss://` -ссылок (SIP002) с query-параметрами (`?plugin=` , `?type=`) или хвостовым `/` порт разбирается корректно (раньше он молча становился `0`); ссылка с действительно некорректным портом пропускается, а не импортируется сломанной.

В русском интерфейсе раздел называется **«Подписки исходящих»**, описание: «Импорт исходящих из удалённых URL подписок (vmess/vless/trojan/ss/…). Теги остаются неизменными для использования в балансировщиках и правилах маршрутизации. Обновление выполняется автоматически.» Раздел расположен на странице Xray, над панелью настройки `outbound` 'ов.

Как это устроено

Подписки хранятся отдельно от шаблона конфигурации Xray. Шаблон **никогда не перезаписывается**: полученные из подписок `outbound`'ы добавляются в итоговую конфигурацию на лету при каждой генерации конфига Xray.

Добавление подписки

В форме «Добавить подписку» доступны следующие поля:

Поле	Ключ	По умолчанию	Назначение
URL подписки	`url`	— (обязательно)	Адрес подписки. Плейсхолдер: «https://... (список ссылок в base64)». Принимается только HTTP(S); адрес проверяется на безопасность.
Примечание	`remark`	пусто	Произвольная метка (плейсхолдер «напр. узлы НК»).
Префикс тега	`tagPrefix`	`subN-`	Префикс, с которого начинаются теги импортируемых `outbound`'ов. Если оставить пустым, панель сама подберёт наименьший свободный номер вида `sub1-`, `sub2-` и т.д.
User-Agent	`userAgent`	пусто (= `3x-ui-outbound-sub/1.0`)	Заголовок `User-Agent` для загрузки подписки — и при обновлениях, и в предпросмотре. Нужен провайдерам, которые отдают ссылки только «своим» клиентам. Пробелы по краям обрезаются; пустое значение — стандартный `3x-ui-outbound-sub/1.0` (он же в плейсхолдере). Это User-Agent **этого outbound-источника**; User-Agent для загрузки внешних подписок самого клиента (вкладка «Links») задаётся отдельной глобальной настройкой `externalSubUserAgent` (см. 10.2).
Интервал обновления	`updateInterval`	600 секунд (10 минут)	Как часто подписка перечитывается. В UI задаётся в часах/минутах.
Включено	`enabled`	да (`true`)	Только включённые подписки попадают в конфиг и обновляются автоматически.
Разрешить приватные адреса	`allowPrivate`	нет (`false`)	Разрешает URL на localhost, LAN и приватные IP. По умолчанию выключено в целях защиты от SSRF — включайте только для доверенного локального источника.
Разрешить небезопасное	`allowInsecure`	нет (`false`)	Не проверять TLS-сертификат сервера подписки (подсказка: «Пропустить проверку TLS-сертификата (allowInsecure / skip-cert-verify).»).
Перед ручными исходящими	`prepend`	нет (`false`)	Если включено, `outbound`'ы этой подписки ставятся **перед** ручными `outbound`'ами шаблона, и один из них может стать `outbound`'ом по умолчанию. Иначе они добавляются **после**.

Кнопка **«Предпросмотр»** (`POST /outbound-subs/parse`) позволяет до сохранения скачать и распарсить URL (с указанными в форме **User-Agent** и **«Разрешить небезопасное»**) и увидеть, какие `outbound`'ы и теги

получатся; ничего в базу при этом не пишется. Если по URL ничего не распознано, выводится «По этому URL исходящих не найдено.»

Порядок нескольких подписок в общем списке `outbound`'ов задаётся приоритетом (`priority`) и меняется стрелками вверх/вниз (`POST /outbound-subs/:id/move`).

Какие форматы подписки принимаются

Тело ответа по URL обрабатывается так:

- Содержимое сначала пробуется как **base64** (стандартный и URL-safe варианты, с автодополнением паддинга и удалением пробелов/переводов строк). Если это base64 — оно декодируется; иначе берётся как есть.
- Затем тело разбивается на строки. Каждая непустая строка, не начинающаяся с `#`, разбирается как ссылка. Нераспознанные строки (комментарии, неподдерживаемые протоколы) молча пропускаются.
- Поддерживаемые схемы ссылок: `vmess://`, `vless://`, `trojan://`, `ss://` (Shadowsocks), `hysteria2://` / `hy2://`, `wireguard://` / `wg://`.

То есть подходит обычная подписка вида «base64-кодированный список ссылок», как у большинства провайдеров.

Стабильные теги

Каждой ссылке вычисляется устойчивая «идентичность» (ядро URI без фрагмента-примечания; для vmess — внутренний JSON без поля `ps`). Соответствие «идентичность → тег» сохраняется, и при следующем обновлении тот же сервер получает тот же тег, даже если изменились примечание или второстепенные параметры. Это специально сделано, чтобы балансировщики и правила маршрутизации продолжали работать после обновлений:

- Точный тег в балансировщике/правиле продолжит указывать на тот же сервер.
- Префиксный/wildcard-селектор (например, `hk-*`) автоматически подхватит новые серверы, которые подписка вернёт позже — это рекомендуемый способ «подписаться на пул».
- Если сервер исчезает из подписки, его тег просто пропадает из итогового массива `outbound`'ов; при наличии `fallbackTag` у балансировщика Xray использует его.
- Если провайдер сменил UUID/хост/учётные данные сервера, идентичность меняется — это считается новым `outbound`'ом. Панель попробует дать ему тег, который стоял на той же позиции в прошлой выгрузке, но только если этот тег не принадлежит серверу, по-прежнему присутствующему в подписке; иначе выдаётся новый тег.
- Сервер, вставленный в подписку между уже известными, получает собственный тег и не забирает тег соседа по позиции (раньше мог забрать, а законный владелец получал суффикс `-N`, и перестановка закреплялась при следующих обновлениях).

Внутри одной выгрузки теги дедуплицируются суффиксом `-N`. Теги из подписок сохраняют не-ASCII символы (например, кириллицу) и остаются читаемыми: буквы и цифры Unicode сохраняются в slug, а пунктуация заменяется на дефис — теги из кириллических имён больше не сводятся к одним цифрам.

Как работает авто-обновление

- Фоновая задача обновления подписок запускается по расписанию **каждые 5 минут**.
- На каждом запуске она перебирает все включённые подписки и обновляет только те, у которых истёк собственный интервал: подписка обновляется, если ещё ни разу не обновлялась либо если с последнего обновления прошло не меньше её `updateInterval`. Таким образом задача проверяет подписки часто, но каждая конкретная подписка перечитывается не чаще своего `updateInterval` (по умолчанию 10 минут). В UI это отражено соответствующей подсказкой.
- Обновление: URL заново проверяется на безопасность как публичный (приватные адреса блокируются, если у подписки не выставлен `allowPrivate`), запрос идёт через прокси-клиент панели с заголовком `User-Agent` из настроек подписки (по умолчанию `3x-ui-outbound-sub/1.0`). Цепочка редиректов ограничена 10 переходами, и каждый хоп тоже проверяется на приватность (защита от SSRF). Ожидается HTTP 200; иначе фиксируется ошибка.
- После успешного парсинга результат сохраняется, проставляется время последнего обновления, очищается ошибка. При ошибке её текст виден в UI как «Последняя ошибка», а ранее полученные `outbound`'ы остаются в силе.
- Если хотя бы одна подписка реально обновилась, задача помечает Xray на перезапуск и шлёт UI-инвалидацию, чтобы интерфейс подтянул новые `outbound`'ы. Фактическая перезагрузка Xray происходит на ближайшем 30-секундном цикле менеджера.

Ручное обновление одной подписки — кнопка **«Обновить сейчас»** (`POST /outbound-subs/:id/refresh`); она тоже помечает Xray на перезапуск. Добавление, изменение, удаление подписки также взводят флаг перезапуска Xray (при удалении его `outbound`'ы выпадают из конфига на следующей перезагрузке). UI подсказывает: «После добавления или обновления перезапустите Xray (или дождитесь следующей автоперезагрузки), чтобы исходящие стали активными.»

Как это попадает в конфиг Xray

При каждой генерации конфигурации Xray активные подписочные `outbound`'ы делятся на две группы — `prepend` (флаг «Перед ручными исходящими») и остальные — и сшиваются с шаблоном: `[prepend подписок] + [outbound'ы шаблона] + [остальные подписки]`. Внутри каждой группы подписки идут по приоритету. Ручные `outbound`'ы из шаблона при этом не затрагиваются; если массив `outbound`'ов шаблона почему-то не парсится, подписочные `outbound`'ы в него не подмешиваются (чтобы не потерять ручные).

Импортированные `outbound`'ы дополнительно отображаются в самой панели `outbound`'ов отдельным блоком **«Из подписок исходящих (только для чтения)»** — редактировать их там нельзя, управление только через раздел «Подписки исходящих».

11.13. Ротация IP в WARP

В 3X-UI можно поднять WARP-outbound — исходящее WireGuard-подключение к Cloudflare WARP (тег `warp` в конфиге Xray). Панель сама регистрирует на серверах Cloudflare устройство-аккаунт, получает WireGuard-ключи и адреса и подставляет их в outbound с тегом `warp`. Через такой outbound трафик выходит в интернет под IP-адресом Cloudflare WARP. Новинка версии 3.3.0 — возможность менять этот исходящий IP вручную или по расписанию, не пересоздавая аккаунт WARP вручную.

Управление находится в разделе **Xray** в карточке WARP (после нажатия «Создать аккаунт WARP» и получения конфига; до этого действия недоступны — панель подскажет «Сначала получите WARP-конфиг»).

Что происходит при смене IP

Кнопка **«Сменить IP»** запускает смену IP. Логика:

1. Генерируется новая пара ключей WireGuard.
2. С новым ключом заново регистрируется устройство WARP на серверах Cloudflare (новый `device_id`, `access_token`, адреса и peer-данные).
3. Новые данные записываются в WARP-outbound конфига Xray: обновляются `secretKey`, `address` (v4 `/32` и v6 `/128`), `reserved` (из `client_id`), а также `publicKey` и `endpoint` у peer.
4. Если ранее был задан лицензионный ключ WARP+ (длиной не менее 26 символов), он автоматически переустанавливается на новый аккаунт. При неудаче это лишь предупреждение в логах — смена IP не отменяется.
5. После успешной смены Xray помечается как требующий перезапуска, чтобы новый outbound вступил в силу.

При успехе интерфейс показывает «IP-адрес WARP успешно изменён!».

Автоматическая ротация по расписанию

В карточке WARP есть переключатель **«Автоматическое обновление IP-адреса»** и поле **«Интервал (дни)»**. Подсказка: «0 — отключить. Автоматически меняет IP-адрес.»

Параметр	Значение
Настройка в БД	`warpUpdateInterval` (целое, ⩾ 0)
Значение по умолчанию	`0` (авто-ротация выключена)
Единица измерения	дни
`0`	отключает автоматическую смену
`> 0`	менять IP каждые N дней

Запись интервала сохраняет `warpUpdateInterval`, и при значении больше 0 сбрасывает «время последнего обновления» на текущий момент — иначе планировщик сменил бы IP уже на ближайшем тике.

Расписание исполняет фоновая задача, запускаемая раз в час — то есть панель раз в час проверяет, не пора ли ротировать. Алгоритм проверки:

- если интервал ⩽ 0 — ничего не делает;
- если «время последнего обновления» равно 0 (например, интервал выставили прямой правкой БД) — это первый запуск: задача лишь фиксирует базовую отметку времени и НЕ меняет IP сразу;
- если с момента последнего обновления прошло не меньше `интервал × 24 × 3600` секунд — выполняется та же смена IP, обновляется отметка времени и планируется перезапуск Xray.

Важная деталь: ручная смена по кнопке «Сменить IP» тоже сбрасывает отметку времени последнего обновления. Поэтому после ручной ротации отсчёт автоматического интервала начинается заново и плановая смена не сработает сразу следом.

«Через прокси панели»

> **Изменено в 3.3.1.** Отдельная настройка «Сетевой прокси панели» (`panelProxy`) убрана. Исходящий трафик самой панели (в том числе запросы к WARP API) теперь направляется через выбранный **outbound для трафика панели** — Xray-outbound или балансировщик (см. раздел 13). Описание ниже относится к версиям до 3.3.1.

Все запросы к API Cloudflare WARP (регистрация, получение конфига, установка лицензии, смена IP) идут не напрямую, а через HTTP-клиент панели с таймаутом 15 секунд. Этот клиент уважает настройку **«Сетевой прокси панели»** (`panelProxy`) из настроек панели.

Из описания настройки: прокси маршрутизирует собственные исходящие запросы панели (обновления geo-баз, проверки версий Xray/панели, Telegram, а теперь и обращения к WARP) — чтобы обходить серверную фильтрацию. Принимаются адреса вида `socks5://` или `http(s)://` , например локальный SOCKS-входящий самого Xray. Если поле пустое или прокси задан неверно — используется прямое подключение (поведение не ломается).

Польза для WARP: если сервер не может напрямую достучаться до `api.cloudflareclient.com` , регистрация и ротация раньше падали. Теперь, указав в `panelProxy` рабочий прокси (в т.ч. собственный inbound Xray), можно гарантировать доступность WARP API и работоспособность как ручной кнопки, так и плановой ротации.

Когда это полезно

- Регулярная смена исходящего IP для outbound, который ходит через WARP, — снижает риск блокировок и трекинга по одному адресу.
- «Освежить» IP вручную, если текущий адрес Cloudflare попал в чёрные списки или работает медленно.
- Серверы, у которых нет прямого доступа к Cloudflare WARP API: маршрутизация запросов через `panelProxy` делает регистрацию и ротацию работоспособными.

11.14. Lua-скрипты маршрутизации и DNS

С 3.10.0 (Xray-core 26.10.10) маршрутизацию и DNS можно передать скриптам на Lua. Сами скрипты панель не редактирует — она задаёт только путь к файлу `.lua` , который должен лежать на сервере. Для этого три поля:

Где	Поле	JSON	Что задаёт
Основное — под «Настройка маршрутизации доменов» (см. 11.2)	**Скрипт маршрутизации**	`routing.script`	Файл с функцией `HandleRoute` . Плейсхолдер `routing.lua` . Шаблон с этим полем сохранить нельзя (см. ниже).

Где	Поле	JSON	Что задаёт
DNS — общие параметры, видны при включённом DNS (см. 11.6)	**Lua-скрипт**	`dns.script`	Файл с функцией `HandleDNSQuery`. Плейсхолдер `dns.lua`.
DNS — окно DNS-сервера	**ID**	`dns.servers[].id`	Имя, по которому DNS-скрипт выбирает этот сервер.

Путь — абсолютный или относительно папки `bin` Xray. Пустое поле удаляет ключ из шаблона: пустой путь ядро отвергает.

- **Скрипт маршрутизации заменяет правила и `domainStrategy`, поэтому ядро не запускается, если рядом с ним есть хотя бы одно правило или `domainStrategy`.** Панель всегда держит первым служебное правило `api → api` (см. 11.11), так что **сохранение шаблона с непустым `routing.script` отклоняется** ошибкой `xray template routing: "script" cannot be used …`. Скрипт, уже лежащий в сохранённом шаблоне, панель при сборке конфигурации отбрасывает и пишет в лог предупреждение, чтобы ядро запускалось; остальные правила остаются. Подсказка поля: *«…направляйте входящий api на исходящий api, иначе остановятся статистика трафика и применение изменений клиентов на лету.»* Пока скрипт задан, над правилами на странице «Маршрутизация» висит предупреждение (см. 11.3).
- **DNS-скрипт отвечает на все запросы сам** и обращается к DNS-серверам только из своего кода — выбирая их по **ID**.
- **DNS-скрипт нельзя сочетать** с переключателями **Отключить резервный DNS**, **Отключить резервный DNS при совпадении**, **Включить параллельные запросы** и с серверами, у которых заданы **Домены**, **Пропустить Fallback** или **Финальный запрос**: ядро такой конфиг отвергает, и сохранение шаблона отклоняется ошибкой `xray template dns: "script" cannot be used with …`. Такой скрипт в уже сохранённом шаблоне при сборке конфигурации отбрасывается с предупреждением в логе; остальные настройки DNS остаются.
- **Xray не запустится**, если файл отсутствует или содержит ошибки, — проверьте путь до сохранения.
- **Скрипт загружается при старте ядра.** Изменение пути в поле при сохранении приводит к полному перезапуску Xray, а после правки самого файла `.lua` перезапустите Xray вручную. Горячее применение правил маршрутизации скрипт не затрагивает.
- Ключи `dns.script` и `id` DNS-серверов, вписанные в JSON вручную, при сохранении со страницы Xray не теряются. Подсказка у поля **Скрипт маршрутизации** в интерфейсе («Xray игнорирует все правила…») описывает прежнее поведение ядра.

---

12. Узлы (мультипанель, master/slave)

Раздел **Узлы** превращает обычную установку 3X-UI в **центральную (мастер-) панель**, которая удалённо наблюдает за другими (дочерними) панелями 3X-UI и управляет ими. Каждый узел — это отдельная установка 3X-UI на своём сервере; мастер обращается к ней по её собственному HTTP API, опрашивает её состояние и синхронизирует на неё inbounds и клиентов, которые им назначены. Это и есть возможность **мультипанели**: вместо того чтобы заходить в каждую панель по отдельности, вы видите все серверы в одном списке и управляете ими централизованно.

Важный принцип: **узел — это не агент, а полноценная панель 3X-UI.** Мастер ничего на ней не «устанавливает» — он лишь подключается к её API по токену. Удаление узла из списка останавливает только мониторинг; сама удалённая панель при этом не затрагивается (подсказка: «Это остановит мониторинг узла. Сама удалённая панель не будет затронута»).

12.1. Сводка вверху списка

Над таблицей узлов выводятся агрегированные счётчики:

Поле	Описание
Всего узлов	Общее число узлов в списке.
В сети	Сколько узлов имеют статус `online`.
Не в сети	Сколько узлов имеют статус `offline`.
Средняя задержка	Усреднённая задержка (ping) до узлов, в миллисекундах.

12.2. Добавление и редактирование узла

Кнопки **Добавить узел** и **Изменить узел** открывают форму с полями узла.

Обязательны (подсказка: «Имя, адрес, порт и токен API обязательны») поля **Имя**, **Адрес**, **Порт** и **API Токен**.

При нажатии «Сохранить» (как при добавлении, так и при изменении) панель **сначала проверяет достижимость узла** с таймаутом 6 секунд. Если узел не отвечает, запись не сохраняется и показывается ошибка. То есть нельзя добавить заведомо недоступный узел.

Поля формы

Поле (RU)	По умолчанию	Допустимые значения	Описание
Имя	— (обязательно)	непустая строка, **уникальная**	Внутреннее имя узла. На колонку имени наложена уникальность — два узла с одинаковым именем создать нельзя. Подсказка-плейсхолдер: `napr. de-frankfurt-1`. При сохранении пробелы по краям обрезаются.
Примечание	пусто	любая строка	Необязательная заметка/описание узла. На работу не влияет.

Поле (RU)	По умолчанию	Допустимые значения	Описание
Схема	`https`	`http` / `https`	Протокол подключения к удалённой панели. Если оставить пустым или указать недопустимое значение, нормализация выставит `https`. Если узел отвечает по обычному HTTP, а схема стоит `https`, панель вернёт понятную подсказку: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Адрес	— (обязательно)	хост или IP	Адрес удалённой панели. Плейсхолдер: `panel.example.com` или `1.2.3.4`. Адрес нормализуется; по умолчанию приватные/локальные адреса запрещены ради защиты от SSRF — см. «Разрешить частный адрес».
Порт	— (обязательно)	целое **1–65535**	Порт веб-панели удалённого узла. Значения вне диапазона отклоняются («node port must be 1-65535»).
Базовый путь	`/`	строка пути	Базовый путь (web base path) удалённой панели, если он задан. Нормализуется: гарантированно начинается и заканчивается на `/` (пустое значение → `/`). К нему панель дописывает `panel/api/server/status` при опросе.
API Токен	— (обязательно)	токен удалённой панели	Bearer-токен для доступа к API узла. Передаётся в заголовке `Authorization: Bearer <token>`. Плейсхолдер: «Токен со страницы Настроек удалённой панели». Подсказка: «Удалённая панель показывает свой токен API в разделе Настройки → Токен API». То есть токен надо создать **на самом узле** (Настройки → Токен API), а сюда вставить.
Включён	`true`	да/нет	Включает мониторинг и синхронизацию узла. Выключенные узлы **не опрашиваются** фоновыми задачами (heartbeat и traffic-sync их пропускают) и не участвуют в массовом обновлении панели.
Разрешить частный адрес	`false`	да/нет	Снимает SSRF-защиту и разрешает подключаться к узлу по приватному/локальному адресу. Подсказка: «Включить только для узлов в частной сети или VPN». Включайте только когда узел действительно находится в приватной сети или доступен через VPN.

Получение и перегенерация токена на стороне узла

Токен берётся на удалённой панели в разделе **Настройки → Токен API**. Там же его можно перевыпустить: кнопка **Сгенерировать токен заново** с предупреждением: «Повторная генерация аннулирует текущий токен. Любая центральная панель, использующая его, потеряет доступ до обновления. Продолжить?». После перегенерации старый токен в мастер-панели перестанет работать — его нужно обновить в форме узла.

Область токена узла. Для узла подходит токен с областью `admin` (по умолчанию, в том числе у `x-ui setting -getApiToken`), `node-sync` или `node-admin` — **«Администратор узла»** (см. 13.9). Для приёма данных мастера с 3.10.0 они работают одинаково полно:

- Мастер помечает каждый свой запрос к узлу заголовком `X-3x-Master-Push`, и узел применяет присланных клиентов и флаг включения inbound'а так же, как при токене `node-sync`. В 3.9.0 узел с `admin`-токеном принимал такую отправку за сохранение устаревшей формы и оставлял прежних клиентов: клиенты, привязанные на мастере («Привязать существующих клиентов…», массовая

привязка, досылка на «грязный» узел), до узла не доходили, хотя отправка числилась успешной и больше не повторялась. Заголовок читает узел, поэтому **обновите узлы до 3.10.0**. Форма в браузере его не шлёт, так что защита от сохранения устаревшей формы для обычных правок не изменилась.
- Токену `node-sync` (и мастеру, подключённому по mTLS) открыты все эндпоинты, которые вызывает мастер. Раньше на узел не доходили порядок inbound'ов в подписке (`/inbounds/:id/subSortIndex`) и массовый сброс трафика нескольких клиентов (`/clients/bulkResetTraffic`), а без `/clients/activeInbounds` атрибуция онлайна молча сводилась к сопоставлению только по email. Обновление панели узла (`/server/updatePanel`) для `node-sync` закрыто: оно доступно токенам `admin` и `node-admin` (права `node-sync` плюс запуск обновления ПО панели). Мастеру, подключённому по mTLS, достаются права `node-sync` ; до `node-admin` они повышаются только запросом с bearer-токеном именно этой области (#6666).

Исходящее подключение (Connection outbound)

Поле **Connection outbound** (Исходящее подключение, `outboundTag`) задаёт, как трафик обращений мастера к API этого узла покидает сервер. Если выбрать в нём тег Xray-outbound, обращения панели к узлу пойдут не напрямую, а через указанный outbound; панель сама добавит в рабочую конфигурацию мост-inbound на loopback и применит изменение на лету, без перезапуска. Подсказка: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Селектор устроен как панельный выбор outbound: теги сгруппированы на **Outbounds** (обычные исходящие) и **Balancers** (балансировщики), blackhole-исходящие из списка скрыты. Пустое значение (плейсхолдер «Direct connection») = прямое подключение к узлу.

Импорт inbound (выбор синхронизируемых inbound-ов)

В форме узла есть настройка **Импорт inbound** (`inboundSyncMode`) с двумя режимами: **Все inbound** (`all` , по умолчанию) и **Выбранные** (`selected`). По умолчанию мастер синхронизирует на узел все inbounds, у которых выбран этот узел; существующие узлы продолжают работать в режиме «Все inbound».

В режиме **Выбранные** под полем появляется мультивыбор тегов inbound. Нажмите **Загрузить inbound** — мастер по введённым (ещё не сохранённым) параметрам подключения запросит у узла список его inbound-ов (эндпоинт `POST /panel/api/nodes/inbounds`) и покажет их теги; отметьте нужные. Панель будет синхронизировать и разворачивать на узел только отмеченные теги, а остальные inbounds, существующие непосредственно на узле, останутся нетронутыми — мастер их не удаляет и ими не управляет.

Если при сохранении формы выборка расширяется — отмечен новый тег или узел переключён с **Выбранные** на **Все inbound**, — новые inbounds узла сначала импортируются в мастер при следующей успешной синхронизации, и лишь после неё мастер снова удаляет с узла inbounds, которых на мастере нет. Поэтому только что отмеченный inbound не удаляется с узла как «удалённый на мастере». Очистка при этом не отключается: inbound, удалённый на мастере, пока узел был недоступен, удаляется и с узла после восстановления связи, в режиме **Выбранные** — в том числе inbound, созданный из панели (на мастере его тег хранится с префиксом `n<id>-` , а на узле — без него). Если же расширить выборку во время такой недоступности, удалённый на мастере inbound не удалится с узла, а импортируется обратно.

Пример: запросить список inbound-ов узла для выборочного импорта. В теле передаются ещё не сохранённые параметры подключения; в ответе — теги доступных на узле inbound-ов:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Проверка TLS (для https-узлов)

Группа полей задаёт, как мастер проверяет HTTPS-сертификат узла. Эти настройки **актуальны только для схемы** `https`; для `http`-узлов они игнорируются.

Проверка TLS — выпадающий список, подсказка: «Как панель проверяет HTTPS-сертификат узла. Закрепление или Пропуск — для самоподписанных сертификатов (только https-узлы)».

Режим (RU)	Значение	По умолчанию	Описание
Проверять (стандартный CA)	`verify`	да (default)	Обычная проверка цепочки сертификата доверенным CA. Подходит для узлов с публичным/Let's Encrypt сертификатом. Используется также для всех `http`-узлов.
Закрепить сертификат (SHA-256)	`pin`	—	Цепочка CA не проверяется, но SHA-256 листового сертификата узла сверяется с сохранённым отпечатком (constant-time сравнение). Сохраняет защиту от MITM для **самоподписанных** сертификатов. Требует заполнить поле отпечатка.
Пропустить проверку	`skip`	—	Проверка сертификата полностью отключается. Предупреждение: «Пропуск проверки убирает защиту от атак «человек посередине» — токен API может быть перехвачен. Лучше закрепить сертификат».

К трём режимам выше в 3.4.0 добавлен четвёртый — **Mutual TLS (client certificate)** (`mtls`), доступный, как и остальные, только для схемы `https`.

Режим (RU)	Значение	По умолчанию	Описание
Mutual TLS (клиентский сертификат)	`mtls`	—	Помимо проверки сертификата узла, мастер дополнительно подтверждает себя узлу **клиентским сертификатом**, выпущенным его собственным CA. Для узла в этом режиме **API-токен становится необязательным** — узел опознаёт мастер по сертификату. При выборе режима показывается подсказка: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Чтобы включить взаимный TLS для узла: на стороне узла задайте режим **Mutual TLS**, скопируйте CA управляющей панели из раздела **Node mTLS** (см. ниже), пропишите его на узле как **доверенный родительский CA** и перезапустите узел.

Если выбрать любое значение, кроме `skip`, `pin` или `mtls`, нормализация принудительно выставит `verify`.

Закрепление сертификата

При выборе **Закрепить сертификат** появляются:

- **SHA-256 закреплённого сертификата** — поле ввода. Принимается отпечаток в **base64** (формат `pinnedPeerCertSha256` из Xray) либо в **hex** с двоеточиями или без (стиль `openssl -fingerprint`). Подсказка: «SHA-256 сертификата узла в base64 или hex. Нажмите «Получить», чтобы считать его с узла сейчас». Плейсхолдер: «SHA-256 в base64 или hex». При выборе `pin` пустой или некорректный отпечаток вызывает ошибку валидации при сохранении.

Пример: один и тот же отпечаток в двух форматах. Поле принимает любой из вариантов — оба означают один сертификат:

```
# base64 (формат pinnedPeerCertSha256 из Xray)
6O7TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex с двоеточиями (стиль openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Если отпечаток ещё неизвестен, нажмите **Получить** — мастер сам считает его с узла по HTTPS и подставит в поле.

- Кнопка **Получить** — подключается к узлу по HTTPS без проверки сертификата и считывает SHA-256 текущего листового сертификата (эндпоинт `POST /certFingerprint`), подставляя его в поле. После успеха — «Текущий сертификат узла получен»; при неудаче — «Не удалось получить сертификат». Доступно только для https-узлов.

Node mTLS (взаимная TLS-аутентификация между панелями)

На странице **Узлы** есть отдельный раздел **Node mTLS** — настройка взаимной TLS-аутентификации, добавляющей к API-токену второй фактор (клиентский сертификат) для вызовов «панель → узел». Взаимный TLS включается по желанию; если поля раздела пусты, узлы работают по прежней схеме — **только с API-токеном** (подсказка: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). В разделе две операции:

- **Скопировать CA этой панели** (`POST /panel/api/nodes/mtls/ca`) — копирует корневой сертификат (CA) этой панели в буфер обмена. Этот CA нужно передать управляемым узлам, чтобы они доверяли клиентскому сертификату панели; на самих узлах после этого выставляется режим проверки TLS **Mutual TLS** (подсказка: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). После копирования — «CA certificate copied to clipboard».
- **Доверенный родительский CA** (`Trusted parent CA`, `POST /panel/api/nodes/mtls/trustCA`) — поле, которое используется, когда сама эта панель выступает узлом для вышестоящей (управляющей) панели. Вставьте сюда CA управляющей панели, чтобы требовать от неё клиентский сертификат, и нажмите **Save trust CA**. Изменение требует **перезапуска панели** (подсказка: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Что показывается по каждому узлу

Колонки таблицы и поля карточки узла (наблюдаемое состояние, заполняется при каждом heartbeat-опросе):

Поле (RU)	Описание
Статус	`online` / `offline` / `unknown` — см. ниже.
CPU	Загрузка процессора удалённого сервера в процентах.
Память	Использование ОЗУ в процентах (вычисляется как `current/total*100`).
Аптайм	Время непрерывной работы сервера (в секундах).
Задержка	Время ответа узла на последний опрос (мс).
Последний пинг	Время последнего успешного heartbeat (unix-секунды; `0` = «никогда»; недавнее значение показывается как «только что»).
Версия Xray	Версия Xray-core, запущенного на узле.
Версия панели	Версия 3X-UI на узле — сравнивается с актуальной для индикатора обновления.
(inbounds)	Сколько inbounds физически размещено на этом узле.
(клиенты)	Число клиентов на inbounds узла.
(онлайн)	Сколько клиентов узла сейчас в сети.
(исчерпано)	Сколько клиентов узла **истекли или исчерпали лимит трафика**. Отключённые вручную клиенты в этот счётчик не входят.
(скорость)	Текущая (живая) скорость передачи на inbound-ах, размещённых на узле.

Счётчики inbounds/клиентов/онлайн привязываются к узлу по его стабильному GUID (`panelGuid`), а не по локальному id, — чтобы клиент на подузле учитывался именно под подузлом, а не под промежуточным узлом, через который он синхронизируется.

Для inbound-ов, размещённых на узле, страница показывает онлайн-клиентов, счётчики и **текущую скорость передачи**. Привязка по стабильному GUID корректно разводит и «клонированные» узлы с одинаковым `panelGuid`.

Статусы узла

Статус	RU	Когда устанавливается
`online`	В сети	Узел ответил `success=true` на опрос `panel/api/server/status`.
`offline`	Не в сети	Узел не ответил, вернул HTTP-ошибку, `success=false` либо нераспознаваемый ответ.
`unknown`	Неизвестно	Начальное значение, пока узел ещё ни разу не опрошен.

При неуспешном опросе текст ошибки сохраняется и показывается понятной формулировкой, что помогает диагностировать причину «offline».

12.5. Действия над узлом

- **Проверить соединение** (`POST /test`) — в форме узла проверяет связь по введённым (ещё не сохранённым) параметрам с таймаутом 6 с. Результат: «Соединение в порядке ({ms} мс)» или «Не удалось подключиться». Удобно для отладки адреса/порта/токена/TLS до сохранения.
- **Проверить сейчас** (кнопка «Проверить сейчас», `POST /probe/:id`) — внеплановый опрос уже сохранённого узла; немедленно обновляет статус и метрики (CPU/память/аптайм/задержку/версии) и записывает heartbeat. При неудаче — «Проверка не удалась».

Пример: проверить и опросить узел через API мастера. «Проверить соединение» испытывает ещё не сохранённые параметры из формы:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGci...", "tlsMode": "verify" }
```

Внеплановый опрос уже сохранённого узла с id 7:

```
POST /panel/api/nodes/probe/7
```

- **Обновить панель** (`POST /updatePanel` с телом `{ids:[…]}`) — запускает на узле его штатный самообновлятель: узел скачивает последний релиз 3X-UI и перезапускается на нём. Кнопка **Обновить выбранные ({count})** выполняет это для нескольких отмеченных узлов сразу. Рядом с узлом показывается индикатор: **Доступно обновление** или **Актуально**, исходя из сравнения версии панели узла с последней. Токен узла должен иметь область `admin` или `node-admin` : токен `node-sync` обновление панели не разрешает, и узел отвечает отказом (см. 12.2).

Пример: обновить несколько узлов одним запросом. В теле передаются id отмеченных узлов; обновятся только включённые и `online` , остальные вернутся как пропущенные.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Ответ вида «Обновление запущено на 2 узлах, 1 не удалось»: узел 12, например, мог быть offline и поэтому пропущен.

- Подтверждение: «Обновить {count} узлов до последней версии? Каждый выбранный узел загрузит последний релиз и перезапустится. Обновляются только включённые узлы в сети».
- **Обновляются только включённые узлы в статусе `online` .** Выключенный узел в результатах помечается «node is disabled», offline — «node is offline». Итог: «Обновление запущено на {ok} узлах, {failed} не удалось». Если не выбрано ни одного подходящего узла — «Выберите хотя бы один включённый узел в сети».

В диалоге подтверждения обновления (как для одиночного узла, так и для массового) есть флажок **Обновить до канала разработки (последний коммит)**. Если его отметить, выбранные узлы установят rolling-сборку dev-latest (последний коммит ветки main) вместо стабильного релиза; при снятом флажке узел обновляется по своему обычному каналу. При включённом флажке под ним выводится предупреждение: «Сборки разработки отслеживают каждый коммит в main и не являются стабильными релизами — автоматического отката нет». Флаг dev пробрасывается через `POST /panel/api/nodes/updatePanel` на узел, и тот запускает обновление именно по dev-каналу.

- **Set Cert from Panel** (вспомогательное, `GET /webCert/:id`) — при создании inbound на узле позволяет подставить пути к **собственному** web-TLS-сертификату узла (а не центральной панели), чтобы файлы существовали именно на узле. Требует, чтобы узел был включён и доступен.
- **Удалить узел** (`POST /del/:id`) — подтверждение: «Удалить узел "{name}"? Это остановит мониторинг узла. Сама удалённая панель не будет затронута». Удаляет запись узла и его накопленную статистику трафика; удалённая панель продолжает работать как обычно. **Удалить узел можно только после того, как с него сняты все inbounds.** Если к узлу всё ещё привязан хотя бы один inbound (через `node_id`), панель отклонит удаление с ошибкой вида «cannot delete node: N inbound(s) still attached to it; detach or delete them first» — сначала открепите или удалите эти inbounds, затем удаляйте узел. Это исключает «осиротевшие» inbounds с висящей ссылкой на удалённый узел.

12.6. История метрик

Кнопка/график истории обращается к `GET /history/:id/:metric/:bucket`. Доступные метрики: **`cpu`** и **`mem`** — они накапливаются при каждом успешном heartbeat. Размер интервала агрегации (`bucket`, в секундах) ограничен белым списком:

Пример: запрос истории. График загрузки CPU узла 7 с агрегацией по 60-секундным интервалам (возвращается до 60 точек):

```
GET /panel/api/nodes/history/7/cpu/60
```

Для памяти и режима «реального времени» (2 с) — соответственно `…/7/mem/60` и `…/7/cpu/2`. Значения вне белого списка отклоняются («invalid metric» / «invalid bucket»).

Bucket (с)	Назначение
2	Режим реального времени
30	Интервалы по 30 с
60	Интервалы по 1 мин
120	Интервалы по 2 мин
180	Интервалы по 3 мин
300	Интервалы по 5 мин

Возвращается до 60 точек. Недопустимая метрика или bucket отклоняются («invalid metric» / «invalid bucket»).

12.7. Как синхронизируются inbounds и клиенты

Inbound «принадлежит» узлу через поле `node_id` (в редакторе inbound выбирается узел). С 3.9.0 на узлы можно разворачивать и **AmneziaWG, TUIC и MTProto** (#6306) — протокол запускает панель самого узла, своим движком/сервером. Поэтому действует порог версии узла: **MTProto — v3.5.0+, AmneziaWG — v3.7.0+, TUIC — v3.8.0+**. С 3.10.0 на узел можно разместить и inbound **MASQUE** (см. 5.15); порог в коде — узел v3.9.1+, но такого релиза не было, так что на практике узел должен быть не ниже 3.10.0. Узел ниже порога (или ещё не сообщивший свою версию) сохранение отклоняет с ошибкой вида «node … runs panel …; … inbounds need … or newer». Панельно-локальными остаются `http`, `mixed`, `tunnel` / `dokodemo` и `tun`: при их выборе узел в форме сбрасывается, а сохранение с узлом отклоняется. У MTProto-узла петлевой порт выделяет сама панель узла, а проброшенные порты AmneziaWG проверяются по inbound'ам узла, а не мастера.

Пример: токен в форме узла. Токен берётся на дочерней панели (Настройки → API Токен) и вставляется в поле **API Токен** мастера. При каждом опросе мастер шлёт его в заголовке:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Если на дочерней панели задан **базовый путь** (web base path), например `/secret/`, мастер сам подставит его перед `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Развёртывание конфигурации (reconcile).** При любом изменении inbound/клиента, привязанного к узлу, узел помечается «грязным». Фоновая задача для каждого включённого узла **в статусе `online`** при наличии изменений разворачивает на узел его inbounds (по `node_id`) и затем сбрасывает флаг «грязного» состояния. Узел, который выключен, offline или «грязный», считается «ожидающим» — деплой на него откладывается до восстановления связи.

   Операции с клиентами (создание, правка, удаление, отвязка, сброс трафика, массовые изменения) мастер, кроме того, сразу передаёт на все узлы, где размещён клиент, причём **параллельно**, а не по очереди: время операции больше не складывается из задержек всех узлов. На ответ каждого узла отводится 4 секунды; узел, не успевший ответить, не задерживает запрос — он остаётся «грязным», и фоновая задача (она запускается каждые 5 секунд) досылает на него изменения. Если операция не удалась на части inbound-ов, на остальных она всё равно применяется: в ошибке перечислены inbound-ы, где не получилось, а Xray перезапускается ради уже применённой части.

   С 3.10.0 узел применяет присланный inbound **на месте**, если изменились только его клиенты (`settings.clients`) у протоколов `vless`, `vmess`, `trojan`, `hysteria` и `masque`: снятых клиентов он удаляет, новых и изменённых добавляет через API Xray поштучно, не пересоздавая обработчик, — слушатель и открытые соединения остальных клиентов на этом порту не обрываются (#6795). Если для ядра ничего не изменилось, узел ничего не делает; любое другое изменение или сбой поштучной операции по-прежнему ведёт к пересозданию inbound'а. AmneziaWG, как MTProto и TUIC, перенастраивается на месте, не опуская туннель. Раньше каждая такая отправка — а мастер шлёт inbound целиком при авто-отключении, продлении или сбросе клиента и заново досылает все inbound'ы после своего перезапуска — рвала все соединения порта. Узлы на старых версиях пересоздают inbound до обновления; на мастере ничего менять не нужно. Присланное мастером

снятие клиента не перезапускает Xray на узле по его собственной настройке «Перезапускать Xray после отключения клиента»: при авто-отключении узлы перезапускает сам мастер по такой же своей настройке.

Если inbound удалили прямо на узле, мастер при следующей досылке **создаёт его заново** — раньше он бесконечно слал обновление по сохранённому id уже несуществующего inbound'а.

2. **Сбор трафика.** Та же задача запрашивает у узла снимок трафика и сливает его в локальную статистику. На основе слитого трафика выполняется проверка исчерпания лимитов/сроков и при необходимости отключение клиентов; счётчик «исчерпано» по узлу отражает именно это. Если узел недоступен, его онлайн-клиенты очищаются.

   Для клиента, привязанного сразу к нескольким панелям, master в той же задаче дополнительно рассылает узлам **суммарный по всем панелям** расход трафика этого клиента (отдельной таблицей на узле, ключ — GUID мастера; перезаписывается при каждой отправке, поэтому сброс на стороне мастера тоже распространяется). На узле в трафике клиента отображается большее из двух значений — локальное или присланное, — а при превышении общей квоты клиент отключается **локально на самом узле** (через тот же механизм перезапуска Xray при авто-отключении, что разрывает уже установленные соединения). Это устраняет ситуацию, когда узел видел только свою долю трафика, недосчитывал расход и продолжал обслуживать клиента, уже исчерпавшего общий лимит. При сбросе трафика, авто-продлении или удалении клиента присланные счётчики очищаются.

При **первой** синхронизации inbound, размещённого на узле (добавление нового узла или повторный импорт inbound), мастер инициализирует счётчики трафика клиентов реальными значениями с узла. Раньше в этой ситуации общий счётчик inbound переносился корректно, а индивидуальные счётчики клиентов обнулялись, и мастер занижал расход клиентов на всю историю, накопленную до подключения узла. Теперь, если inbound создаётся в той же синхронизации, новая строка `client_traffics` наследует значение счётчика с узла (baseline выставляется равным ему, поэтому следующая дельта равна нулю и трафик не учитывается дважды). Засев счётчика применяется только для inbound, созданного в этом же проходе: клиент, заново появившийся под уже существующим inbound, по-прежнему стартует с нуля (защита от «фантомного» трафика), а только что удалённый клиент не «воскресает» при пересоздании его inbound.

Слияние снимков узла защищено и от других расхождений:

- **Удалённые клиенты не «воскресают».** Email удалённого клиента (в том числе при массовом удалении) 90 секунд отфильтровывается из снимков узлов, а массовое удаление передаётся на узел для всех выбранных клиентов, даже если в настройках inbound-а на мастере записи уже нет, — снимок, снятый до удаления, не вернёт клиента. Если тот же email сразу создать заново, пометка снимается, и новый клиент не пропадает из inbound-а на эти 90 секунд. Это верно и для клиента, который вернулся через повторное добавление inbound'а узла: пометка снимается после фиксации записи, так что очередной снимок не отвяжет клиента и мастер не удалит его через 15 минут как «осиротевшего».
- **Отключение клиента узлом не «залипает» без причины** (#6870). Выключенным, о котором сообщил узел, клиент остаётся на мастере, только когда по данным мастера у него исчерпаны срок или общая квота либо расход, посчитанный самим узлом, уже тратит квоту мастера. Если мастер считает клиента разрешённым, такое отключение признаётся устаревшим (снимок после сброса квоты, лишнее срабатывание отключения недействительных клиентов) и клиент включается обратно, а не переключается туда и обратно на каждом такте.

- **Узел не может «забрать» чужого клиента.** Email клиента в панели уникален, поэтому клиент из снимка узла не принимается, если на мастере клиент с таким email привязан только к inbound-ам вне этого узла (локальным или другого узла): иначе мастер перезаписал бы его UUID, и настоящий пользователь потерял бы доступ. В журнал один раз пишется предупреждение с текстом `rename one side to remove the duplicate email` — переименуйте клиента на одной из сторон.
- **Правка клиента, размещённого на нескольких узлах.** Перед рассылкой правки все узлы, где размещён клиент, помечаются «грязными», и снимок, пришедший посреди операции, пропускается: переименование не оставляет копию клиента под старым email, а изменённый срок не откатывается. Окно для такого расхождения этим сужено, но не закрыто полностью.
- **Принятые inbounds с другим именем.** Если inbound узла принят под тегом мастера, эта связь сохраняется при обновлении кэша идентификаторов, и мастер не создаёт на узле дубликат inbound-а на том же порту. Тег, о котором сообщает сам узел, при этом важнее сохранённой связи, поэтому операции не уходят на другой inbound узла, получивший прежнее имя.
- **Пустой снимок не удаляет клиентов** (#6734). Узел, только что удалённый, сброшенный, перезапущенный или ответивший до загрузки конфигурации, может сообщить inbound без единого клиента. Раньше мастер верил такому снимку: отвязывал всех клиентов inbound'а, а после льготного периода удалял их совсем — удаление одного узла могло стереть клиентов всей панели. Теперь снимок с нулём клиентов для inbound'а, у которого на мастере клиенты есть, не считается авторитетным: настройки inbound'а на мастере не перезаписываются, связи клиентов и базовые значения трафика не трогаются, узел помечается «грязным», и мастер заново отправляет на него своих клиентов. Следствие: последнего клиента inbound'а удаляйте с мастера — удаление прямо на узле мастер отменит повторной отправкой (удаление одного из нескольких клиентов узла по-прежнему подхватывается).
- **Статистика отвязанных клиентов игнорируется** (#6724). При отвязке на мастере узел сохраняет строку трафика клиента и продолжает сообщать его под этим inbound'ом. Раньше мастер принимал эти устаревшие данные: переносил к себе старый расход, по нему отключал клиента, а прежняя квота каждые 5 секунд возвращалась обратно. Теперь из снимка учитываются только клиенты, перечисленные в настройках inbound'а (если настройки не разбираются — как раньше, все), и для email, которого на мастере уже нет, новая строка трафика не создаётся.
- **История трафика сохраняется при замене inbound'а** (#6749). Если узел сообщает inbound под тегом, который не совпадает ни с тегом мастера, ни с известными псевдонимами, мастер принимает его как новый, а старый удаляет. Раньше вместе со старым удалялись строки трафика его клиентов, и на следующем такте они создавались заново с нулями — учёт квот терял весь расход. Теперь строки клиентов, которых узел по-прежнему сообщает, переносятся на новый inbound; удаляются только строки тех, кого узел больше не сообщает вовсе. Созданный с мастера inbound узла, ещё не сопоставленный по тегу, при переименовании на первой синхронизации распознаётся как прежний, а не заменяется.

1. **Heartbeat.** Отдельная фоновая задача периодически опрашивает все **включённые** узлы (с ограничением параллелизма) через `panel/api/server/status`, обновляет статус/метрики/версии и, при наличии веб-клиентов, рассылает обновлённое дерево узлов по WebSocket. Статус узла становится `offline` сразу после неудачного опроса, а уведомление `node.down` уходит, когда число неудачных проверок подряд достигает порога из настроек уведомлений (14.5): рядом с событием «Down» у группы **Nodes** стоит поле `nodeDownThreshold` (1–100, по умолчанию `1`, подпись *«неудачных проверок подряд»*). `node.down` отправляется один раз за простой, `node.up` — только если `node.down` был отправлен. Факт отправки хранится в базе (`nodes.down_notified`), поэтому перезапуск панели внутри

окна порога не теряет и не дублирует уведомление; узлы, бывшие offline при обновлении, считаются уже оповещёнными (#6768).

С 3.10.2 при слиянии трафика с узлов (раз в 5 секунд) в базу пишутся только изменившиеся записи. Раньше перезаписывались строки всех клиентов, о которых сообщил узел, даже простаивающих, а общая запись клиента сразу на нескольких inbound'ах (например, с flow и без него, или WireGuard плюс любой другой) переключалась туда и обратно — на больших установках это давало лишнюю нагрузку на базу.

12.8. Цепочки узлов (подузлы / транзитивные узлы)

Топология может быть не плоской: узел сам может быть мастером для своих узлов. Такие нижестоящие панели показываются у вас как **Подузлы** — это **проекции «только для чтения»**, полученные от прямого узла.

- Подсказка: «Только для чтения: подчинённый узел, доступный через {parent}. Управляйте им из собственной панели {parent}». То есть подузел нельзя здесь редактировать, удалять или обновлять — все операции с ним выполняются из панели его прямого родителя.
- Идентичность подузла определяется его GUID; благодаря этому онлайн-клиенты и inbounds учитываются именно под тем физическим узлом, который их хостит, даже в цепочке `Node1 → Node2 → Node3` (мастер «проходит» на один уровень вглубь через каждый прямой узел).
- Если прямой узел становится недоступен, его кэш подузлов очищается, и подузлы пропадают из дерева до восстановления связи.

12.9. Узлы: новое в 3.7.0

- **Клонирование inbound'ов на узлы.** Выбранные inbound'ы разворачиваются на несколько узлов одной операцией; в отчёте — сколько клонировано и сколько не вышло («{ok} cloned, {failed} failed»).
- **Токен API узла можно хранить зашифрованным** (опция при добавлении/редактировании узла): в базе не остаётся читаемого токена, что важно, если бэкап базы уезжает наружу.
- **Синхронизация перестала ломать то, чего не трогала.** Совпадающий уже развёрнутый inbound теперь **усыновляется**, а не пересоздаётся (раньше пересоздание рвало клиентов); выключенные inbound'ы, о которых снимок узла физически не может сообщить, **больше не удаляются**; устаревшая синхронизация сроков **не отменяет продления** клиента; удаление клиентов, которых синхронизация удалять не собиралась, исправлено; отметка об усыновлении не ставится, если усыновлять было нечего.
- **mTLS стал строже и живее.** Проверяется **каждый** сертификат в доверенном наборе, а не только первый; ротация мастер-сертификата применяется **без перезапуска панели** — кнопка «Reload master mTLS credential» (`POST /panel/api/nodes/mtls/reloadClient`); мастер-учётные данные записываются атомарно, тихая перевыдача исключена.
- **Ошибки перестали теряться:** сбой сохранения heartbeat'а и удаление inbound'а по снимку узла теперь пишутся в лог централизованно.
- **Точнее считается онлайн** в мультипанели: клиент, подключённый к узлу, больше не пропадает из подсчёта.

12.10. Исправления 3.5.0 (стабильность узлов)

Пакет исправлений в 3.5.0, заметных оператору узлов:

- **Сохранение клиента без изменений больше не рушит трафик узла.** Раньше любое сохранение клиента (даже «открыл — сохранил») отправляло на узел полное обновление inbound с пересозданием обработчика Xray — узел числился онлайн, но не пропускал трафик до ручного перезапуска. Теперь no-op сохранения ничего не делают, а реальные правки уходят одним лёгким обновлением без пересоздания.
- **Host-переопределения узла принимаются в мастер**: при первом принятии inbound узла его строки Hosts (SNI, отпечаток, ALPN и т. д.) импортируются, и подписки несут правильные TLS-параметры.
- **Авто-продление на узле открывает свежее окно квоты** (пакет вида «100 ГБ каждые 30 дней» снова выдаёт полный объём).
- **Удаление клиента на мастере полностью удаляет его на узлах** (запись + трафик), а не только отвязывает; остатки прежних массовых удалений можно убрать действием «Удалить непривязанных клиентов» на узле.
- **Inbound'ы узла не выметаются до первого принятия**: сохранение свежедобавленного узла больше не может удалить его живые inbound'ы.
- **Один некорректный inbound не останавливает синхронизацию трафика узла** (заодно легаси-inbound'ы `socks` на узлах переименовываются в `mixed`).
- **Проверка конфликта портов ограничена своим узлом** — правка адреса прослушивания узлового inbound больше не отклоняется из-за такого же порта на другом узле («port … already used by inbound …»).

12.11. Узлы: новое в 3.3.0

В версии 3.3.0 раздел **Узлы** получил три заметных улучшения: корректную атрибуцию трафика и онлайн-клиентов в многоуровневых (multi-hop) топологиях, синхронизацию client-IP между узлами и отдельный индикатор статуса для случая, когда панель узла жива, а ядро Xray на ней упало. Слова inbound/outbound далее не переводятся.

1. Multi-hop: правильная атрибуция трафика по цепочке под-узлов

Раньше счётчики (число inbound, онлайн-клиентов, исчерпанных) считались на уровне «прямого» узла. Если у вас цепочка вида `Мастер → Узел1 → Узел2 → Узел3`, всё, что физически живёт на `Узел2` / `Узел3`, ошибочно приписывалось `Узел1`, через который оно дошло до мастера. В 3.3.0 атрибуция идёт по реальному источнику.

Как это устроено:

- **Под-узлы становятся видны как отдельные строки.** Каждая панель публикует список своих прямых узлов; включаются только узлы с известным `Guid` — стабильная идентичность нужна, чтобы приписать узел на один «прыжок» вверх. Мастер периодически (из heartbeat-задания) подтягивает эти списки и кэширует их, а затем добавляет к прямым узлам «транзитивные» под-узлы.

- **Транзитивные узлы — только для чтения.** В UI они помечаются как **«Подузел»** с подсказкой: *«Только для чтения: подчинённый узел, доступный через {родитель}. Управляйте им из собственной панели*

{родитель}.» Кнопок управления у такой строки нет — узлом управляют с панели его непосредственного родителя.

- **Иерархия через GUID.** У прямого узла `ParentGuid` — это GUID самого мастера; у транзитивного — GUID его родительского узла. Так строится дерево.
- **Источник правды для счётчиков — `origin_node_guid` на inbound.** Это `panelGuid` узла, который физически держит этот inbound. Оно проставляется при синхронизации inbound с узла и **сохраняется как есть при дальнейших прыжках**, поэтому глубоко вложенный inbound приписывается реальному узлу, а не промежуточному. По этому GUID пересчитываются счётчики числа inbounds, онлайн-клиентов и исчерпанных клиентов. Логика выбора ключа:

Состояние inbound	Чему приписывается
`origin_node_guid` задан	этому GUID (реальный узел-источник)
пусто, но задан `node_id`	синтетический GUID узла (старый билд, ещё не сообщил свой `panelGuid`)
пусто и `node_id` пуст	собственному GUID мастера (inbound на локальном Xray)

  Онлайн-клиенты так же группируются по GUID, поэтому каждая строка узла показывает только тех, кто реально подключён именно к нему.

Что видит пользователь: в плоской топологии (узлы напрямую под мастером) ничего не меняется — счётчики по GUID и по `id` совпадают. Но как только появляется цепочка узлов, в списке возникают строки-«Подузлы», и числа inbound/онлайн/исчерпанных у каждого узла теперь отражают именно его собственную нагрузку, а не сумму всего, что прошло через него транзитом.

2. Синхронизация client-IP из access.log между узлами

Лимит по IP (`limitIp` у клиента) опирается на адреса, которые Xray пишет в свой access.log. Раньше каждый узел видел только подключения к себе, поэтому ограничение «не более N IP на клиента» не работало в кластере: клиент мог подключиться к разным узлам и обойти лимит. В 3.3.0 наблюдённые IP синхронизируются по всему кластеру.

Как это работает:

- На каждом узле фоновое задание парсит access.log, извлекая по каждой строке IP, email клиента и метку времени, и складывает их в локальную таблицу (по одной записи на email, IP хранятся как JSON-массив `{ip, timestamp}`). Локальные адреса `127.0.0.1` и `::1` отбрасываются.
- Синхронизация **раз в 10 секунд** выполняет двусторонний обмен по каждому включённому узлу в сети: тянет IP с узла и сливает их в локальную таблицу, а затем отдаёт узлу сводную картину мастера.
- Слияние объединяет старые и входящие наблюдения **без двойного учёта** одного IP, увиденного на нескольких узлах, и **без воскрешения устаревших** записей: применяется тот же порог давности, что и в локальном задании — **30 минут**. По каждому IP сохраняется самая свежая метка времени. Записи с чужих узлов получают новый локальный id (id-пространства узлов независимы); конкурентная вставка одного и того же email защищена от дублей.
- При подсчёте лимита «живым» считается IP, который либо замечен в текущем сканировании локально, либо имеет очень свежую метку из синхронизированной базы (**в пределах 2 минут**). Именно это и заставляет лимит работать в масштабе всего кластера, даже если адрес был замечен на другом узле.

При превышении лимита самые старые «живые» IP отправляются в журнал fail2ban, а соединения принудительно разрываются (remove/re-add клиента через Xray API).

Что видит пользователь: ограничение по числу IP теперь действует на весь кластер, а не на каждый узел по отдельности; в панели по клиенту видны IP, замеченные на любом узле (в пределах окна в 30 минут). Отдельной кнопки/настройки для этого нет — синхронизация идёт автоматически в фоне, при условии что у узла включён и доступен access.log (для самого лимита также нужен Fail2Ban на узле).

3. Отдельный индикатор статуса: панель узла онлайн, но Xray упал

Раньше статус узла был, по сути, «в сети / не в сети». Если панель узла отвечала, узел считался онлайн — даже когда ядро Xray на нём не работало, и клиенты фактически не могли подключиться. В 3.3.0 здоровье панели и здоровье ядра Xray разделены.

Как это устроено:

- При опросе узла мастер берёт из ответа удалённого `/panel/api/server/status` поля `xray.state` и `xray.errorMsg` и сохраняет их в узле. Эти поля заполняются даже при успешном пинге панели, когда ядро нездорово, — именно чтобы отличить доступность панели от состояния Xray.
- Значения `xray.state`: `"running"` (работает), `"stop"` (остановлен), `"error"` (ошибка).
- Эти значения транслируются в статусы узла. К привычным добавились новые:

Ключ статуса	Русская подпись	Когда показывается
`online`	«В сети»	панель отвечает, Xray работает (`running`)
`offline`	«Не в сети»	панель недоступна / пинг не прошёл
`unknown`	«Неизвестно»	состояние ещё не определено
`xrayError`	«Ошибка Xray»	панель онлайн, но ядро Xray в состоянии `error` (есть `errorMsg`)
`xrayStopped`	«Остановлен»	панель онлайн, но Xray остановлен (`stop`)

- Для подобного состояния в UI используется **отдельный фиолетовый индикатор** (цвет, отличный от зелёного «в сети» и красного «не в сети»). Фиолетовый прямо сигналит: до узла дозвониться можно, проблема — в самом ядре Xray, а не в сети или в самой панели.

Что видит пользователь: вместо вводящего в заблуждение «зелёного» при упавшем ядре узел подсвечивается **фиолетовым** со статусом **«Ошибка Xray»** или **«Остановлен»**. Это сразу показывает, что чинить надо Xray на узле (перезапустить ядро, посмотреть `errorMsg`), а не разбираться с доступностью самого узла. Тот же `xrayState` / `xrayError` прокидывается и в транзитивные под-узлы (см. пункт 1), так что некорректное состояние ядра видно по всей цепочке.

---

13. Настройки панели

Раздел «Настройки» (заголовок страницы — **Настройки**, англ. *Panel Settings*) управляет поведением самой веб-панели 3X-UI: на каком адресе и порту она слушает, как защищена, как взаимодействует с Telegram- и Discord-ботами и внешними сервисами, в каком часовом поясе выполняет запланированные задачи. Каждый параметр хранится в таблице `settings` базы данных в виде пары «ключ — значение»; если значение в БД отсутствует, применяется значение по умолчанию.

> **Важно — применение изменений.** Любое изменение на этой странице нужно сохранить кнопкой **Сохранить** (*Save*), а затем перезапустить панель, чтобы изменения вступили в силу. Дословная подсказка: «Сохраните изменения и перезапустите панель для их применения.» При сохранении показывается уведомление «Настройки изменены».

13.1. Сохранение и перезапуск панели

Элемент	Назначение
Сохранить (*Save*)	Записывает все поля формы в БД (`POST /panel/setting/update`). Перед записью значения проходят валидацию — некорректные адреса, порты или пути будут отклонены, и панель вернёт ошибку.
Перезапустить панель (*Restart Panel*)	Перезапускает веб-сервер панели (`POST /panel/setting/restartPanel`). Перезапуск происходит с задержкой 3 секунды. Подсказка: «Вы уверены, что хотите перезапустить панель? Подтвердите, и перезапуск произойдёт через 3 секунды. Если панель будет недоступна, проверьте лог сервера». При успехе — «Панель успешно перезапущена».
Восстановить настройки по умолчанию (*Reset to Default*)	Удаляет все сохранённые в БД настройки, после чего панель использует значения по умолчанию. Учётные данные администратора не сбрасываются этой операцией. Пути подписок (`subPath` , `subJsonPath` , `subClashPath`) не возвращаются к `/sub/` , `/json/` и `/clash/` , а генерируются заново случайными, поэтому ранее выданные ссылки подписок перестанут работать.

Перезапуск выполняется отправкой процессу панели сигнала `SIGHUP` (или через зарегистрированный хук перезапуска). На Windows автоматический перезапуск через сигнал не поддерживается. **Изменения параметров прослушивания (IP, порт, путь, домен, сертификаты, часовой пояс) применяются только после перезапуска панели.**

13.2. Общие настройки (вкладка «Панель» / *General*)

Язык интерфейса (*Language*)

Язык веб-интерфейса панели. Доступные языки: `en-US` (английский), `ru-RU` (русский), `zh-CN` , `zh-TW` , `fa-IR` , `ar-EG` , `es-ES` , `id-ID` , `ja-JP` , `pt-BR` , `tr-TR` , `uk-UA` , `vi-VN` , `sk-SK` (словацкий, в списке — «Slovenčina»). Это настройка отображения и не влияет на работу Xray.

Тип календаря (*Calendar Type*)

- **Ключ:** `datepicker`
- **Значение по умолчанию:** `gregorian` (григорианский).
- **Назначение:** тип календаря, используемый в выборе дат (например, при задании срока действия клиентов). Подсказка: «Запланированные задачи будут выполняться в соответствии с этим календарём.» Альтернативное значение — персидский (джалали) календарь, что востребовано в иранской аудитории панели.

Размер нумерации страниц (*Pagination Size*)

- **Ключ:** `pageSize`
- **Значение по умолчанию:** `25`
- **Допустимые значения:** целое от `0` до `1000`.
- **Назначение:** число строк на странице в таблицах (списках подключений/inbound). Подсказка: «Определить размер страницы для таблицы подключений. Установите 0, чтобы отключить» — при `0` постраничный вывод отключается, и все записи показываются единым списком.
- **Перезапуск панели не требуется** (настройка отображения).

Перезапускать Xray после отключения клиента (*Restart Xray After Client Disable*)

- **Ключ:** `restartXrayOnClientDisable`
- **Значение по умолчанию:** `true`
- **Назначение:** Xray перезапускается, чтобы разорвать уже установленные соединения клиента, который перестал обслуживаться, — **автоматически** (по истечении срока действия или лимита трафика) **или вручную** (отключение или удаление клиента, в т.ч. массовое). Подсказка: «Когда клиент перестаёт обслуживаться — автоматически (по сроку или лимиту трафика) или вручную — перезапускать Xray, чтобы его активные соединения оборвались.» Переключатель находится на вкладке «Панель» (*General*) рядом с прочими общими настройками.
- **Изменение в 3.8.5.** Раньше (*Restart Xray After Auto Disable*) настройка срабатывала только при автоотключении: снятие клиента через API ядра (`RemoveUser`) удаляет лишь учётные данные и не разрывает уже установленную сессию (VLESS, VMess, Trojan, Shadowsocks её сохраняют), поэтому вручную отключённый или удалённый клиент мог продолжать передавать трафик. Теперь ручное отключение/удаление клиента (в форме клиента и при массовом отключении) при включённой настройке тоже вызывает перезапуск Xray. Действует только для локальных inbound'ов панели, выполняющей операцию, — правка клиента на узле не заставляет мастер перезапускать свой собственный Xray.

Модель примечания и символ разделения (*Remark Model & Separation Character*)

- **Ключ:** `remarkModel`
- **Значение по умолчанию:** `-ieo`

- **Назначение:** задаёт, как формируется имя (remark) конфигурации в подписке. Строка состоит из **первого символа** — разделителя, и далее **последовательности букв-порядка**:
  - `i` — примечание inbound (*inbound remark*);
  - `e` — email клиента;
  - `o` — дополнительная метка (*extra*).

  При значении по умолчанию `-ieo` разделителем служит `-`, а порядок частей: inbound → email → extra (например, `MyInbound-user@mail-extra`). Пустые части пропускаются. Поле «Пример примечания» (*Sample Remark*) в интерфейсе показывает предпросмотр формируемого имени. Включение email в имя дополнительно зависит от параметра «Включать Email в название» в настройках подписки (раздел о подписке).

Пример: как значение `remarkModel` влияет на имя конфигурации. Допустим, inbound называется `VLESS-Reality`, email клиента — `alex@vpn`, а дополнительная метка — `RU`. Тогда:

Значение поля	Итоговое имя (remark)
`-ieo` (по умолчанию)	`VLESS-Reality-alex@vpn-RU`
`_ie`	`VLESS-Reality_alex@vpn`
`-ei`	`alex@vpn-VLESS-Reality`
`o` (пробел-разделитель, только метка)	`RU`

Первый символ строки — всегда разделитель; остальные буквы задают, какие части и в каком порядке войдут в имя.

Кандидаты сканирования Reality (*Reality scan candidates*)

- **Ключ:** `realityScanCandidates`
- **Значение по умолчанию:** прежний встроенный список из десяти целей — `www.cloudflare.com:443`, `www.microsoft.com:443`, `www.amazon.com:443`, `aws.amazon.com:443`, `www.samsung.com:443`, `www.nvidia.com:443`, `www.amd.com:443`, `www.intel.com:443`, `www.sony.com:443`, `dl.google.com:443`.
- **Формат:** цели `host:port` и/или диапазоны CIDR через запятую (многострочное поле).
- **Назначение:** список, который сканер целей REALITY (кнопка **«Найти цели»** в настройках REALITY inbound-а, см. 7.4) проверяет, когда поле поиска оставлено пустым; тот же список берёт `POST /panel/api/server/scanRealityTargets`, если цели в запросе не переданы. Подсказка: «Список host:port (или CIDR) через запятую — используется по умолчанию в «Найти цели», если поиск пуст. Настройте под часто используемые цели.» Если оставить поле пустым, сканер использует встроенный список.
- **Перезапуск панели не требуется** — список читается при каждом сканировании.

13.3. Доступ к панели: IP, порт, путь, домен, сертификат

Эта группа определяет сетевую точку входа панели. **Все изменения здесь применяются только после перезапуска панели.**

Поле	Ключ	Значение по умолчанию	Описание
IP-адрес для управления панелью (*Listen IP*)	`webListen`	`""` (пусто)	IP, на котором слушает веб-панель. Пусто = слушать на всех IP. Подсказка: «Оставьте пустым для подключения с любого IP». Если задан, должен быть корректным IP-адресом (иначе сохранение отклоняется).
Домен панели (*Listen Domain*)	`webDomain`	`""` (пусто)	Доменное имя панели для проверки запроса по домену. Пусто = принимать подключения с любых доменов и IP. Подсказка: «Оставьте пустым для подключения с любых доменов и IP.»
Порт панели (*Listen Port*)	`webPort`	`2053`	Порт, на котором работает панель. Подсказка: «Порт, на котором работает панель». Допустимо `1–65535`. Порт должен быть свободным; панель и служба подписки не могут одновременно использовать одинаковую пару `IP:порт`.
URI-путь (*URI Path*)	`webBasePath`	`/`	Базовый путь URL панели (basePath). Подсказка: «Должен начинаться с '/' и заканчиваться '/'». При сохранении панель автоматически добавляет ведущий и завершающий `/`, если они отсутствуют. Запрещённые символы в пути отклоняются.

Сертификат панели (TLS / HTTPS)

Поле	Ключ	Значение по умолчанию	Описание
Путь к файлу публичного ключа сертификата панели (*Public Key Path*)	`webCertFile`	`""`	Полный путь к файлу сертификата (цепочки). Подсказка: «Введите полный путь, начинающийся с '/'».
Путь к файлу приватного ключа сертификата панели (*Private Key Path*)	`webKeyFile`	`""`	Полный путь к файлу приватного ключа. Подсказка: «Введите полный путь, начинающийся с '/'».

Если задан **хотя бы один** из путей сертификата/ключа, панель при сохранении пытается загрузить пару «сертификат + ключ»; при ошибке (несуществующий файл, несоответствие ключа и сертификата) сохранение отклоняется. Когда заданы оба корректных пути, панель переходит на HTTPS. Оба поля пустые = панель работает по обычному HTTP.

> **Предупреждения безопасности** (*Security warnings*). Панель показывает блок «Ваша панель может быть открыта:» с предупреждениями, если обнаруживает небезопасную конфигурацию:
>
> - работа по обычному HTTP — «настройте TLS для продакшна»;
> - стандартный порт 2053 — «измените его на случайный»;
> - базовый путь по умолчанию `/` — «измените его на случайный»;
> - стандартный путь подписки `/sub/` и JSON-подписки `/json/` — «измените его». Новая установка и сброс настроек сразу задают случайные пути подписок, поэтому это предупреждение появляется в основном на панелях, обновлённых со старых версий. Это рекомендации, а не блокировки.

13.4. Сессия, прокси панели и доверенные прокси (вкладка «Прокси и сервер» / *Proxy and Server*)

Продолжительность сессии (*Session Duration*)

- **Ключ:** `sessionMaxAge`
- **Значение по умолчанию:** `360` (минут, т.е. 6 часов).
- **Допустимые значения:** от `1` до `525600` минут (1 год).
- **Назначение:** как долго администратор остаётся авторизованным без повторного входа. Единица — **минута**. Подсказка: «Продолжительность сессии в системе (значение: минута)».

Outbound для трафика панели (*Panel Traffic Outbound*)

- **Ключ:** `panelOutbound`
- **Значение по умолчанию:** `""` (пусто = прямое подключение).
- **Назначение:** задаёт Xray-**outbound**, через который панель отправляет **собственные запросы** — проверки версий и загрузку панели/Xray, обращения к Telegram, обычное обновление geo-файлов — чтобы обойти серверную фильтрацию GitHub/Telegram. Поле представляет собой **выпадающий список**: в нём перечислены outbound'ы из шаблона конфигурации Xray, outbound'ы из подписок на outbound, а также маршрутные **балансировщики** (отдельной группой). Outbound'ы типа `blackhole` из списка исключены — маршрутизировать загрузку в «чёрную дыру» бессмысленно. Дословная подсказка: «Маршрутизирует собственные запросы панели — проверки версий и загрузки панели/Xray, Telegram и обычное обновление geo-файлов — через этот исходящий Xray для обхода серверной фильтрации GitHub/Telegram. Локальный мост-входящий добавляется в работающую конфигурацию автоматически и применяется на лету. Встроенное в Xray автообновление Geodata не затрагивается; у него свой исходящий для загрузки. Оставьте пустым для прямого подключения.»

> **Как это работает.** При выборе outbound'а панель сама добавляет в рабочий конфиг служебный loopback-inbound (SOCKS-мост с тегом `panel-egress`) и правило маршрутизации, которое заворачивает собственный HTTP-трафик панели на выбранный outbound. Если выбран балансировщик, в правило подставляется `balancerTag`, и трафик панели распределяется между его участниками. Мост и правило применяются **на лету**, без полного перезапуска панели. Оставьте поле пустым для прямого соединения. Встроенное в Xray автообновление geo-данных этой настройкой **не затрагивается** — у него собственный outbound внутри Xray-роутинга.

- **Формат:** `socks5://` (или `socks5h://`) либо `http(s)://`, при необходимости с авторизацией вида `socks5://user:pass@host:port`. Поддерживаемые схемы строго: `socks5`, `socks5h`, `http`, `https` — иные схемы считаются недопустимыми, и панель тогда откатывается на прямое подключение. Типичный пример — локальный SOCKS-входящий самого Xray.
- Дословная подсказка: «Маршрутизирует исходящие запросы самой панели (обновления geo, проверки версий Xray/панели, Telegram) через этот прокси для обхода серверной фильтрации GitHub/Telegram. Принимает socks5:// или http(s)://, напр. локальный SOCKS-входящий Xray. Оставьте пустым для прямого подключения.»
- Невалидный прокси не приводит к ошибке сохранения — панель просто использует прямое соединение и пишет предупреждение в лог.

Пример значений поля. Если на сервере уже поднят локальный SOCKS-входящий Xray на порту `10808`, направьте через него собственные запросы панели:

```
socks5://127.0.0.1:10808
```

Для внешнего HTTP-прокси с авторизацией:

```
http://user:pass@proxy.example.com:8080
```

После сохранения и перезапуска панель будет тянуть обновления geo-баз, проверять версии и обращаться к Telegram уже через указанный прокси.

Доверенные CIDR прокси (*Trusted proxy CIDRs*)

- **Ключ:** `trustedProxyCIDRs`
- **Значение по умолчанию:** `127.0.0.1/32,::1/128` (только локальный хост).
- **Формат:** список IP-адресов или CIDR-подсетей через запятую (например `10.0.0.0/8, 192.168.1.5`). Каждый элемент проверяется как IP или CIDR — некорректное значение отклоняется при сохранении.
- **Назначение:** перечисляет источники, которым разрешено устанавливать заголовки `X-Forwarded-Host`, `X-Forwarded-Proto` и заголовок реального IP клиента. Дословная подсказка: «IP/CIDR через запятую, которым разрешено устанавливать заголовки forwarded host, proto и client IP.» Нужно настраивать, если панель работает за обратным прокси (nginx, Caddy и т.п.), чтобы корректно определять клиентские IP и схему.

Пример: панель за обратным прокси. Если nginx стоит на том же хосте и проксирует запросы на панель, оставьте доверие только локальному хосту (значение по умолчанию):

```
127.0.0.1/32,::1/128
```

Если прокси находится на отдельном сервере во внутренней сети `10.0.0.0/8`, добавьте его подсеть, иначе панель проигнорирует переданные им заголовки и будет видеть IP прокси вместо реального клиента:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Пример соответствующего блока nginx, который передаёт реальный IP и схему:

```
proxy_set_header X-Real-IP        $remote_addr;
proxy_set_header X-Forwarded-For  $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

Как панель выбирает адрес клиента. Для запроса от доверенного прокси панель разбирает `X-Forwarded-For` справа налево, пропуская адреса из этого списка, и берёт первый недоверенный. `X-Real-IP` учитывается, только если совпадает с этим адресом или `X-Forwarded-For` нет; при расхождении используется адрес прокси. Поэтому прокси обязан перезаписывать `X-Real-IP`, а не пересылать значение

клиента. nginx в примере выше это делает; Caddy и HAProxy присланный клиентом `X-Real-IP` не трогают, и в Caddy его нужно перезаписать явно:

```
panel.example.com {
    reverse_proxy 127.0.0.1:2053 {
        header_up X-Real-IP {remote_host}
    }
}
```

13.5. Telegram-бот (вкладка «Telegram-бот» / *Telegram Bot*)

Включить Telegram бота (*Enable Telegram Bot*)

- **Ключ:** `tgBotEnable`
- **Тип/по умолчанию:** логический, `false`.
- **Назначение:** включает работу Telegram-бота. Подсказка: «Доступ к функциям панели через Telegram-бота».

Telegram-токен (*Telegram Token*)

- **Ключ:** `tgBotToken`
- **По умолчанию:** `""`.
- **Назначение:** токен бота. Подсказка: «Необходимо получить токен у менеджера ботов Telegram @botfather».
- **Особенность безопасности:** токен относится к секретным значениям. В ответе панели на чтение настроек он не возвращается (поле очищается, отдаётся только флаг «настроен/не настроен»). Если при сохранении поле оставить пустым, прежний сохранённый токен **сохраняется** (не затирается).

Язык Telegram-бота (*Telegram Bot Language*)

- **Ключ:** `tgLang`
- **По умолчанию:** `en-US`.
- **Назначение:** язык сообщений бота (независимо от языка веб-интерфейса). Список доступных языков совпадает с языками панели.

User ID администратора бота (*Admin Chat ID*)

- **Ключ:** `tgBotChatId`
- **По умолчанию:** `""`.
- **Формат:** один или несколько числовых Telegram User ID **через запятую**. После ID можно через двоеточие указать тему форума: `chatId:topicId`, например `-1001234567890:42`. Тогда уведомления и отчёты для этого чата уходят в указанную тему форум-супергруппы, а не в General. Повторы одного чата сливаются в одну запись; два разных `topicId` для одного чата, нечисловой или неположительный `topicId` не принимаются: при запуске бота в логе появляется предупреждение «Failed to parse admin ID from Telegram bot chat ID», и бот не стартует.

- **Назначение:** получатели уведомлений и администраторы, которым разрешено управлять панелью через бота. Подсказка: «Один или несколько User ID администратора(-ов) Telegram-бота. Для получения User ID используйте @userinfobot или команду '/id' в боте.»

Проверка бота: тестовое сообщение и предупреждения

Кнопка **«Отправить тестовое сообщение»** (`POST /panel/api/setting/testTgBot`) шлёт тест администраторам и заодно запрашивает возможности бота (`POST /panel/api/setting/tgBotCapabilities` , работает только при включённом и запущенном боте). По ответу под настройками может появиться предупреждение:

- **«Inline mode is disabled»** — у бота выключен inline-режим. Списки бота открываются через inline-поиск, поэтому нужно включить его в @BotFather командой `/setinline` , иначе кнопки поиска ничего не делают. Кнопка **«Check again»** повторяет проверку. Текст этого предупреждения в русской локали не переведён.
- **«Режим приватности включён»** (Group Privacy) — бот работает только в личных сообщениях и группы игнорирует намеренно; приватность в группах решает лишь, может ли бот читать сообщения группы, а бот этого не использует. Это сведение, а не ошибка.

Частота уведомлений (*Notification Time*)

- **Ключ:** `tgRunTime`
- **По умолчанию:** `@daily` (раз в сутки).
- **Формат:** значение собирается конструктором расписания (см. «Конструктор расписания» в 14.5): интервал `@every N` с единицей **Секунды**, **Минуты** или **Часы**, пресеты `@hourly` , `@daily` , `@weekly` , `@monthly` либо **Произвольный (crontab)** — выражение из **6 полей**, первое из которых — секунды, или (с 3.10.0) привычное выражение системного cron из **5 полей**, где младшее — минуты (секунды тогда равны 0). Нераспознаваемое расписание с 3.10.0 не сохраняется: настройки отклоняются с ошибкой `tgRunTime: cron schedule <…> is not valid` (так же проверяются `discordRunTime` и расписание синхронизации LDAP `ldapSyncCron` , см. 2.7). Подсказка: «Как часто бот Telegram отправляет периодические отчёты. Выберите готовый интервал или «Произвольный», чтобы ввести выражение crontab.» Управляет периодическими отчётами бота. Тот же конструктор используется для поля **Частота уведомлений от Discord-бота** (`discordRunTime` , см. 14.8).

Примеры значений поля.

Значение	Когда бот шлёт отчёт
`@daily`	раз в сутки в полночь (по умолчанию)
`@hourly`	каждый час
`@every 6h`	каждые 6 часов
`0 0 9 * * *`	ежедневно в 09:00
`0 30 8 * * 1`	каждый понедельник в 08:30
`30 8 * * 1`	то же в формате из 5 полей (с 3.10.0)

Время считается в часовом поясе из настройки «Часовой пояс» (п. 13.6).

SOCKS-прокси (*SOCKS Proxy*)

- **Ключ:** `tgBotProxy`
- **По умолчанию:** `""`.
- **Назначение:** SOCKS5-прокси отдельно для подключения бота к Telegram. Подсказка: «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству.» Применяется именно к трафику бота (отличается от общего «Сетевого прокси панели» из п. 13.4).

Telegram API Server (*Telegram API Server*)

- **Ключ:** `tgBotAPIServer`
- **По умолчанию:** `""` (использовать стандартный сервер `api.telegram.org`).
- **Формат:** URL `http(s)://…`; при сохранении проходит проверку корректности URL — невалидный адрес отклоняется. Подсказка: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию.» Нужен для самостоятельно развёрнутого Telegram Bot API server.

Уведомления бота (группа «Уведомления» / *Notifications*)

Поле	Ключ	По умолчанию	Описание
Резервное копирование базы данных (*Database Backup*)	`tgBotBackup`	`false`	Отправлять в Telegram файл резервной копии БД вместе с отчётом. Подсказка: «Отправлять уведомление с файлом резервной копии базы данных».
Уведомление о входе (*Login Notification*)	`tgBotLoginNotify`	`true`	Уведомлять при попытке входа в панель. Подсказка: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель.»
Задержка уведомления об истечении сессии (*Expiration Date Notification*)	`expireDiff`	`0`	За сколько **дней** до истечения срока действия клиента слать уведомление. `0` — отключено. Допустимо `>= 0`. Подсказка: «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)».
Порог трафика для уведомления (*Traffic Cap Notification*)	`trafficDiff`	`0`	Порог остатка трафика для уведомления. Подсказка: «Получение уведомления об исчерпании трафика до достижения порога (значение: ГБ)». Допустимо `0–100`.
Порог нагрузки на ЦП (*CPU Load Notification*)	`tgCpu`	`80`	Уведомлять администраторов, если загрузка CPU превышает порог (в **%**). Допустимо `0–100`. Подсказка: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)».

Порог неудачных проверок. В карточках событий рядом с «Down» групп Outbound и Nodes есть числовое поле с подписью «неудачных проверок подряд» (1–100). Уведомление `outbound.down` / `node.down`

уходит, только когда серия неудачных проверок достигла порога, и **один раз за простой**; «Up» отправляется лишь после ранее отправленного «Down». Ключи: `outboundDownThreshold` (по умолчанию `3`) и `nodeDownThreshold` (по умолчанию `1` — сообщать с первой же неудачи, как раньше). На нестабильном канале порог выше 1 не даёт потерянному heartbeat порождать пару «Down»/«Up» (#6768). Факт отправленного `node.down` хранится в `nodes.down_notified`, поэтому перезапуск панели внутри окна порога не теряет сообщение и не дублирует уже отправленные. Сами события описаны в 14.5.

13.6. Дата и время (вкладка «Дата и время» / *Date and Time*)

Часовой пояс (*Time Zone*)

- **Ключ:** `timeLocation`
- **Значение по умолчанию:** `Local` (системный часовой пояс сервера).
- **Формат:** имя зоны из базы IANA tz (например, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Назначение:** часовой пояс, в котором панель выполняет запланированные задачи (отчёты бота, сброс/проверки трафика, истечение сроков). Подсказка: «Запланированные задачи выполняются в соответствии со временем в этом часовом поясе».
- **Валидация:** при сохранении зона проверяется — несуществующая зона отклоняется. Если позже в БД окажется некорректное значение, панель в рантайме откатится на `Local`, а при недоступности и его — на `UTC`.

13.7. Внешний трафик и поведение Xray (вкладка «Внешний трафик» / *External Traffic*)

Поле	Ключ	По умолчанию	Описание
Информация о внешнем трафике (*External Traffic Inform*)	`externalTrafficInformEnable`	`false`	Уведомлять внешний API при каждом обновлении трафика. Подсказка: «Уведомлять внешний API при каждом обновлении трафика.»
URI информации о внешнем трафике (*External Traffic Inform URI*)	`externalTrafficInformURI`	`""`	URL, на который панель отправляет обновления трафика. Проходит проверку корректности URL при сохранении. Подсказка: «Обновления трафика отправляются на этот URI».

Поле	Ключ	По умолчанию	Описание
Перезапускать Xray после отключения клиента (*Restart Xray After Client Disable*)	`restartXrayOnClientDisable`	`true`	Перезапускать Xray, когда клиент перестаёт обслуживаться — автоматически (по истечении срока или лимита трафика) или вручную (отключение/удаление, в т.ч. массовое). С 3.8.5 распространяется и на ручное отключение/удаление (раньше — только автоотключение). Подсказка: «Когда клиент перестаёт обслуживаться — автоматически (по сроку или лимиту трафика) или вручную — перезапускать Xray, чтобы его активные соединения оборвались.» **Переключатель находится на вкладке «Панель» (*General*)** — см. п. 13.2; здесь приведён для полноты.

Отказ в конфигурации, которую ядро не может забиндить (с 3.8.5). Если сгенерированная конфигурация назначает двум inbound'ам один и тот же адрес и порт с пересекающимся транспортом (TCP+TCP или UDP+UDP; сочетание разных транспортов на одном порту, например TCP и UDP, — штатно и конфликтом не считается), панель отклоняет такую конфигурацию на единой точке перезапуска Xray, **не** останавливая уже работающее ядро. Раньше промах бинда убивал сам процесс Xray, а секундный цикл перезапуска-наблюдателя раз за разом повторял ту же ошибку, укладывая разом все протоколы. Причина отказа (вида «config refused: …») отображается на панели рядом со статусом Xray, пока ядро продолжает работать на прежней (последней успешно применённой) конфигурации. Коллизия, которую уже обслуживает работающее ядро, не считается ошибкой — отклоняются только новые пересечения адреса и порта.

13.8. Прочее: шаблон конфигурации Xray и проверочный URL

Шаблон конфигурации Xray (*xrayTemplateConfig*)

- **Ключ:** `xrayTemplateConfig`
- **По умолчанию:** встроенный (embedded) JSON-шаблон, поставляемый со сборкой.
- **Назначение:** базовый JSON-шаблон конфигурации Xray-core, поверх которого панель достраивает inbound/outbound. Это значение **не отдаётся** в обычной выдаче всех настроек и редактируется на отдельной странице конфигурации Xray, а не в общем списке полей настроек панели. Стандартный шаблон по умолчанию доступен через `GET /panel/setting/getDefaultJsonConfig`.

URL проверки исходящих (*xrayOutboundTestUrl*)

- **Ключ:** `xrayOutboundTestUrl`
- **По умолчанию:** `https://www.google.com/generate_204`
- **Назначение:** URL, используемый при проверке работоспособности исходящих (outbound) подключений. При установке проходит санитизацию как HTTP(S)-URL.

13.9. Учётная запись администратора и API-токены

Эти параметры находятся на смежной вкладке («Учётная запись» / *Authentication*) и подробно рассмотрены в разделе о безопасности; здесь — краткая сводка ключей.

- **Смена учётных данных** (поля «Текущий логин», «Текущий пароль», «Новый логин», «Новый пароль») сохраняется отдельным запросом `POST /panel/setting/updateUser`. Требуется верный текущий логин и пароль; новые логин и пароль не должны быть пустыми. Сообщения: «Вы успешно изменили учётные данные администратора.» / «Неверное имя пользователя или пароль».
- **Двухфакторная аутентификация (2FA)** — ключи `twoFactorEnable` (по умолчанию `false`) и секретный `twoFactorToken`. Токен — секрет: при включённом 2FA пустое поле при сохранении не затирает существующий токен. При **первом** включении 2FA панель инвалидирует текущие сессии (повышается «эпоха входа»).
- **API-токены** управляются отдельными эндпоинтами (`/panel/setting/apiTokens…`): список, создание (`apiTokens/create`), удаление, включение/выключение. Сам токен показывается **только один раз при создании** и не хранится в читаемом виде: «Скопируйте этот токен сейчас. В целях безопасности он не хранится в читаемом виде и больше не будет показан.»

Область действия и срок годности токена (3.7.0). У каждого токена теперь есть **scope**:

Scope	Что разрешено
`admin`	Всё, как и раньше. Значение по умолчанию; все ранее выданные токены остались `admin`
`monitor`	Только чтение и только состояние: `GET /panel/api/server/status`, истории метрик (`/server/history/…`, `/server/cpuHistory/…`, `/server/xrayMetrics…`, `/server/xrayObservatory…`), `getXrayVersion`, `getPanelUpdateInfo`, история метрик узла
`node-sync`	Ровно то, что нужно центральной панели: список/добавление/изменение/удаление inbound'ов и их порядок в подписке (`subSortIndex`), добавление/изменение/удаление/отвязка клиентов, массовый сброс трафика клиентов (`bulkResetTraffic`), активные inbound'ы клиентов (`activeInbounds`, с 3.10.0), рестарт Xray, `GET /server/status`. Обновить панель узла (`/server/updatePanel`) такой токен не может
`node-admin`	«Администратор узла»: всё, что у `node-sync`, и дополнительно запуск обновления панели узла (`POST /panel/api/server/updatePanel`), больше ничего. Для центральной панели, которой нужно обновлять узлы по API без выдачи полного `admin` (#6666)

В форме создания токена область выбирается в поле «Область доступа»: «Администратор», «Мониторинг», «Синхронизация узла» или «Администратор узла». На соединении mTLS с проверенным клиентским сертификатом центральная панель остаётся в роли `node-sync`; до `node-admin` её повышает только явно предъявленный Bearer-токен этой области. Токен ищется при каждом запросе по хэшу через индекс `idx_api_tokens_token` (#6915), поэтому стоимость проверки не растёт с числом токенов.

Запрос вне своей области получает **HTTP 403** с сообщением `this API token is not permitted to access this endpoint`. Дополнительно у токена есть необязательный **срок годности** (`expiresAt`): после его наступления токен перестаёт аутентифицировать. Приложениям, которые управляют панелью (в том числе мобильному менеджеру), нужен токен `admin` — `monitor` не сможет ничего изменить.

Если в заголовке `Authorization: Bearer` передан неверный, отключённый или просроченный токен, API отвечает **HTTP 401** — так ошибку токена можно отличить от неверного базового пути, который по-прежнему даёт 404. Запрос без заголовка авторизации и без сессии, как и раньше, получает 404.

Замена TOTP-секрета требует 2FA-кода (3.7.0). Перезаписать сохранённый секрет двухфакторной аутентификации теперь можно только с действующим кодом: угнанная сессия не сможет тихо перевесить 2FA на свой секрет.

Подробности по 2FA, паролям, LDAP-синхронизации и форматам подписки (JSON/Clash, fragmentation, noises, mux) вынесены в соответствующие отдельные разделы руководства.

13.10. Изменения API в 3.3.0 (важно для интеграций)

В версии 3.3.0 изменилась структура путей серверного API. Если у вас есть внешние интеграции (скрипты, боты, центральные панели, CI-задачи), которые обращаются к панели по HTTP, их **нужно поправить**, иначе они перестанут работать.

⚠️ BREAKING CHANGE: эндпоинты `/panel/setting/*` и `/panel/xray/*` переехали под `/panel/api`

Раньше управление настройками панели и конфигурацией Xray жило отдельно, под путями `/panel/setting/*` и `/panel/xray/*`. Теперь оба набора зарегистрированы внутри общей API-группы `/panel/api`. Старые пути **удалены полностью** — запрос по ним вернёт 404.

Зачем это сделано: вся группа `/panel/api` проходит через единую проверку доступа, то есть эти эндпоинты теперь принимают тот же заголовок `Authorization: Bearer <token>`, что и остальной API. API-токен — это полноценный администраторский доступ, и таким образом вся поверхность API стала единообразной.

Что НЕ изменилось: страницы веб-интерфейса (SPA-маршруты) `/panel/settings` и `/panel/xray` остались на месте — речь только о серверных API-эндпоинтах.

Таблица соответствия путей (старый → новый)

Префикс ко всем путям ниже — просто добавилось `api/` после `/panel/`.

Было (≤ 3.2.x)	Стало (3.3.0)	Метод
`/panel/setting/all`	`/panel/api/setting/all`	POST
`/panel/setting/defaultSettings`	`/panel/api/setting/defaultSettings`	POST
`/panel/setting/update`	`/panel/api/setting/update`	POST
`/panel/setting/updateUser`	`/panel/api/setting/updateUser`	POST
`/panel/setting/restartPanel`	`/panel/api/setting/restartPanel`	POST
`/panel/setting/getDefaultJsonConfig`	`/panel/api/setting/getDefaultJsonConfig`	GET
`/panel/setting/apiTokens`	`/panel/api/setting/apiTokens`	GET

Было (⩽ 3.2.x)	Стало (3.3.0)	Метод
`/panel/setting/apiTokens/create`	`/panel/api/setting/apiTokens/create`	POST
`/panel/setting/apiTokens/delete/:id`	`/panel/api/setting/apiTokens/delete/:id`	POST
`/panel/setting/apiTokens/setEnabled/:id`	`/panel/api/setting/apiTokens/setEnabled/:id`	POST
`/panel/xray/`	`/panel/api/xray/`	POST
`/panel/xray/update`	`/panel/api/xray/update`	POST
`/panel/xray/getDefaultJsonConfig`	`/panel/api/xray/getDefaultJsonConfig`	GET
`/panel/xray/getXrayResult`	`/panel/api/xray/getXrayResult`	GET
`/panel/xray/getOutboundsTraffic`	`/panel/api/xray/getOutboundsTraffic`	GET
`/panel/xray/resetOutboundsTraffic`	`/panel/api/xray/resetOutboundsTraffic`	POST
`/panel/xray/testOutbound`	`/panel/api/xray/testOutbound`	POST
`/panel/xray/warp/:action`	`/panel/api/xray/warp/:action`	POST
`/panel/xray/nord/:action`	`/panel/api/xray/nord/:action`	POST
`/panel/xray/outbound-subs` (и `/outbound-subs/*`)	`/panel/api/xray/outbound-subs` (и `/outbound-subs/*`)	GET/POST/DELETE

Сами имена под-путей, тела запросов и форматы ответов не менялись — изменился **только префикс**.

Как починить существующие интеграции

1. Найдите в своих скриптах/конфигах все вхождения `/panel/setting/` и `/panel/xray/`.
2. Замените префикс: добавьте `api/` сразу после `/panel/` (например, `/panel/setting/all` → `/panel/api/setting/all`).
3. Тела запросов, параметры и формат ответов править не нужно — меняется только URL.
4. Поскольку настройки и Xray-конфигурация теперь под `/panel/api`, к ним можно (и нужно) обращаться по тому же API-токену `Authorization: Bearer <token>`, что и к `/panel/api/inbounds/*` и прочим эндпоинтам. Не забывайте про CSRF-middleware, которое включено на всю группу `/panel/api`.

Пример: чтение всех настроек через API. Раньше (⩽ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
  -H "Authorization: Bearer <token>"
```

Теперь (3.3.0) — добавили `api/` после `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
  -H "Authorization: Bearer <token>"
```

Аналогично перезапуск панели: `POST /panel/api/setting/restartPanel`. Старый путь `/panel/setting/restartPanel` теперь вернёт 404.

Типизированный API: схемы и документация (Swagger / OpenAPI)

В 3.3.0 OpenAPI-спецификация стала полноценно типизированной. Раньше типизированные ответы описывались пустым объектом `{}`; теперь компоненты и схемы (`components.schemas`) генерируются прямо из моделей данных. Благодаря этому:

- Swagger UI показывает реальные модели данных, а не безликие заглушки.
- Внешние генераторы (`openapi-generator` и т. п.) могут построить по спецификации готовые клиенты на нужном языке.
- К каждому типизированному ответу прикреплён `$ref` на конкретную модель и приложены примеры ответов.

Где смотреть документацию API:

- **Встроенная страница Swagger.** В меню панели — пункт **«Документация API»** (SPA-маршрут `/panel/api-docs`). Здесь интерактивно перечислены все эндпоинты с описаниями, телами запросов и примерами ответов.
- **Сырая OpenAPI 3.0-спецификация** отдаётся по адресу `/panel/api/openapi.json`. Этот URL можно скормить напрямую в Postman, Insomnia или `openapi-generator`. Спецификация встроена в бинарник на этапе сборки; при работе панели под нестандартным `webBasePath` поле `servers` в спецификации автоматически переписывается под текущий базовый путь, чтобы кнопка «Try it out» и внешние генераторы били по правильному префиксу.

13.11. Discord-бот (вкладка «Discord бот» / *Discord Bot*)

Вкладка открывается пунктом **Discord бот** в боковом меню, в группе настроек (адрес `/panel/settings#discord`), и устроена так же, как вкладка Telegram-бота: внутри две вкладки — **Панель** и **Уведомления**. Здесь — сводка ключей; настройка бота в Discord, назначение полей, события и команды описаны в 14.8.

Внутренняя вкладка	Поле	Ключ	По умолчанию
Панель	Включить уведомления в Discord	`discordBotEnable`	`false`
Панель	Токен Discord бота	`discordBotToken`	`""` (секрет)
Панель	ID канала	`discordChannelId`	`""`
Панель	ID администраторов	`discordAdminIds`	`""`
Панель	Язык Discord-бота	`discordLang`	`en-US`
Уведомления	Частота уведомлений от Discord-бота	`discordRunTime`	`@daily`
Уведомления	Резервное копирование базы данных	`discordBotBackup`	`false`

Внутренняя вкладка	Поле	Ключ	По умолчанию
Уведомления	Уведомления Discord (выбор событий)	`discordEnabledEvents`	`login.attempt,cpu.high`
Уведомления	Пороги «Превышение порога CPU (%)» и «Превышение порога памяти (%)»	`discordCpu`, `discordMemory`	`80`, `80`

Кнопка **Отправить тестовое уведомление** (`POST /panel/api/setting/testDiscord`) находится на вкладке **Панель** и доступна, когда включён переключатель бота. Большая часть изменений на этой вкладке применяется сразу после сохранения, без перезапуска панели (подробности — в 14.8).

---

14. Telegram-бот

Панель 3X-UI содержит встроенного Telegram-бота, через которого можно получать уведомления о состоянии сервера и клиентов, а также управлять отдельными клиентами прямо из мессенджера. Бот работает по технологии long polling (постоянный опрос Telegram), поэтому ему не требуется внешний домен или открытый порт — достаточно исходящего доступа к серверам Telegram. Уведомления о тех же событиях, отчёты по расписанию и основные команды доступны и в Discord — см. 14.8.

Бот различает три уровня доступа:

- **Администратор** — пользователь, чей Telegram User ID указан в настройках бота (поле «User ID администратора бота»). Имеет доступ ко всем функциям: статистике сервера, бэкапу, управлению клиентами, перезапуску Xray.
- **Клиент** — любой другой пользователь, чей Telegram User ID привязан к конкретному клиенту входящего подключения (поле `tgId` клиента). Видит только информацию о собственных подписках.
- **Посторонний** — аккаунт, который не указан в настройках и не привязан ни к одному клиенту. Для него доступны только `/start` и `/id`: бот сообщает ему его User ID, чтобы он мог передать этот номер администратору.

Бот работает **только в личных сообщениях**. В группе или канале он не выполняет команды и нажатия, а отвечает «🤖 Бот работает только в личных сообщениях. Открой его в личке.» Группа с темами (форум) годится лишь как адресат уведомлений администратора — см. «Уведомления в тему форума» в 14.1.

Пример: привязка клиента к Telegram. Чтобы пользователь получал статистику по своей подписке, его числовой Telegram User ID записывается в поле `tgId` клиента. В JSON-настройках клиента это выглядит так:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

После этого пользователь с User ID `123456789` сможет запросить у бота `/usage ivan` и увидеть свою статистику. Тот же ID администратор может проставить через кнопку «👤 Установить пользователя Telegram» в карточке клиента — вручную в JSON лезть не обязательно.

14.1. Включение и настройка бота

Все параметры бота задаются в панели в разделе **Настройки** → **Telegram-бот**. После изменения настроек достаточно их сохранить — панель применяет их сразу, перезапуск панели не требуется. Если изменены флаг включения (`tgBotEnable`), токен, User ID администраторов или адрес API-сервера, панель автоматически останавливает и заново запускает бота с новыми параметрами. Прежнее правило о необходимости перезапуска панели после смены токена больше не действует.

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Включить Telegram бота	`tgBotEnable`	`false`	Главный переключатель. Подсказка: «Доступ к функциям панели через Telegram-бота». Пока выключен, бот не запускается и задания уведомлений не планируются.
Telegram-токен	`tgBotToken`	(пусто)	Токен бота. Подсказка: «Необходимо получить токен у менеджера ботов Telegram @botfather». Без непустого токена бот не стартует.
SOCKS-прокси	`tgBotProxy`	(пусто)	Прокси для подключения к Telegram. Подсказка (RU): «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству».
Telegram API Server	`tgBotAPIServer`	(пусто)	Альтернативный API-сервер Telegram. Подсказка: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию».
User ID администратора бота	`tgBotChatId`	(пусто)	Один или несколько Telegram User ID администраторов через запятую. К ID можно добавить тему форума через двоеточие: `chatId:topicId`. Подсказка: «Для получения User ID используйте @userinfobot или команду `/id` в боте. Чтобы отправлять в тему форума, добавьте ':topicId', например -1001234567890:42».
Частота уведомлений для администраторов от бота	`tgRunTime`	`@daily`	Расписание периодического отчёта в формате crontab. Подсказка: «Укажите интервал уведомлений в формате Crontab».
Резервное копирование базы данных	`tgBotBackup`	`false`	Подсказка: «Отправлять уведомление с файлом резервной копии базы данных». Прикладывает бэкап к периодическому отчёту.
Уведомление о входе	`tgBotLoginNotify`	`true`	Подсказка: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель».
Порог нагрузки на ЦП для уведомления	`tgCpu`	`80`	Порог загрузки CPU в процентах (валидация 0–100). Подсказка: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)». При значении 0 проверка CPU отключена.
Язык Telegram-бота	—	—	Язык, на котором бот формирует все сообщения.

Получение токена через @BotFather

1. Откройте в Telegram диалог с **@BotFather**.
2. Отправьте команду `/newbot` и следуйте инструкциям (имя бота и уникальное `username`, оканчивающееся на `bot`).
3. BotFather выдаст токен вида `123456789:AA...`. Скопируйте его в поле **Telegram-токен**.

4. Включите inline-режим бота (команда `/setinline`, подробности — в следующем подразделе): без него списки inbound и клиентов не откроются.

Включение inline-режима в @BotFather

Списки inbound и клиентов бот показывает через inline-режим Telegram. Кнопки «🔍 Find inbound…» и «🔍 Find client…» открывают в этом же чате поиск по боту: в выдаче карточки inbound или клиентов (с трафиком, сроком и статусом онлайн), выбор карточки открывает соответствующий экран. Администратору доступны списки inbound и всех клиентов (в том числе клиентов одного inbound), клиенту — только его собственные подписки.

Telegram включает inline-режим только в @BotFather, панель сделать это за вас не может: отправьте `/setinline`, выберите своего бота и введите любой текст-подсказку для строки поиска. Пока режим выключен, вместо кнопок поиска бот показывает кнопку «⚠️ Inline mode is off», которая открывает экран с инструкцией и кнопкой «Check again» (повторная проверка).

Получение User ID администратора

User ID — это числовой идентификатор аккаунта (не username). Узнать его можно двумя способами:

- Написать боту **@userinfobot**.
- Запустить уже настроенного бота и отправить ему команду **`/id`** — он вернёт ваш ID.

Полученное число впишите в поле **User ID администратора бота**. Чтобы назначить нескольких администраторов, перечислите их ID через запятую (например, `11111111,22222222`). Каждый ID проверяется как целое число; некорректное значение приведёт к ошибке запуска бота.

Пример: значение поля «User ID администратора бота». Один администратор — просто число:

```
123456789
```

Два администратора через запятую (пробелы можно не ставить):

```
123456789,987654321
```

Каждое значение должно быть целым числом (необязательная тема форума после двоеточия описана ниже). Запись вида `@username` или `123 456` (с пробелом внутри числа) не подойдёт — бот не запустится.

Уведомления в тему форума

Если администратор — группа-форум (супергруппа с включёнными темами), уведомления можно направить не в общую тему General, а в конкретную: добавьте после ID чата ID темы через двоеточие.

```
123456789,-1001234567890:42
```

Здесь первый администратор — личный чат, а второй — тема `42` форума `-1001234567890`. Уведомления, периодический отчёт, документы (бэкап и лог банов) и сообщения бота для этого администратора уходят в эту тему. Правила записи:

- ID темы — положительное целое число; тема `1` (General) равнозначна записи без темы.
- Для одного чата нельзя указать две разные темы — это ошибка настройки, бот не запустится.
- Повторяющиеся записи одного и того же чата сворачиваются в одну.

Управлять ботом (меню, кнопки, команды) по-прежнему нужно в личных сообщениях: в группе он не отвечает, она только принимает уведомления.

Прокси

Поддерживаются схемы `socks5://`, `http://` и `https://`. Если поле прокси оставлено пустым, бот пытается использовать общий прокси панели (если он задан и его схема поддерживается). URL с неподдерживаемой схемой или некорректным синтаксисом игнорируется — бот соединяется напрямую. Прокси полезен, когда прямой доступ к API Telegram с сервера заблокирован.

Уведомления по электронной почте (SMTP)

Помимо Telegram, те же события можно получать письмами. Канал настраивается в разделе **Настройки** → **Email** на вкладке **SMTP Settings**:

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Enable Email Notifications	`smtpEnable`	`false`	Главный переключатель email-уведомлений через SMTP.
SMTP Host	`smtpHost`	(пусто)	Хост SMTP-сервера (например, `smtp.gmail.com`).
SMTP Port	`smtpPort`	`587`	Порт SMTP-сервера.
SMTP Username	`smtpUsername`	(пусто)	Имя пользователя для аутентификации SMTP. Используется и как адрес отправителя (From).
SMTP Password	`smtpPassword`	(пусто)	Пароль для аутентификации SMTP. Хранится скрыто; если пароль уже задан, поле показывает признак «настроен», и его можно оставить пустым, чтобы сохранить текущий.
Recipients	`smtpTo`	(пусто)	Список получателей через запятую (например, `admin@example.com, ops@example.com`).
Encryption	`smtpEncryptionType`	`starttls`	Тип шифрования соединения: `none` (без шифрования), `starttls` (STARTTLS) или `tls` (неявный TLS).

Кнопка **Отправить тестовое письмо** (*Send Test Email*) отправляет пробное письмо и показывает результат по этапам: **Подключение** (*Connection*), **Аутентификация** (*Authentication*) и **Отправка** (*Send*). Если что-то не так, диагностика указывает этап и причину понятной фразой — например, «Ошибка аутентификации — проверьте логин и пароль» или «Сервер требует STARTTLS — измените тип шифрования», — что упрощает

подбор параметров. Ошибку, которую панель не распознала, она выводит как «Ошибка SMTP — подробности в журнале панели».

На второй вкладке (**Notifications**) выбираются события, о которых будут приходить письма, — теми же карточками-группами, что и для Telegram (см. «Шина событий и выбор уведомлений» в разделе 14.5).

API-сервер Telegram

По умолчанию бот обращается к официальному API Telegram. В поле **Telegram API Server** можно указать адрес собственного Bot API сервера (`telegram-bot-api`). URL проверяется на безопасность; заблокированный или некорректный адрес отбрасывается, и используется сервер по умолчанию.

Проверка бота

Кнопка **Отправить тестовое сообщение** рядом с настройками бота отправляет пробное сообщение администраторам и показывает результат: «Тестовое сообщение отправлено в Telegram», либо причину сбоя («Telegram бот не включен», «Telegram бот не запущен», «Тест Telegram не удался»). Заодно панель опрашивает работающего бота (`POST /panel/api/setting/tgBotCapabilities`) и при необходимости показывает под полями предупреждения:

- **Inline mode is disabled** — у бота выключен inline-режим (`/setinline` в @BotFather, см. выше), и кнопки поиска в боте не сработают; кнопка **Check again** повторяет проверку после включения.
- **Режим приватности включён** — справочное предупреждение: бот работает только в личных сообщениях и группы игнорирует намеренно, поэтому режим приватности в группах ни на что не влияет.

14.2. Главное меню и кнопки

Меню вызывается командой `/start` — это единственная команда, зарегистрированная в меню «/» Telegram; остальные команды (см. 14.3) работают, если набрать их вручную.

Интерфейс бота — **один «живой» экран на чат**: сообщение с картинкой (аватар бота либо чёрная плитка), текстом в подписи и inline-клавиатурой. Нажатие кнопки не присылает новое сообщение, а **правит** текущее; новое бот отправляет только тогда, когда править нечего (экран удалён или слишком старый). Над текстом стоит путь вида «Panel › Server». Экран, кроме главного, заканчивается кнопкой «⬅️ Back» — возврат в главное меню. Текст, не помещающийся в подпись (1024 символа), листается: внизу появляется строка «‹ Page 2 of 5 ›» с кнопками перехода. Разовые сообщения (уведомления, бэкап, файлы) приходят отдельными сообщениями с кнопкой «🙈 Hide», которая их удаляет.

Набор кнопок зависит от того, администратор вы или клиент.

Меню администратора

Главный экран показывает сводку по панели (хост, версия, число inbound, клиентов и онлайн, общий трафик) и четыре раздела плюс «🔄 Обновить». Остальные функции — на один экран глубже:

Раздел	Кнопки
🖲 Сервер	🖥 Состояние сервера, 🔌 Входящие подключения, 🟢 Онлайн
👥 Клиенты	👥 Все клиенты, ➕ Новый клиент, 📢 Рассылка
🚦 Трафик	📈 Отсортированный отчет об использовании трафика, ⚠️ Скоро конец
⚙️ Обслуживание	🗄 Бэкап БД, 📄 Лог банов, ♻️ Restart Xray, ♻️ Сбросить весь трафик

Кнопка	Действие
📈 Отсортированный отчет об использовании трафика	Перечисляет всех клиентов, отсортированных по трафику, с расходом каждого — одним экраном, который при большом числе клиентов листается страницами; «лишние» email без данных помечаются « ❗ Нет результатов».
🖥 Состояние сервера	Сводка по серверу (см. раздел 14.5). Кнопка «🔄 Обновить» перерисовывает данные.
♻️ Сбросить весь трафик	Сбрасывает счётчики трафика **всех** клиентов. Запрашивает подтверждение («Вы уверены? 🤔»), затем присылает один отчёт (при большом числе клиентов — разбитый на части), где по каждому клиенту указано «✅ Успешно» либо «❌ Неудача», а в конце — «🔚 Сброс трафика завершён для всех клиентов». Так бот не упирается в ограничение Telegram на частоту сообщений даже на панели с сотнями клиентов.
♻️ Restart Xray	Перезапускает ядро Xray; результат («✅ Ядро Xray успешно перезапущено» или причина сбоя) всплывает подсказкой, экран остаётся на месте.
🗄 Бэкап БД	Высылает файл базы данных и `config.json` (см. раздел 14.6).
📄 Лог банов	Высылает файл лога забаненных по IP-лимиту адресов.
🔌 Входящие подключения	Сводка по всем inbound: Remark, порт, трафик, число клиентов, дата окончания. Кнопка «🔍 Find inbound…» открывает поиск по inbound (см. ниже).
⚠️ Скоро конец	Сводка по inbound и клиентам, у которых скоро исчерпается трафик или срок (см. раздел 14.5).
🟢 Онлайн	Количество и список клиентов, находящихся онлайн (рядом с email — Remark inbound). Кнопка «🔄 Обновить».
👥 Все клиенты	Открывает поиск по всем клиентам панели (кнопка «🔍 Find client…», см. ниже); выбор клиента открывает его карточку.
➕ Новый клиент	Запускает мастер добавления клиента (черновик → привязка inbound → подтверждение).
📢 Рассылка	Массовая рассылка клиентам — то же, что команда `/broadcast` (см. раздел 14.3).

Списки через поиск (inline-режим)

Списки не выводятся в сообщении, а открываются поиском бота в текущем чате; для этого у бота должен быть включён inline-режим (см. раздел 14.1). После нажатия «🔍 Find inbound…» или «🔍 Find client…» в строке ввода появляется `@бот` с запросом, а ниже — карточки результатов. Можно набрать часть названия для фильтра; длинный список подгружается порциями. Выбор карточки открывает экран: для inbound — список его клиентов, для клиента — карточку клиента (раздел 14.4). Выбранный элемент приходит в чат как сообщение-маркер вида `inb:3` или `cl:email`; бот сразу его удаляет и рисует экран. Список inbound доступен только администратору, а клиент в выдаче видит лишь свои подписки.

Меню клиента

Клиенту доступен ограниченный набор: на главном экране — сводка по всем привязанным к его Telegram User ID подпискам (расход, срок, статус). Дальше:

Кнопка	Действие
🔗 Links	Если подписка одна: экран со ссылкой на подписку и JSON-подписку, а также кнопками «🔗 Individual links» (индивидуальные ссылки для приложений) и «▪ QR code».
▪ QR code	QR-коды подписки: все коды собраны в **одну картинку-лист**, а в подписи идёт нумерованная легенда «▪ QR-коды:» — «1. Подписка», «2. JSON-подписка», «3. Ссылка для приложения …». Кнопка «🔗 Individual links» — переход к ссылкам.
email клиента	Если подписок несколько, вместо «Links» и «QR code» показывается строка на каждую подписку (её email); выбор открывает экран ссылок этой подписки.

Если у пользователя нет ни одного клиента с его Telegram User ID, бот отвечает: «❌ Ваша конфигурация не найдена! 💭 Пожалуйста, попросите администратора использовать ваш Telegram User ID в конфигурации. 🆔 Ваш User ID: …». Этот ID нужно передать администратору, чтобы тот вписал его в поле клиента.

Кнопки ссылок (подписка, индивидуальные ссылки, QR-код) клиент может использовать только для своих подписок: бот проверяет, что email из кнопки принадлежит клиенту, привязанному к Telegram User ID нажавшего. Пересланная карточка, кнопка в групповом чате или клавиатура, оставшаяся после того, как администратор снял привязку, на нажатие отвечают « ❗ Ошибка в операции.» Администраторов эта проверка не касается.

14.3. Команды бота

Кнопки удобнее команд, поэтому в меню «/» Telegram зарегистрирована только `/start`, а остальные команды работают, если набрать их в чате. Кнопки «Команды» (справки по командам) в меню больше нет.

У бота **три уровня доступа**, определяемые по отправителю: **посторонний** (любой аккаунт, которого админ не связал с клиентом) — доступны только `/start` и `/id`; **клиент** (аккаунт, привязанный к клиенту по Telegram ID) — `/start`, `/help`, `/status`, `/id` и свой `/usage`; **администратор** (ID из настроек бота) — все команды, включая `/broadcast`. Команда, добавленная в бот позже, остаётся админской, пока её явно не откроют нижним уровням. Авторизация учитывает только личные чаты (#6518); поиск клиента по Telegram ID идёт по индексированной таблице клиентов и не замедляется на панели с большим числом клиентов (#6887).

Команда	Описание	Доступ	Что делает
`/start`	Показать главное меню	все	Главное меню по уровню доступа (см. 14.2); постороннему — только его User ID и просьба передать его администратору. Вариант `/start <токен>` привязывает аккаунт по ссылке-приглашению (см. 14.4).
`/help`	Справка по боту	клиент, админ	Выводит общее приветствие и предложение выбрать пункт меню.
`/status`	Проверить статус бота	клиент, админ	Администратору открывает экран «Состояние сервера»; клиенту отвечает «✅ Бот в порядке!» — без сведений о сервере.

Команда	Описание	Доступ	Что делает
`/id`	Показать ваш Telegram ID	все	Возвращает «🆔 Ваш User ID: `…` ». Удобно для получения собственного User ID.
`/usage`	Показать расход клиента: /usage email	клиент, админ	Расход подписки (см. ниже).
`/inbound`	Поиск inbound: /inbound remark	админ	Ищет inbound по Remark.
`/restart`	Перезапустить ядро Xray	админ	Перезапускает Xray.
`/clearall`	Сбросить трафик всех клиентов	админ	Запрашивает подтверждение («Вы уверены? 🤔», кнопка «✅ Подтвердить сброс трафика?») и обнуляет трафик всех клиентов.
`/broadcast`	Массовая рассылка всем клиентам	админ	(#6510) Бот просит сообщение (текст, фото, видео, файл или альбом), показывает предпросмотр и спрашивает подтверждение с числом получателей. Рассылка копируется 1-в-1 (без пометки «переслано», личность админа скрыта) всем клиентам с привязанным Telegram ID, кроме админов; есть прогресс и отмена. Недоступные получатели (не нажавшие Start / заблокировавшие бота) помечаются «Пропущено».

Детали команд-аргументов:

- **`/usage [Email]`** — поиск клиента по email.
  - Для **администратора** открывает карточку клиента (с кнопками управления).
  - Для **клиента** показывает только его собственную подписку с указанным email (по привязке Telegram User ID). Без аргумента бот просит указать email: «❗ Пожалуйста, укажите email для поиска».
- **`/inbound [имя подключения]`** — только для администратора. Ищет inbound по Remark и выводит его параметры со статистикой всех клиентов. Без аргумента (или для клиента) — «❗ Неизвестная команда».
- **`/restart`** — только для администратора. Перезапускает Xray Core. Возможные ответы: «✅ Ядро Xray успешно перезапущено», «❗ Xray Core не запущен» (если ядро не работает), «❗ Ошибка при перезапуске Xray-core. <Ошибка>». Любые аргументы после `/restart` приводят к сообщению о неизвестной команде с подсказкой `/restart`.

В групповых чатах бот команды не выполняет (см. начало раздела 14), поэтому суффикс `/команда@botusername` имеет смысл только в личном чате.

Сообщения **бэкапа** и **бан-лога** начинаются со строки `Hostname==…` — удобно, когда один бот обслуживает несколько панелей.

14.4. Управление клиентом (только администратор)

Карточку клиента администратор открывает из поиска («👥 Все клиенты» или список клиентов выбранного inbound), командой `/usage email`, а также автоматически после создания клиента мастером «➕ Новый клиент». Она показывает сведения о клиенте (email, привязанные inbound, статус «Активен», статус

соединения, дата окончания, расход трафика) и inline-кнопки управления; «⬅️ Back» возвращает в главное меню:

Кнопка	Назначение
🔄 Обновить	Перечитать карточку клиента.
📈 Сбросить трафик	Обнулить счётчик трафика клиента. Требует подтверждения «✅ Подтвердить сброс трафика?».
🚧 Лимит трафика	Задать лимит трафика. Готовые значения: ♾ Безлимит (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB или «🔢 Своё» — ввод числа на встроенной цифровой клавиатуре (кнопки 0–9, «🔄» — сброс к 0, «⬅️» — стереть последнюю цифру, «✅ Подтвердить: N»). Значение задаётся в гигабайтах.
📅 Изменить дату окончания	Готовые варианты: ♾ Безлимит, «🔢 Своё», добавить 7/10/14/20 дней, 1/3/6/12 месяцев. Положительное число продлевает срок (прибавляет дни к текущей дате окончания либо к «сейчас», если срок уже истёк); 0 снимает ограничение по сроку.
🔢 Лог IP	Показывает зафиксированные IP-адреса клиента (с отметками времени, если есть). Из лога доступны «🔄 Обновить» и «❌ Очистить IP» (с подтверждением «✅ Подтвердить очистку IP?»).
🔢 Лимит IP	Ограничение одновременных IP. Варианты: ♾ Безлимит (0), 1–10 или «🔢 Своё» (цифровая клавиатура).
👤 Установить пользователя Telegram	Показывает текущий привязанный Telegram User ID клиента; позволяет очистить привязку («❌ Удалить пользователя Telegram» с подтверждением). Привязка нового пользователя выполняется через системный выбор контакта Telegram.
🔗 Ссылка-приглашение	(3.9.0) Создаёт ссылку `https://t.me/<бот>?start=…` : **первый**, кто её откроет, привязывается к этому клиенту. Ссылка одноразовая, одинаково отвечает на неверный и уже использованный токен (подбор невозможен), ограничена 5 попытками в час на аккаунт. Ручная привязка по `/id` сохранена (#6518).
🔘 Вкл./Выкл.	Включает или отключает клиента. Требует подтверждения «✅ Подтвердить вкл/выкл пользователя?».

> **Ограничение 3.11.0.** Кнопок «🔗 Ссылки» / «◼️ QR-код» в карточке клиента у администратора нет, а прежнее админ-меню ссылок удалено: ссылки и QR-коды чужих клиентов бот администратору не выдаёт — берите их в панели (раздел 8). Свои подписки администратор видит, как клиент, если его Telegram ID привязан к клиенту (#6984).

Все операции, изменяющие конфигурацию (лимит трафика/IP, дата окончания, привязка/отвязка Telegram-пользователя, вкл/выкл), при необходимости помечают Xray на перезапуск, чтобы изменения вступили в силу. После успешной операции бот выводит подтверждение вида «✅ : …» и заново показывает карточку.

Любой ввод цифр в мастерах ограничен значением < 999999.

Если у клиента срок в режиме «после первого использования», карточка показывает его числом дней — в том числе у отключённого клиента.

Мастер «➕ Новый клиент». Мастер начинается не с выбора inbound, а с **черновика**: бот сразу показывает карточку с подставленными значениями (случайный email, безлимитные трафик и срок), а меняют их кнопками «⚙️📧 Email», «⚙️💬 Комментарий», «🚧 Лимит трафика», «📅 Изменить дату окончания», «🔢

Лимит IP», «👤 Установить пользователя Telegram». Inbound выбираются кнопкой «Привязать инбаунд (N)» (N — сколько выбрано): откроется список с отметками ☐ / ✅, клиента можно привязать сразу к нескольким inbound; протоколы без клиентов (Tunnel, Mixed, HTTP) в список не попадают. «✅ Готово» возвращает к карточке; без хотя бы одного выбранного inbound клиента создать нельзя. Завершают мастер кнопки «Добавить включенным ✅» и «Добавить отключенным ☑», а «❌ Отмена» отбрасывает черновик; после создания открывается карточка нового клиента. Черновик хранится отдельно для каждого чата, поэтому два администратора могут заводить клиентов одновременно, не перезаписывая значения друг друга. Карточка черновика оформлена HTML-разметкой, а введённые значения (email, комментарий, Telegram ID, примечания inbound) экранируются — символ `<` в них не ломает сообщение. Срок действия задаётся кнопками «♾ Безлимит», «🔢 Своё» и «7 Дней» … «12 Месяцев»: повторный выбор **заменяет** срок, а не прибавляет к нему, и срок отсчитывается с первого использования клиента. Кнопки «Добавить …» в меню «📅 Изменить дату окончания» существующего клиента, наоборот, продлевают текущий срок.

14.5. Уведомления и отчёты

Уведомления отправляются всем администраторам (всем User ID из `tgBotChatId`); если для администратора указана тема форума (`chatId:topicId`), они приходят в неё.

Шина событий и выбор уведомлений

Уведомления построены на единой шине событий, а каналов доставки три — **Telegram**, **электронная почта (SMTP)** и **Discord** (см. 14.8). Для каждого канала отдельно выбирается, о каких событиях оповещать: в **Настройки** → **Telegram-бот**, **Настройки** → **Email** и **Настройки** → **Discord бот** это делается на внутренней вкладке **Уведомления** (*Notifications*).

События сгруппированы карточками; у каждой группы есть мастер-переключатель со счётчиком включённых событий (n/всего) и промежуточным состоянием, когда выбрана только часть. Доступны группы:

- **Outbound** — «Down» (`outbound.down`) и «Up» (`outbound.up`): падение и восстановление outbound.
- **Xray Core** — «Crash» (`xray.crash`): аварийное завершение ядра Xray.
- **Nodes** — «Down» (`node.down`) и «Up» (`node.up`): узел стал недоступен или восстановился.
- **System** — «CPU high (%)» (`cpu.high`) и «Memory high (%)» (`memory.high`): высокая загрузка процессора и оперативной памяти. У обоих событий рядом находится инлайн-поле порога в процентах.
- **Security** — «Login attempt» (`login.attempt`): попытка входа в панель.

Набор включённых событий хранится раздельно: для Telegram — в `tgEnabledEvents`, для Email — в `smtpEnabledEvents`, для Discord — в `discordEnabledEvents`. По умолчанию во всех трёх каналах включены «Login attempt» и «CPU high» (значение `login.attempt,cpu.high`).

Как уведомления приходят в чат

Каждое уведомление — отдельное сообщение с кнопкой «🙈 Hide» (удаляет его). Для одного чата и одного вида события существует **одна живая карточка**: повтор события в течение 30 минут не плодит сообщения, а правит эту карточку и добавляет счётчик вида «🔁 repeated 3× · last 14:05:09». Через 30 минут (или если карточку уже нельзя править) приходит новая. Успешный и неудачный входы в панель — разные виды,

поэтому у них разные карточки. Отдельного ограничителя частоты уведомлений больше нет: его заменила эта свёртка повторов.

Уведомление о входе в панель

Управляется галочкой **Уведомление о входе** (`tgBotLoginNotify` , по умолчанию включено). При каждой попытке входа в веб-панель администраторам приходит сообщение:

- При успехе: «✅ Успешный вход в панель.» + хост, имя пользователя, IP, время.
- При неудаче: « ❗ Ошибка входа в панель.» + хост, **причина** (например, «Ошибка 2FA» при неверном втором факторе), имя пользователя, IP, время.

Превышение нагрузки на CPU и память

Раз в минуту панель проверяет загрузку процессора и оперативной памяти. Если порог **`tgCpu`** > 0 и средняя загрузка CPU за минуту его превышает, администраторам уходит: «🔴 Загрузка процессора составляет N%, что превышает пороговое значение M%». Аналогично проверяется загрузка RAM против порога **`tgMemory`** (по умолчанию 80%) — событие «Memory high (%)».

Оба порога задаются инлайн-полями рядом с событиями «CPU high (%)» и «Memory high (%)» в группе **System** на вкладке **Уведомления** (см. «Шина событий и выбор уведомлений» выше). Для канала Email действуют отдельные ключи `smtpCpu` и `smtpMemory` , для Discord — `discordCpu` и `discordMemory` . При значении порога 0 соответствующая проверка отключена. Задача проверки CPU и памяти планируется при запуске панели и только если хотя бы одному включённому каналу нужно это событие с ненулевым порогом: если при запуске панели оно не было нужно ни одному каналу, после его включения перезапустите панель.

Периодический отчёт (по расписанию)

Планируется по cron-выражению из поля **Частота уведомлений** (`tgRunTime` , по умолчанию `@daily`). Если значение пустое, используется `@daily` . С 3.10.0 расписание проверяется при сохранении настроек: нераспознаваемое выражение отклоняется с ошибкой `tgRunTime: cron schedule <…> is not valid` . Если в базе осталось старое значение, которое не разбирается, отчёт больше не пропадает: он идёт по расписанию по умолчанию (`@daily`), а в журнале панели появляется предупреждение `Telegram notify: cron schedule <…> is not valid: …; using default @daily` . (До 3.10.0 такое выражение, например crontab из 5 полей, сохранялось без ошибки, но отчёт молча не планировался.) Отчёт включает:

Конструктор расписания

Поле **Частота уведомлений для администраторов от бота** задаётся не вводом строки вручную, а через конструктор расписания. Сначала в выпадающем списке выбирается режим:

- **`@every` — повторять с интервалом** — появляются поле числа и выбор единицы (**Секунды** / **Минуты** / **Часы**); итог собирается в выражение вида `@every 6h` .
- **`@hourly` — каждый час**, **`@daily` — каждый день в 00:00**, **`@weekly` — каждую неделю**, **`@monthly` — каждый месяц** — готовые пресеты, которые сохраняются как соответствующий макрос (`@hourly` , `@daily` , `@weekly` , `@monthly`).

- **Произвольный (crontab)** — поле для собственного crontab-выражения. Произвольное выражение состоит из **6 полей**: секунда, минута, час, день месяца, месяц, день недели (например, `0 30 8 * * *` — каждый день в 08:30:00). С 3.10.0 секунды необязательны — принимается и стандартный crontab из **5 полей** (`30 8 * * *` — то же самое). При переключении на этот режим поле заполняется crontab-эквивалентом текущего выбора, чтобы было от чего отталкиваться.

Пример: значения поля «Частота уведомлений» (`tgRunTime`). Поддерживаются как готовые сокращения, так и полный crontab-формат:

Значение	Когда срабатывает
`@daily`	раз в сутки в полночь (значение по умолчанию)
`@hourly`	каждый час
`@every 6h`	каждые 6 часов
`0 0 9 * * *`	каждый день в 09:00
`0 0 9 * * 1`	каждый понедельник в 09:00
`0 0 */12 * * *`	каждые 12 часов (в 00:00 и 12:00)
`0 9 * * *`	каждый день в 09:00 (5 полей, с 3.10.0)

Порядок полей в 6-польном выражении: секунда, минута, час, день месяца, месяц, день недели. Отчёты планирует не системный cron, а встроенный планировщик панели (Go-библиотека `robfig/cron`, запущенная с поддержкой секунд). До 3.10.0 он не принимал привычное выражение из 5 полей, где младшее поле — минуты: оно сохранялось без ошибки, но отчёт не планировался. С 3.10.0 расписания, вводимые администратором (отчёты Telegram и Discord, синхронизация LDAP), разбираются с необязательным столбцом секунд: 5 полей — минута, час, день месяца, месяц, день недели, — 6 полей — с секундами впереди; макросы `@every` / `@daily` и т. п. работают как прежде.

1. Строку «🈁 Запланированные отчёты: <расписание>» и текущие дату/время.
2. **Состояние сервера** (см. ниже).
3. Блок «Скоро конец» по inbound и клиентам.
4. Персональные уведомления клиентам с привязанным Telegram User ID — каждому клиенту-неадмину уходит список его подписок, у которых скоро исчерпание трафика/срока (с учётом отключённых); это отдельные сообщения с кнопкой «🙈 Hide».
5. Если включено **Резервное копирование базы данных** (`tgBotBackup`) — бэкап БД администраторам.

Для администратора отчёт приходит одним блоком и **заменяет прежний**: бот удаляет сообщение предыдущего отчёта, поэтому в чате остаётся только последний.

Состояние сервера содержит: хост, версию 3X-UI и Xray, IPv4/IPv6, время работы (в днях), среднюю нагрузку (Load1/2/3), RAM (текущая/всего), число онлайн-клиентов, счётчики TCP/UDP-соединений, суммарный сетевой трафик (↑/↓) и статус Xray.

«Скоро конец» показывает:

- по inbound: число отключённых и число «скоро исчерпающихся»;
- по клиентам: то же, плюс карточки клиентов, у которых скоро конец (трафик или срок).

Пороги «скоро исчерпания» берутся из общих настроек панели: запас по трафику (в ГБ) и запас по сроку (в днях). «Исчерпывающимся» считается inbound/клиент, у которого до лимита трафика осталось меньше порога ИЛИ до даты окончания осталось меньше порога.

14.6. Бэкап и логи

- **Бэкап БД** (кнопка «📂 Бэкап БД» или галочка в периодическом отчёте): бот присылает файл базы данных (`x-ui.db`, либо `x-ui.dump` для PostgreSQL) **одним сообщением с подписью** — в подписи имя хоста, время бэкапа и сводка «Скоро конец» (если подпись не влезает в 1024 символа, сводка уходит следующим сообщением) — и отдельным документом файл конфигурации Xray `config.json`.

Имя файла бэкапа, который присылает бот, формируется по адресу сервера: используется значение **webDomain**, а если оно не задано — публичный IP сервера. Это помогает понять, с какого сервера пришёл файл, когда бэкапы собираются с нескольких панелей. Если адрес определить не удалось, применяется обобщённое имя.

- **Лог банов** (кнопка «📄 Лог банов»): присылает сообщение с именем хоста и временем, затем файл лога IP-адресов, забаненных по превышению лимита IP (журнал `3xipl-banned.prev.log`). Если файла нет, бот отвечает «❗ Нет результатов»; пустой файл Telegram не принимает, и тогда приходит только сообщение с именем хоста. В 3.11.0 бот присылает только этот, предыдущий, журнал; текущий `3xipl-banned.log` (до 3.10.2 он приходил вторым файлом) смотрите на сервере (#6990).

14.7. Особенности работы

- **Длинные сообщения**: экран бота — подпись к картинке (лимит Telegram 1024 символа), поэтому длинный текст листается страницами по границам строк, а внизу появляется пагинатор «‹ Page X of Y ›». Отдельные сообщения (уведомления, отчёты) не влезают в одно при ~2000 символах и делятся на части по строкам; таблицы, списки клиентов и конфиги не рвутся посреди строки.
- **Один экран на чат**: нажатия правят одно сообщение, а не засоряют чат; лишние сообщения удаляются кнопкой «🙈 Hide».
- **Только личные чаты**: группы и каналы бот игнорирует, отвечая «🤖 Бот работает только в личных сообщениях. Открой его в личке.»
- **Параллельность**: команды и нажатия кнопок обрабатываются конкурентно (пул до 10 одновременных обработчиков).
- **Надёжность отправки**: при ошибках соединения сообщения отправляются повторно с экспоненциальной задержкой (1с/2с/4с, до 3 попыток).
- **Кэширование**: данные «Состояние сервера» кэшируются, чтобы частые нажатия «Обновить» не нагружали систему.
- **Перезапуск бота**: при сохранении настроек, затрагивающих бота (флаг включения, токен, User ID администраторов или адрес API-сервера), панель сама останавливает предыдущий цикл опроса и запускает новый с актуальными параметрами — перезагрузка панели для этого не нужна. Одновременно работает только один экземпляр приёма обновлений.
- **Ответ на любое нажатие**: кнопка, которую бот не может обработать, больше не «крутится» до таймаута Telegram — нераспознанное нажатие получает ответ «❗ Ошибка в операции.». Данные длинных кнопок панель хранит у себя 20 минут и не сохраняет между перезапусками; на нажатие такой устаревшей кнопки бот отвечает «❌ Запрос не найден. Пожалуйста, повторите команду.»

- **Сбой обработчика не роняет панель**: паника при обработке команды или кнопки перехватывается и пишется в журнал; нажатие старой кнопки со списком клиентов уже удалённого inbound-а получает ответ об ошибке.
- **Кнопка «🔄 Обновить» без изменений в данных** не засоряет журнал предупреждениями `message is not modified` — они пишутся на уровне Debug.

14.8. Discord-бот

Кроме Telegram, панель работает с ботом Discord. Он выполняет три задачи:

- **уведомления о событиях** — те же события шины, что у Telegram и Email (см. 14.5), приходят в канал карточками (embed);
- **отчёт по расписанию** о состоянии сервера, при желании — с резервной копией базы данных;
- **текстовые команды** (`!status` , `!backup` , `!restart` и другие), которые администраторы пишут в тот же канал.

Уведомления, отчёты и ответы на команды панель отправляет через REST API Discord (v10) по HTTPS, а команды принимает через постоянное WebSocket-соединение с Discord Gateway. Оба соединения исходящие: домен и открытый порт не нужны. Если задан **Исходящий для трафика панели** (`panelOutbound` , см. 13.4), трафик бота идёт через него; когда Xray не запущен, панель соединяется напрямую и пишет предупреждение в журнал. Discord-бот не зависит от Telegram-бота — их можно включить одновременно.

Создание бота в Discord

1. В Discord Developer Portal создайте приложение (**New Application**).
2. На вкладке **Bot** получите токен (**Reset Token**) и скопируйте его — это значение поля **Токен Discord бота**. Не путайте токен с Client Secret и Application ID.
3. Там же, в блоке **Privileged Gateway Intents**, включите **Message Content Intent**. Без него Discord закрывает соединение с Gateway (код 4014) и команды не работают; уведомления и отчёты при этом отправляются.
4. В разделе **OAuth2** → **URL Generator** отметьте scope `bot` и права **View Channels**, **Send Messages**, **Embed Links** и **Attach Files** (последнее нужно для бэкапов), откройте полученную ссылку и добавьте бота на свой сервер.
5. В клиенте Discord включите режим разработчика (**Настройки пользователя** → **Расширенные** → **Режим разработчика**). Правый клик по каналу → **Копировать ID канала** даёт значение для **ID канала**, правый клик по своему имени → **Копировать ID пользователя** — для **ID администраторов**.

Настройки в панели

Все параметры задаются в **Настройки** → **Discord бот** (сводка ключей — в 13.11). Первые пять полей таблицы находятся на внутренней вкладке **Панель**, остальные — на вкладке **Уведомления**.

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Включить уведомления в Discord	`discordBotEnable`	`false`	Главный переключатель. Подсказка: «Отправлять оповещения о событиях в канал Discord через бота». Пока выключен, не отправляются уведомления и отчёты и не устанавливается соединение с Gateway.
Токен Discord бота	`discordBotToken`	(пусто)	Токен бота. Хранится как секрет: панель его не возвращает, а в описании поля показывает «Токен настроен. Введите новый токен для замены.». Пустое поле при сохранении оставляет прежний токен, кнопка **Очистить** с последующим сохранением удаляет его. Префикс `Bot ` в начале значения можно не убирать.
ID канала	`discordChannelId`	(пусто)	Канал для уведомлений, отчётов и ответов на команды. Команды бот читает только в этом канале; личные сообщения и другие каналы игнорируются.
ID администраторов	`discordAdminIds`	(пусто)	ID пользователей Discord через запятую, которым разрешены команды. Подсказка: «… Сообщения остальных игнорируются, а пустой список отключает команды.» На уведомления и отчёты не влияет.
Язык Discord-бота	`discordLang`	`en-US`	Язык тестового сообщения, уведомлений, отчётов и ответов на команды.
Частота уведомлений от Discord-бота	`discordRunTime`	`@daily`	Расписание отчёта. Задаётся тем же конструктором, что у Telegram-бота: произвольный crontab — 6 полей (первое — секунды) или, с 3.10.0, 5 полей без секунд; нераспознаваемое значение не сохраняется, а неразбираемое старое заменяется на `@daily` с предупреждением в журнале.
Резервное копирование базы данных	`discordBotBackup`	`false`	Прикладывать к отчёту по расписанию и к ответу на `!report` файл базы данных и `config.json`.

Поле (UI)	Ключ настройки	Значение по умолчанию	Описание
Уведомления Discord	`discordEnabledEvents`	`login.attempt,cpu.high`	Выбор событий — те же карточки-группы, что для Telegram (см. 14.5).
Пороги CPU и памяти (в группе **Система**)	`discordCpu`, `discordMemory`	`80`, `80`	Порог в процентах (0–100) для событий `cpu.high` и `memory.high`; значение 0 отключает событие.

Перезапуск панели после сохранения обычно не нужен. Если изменились флаг включения или расписание, а при включённом боте — токен или ID канала, панель сама перепланирует отчёт и запускает или останавливает соединение с Gateway. ID администраторов, язык, набор событий, пороги и флаг бэкапа читаются в момент отправки. Исключение — проверка CPU и памяти: если при запуске панели эти события не были нужны ни одному каналу, после их включения перезапустите панель (см. 14.5).

Кнопка **Отправить тестовое уведомление** шлёт в канал карточку «Тестовое уведомление Discord от 3x-ui». Она использует **сохранённые** настройки, поэтому сначала нажмите **Сохранить**. Результат выводится под кнопкой: «Тестовое уведомление успешно отправлено», «Discord-бот не включен» (переключатель не сохранён включённым) или «Тест Discord не удался: …» с причиной (см. «Неполадки» ниже).

Уведомления о событиях

Событие	Карточка	Содержимое
`outbound.down`	«Исходящее недоступно» (красная)	тег outbound, ошибка, задержка
`outbound.up`	«Исходящее восстановлено» (зелёная)	тег outbound, задержка
`node.down`	«Узел недоступен» (красная)	имя узла, ошибка Xray на узле (если есть)
`node.up`	«Узел снова в сети» (зелёная)	имя узла, задержка
`xray.crash`	«Сбой ядра Xray» (красная)	текст ошибки
`cpu.high`	«Превышен порог CPU» (оранжевая)	текущая загрузка и порог
`memory.high`	«Превышен порог памяти» (оранжевая)	текущее использование и порог
`login.attempt`	«Успешный вход» (зелёная) или «Неудачный вход» (красная)	имя пользователя, IP, время, причина неудачи; пароль не передаётся

Одно и то же событие от одного источника (например, падение одного outbound) отправляется не чаще раза в минуту; попытки входа не ограничиваются. В подписи карточки указывается имя хоста сервера.

Отчёт по расписанию и бэкап

Отчёт «📊 Отчёт о состоянии 3x-ui» отправляется по расписанию `discordRunTime`, пока бот включён и Xray запущен. В нём: хост, версии панели и Xray (с состоянием ядра), время работы, средняя нагрузка, память, сетевой трафик (↑, ↓ и всего), соединения TCP/UDP, число клиентов онлайн, счётчики inbound-ов и клиентов в формате «Всего | Почти исчерпано | Отключено», а также IPv4/IPv6-адреса интерфейсов. «Почти исчерпанными» считаются те же inbound-ы и клиенты, что в блоке

«Скоро конец» Telegram-бота: пороги берутся из настроек `expireDiff` и `trafficDiff`. Пустое расписание заменяется на `@daily`; выражение, которое планировщик не принимает, в расписание не попадает, а в журнале появляется предупреждение `failed to schedule runtime`.

Если включено **Резервное копирование базы данных**, следом отдельным сообщением приходят файл базы данных (`<домен или IP>_ГГГГ-ММ-ДД_ЧЧММСС.db`, для PostgreSQL — `.dump`) и `config.json`. Отчёт и файлы отправляются разными сообщениями: если база превысит лимит Discord на размер вложений, отклонено будет только сообщение с бэкапом, а отчёт дойдёт.

Команды

Команды — обычные текстовые сообщения в канале из поля **ID канала**, а не зарегистрированные slash-команды Discord. Префикс — `!` или `/` (`/status` равнозначно `!status`), регистр не важен. Бот выполняет команду, только если её автор указан в **ID администраторов**; сообщения других пользователей и ботов, а также неизвестные команды он молча игнорирует.

Команда	Что делает
`!status`	Карточка «⚡ Состояние сервера 3x-ui»: версии панели и Xray, время работы, нагрузка, память, клиенты онлайн, соединения TCP/UDP, отправлено и получено.
`!report`	Сразу отправляет отчёт, как по расписанию; бэкап прикладывается, только если включено **Резервное копирование базы данных**.
`!backup`	Отправляет файл базы данных и `config.json` — всегда, независимо от `discordBotBackup`.
`!usage <email>`	Карточка клиента: статус, загрузка, скачивание, всего использовано, лимит и срок действия (дата, число дней для режима «после первого использования» или «Безлимит»). Email сравнивается без учёта регистра. Без аргумента бот подсказывает формат команды, для неизвестного email сообщает, что клиент не найден.
`!inbounds`	Список inbound-ов: протокол, порт, число клиентов, трафик и состояние. Длинный список разбивается на несколько сообщений в пределах лимитов Discord.
`!restart`	Перезапускает ядро Xray: «🔄 Перезапуск ядра Xray…», затем «✅ Ядро Xray успешно перезапущено.» или текст ошибки.
`!help`	Список команд (то же делает `!start`).

> **Внимание.** `!backup` и бэкапы в отчётах публикуют в канал всю базу данных панели. Используйте приватный канал, который видят только администраторы.

Соединение с Gateway и неполадки

При обрыве соединения с Gateway панель переподключается через 5 секунд; если Discord перестаёт отвечать на heartbeat-сообщения, «зависшее» соединение закрывается и тоже устанавливается заново. Если Discord закрывает соединение кодом 4004 (неверный токен) или 4010–4014 (в том числе 4014 — не включён Message Content Intent), панель больше не переподключается и пишет в журнал `Discord Gateway closed for good`. Соединение восстановится после перезапуска панели или после сохранения настроек,

которое перезапускает бота: новый токен или ID канала, изменённое расписание, выключение и повторное включение бота.

- **«… (401): invalid bot token»** — неверный токен: скопирован Client Secret или Application ID вместо токена с вкладки **Bot**.
- **«… (403): bot lacks permissions for channel»** — у бота нет прав в канале: проверьте **View Channels**, **Send Messages**, **Embed Links** и **Attach Files** для его роли.
- **«… (404): channel not found»** — неверный ID канала или бот не добавлен на сервер, которому принадлежит канал.
- **Бот не отвечает на команды** — проверьте, что ваш ID есть в **ID администраторов**, команда отправлена в канал из **ID канала**, а в Developer Portal включён **Message Content Intent**; после включения intent перезапустите панель.
- **Discord недоступен с сервера** — задайте **Исходящий для трафика панели** (`panelOutbound`): через него идут и REST-запросы, и соединение с Gateway.

---

15. Гео-базы (geoip / geosite и кастомные)

Гео-базы — это бинарные файлы `.dat`, которые Xray-core использует для маршрутизации и фильтрации трафика по принадлежности к стране (IP-диапазоны) или по категории доменов. Панель умеет загружать и обновлять как стандартный набор гео-файлов, так и произвольные пользовательские источники, указанные по URL. Все файлы хранятся в каталоге `bin` рядом с бинарником Xray (путь по умолчанию `bin`, переопределяется переменной окружения `XUI_BIN_FOLDER`).

15.1. Что такое geoip.dat и geosite.dat

- **geoip.dat** — база соответствий «IP-адрес → код страны/региона». Применяется в правилах маршрутизации в виде `geoip:<код>`, например `geoip:ru`, `geoip:cn`, а также для специальных меток `geoip:private` (приватные/локальные сети). По смыслу это ответ на вопрос «в какой стране находится этот IP».
- **geosite.dat** — база соответствий «домен → категория/список». Применяется в виде `geosite:<категория>`, например `geosite:category-ads-all` (рекламные домены), `geosite:google`, `geosite:ru`. По смыслу это сгруппированные списки доменов.

Эти файлы нужны, чтобы строить правила вида «весь трафик на российские IP/домены — напрямую, остальное — через outbound» и подобные. Сами правила задаются в разделе маршрутизации Xray; гео-базы лишь поставляют для них данные. Без актуальных гео-файлов правила, ссылающиеся на `geoip:` / `geosite:`, не сработают или будут опираться на устаревшие списки.

Пример: правило «российские домены и IP — напрямую». Такое правило в разделе маршрутизации направляет весь трафик к российским ресурсам в outbound с тегом `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Стандартные гео-файлы и их обновление

Панель содержит фиксированный «белый список» (allowlist) из шести стандартных файлов с жёстко прописанными источниками загрузки. Обновление выполняется через `POST /panel/api/server/updateGeofile/:fileName` (или без имени файла — для обновления всех сразу).

Пример: обновление одного файла и всех сразу через API. Обновить только `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
  -H 'Cookie: 3x-ui=<session-cookie>'
```

Обновить все шесть стандартных файлов одним запросом (имя файла не указано):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
  -H 'Cookie: 3x-ui=<session-cookie>'
```

Успешный ответ:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Имя файла	Источник (репозиторий релизов)
`geoip.dat`	`github.com/Loyalsoldier/v2ray-rules-dat` (geoip.dat)
`geosite.dat`	`github.com/Loyalsoldier/v2ray-rules-dat` (geosite.dat)
`geoip_IR.dat`	`github.com/chocolate4u/Iran-v2ray-rules` (geoip.dat)
`geosite_IR.dat`	`github.com/chocolate4u/Iran-v2ray-rules` (geosite.dat)
`geoip_RU.dat`	`github.com/runetfreedom/russia-v2ray-rules-dat` (geoip.dat)
`geosite_RU.dat`	`github.com/runetfreedom/russia-v2ray-rules-dat` (geosite.dat)

Особенности обновления стандартных файлов:

- **Кнопка обновления одного файла.** Перед загрузкой выводится подтверждение: «Вы действительно хотите обновить геофайл?» с пояснением «Это обновит файл #filename#.» (англ. *Do you really want to update the geofile? This will update the #filename# file.*). При успехе всплывает уведомление «Геофайлы успешно обновлены» (англ. *Geofile updated successfully*).
- **Кнопка «Обновить все»** (англ. *Update all*) загружает все шесть файлов. Подтверждение: «Это обновит все геофайлы.» (англ. *This will update all geofiles.*).
- **Условная загрузка.** Если локальный файл уже есть, в запрос подставляется заголовок `If-Modified-Since` с временем модификации файла. Ответ сервера `304 Not Modified` означает, что файл не менялся — повторно он не скачивается, обновляется только метка времени файла.
- **Проверка SHA-256.** Все три источника публикуют рядом с каждым `.dat` файл `<имя>.sha256sum`. Панель сначала узнаёт тег текущего релиза (по перенаправлению ссылки `releases/latest/download/…`) и затем скачивает и файл, и его контрольную сумму из **этого** релиза — чтобы они не оказались из разных релизов, если источник выпустит новый прямо во время загрузки. Нужная строка в `.sha256sum` ищется по базовому имени файла (источники записывают и `geoip.dat`, и `release/geoip.dat`). При несовпадении файл не устанавливается, а ошибка сообщает *«does not match the published SHA-256 checksum, so the download is corrupted or has been tampered with»* с ожидаемой и полученной суммой; если `.sha256sum` не удалось получить или в нём нет строки для файла, файл тоже не ставится.
- **Всё или ничего в пределах источника.** Скачанные файлы сначала складываются во временную папку внутри `bin` и переносятся на место только после проверки. Файлы одного репозитория (пара geoip/geosite) устанавливаются вместе: если один не скачался или не прошёл проверку, из этого репозитория не ставится ничего. Остальные репозитории обрабатываются независимо, их ошибки собираются в одно сообщение.
- **Безопасность имени файла.** Принимаются только имена из allowlist; имя проверяется на отсутствие `..`, разделителей пути `/` и `\`, абсолютных путей и обязано соответствовать шаблону `^[a-zA-Z0-9._-]+\.dat$`. Любое имя вне списка отклоняется с ошибкой «Invalid geofile name».

- **Перезапуск Xray.** Xray-core перезапускается, только если хотя бы один файл действительно установлен. Если все источники ответили `304 Not Modified` или ни один файл не прошёл проверку, перезапуска нет и активные соединения не рвутся. Если перезапуск не удался, в сообщение об ошибке добавляется соответствующая строка.

Обновление гео-баз из командной строки (x-ui)

Гео-базы можно обновлять и без панели — через интерактивное меню `x-ui` (пункт обновления гео-файлов) либо неинтерактивной командой `x-ui update-all-geofiles`. Для каждого файла набора (geoip/geosite, включая наборы IR и RU) выводится отдельный статус: «обновлён», «уже актуален» или «ошибка загрузки». При неудачной загрузке ложное сообщение об успехе не печатается. Перезапуск Xray (а значит и разрыв активных соединений) происходит только если хотя бы один файл действительно был обновлён; если ни один файл не изменился (все вернули `304 Not Modified`), панель и Xray не перезапускаются.

15.3. Авто-обновление гео-данных средствами Xray (Geodata Auto-Update)

Дополнительные `.dat`-источники по произвольному URL добавляются не средствами панели, а через нативную секцию `geodata` Xray-core. Соответствующий раздел вынесен в модальное окно обновлений Xray (Дашборд → обновления Xray, `xrayUpdates`) — это вкладка «Автообновление Geodata» (англ. *Geodata Auto-Update*). Панель здесь лишь редактирует ключ `geodata` в шаблоне конфигурации Xray; скачиванием, проверкой и горячей перезагрузкой файлов занимается само ядро Xray.

В верхней части раздела показана подсказка: «Xray скачивает эти файлы по расписанию и перезагружает их без перезапуска. URL должны быть HTTPS. Файл должен уже существовать в папке bin, прежде чем Xray сможет его обновлять.» (англ. *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Поля раздела

- **Расписание (cron)** (англ. *Schedule (cron)*) — строка cron из 5 полей; значение по умолчанию `0 4 * * *` (ежедневно в 04:00). При сохранении проверяется, что строка содержит ровно 5 полей, иначе выводится ошибка «Cron должен содержать 5 полей, напр. 0 4 * * *».
- **Скачивать через outbound (необязательно)** (англ. *Download through outbound (optional)*) — раскрывающийся список с тегами доступных outbound (плюс outbound подписок), через который Xray будет загружать файлы; outbound с протоколом `blackhole` в список не попадают. Поле можно оставить пустым — тогда используется прямое подключение. Этот выбор не зависит от outbound для собственных запросов панели (см. §11): у авто-обновления geodata свой отдельный outbound для загрузки.
- **Список файлов** — каждая строка задаёт пару «URL + Имя файла» (англ. *File name*). URL должен начинаться с `https://` (иначе «Для каждого файла нужен HTTPS URL.»). Имя файла указывается простым, без путей и разделителей — только символы `^[A-Za-z0-9._-]+$` (иначе «Имя файла должно быть простым, например geosite_custom.dat (без путей).»). При вводе URL панель пытается подставить имя файла автоматически из последнего сегмента пути. Кнопка «Добавить файл» (англ. *Add file*) добавляет строку, кнопка-корзина удаляет её.
- **Использовать штатные источники** (англ. *Use standard sources*) — добавляет в список шесть стандартных файлов из §15.2 (`geoip.dat`, `geosite.dat`, `geoip_IR.dat`, `geosite_IR.dat`, `geoip_RU.dat`,

`geosite_RU.dat`) со ссылками вида `https://github.com/<репозиторий>/releases/latest/download/geoip.dat` (или `geosite.dat`). Добавляются только файлы, которых в списке ещё нет (сравнение по имени файла); уже заданные строки, в том числе собственные источники, не заменяются. Кнопка неактивна, если панель не прислала список штатных источников. Чтобы изменения вступили в силу, нажмите «Сохранить и перезапустить Xray».

Если список пуст, показывается подсказка: «Файлы не настроены. В правилах маршрутизации файлы указываются как ext:geosite_custom.dat:category.» (англ. *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*).

Сохранение

Кнопка «Сохранить и перезапустить Xray» (англ. *Save & Restart Xray*) выводит подтверждение «Сохранить настройки geodata?» с пояснением «Шаблон конфигурации Xray будет обновлён, а Xray перезапущен.» (англ. *Save geodata settings? This updates the Xray config template and restarts Xray.*). После сохранения ключ `geodata` записывается в шаблон конфигурации (`POST /panel/api/xray/update`) и Xray перезапускается (`POST /panel/api/server/restartXrayService`). Если список файлов пуст, ключ `geodata` из шаблона удаляется.

Важные особенности:

- **Файл должен уже существовать в `bin`.** Xray обновляет только те `.dat`-файлы, которые на момент запуска уже присутствуют в папке `bin`. Поэтому новый кастомный файл сначала кладут в `bin` вручную (или хотя бы создают там пустую/устаревшую версию под нужным именем), и лишь затем Xray начинает поддерживать его в актуальном состоянии по расписанию.
- **Горячая перезагрузка.** После планового скачивания Xray перечитывает обновлённые базы без полного перезапуска процесса.
- **Совместимость.** Ранее скачанные гео-файлы (как стандартные, так и кастомные) продолжают работать в правилах маршрутизации с синтаксисом `ext:` без изменений.

Если список пуст, отображается подсказка: «Пользовательских источников geo пока нет — нажмите «Добавить», чтобы создать» (англ. *No custom geo sources yet — click Add to create one*).

Колонки таблицы и поля источника

Поле (UI)	JSON	Значение по умолчанию	Описание
Тип (*Type*)	`type`	— (обязательно)	Тип ресурса: только `geosite` или `geoip`. Определяет имя итогового файла.
Псевдоним (*Alias*)	`alias`	— (обязательно)	Короткий идентификатор источника. Из него и типа строится имя файла.
URL (*URL*)	`url`	— (обязательно)	Прямая ссылка на `.dat`-файл (http/https).
Включено (*Enabled*)	—	—	Признак активности источника в списке.
Обновлено (*Last updated*)	`lastUpdatedAt`	`0`	Время последнего успешного обновления (Unix-время; `0` — ещё не обновлялся).

Поле (UI)	JSON	Значение по умолчанию	Описание
Маршрутизация (ext:...) (*Routing (ext:...)*)	—	—	Готовая строка для правил маршрутизации: `ext:<файл.dat>:tag`.
Действия (*Actions*)	—	—	Кнопки «Изменить», «Удалить», «Обновить сейчас».

Дополнительно в БД хранятся служебные поля: `localPath` (фактический путь к файлу в каталоге `bin`), `lastModified` (значение заголовка `Last-Modified` от сервера, используется для условной загрузки), `createdAt` и `updatedAt`.

Именование файлов

Имя итогового файла формируется автоматически из типа и псевдонима:

- тип `geoip` → `geoip_<alias>.dat`;
- тип `geosite` → `geosite_<alias>.dat`.

Например, источник с типом `geosite` и псевдонимом `myads` создаст файл `geosite_myads.dat`.

Пример: добавление источника через API. Добавить свой список рекламных доменов как `geosite`-ресурс с псевдонимом `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
  -H 'Cookie: 3x-ui=<session-cookie>' \
  -H 'Content-Type: application/json' \
  -d '{
    "type": "geosite",
    "alias": "myads",
    "url": "https://example.com/lists/myads.dat"
  }'
```

Панель скачает файл в каталог `bin` как `geosite_myads.dat`, сохранит запись и перезапустит Xray.

Кнопки и действия

- **Добавить** (англ. *Add*) — открывает форму «Добавить источник» (англ. *Add custom geo*). Кнопка сохранения — «Сохранить» (англ. *Save*). API: `POST /add`.
- **Изменить** (англ. *Edit*) — форма «Изменить источник» (англ. *Edit custom geo*). API: `POST /update/:id`. При смене типа или псевдонима старый файл удаляется, новый скачивается заново.
- **Удалить** (англ. *Delete*) — подтверждение «Удалить этот пользовательский источник?» (англ. *Delete this custom geo source?*). Удаляет запись из БД и сам `.dat`-файл. API: `POST /delete/:id`. При успехе: «Пользовательский geo-файл «<имя>» удалён».
- **Обновить сейчас** (англ. *Update now*) — перекачивает конкретный источник и обновляет метку времени. API: `POST /download/:id`. При успехе: «Geofile «<имя>» обновлён».
- **Обновить все** — обновляет все пользовательские источники сразу. API: `POST /update-all`. При полном успехе: «Все пользовательские источники обновлены» (англ. *All custom geo sources updated*). Если хотя бы один источник не обновился, операция считается частично неуспешной с сообщением

«Не удалось обновить один или несколько пользовательских источников» (англ. *One or more custom geo sources failed to update*), а в ответе перечисляются успешные и неуспешные источники.

После любого из действий (добавление, изменение, удаление, обновление, обновление всех при наличии успехов) Xray-core перезапускается.

Пошагово: добавление источника

1. Нажмите «Добавить».
2. В поле «Тип» выберите `geosite` или `geoip`.
3. В поле «Псевдоним» введите идентификатор (только строчные латинские буквы, цифры, `-` и `_`; подсказка-плейсхолдер: `a-z 0-9 _ -`).
4. В поле «URL» укажите прямую ссылку на `.dat`-файл (должна начинаться с `http://` или `https://`).
5. Нажмите «Сохранить». Панель сразу скачает файл в каталог `bin`, сохранит запись и перезапустит Xray.

15.4. Просмотр содержимого гео-баз (3.7.0)

Раньше гео-базы можно было только обновлять — что внутри `geosite.dat`, приходилось смотреть сторонними утилитами. Теперь панель показывает их сама, в том числе прямо из формы правила маршрутизации:

- **Список файлов** в папке Xray с числом категорий, размером и датой обновления (`GET /panel/api/xray/geodata/files`). Пусто — «No geo databases in the Xray folder»: файлы появляются после того, как Xray скачает `geosite.dat` и `geoip.dat`.
- **Список категорий** файла (`/geodata/categories`) и **постраничный просмотр записей** (`/geodata/entries`) — доменов или подсетей, с поиском и счётчиком «Showing {from}–{to} of {total}».
- **Проверка значений** против базы (`POST /panel/api/xray/geodata/validate`): панель отвечает не «неверно», а конкретно — категории нет в базе, атрибута нет (правило не совпадёт ни с чем), не тот тип базы для этого поля, файла базы нет.

Ограничения: повреждённый или не тот файл распознаётся («Damaged or not a geosite/geoip database»), а слишком большой для просмотра — отклоняется («Too large to browse»).

15.5. Валидация и ограничения

При создании и изменении источника выполняются строгие проверки. Сообщения об ошибках:

Условие	Сообщение (RU)	Сообщение (EN)
Тип не `geosite` / `geoip`	Тип должен быть geosite или geoip	*Type must be geosite or geoip*
Пустой псевдоним	Укажите псевдоним	*Alias is required*
Недопустимые символы в псевдониме (не `^[a-z0-9_-]+$`)	Псевдоним содержит недопустимые символы	*Alias must match allowed characters*
Псевдоним зарезервирован	Этот псевдоним зарезервирован	*This alias is reserved*
Пустой URL	Укажите URL	*URL is required*

Условие	Сообщение (RU)	Сообщение (EN)
URL не парсится	Некорректный URL	*URL is invalid*
Схема не http/https	URL должен использовать http или https	*URL must use http or https*
Пустой/некорректный хост, либо заблокирован SSRF-защитой	Некорректный хост URL	*URL host is invalid*
Дубликат «тип + псевдоним»	Такой псевдоним уже используется для этого типа	*This alias is already used for this type*
Источник не найден	Источник не найден	*Custom geo source not found*
Ошибка скачивания	Ошибка загрузки	*Download failed*

Подсказки в форме (валидация на клиенте): «Псевдоним: только a-z, цифры, - и _» (*Alias may only contain lowercase letters, digits, - and _*) и «URL должен начинаться с http:// или https://» (*URL must start with http:// or https://*).

Дополнительные технические ограничения:

- **Зарезервированные псевдонимы.** Нельзя использовать псевдонимы, конфликтующие со стандартными файлами. Зарезервированы (сравнение регистронезависимое, дефис приравнивается к подчёркиванию): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Например, `geosite-ru` будет отклонён как `geosite_ru`.
- **SSRF-защита.** Хост URL резолвится в IP, и если он указывает на приватный/внутренний адрес, загрузка блокируется (пользователь видит «Некорректный хост URL»). Это не даёт использовать панель для обращений к внутренним сервисам.
- **Защита от path traversal.** Конечный путь файла обязан находиться внутри каталога `bin` (с разрешением симлинков); попытка выйти за его пределы отклоняется.
- **Минимальный размер файла.** Скачанный файл считается валидным только если он не меньше 64 байт; слишком маленький файл отвергается с ошибкой загрузки.
- **Прокси и условная загрузка.** Если в настройках панели задан прокси, скачивание идёт через него; в остальных случаях используется прямое соединение с SSRF-безопасным транспортом. Как и для стандартных файлов, применяется `If-Modified-Since` / `304 Not Modified` (повторно неизменённый файл не качается). Таймаут загрузки — 10 минут, проба доступности URL (HEAD, при необходимости — частичный GET) — 12 секунд.

15.6. Автопроверка при старте панели

При запуске панель проходит по всем пользовательским источникам и для каждого проверяет наличие и целостность локального файла (файл отсутствует, является каталогом или меньше 64 байт). Если файл отсутствует или повреждён, выполняется проба источника и попытка повторной загрузки. Это гарантирует, что после переустановки или потери каталога `bin` пользовательские гео-файлы будут восстановлены автоматически.

15.7. Использование гео-баз в правилах маршрутизации

В правилах маршрутизации Xray гео-базы используются в полях вроде `domain` / `ip` через префиксы:

- **geoip:** для IP-баз — `geoip:<код>`. Примеры: `geoip:ru`, `geoip:cn`, `geoip:private`. Берётся из `geoip.dat` (или `geoip_RU.dat` и т. п., если правило указывает на конкретный файл).
- **geosite:** для доменных баз — `geosite:<категория>`. Примеры: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Берётся из `geosite.dat`.

Пример: блокировка рекламы через geosite. Правило, отправляющее все рекламные домены в «чёрную дыру» (предполагается outbound с тегом `blocked` и протоколом `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Для **пользовательских** файлов используется синтаксис внешнего файла `ext:`. Подсказка в UI: «В правилах маршрутизации используйте значение как ext:файл.dat:тег (замените тег).» (англ. *In routing rules use the value column as ext:file.dat:tag (replace tag).*). Формат:

```
ext:<имя_файла.dat>:<тег>
```

где `<имя_файла.dat>` — это `geoip_<alias>.dat` или `geosite_<alias>.dat`, а `<тег>` — конкретный список/категория внутри файла. Панель в колонке «Маршрутизация (ext:...)» подсказывает готовый шаблон вида `ext:geosite_myads.dat:tag` — остаётся заменить `tag` на нужный тег. Имя такого файла задаётся в разделе «Автообновление Geodata» (см. §15.3) в поле «Имя файла» — например `geosite_custom.dat`; на него ссылаются в правилах как `ext:geosite_custom.dat:category`.

Пример: правило на основе пользовательского файла. Если добавлен источник типа `geosite` с псевдонимом `myads`, а внутри `.dat`-файла список помечен тегом `ads`, правило маршрутизации выглядит так:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Для IP-источника (тип `geoip`, псевдоним `mycorp`, тег `office`) поле будет `"ip": ["ext:geoip_mycorp.dat:office"]`.

---

16. Эксплуатация: бэкапы, логи, обновление, CLI

Этот раздел охватывает повседневное обслуживание панели: создание и восстановление резервных копий базы данных, просмотр журналов (логов) панели и Xray, перезапуск и остановку сервисов, обновление Xray и самой панели, периодические задачи (cron) и удаление панели. Часть операций выполняется из веб-интерфейса (вкладки на странице «Дашборд» и «Настройки панели»), часть — из консольного меню `x-ui` на сервере.

16.1. Резервное копирование и восстановление базы данных

Все данные панели (входящие/inbound, клиенты, группы, узлы, настройки) хранятся в одной базе. Управление бэкапами доступно на странице **«Дашборд»** во вкладке **«Резервная копия»**, заголовок блока — **«Бэкап и восстановление»**. С 3.9.0 восстановление из SQL-дампа изолировано: оно идёт на одном соединении с запретом присоединять другие БД, поэтому вредоносный или повреждённый дамп не может открыть или создать посторонний файл.

Панель поддерживает два движка БД, и поведение бэкапа от этого зависит:

- **SQLite** (вариант по умолчанию) — данные лежат в файле `x-ui.db`.
- **PostgreSQL** — если панель настроена на PostgreSQL, в блоке отображается подсказка:

> «Эта панель работает на PostgreSQL. „Резервная копия“ скачивает архив pg_dump (.dump), а „Восстановление“ загружает его обратно через pg_restore. Восстановление также принимает базу SQLite (.db) или миграционный дамп SQLite и импортирует их данные в PostgreSQL. На сервере должны быть установлены клиентские инструменты PostgreSQL (pg_dump и pg_restore).»

Экспорт (создание копии)

Кнопка **«Экспорт базы данных»** (англ. `Back Up`) скачивает файл резервной копии на ваше устройство.

Движок БД	Имя файла	Что происходит на сервере
SQLite	`{host}_ГГГГ-ММ-ДД_ЧЧММСС.db`	Сначала выполняется checkpoint WAL, чтобы файл содержал последние записи, затем файл целиком читается и отдаётся на скачивание
PostgreSQL	`{host}_ГГГГ-ММ-ДД_ЧЧММСС.dump`	Запускается `pg_dump`, архив отдаётся на скачивание

С версии 3.4.2 имя скачиваемого файла резервной копии содержит адрес, по которому вы открыли панель, и **дату-время**: `{host}_ГГГГ-ММ-ДД_ЧЧММСС` с расширением `.db` (SQLite) или `.dump` (PostgreSQL) — например `panel.example.com_2026-06-27_000000.db`. Так файлы группируются по серверу и сортируются по времени, а несколько копий за один день не перезаписывают друг друга. То же имя используется в бэкапах, которые присылает бот Telegram (`host` берётся из домена/IP панели, запасной вариант — `x-ui`).

Подсказки в интерфейсе:

- SQLite: «Нажмите, чтобы скачать файл .db, содержащий резервную копию вашей текущей базы данных на ваше устройство.»
- PostgreSQL: «Нажмите, чтобы скачать дамп PostgreSQL (.dump) текущей базы данных на ваше устройство.»

Технически экспорт — это запрос `GET /panel/api/server/getDb`. Имя вложения формируется сервером (`Content-Disposition`) в зависимости от движка.

Имя файла резервной копии формируется по адресу сервера, а не фиксированным `x-ui.db` / `x-ui.dump`. При скачивании из браузера оно берётся из адреса панели в адресной строке (хост запроса), иначе — из настроенного веб-домена, а при его отсутствии — из публичного IP сервера (сначала IPv4, затем IPv6), с откатом на `x-ui`. Так бэкапы с разных серверов легко различать. Расширение остаётся `.db` для SQLite и `.dump` для PostgreSQL; бэкапы через Telegram именуются по тому же домену/IP.

Пример: скачивание бэкапа через API. Тот же экспорт можно получить запросом из консоли — например, для скрипта автоматического резервного копирования. Нужна авторизованная сессия (cookie входа):

```
# 1) Логинимся и сохраняем cookie сессии
curl -s -c cookies.txt \
     -d 'username=admin&password=admin' \
     https://panel.example.com:2053/panel/login

# 2) Скачиваем файл базы (имя задаёт сервер: x-ui.db или x-ui.dump)
curl -s -b cookies.txt -OJ \
     https://panel.example.com:2053/panel/api/server/getDb
```

Если панель открыта по базовому пути (Web Base Path), его нужно добавить в URL: `…:2053/<base_path>/panel/api/server/getDb`.

Импорт (восстановление)

Кнопка **«Импорт базы данных»** (англ. `Restore`) открывает выбор файла и загружает его на сервер для восстановления (`POST /panel/api/server/importDB`, поле формы `db`).

Подсказки в интерфейсе:

- SQLite: «Нажмите, чтобы выбрать и загрузить файл .db или миграционный дамп (.dump) с вашего устройства для восстановления базы данных.»
- PostgreSQL: «Нажмите, чтобы выбрать и загрузить бэкап PostgreSQL (.dump), базу SQLite (.db) или миграционный дамп SQLite для восстановления базы данных. Это заменит все текущие данные.»

С 3.5.0 тип файла определяется по содержимому (`PGDMP` — архив pg_dump; заголовок «SQLite format 3» — база `.db`; ведущие `PRAGMA` / `BEGIN TRANSACTION` — текстовый SQL-дамп), а не по расширению; диалог выбора принимает `.dump,.db` на обеих СУБД. SQLite-файлы, загруженные на PostgreSQL-панель, импортируются в PostgreSQL одной транзакцией (см. 3.14); перед импортом файл проверяется как настоящая панельная БД, старые схемы автоматически домигрируются. Перед восстановлением `pg_dump`-архива панель заранее (до остановки Xray) проверяет его читаемость: если дамп создан более новым PostgreSQL, выводится ошибка с точной командой — например, «…run 'x-ui pgclient 17'…».

Процесс импорта для SQLite (важно понимать, что он атомарный и с откатом):

1. Загруженный файл проверяется на формат — это должна быть валидная база SQLite; иначе возвращается ошибка «Invalid db file format».
2. Файл сохраняется во временный `x-ui.db.temp` и проходит проверку целостности.
3. **Xray останавливается** перед заменой БД.
4. Текущая база переименовывается в резервную `x-ui.db.backup` (fallback).
5. Временный файл перемещается на место рабочей БД, выполняется инициализация и миграции схемы, затем миграция inbound.
6. **Если любой шаг завершился ошибкой** — выполняется откат: восстанавливается прежняя база из `x-ui.db.backup`, и Xray перезапускается на старых данных.
7. При успехе fallback-файл удаляется, и **Xray автоматически перезапускается** уже на восстановленных данных.

Сообщения интерфейса по итогу:

Результат	Текст
Успех	«База данных успешно импортирована»
Ошибка импорта	«Произошла ошибка при импорте базы данных»
Ошибка чтения файла	«Произошла ошибка при чтении базы данных»

> Восстановление полностью заменяет текущие данные. Поскольку Xray в процессе кратковременно останавливается, существующие подключения клиентов на время импорта прерываются. Через 3 секунды после успешного импорта (на SQLite и на PostgreSQL) перезапускается и сама панель, чтобы пути подписок и другие параметры из загруженного файла, читаемые только при старте, сразу вступили в силу; страница ждёт 5 секунд и перезагружается (см. 3.14).

Файл миграции между движками (SQLite ⇄ PostgreSQL)

Отдельно от обычного бэкапа есть функция **«Скачать файл миграции»** (`Download Migration`, запрос `GET /panel/api/server/getMigration`). Она формирует переносимый файл для перехода на другой движок БД:

С 3.5.0 кнопка показывается **только на PostgreSQL-панелях** (на SQLite обычный бэкап `.db` теперь восстанавливается прямо в PostgreSQL — отдельный экспорт не нужен):

Текущий движок	Что скачивается	Имя файла	Назначение
PostgreSQL	База SQLite, собранная из данных PostgreSQL	`x-ui.db`	Перевести панель обратно на SQLite

Подсказка: «Нажмите, чтобы скачать базу данных SQLite (.db), собранную из ваших данных PostgreSQL и готовую для запуска панели на SQLite.»

Преобразование `.db ⇄ .dump` для SQLite можно выполнить и из CLI командой `x-ui migrateDB [file]` (см. раздел 16.7). Также с 3.5.0 миграция между движками стала **транзакционной и без потерь** (переносятся в том числе группы клиентов и глобальные счётчики трафика; неудачный импорт не меняет целевую БД), а

для обновления клиентских инструментов PostgreSQL появилась команда **`x-ui pgclient [версия]`** и пункт **10. Install/Upgrade client tools (pg_dump/pg_restore)** в меню PostgreSQL (при необходимости подключается официальный репозиторий PostgreSQL).

Бэкап через Telegram-бот

Если настроен Telegram-бот (см. раздел про уведомления), он умеет присылать резервную копию прямо в чат администратора. Бэкап через Telegram включает **два файла**: саму базу данных (`x-ui.db`, либо `x-ui.dump` на PostgreSQL) и конфигурацию Xray `config.json`. Сообщению предшествует строка «🗓 Время резервного копирования: …».

Есть два способа получить бэкап в Telegram:

1. **По запросу.** Кнопка **«📂 Бэкап БД»** в меню бота — бот немедленно отправляет файлы в текущий чат.
2. **Автоматически с отчётом.** В настройках бота есть переключатель **«Резервное копирование базы данных»** (`Database Backup`) с описанием «Отправлять уведомление с файлом резервной копии базы данных». Когда он включён, при каждой периодической рассылке отчёта бот после отчёта присылает резервную копию всем администраторам. Период отправки отчёта задаётся расписанием cron бота (см. раздел 16.6). Между файлами и между администраторами бот делает паузы, чтобы не превышать лимиты Telegram.

> Бэкап через бота отправляется только если бот запущен; на PostgreSQL он также требует наличия `pg_dump` на сервере.

Бэкап через Discord-бот

Те же два файла — базу (с именем по домену или IP сервера) и `config.json` — присылает и Discord-бот (см. 14.8): немедленно по команде `!backup` (или `/backup`) в настроенном канале, а при включённом переключателе **«Резервное копирование базы данных»** (`discordBotBackup`, по умолчанию выключен) — ещё и после каждого периодического отчёта (расписание «Частота уведомлений от Discord-бота», по умолчанию `@daily`) и по команде `!report`. Команды принимаются только от пользователей из списка «ID администраторов». Файлы уходят отдельным сообщением после отчёта, поэтому бэкап, не укладывающийся в лимит вложений Discord, не мешает доставке самого отчёта.

16.2. Просмотр логов

В панели два независимых средства просмотра логов, оба открываются с вкладки **«Логи»** на «Дашборде». Каждое окно умеет обновляться (иконка «обновить» в заголовке) и скачивать показанное в файл `x-ui.log` (кнопка с иконкой загрузки). С 3.9.0 просмотр системного лога ограничен 15-секундным таймаутом `journalctl` (#6689), а меню `x-ui` читает состояние службы через `systemctl show` вместо разбора журнала — на хостах с большими логами меню и просмотр больше не подвисают (#6629).

Логи панели (приложения / syslog)

Окно логов панели (`POST /panel/api/server/logs/{count}`). Элементы управления:

Элемент	Значение по умолчанию	Описание
Количество строк	`20`	Выпадающий список: 20 / 50 / 100 / 500 / 1000
Уровень	`Info`	Минимальный уровень: Debug / Info / Notice / Warning / Error
SysLog (галочка)	выключено	Откуда брать логи: из буфера приложения или из системного журнала
Автообновление (галочка)	выключено	Перечитывать лог каждые 5 секунд (см. ниже)

Поведение зависит от галочки **SysLog**:

- **Выключено (по умолчанию):** логи берутся из внутреннего кольцевого буфера панели, отфильтрованные по выбранному уровню. Записи отображаются с уровнем (DEBUG / INFO / NOTICE / WARNING / ERROR) и источником: `X-UI:` — сообщения самой панели, `XRAY:` — проброшенные сообщения Xray.

> Простые уведомления без штампа времени и уровня (например, системное сообщение «Syslog is not supported» на Windows) теперь показываются целиком, как есть. Распознаётся строго формат `YYYY/MM/DD LEVEL — тело`; всё прочее выводится без разбора, поэтому такие строки больше не обрезаются (раньше первые три слова ошибочно трактовались как дата/время/уровень).

- **Включено:** панель выполняет на сервере `journalctl -u x-ui --no-pager -n <count> -p <level>`, то есть показывает системный журнал службы `x-ui`. Допустимое количество строк — от 1 до 10000; уровень принимает значения syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). На Windows режим SysLog не поддерживается — будет показано предупреждение, что нужно снять галочку и использовать логи приложения. Если `systemd`/служба недоступны, появится сообщение об ошибке запуска `journalctl`.

Пример: тот же журнал из консоли сервера. Когда панель недоступна (например, не стартует), системный журнал можно прочитать напрямую — это ровно та команда, которую панель выполняет в режиме SysLog:

```
# последние 100 строк уровня warning и выше
journalctl -u x-ui --no-pager -n 100 -p warning

# следить за журналом в реальном времени
journalctl -u x-ui -f
```

> Уровень в этом окне фильтрует **вывод**. Какой минимальный уровень вообще пишется в консоль/syslog, определяется уровнем логирования панели (переменная окружения, по умолчанию `Info`; в файл панель всегда пишет на уровне `DEBUG`).

Логи доступа Xray (журнал доступа)

Отдельное окно для access-лога Xray (`POST /panel/api/server/xraylogs/{count}`). Оно разбирает строки журнала доступа Xray и показывает их таблицей: **Date, From, To, Inbound, Outbound, Email**.

Начиная с 3.4.1 это окно и кнопка его вызова на карточке статуса Xray подписаны **«Логи доступа»** (`Access Logs`) — раньше они назывались просто «Логи». Переименование сделано, чтобы не путать просмотрщик access-лога Xray с просмотрщиком логов самой панели, у которого прежде было такое же название.

Элемент	Значение по умолчанию	Описание
Количество строк	`20`	20 / 50 / 100 / 500 / 1000
Фильтр	пусто	Текстовый поиск по подстроке (применяется по нажатию Enter)
Автообновление (галочка)	выключено	Перечитывать лог каждые 5 секунд (см. ниже)
Direct (галочка)	включено	Показывать прямые соединения (трафик через freedom-outbound)
Blocked (галочка)	включено	Показывать заблокированные соединения (трафик в blackhole-outbound)
Proxy (галочка)	включено	Показывать проксированный трафик

Тип события определяется автоматически по тегу исходящего соединения в строке лога: соответствие тегам freedom → «DIRECT» (зелёный), blackhole → «BLOCKED» (красный), всё остальное → «PROXY» (синий). Строки `api -> api` и пустые строки пропускаются.

Автообновление. В обоих окнах логов (и «Логи», и «Логи доступа») есть флажок **«Автообновление»** (`Auto Update`). Если его включить, содержимое лога автоматически перечитывается каждые 5 секунд с сохранением всех текущих настроек окна — выбранного числа строк, уровня/фильтра и галочек Direct / Blocked / Proxy. Опрос прекращается, как только окно закрыто или флажок снят.

> Чтобы это окно показывало записи, у Xray должен быть включён **журнал доступа** с путём к файлу (не `none`) — см. ниже. Если access-лог отключён или файл недоступен, окно будет пустым («No Record…»).

Отчёт о сбое ядра Xray

Если ядро Xray аварийно завершается, панель сохраняет отчёт в файл `core_crash_<дата>_<время>_<суффикс>.log` в каталоге логов панели (`XUI_LOG_FOLDER` , в Linux по умолчанию `/var/log/x-ui`), а строку с причиной показывает в результате Xray. С 3.10.2 отчёт пишется целиком — от заголовка `panic:` или `fatal error:` до конца трассировки (раньше в файл попадали только первые 7 байт «panic: »); у каждого отчёта своё имя, поэтому сбои в одну секунду не затирают друг друга. Обычные строки лога, где встречаются слова panic, exception или stack trace, за сбой больше не принимаются и остаются в логе.

Отказы CSRF в логе панели

Запрос из браузера, отклонённый CSRF-проверкой (нет токена или он неверный), получает ответ 403 и оставляет в логе панели предупреждение: `CSRF validation failed, request rejected: method=POST path=… IP=203.0.113.45`. Адрес `IP` определяется по списку доверенных прокси (см. 2.3), поэтому за настроенным обратным прокси в нём виден реальный клиент, а не прокси. Типичные причины: устаревшая вкладка после истечения сессии и скрипт, который ходит с сессионной cookie без CSRF-токена (для скриптов используйте API-токен: запросы с Bearer-токеном CSRF-проверку не проходят). Найти записи: `journalctl –u x–ui | grep 'CSRF validation failed'` (#6692).

16.3. Уровень и настройка логирования Xray

Параметры журналирования самого Xray задаются на странице **«Конфигурации Xray»** в блоке **«Лог»** (`Log`) с предупреждением:

> «Логи могут замедлять работу сервера. Включайте только нужные вам виды логов при необходимости!»

Поле	Перевод	Значение по умолчанию	Описание
Уровень логов (`logLevel`)	Log Level	`warning`	Уровень детализации лога ошибок Xray. Допустимые значения: `debug` , `info` , `notice` , `warning` , `error` . Подсказка: «Уровень журнала для журналов ошибок, указывающий информацию, которую необходимо записать.»
Логи доступа (`accessLog`)	Access Log	`none`	Путь к файлу журнала доступа. Спецзначение `none` отключает логи доступа. Подсказка: «Путь к файлу журнала доступа. Специальное значение „none“ отключает логи доступа.»
Логи ошибок (`errorLog`)	Error Log	пусто (путь по умолчанию)	Путь к файлу логов ошибок; `none` отключает их. Подсказка: «Путь к файлу логов ошибок. Специальное значение „none“ отключает логи ошибок.»
Логи DNS (`dnsLog`)	DNS Log	`false` (выкл.)	Включить логирование DNS-запросов. Подсказка: «Включить логи запросов DNS».
Маскировка адреса (`maskAddress`)	Mask Address	пусто (выкл.)	При активации реальный IP-адрес автоматически заменяется на маскировочный в логах. Подсказка: «При активации реальный IP-адрес заменяется на маскировочный в логах.»

> Так как по умолчанию **«Логи доступа» =** `none` , окно «Логи Xray» (раздел 16.2) изначально пусто. Чтобы оно заработало, задайте здесь путь к access-логу и перезапустите Xray.

> Обратите внимание: пустой access-лог влияет только на это окно. Список онлайн-клиентов на «Дашборде» и лимит количества IP в форме клиента **не зависят** от access-лога — панель определяет онлайн-клиентов и считает их IP-адреса через online-stats API ядра Xray (статистику по соединениям).

> На старых версиях ядра, где этого API нет, панель автоматически возвращается к прежнему способу (чтение access-лога), и тогда для лимита IP путь к access-логу здесь по-прежнему нужен.

> **Лимит по количеству IP и fail2ban.** Само ограничение по количеству IP у клиента (поле «IP Limit» в форме клиента и при массовом добавлении) применяется на сервере только если установлен **fail2ban** — именно он банит превышающие лимит адреса. Панель проверяет наличие fail2ban (`GET /panel/api/server/fail2banStatus`); если его нет, поле «IP Limit» становится недоступным с поясняющей подсказкой (на Windows — отдельным сообщением), а ранее заданные лимиты на таких серверах автоматически обнуляются, поскольку всё равно не действовали. Блокировка fail2ban распространяется и на TCP, и на UDP. Если применение лимитов выключено (`XUI_ENABLE_FAIL2BAN=false`), панель всё равно продолжает **записывать** IP-адреса клиентов, и список адресов в карточке клиента остаётся актуальным; отключается только блокировка (#6830). С 3.5.0 «мёртвое» соединение (клиент пропал без корректного закрытия TCP) фиксируется **один раз**, а не при каждом 10-секундном сканировании: строка `[LIMIT_IP]` и цикл отключения повторяются только при реальном переподключении, поэтому счётчик fail2ban не накручивается и завышать `maxretry` больше не нужно (формат лога и failregex не менялись). На обычных серверах fail2ban теперь ставится автоматически при установке и обновлении панели (см. раздел 16.5).

Пример: блок `log` , при котором окно «Логи Xray» начнёт показывать записи. В JSON-конфигурации Xray это выглядит так:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Главное — заменить `"access": "none"` на путь к файлу (например, `"./access.log"`). После сохранения перезапустите Xray, и таблица в окне «Логи Xray» наполнится строками.

16.4. Управление Xray: остановка и перезапуск

Состоянием Xray управляют с карточки Xray на «Дашборде». Текущее состояние отображается одним из значений: **Запущен** (`Running`), **Остановлен** (`Stopped`), **Неизвестно** (`Unknown`), **Ошибка** (`Error`). При ошибке доступна всплывающая подсказка «Ошибка при запуске Xray».

Кнопка	Перевод	Эндпоинт	Действие
Стоп	`Stop`	`POST /panel/api/server/stopXrayService`	Останавливает процесс Xray. При успехе — уведомление-предупреждение «Xray service has been stopped».

Кнопка	Перевод	Эндпоинт	Действие
Перезапуск	`Restart`	`POST /panel/api/server/restartXrayService`	Перезапускает (или запускает) Xray с применением текущей конфигурации. При успехе — уведомление «Xray service has been restarted successfully».

После любой из операций панель рассылает новое состояние по WebSocket, поэтому статус на «Дашборде» обновляется без перезагрузки страницы. Если операция завершилась ошибкой, состояние Xray становится «Ошибка», и текст ошибки попадает в уведомление.

> Помимо ручного перезапуска, панель сама проверяет, не нужно ли перезапустить Xray (фоновая задача каждые 30 с) и не упал ли процесс (проверка каждую секунду) — см. раздел 16.6.

Монитор здоровья туннеля (автоперезапуск Xray)

В 3.4.1 появился необязательный **монитор здоровья туннеля**. Если он включён, панель периодически проверяет доступность заданного URL и после нескольких неудачных проверок подряд автоматически перезапускает ядро Xray — это помогает восстановить туннель, переставший пропускать трафик. По умолчанию монитор **отключён** и настраивается **только переменными окружения службы** (в веб-интерфейсе его настроек нет — так задумано авторами).

Включает монитор переменная `XUI_TUNNEL_HEALTH_MONITOR=true`. Переменную `XUI_TUNNEL_HEALTH_PROXY` следует направить на локальный xray-inbound (например `socks5://127.0.0.1:1080`) — тогда проба идёт через сам Xray и проверяет именно туннель; без неё проверяется лишь связность хоста, и перезапуск не починит проблему сетевого подключения сервера. Остальные переменные задают параметры проверки:

Переменная	Назначение	По умолчанию
`XUI_TUNNEL_HEALTH_MONITOR`	Включить монитор (вкл/выкл)	`false`
`XUI_TUNNEL_HEALTH_PROXY`	Прокси, через который идёт проба (укажите локальный xray-inbound)	пусто
`XUI_TUNNEL_HEALTH_URL`	URL, который проверяется	`https://www.cloudflare.com/cdn-cgi/trace`
`XUI_TUNNEL_HEALTH_INTERVAL`	Интервал между проверками	`30s`
`XUI_TUNNEL_HEALTH_TIMEOUT`	Таймаут одной проверки	`10s`
`XUI_TUNNEL_HEALTH_FAILURES`	Число неудач подряд до перезапуска	`3`
`XUI_TUNNEL_HEALTH_COOLDOWN`	Минимальная пауза между перезапусками	`5m`

> Перезапуск Xray разрывает соединения всех подключённых клиентов, поэтому имеет смысл оставлять интервал и порог числа неудач достаточно большими, чтобы случайный сбой одной пробы не приводил к лишним перезапускам.

16.5. Перезапуск и обновление панели

Перезапуск панели

На странице **«Настройки панели»** есть действие **«Перезапустить панель»** (`Restart Panel` , `POST /panel/api/setting/restartPanel`). При подтверждении панель перезапускается **через 3 секунды**.

Сообщения:

- Подтверждение: «Вы уверены, что хотите перезапустить панель? Подтвердите, и перезапуск произойдёт через 3 секунды. Если панель будет недоступна, проверьте лог сервера.»
- Успех: «Панель успешно перезапущена».

Технически на Linux перезапуск выполняется отправкой сигнала `SIGHUP` процессу панели (либо через зарегистрированный хук). На Windows отправка `SIGHUP` не поддерживается.

Самообновление панели (Update Panel)

На «Дашборде» доступна функция **«Обновить панель»** (`Update Panel`) — обновление 3X-UI до последнего релиза прямо из веб-интерфейса.

Перед обновлением панель сверяет версии (`GET /panel/api/server/getPanelUpdateInfo`), запрашивая последний релиз 3x-ui с GitHub:

Поле	Перевод
Текущая версия панели	Current panel version
Последняя версия панели	Latest panel version
Панель обновлена / «Обновлено»	Panel is up to date / Up to date — показывается, если новой версии нет

Запуск обновления — `POST /panel/api/server/updatePanel` . Диалог подтверждения:

- «Вы действительно хотите обновить панель?»
- «Это обновит 3X-UI до версии #version# и перезапустит сервис панели.»

После запуска — всплывающее сообщение «Обновление панели началось» (`Panel update started`); при неудачной проверке версий — «Проверка обновления панели не удалась» (`Panel update check failed`).

Что происходит на сервере: самообновление поддерживается **только на Linux** (на других ОС вернётся ошибка «panel web update is supported only on Linux installations»). Панель скачивает официальный скрипт `update.sh` с GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) и запускает его в отдельном процессе: предпочтительно через `systemd-run` в отдельном юните (`x-ui-web-update-<timestamp>`), а при отсутствии systemd — как отдельный отсоединённый процесс. По завершении скрипт обновляет компоненты и перезапускает службу панели. Для запуска требуется `bash` . Перед распаковкой `update.sh` сверяет SHA-256 архива с опубликованным файлом `.sha256` : несовпадение или неудачная загрузка суммы прерывают обновление ещё до остановки панели, а у старых релизов без файла суммы выводится только предупреждение (подробнее — в 1.3). Скрипт меню `x-ui.sh` , `x-ui.rc` и юниты службы берутся из тега устанавливаемого релиза (для dev-канала — из `main`); если нужного файла для тега нет,

обновление отменяется, а текущая установка остаётся нетронутой. Процесс `mtg` (MTProto) перед заменой файлов останавливается — после старта панель запускает его заново (TUIC с 3.9.0 встроен в панель, отдельного процесса нет). Запасной путь (отдельный процесс без `systemd-run`) внутри песочницы службы (с 3.10.0, см. ниже) работать не может: дочерний процесс наследует её ограничения. Поэтому `update.sh` заранее проверяет запись в каталог установки и `/usr/bin` и при отказе сразу останавливается с сообщением «ERROR: <каталог> is not writable for this process …», советуя обновиться из веб-интерфейса (через `systemd-run`) или командой `x-ui update` в консоли.

Миграция базы до запуска службы (с 3.10.0). `update.sh` выполняет `x-ui migrate` (сообщение «Migrating database…») **до** запуска службы — и на systemd, и на OpenRC (#6729). Раньше служба стартовала, и одновременно с ней команды скрипта (`x-ui setting -show`, `x-ui migrate`) прогоняли те же миграции схемы на той же базе; на обновлении с новыми колонками или таблицами один из процессов падал — например, при переходе 3.8.5 → 3.9.0 служба останавливалась с ошибкой «duplicate column name: exclude_from_sub» и поднималась лишь через 5 секунд благодаря `Restart=on-failure`.

Обновление не трогает TLS (с 3.10.0). Раньше на панели без сертификата `update.sh` открывал меню SSL. При веб-обновлении ответить на вопросы некому: пустой ответ выбирал выпуск сертификата Let's Encrypt для IP, панель останавливалась, а при неудаче удалялись `/root/cert/ip` и состояние acme.sh, которыми могли пользоваться и другие службы; если не удавалось определить публичный IP, скрипт бесконечно ждал его ввода. Теперь обновление только обновляет панель: настройка SSL из `update.sh` убрана, IP для итоговой ссылки определяется без вопросов, а «Access URL» показывает реальную схему — `https://` только при настроенном сертификате. Без сертификата выводится «No SSL certificate is configured; the panel is served over HTTP.» с подсказкой настроить его через `x-ui` → «SSL Certificate Management». TLS по-прежнему настраивается из меню `x-ui` (пункт 20) и при установке.

Если в ходе обновления скрипт сгенерировал новый случайный базовый путь веб-панели (Web Base Path), служба `x-ui` перезапускается автоматически, чтобы новый путь сразу заработал. (Без перезапуска сервер продолжал бы отдавать старый путь, а интерфейс показывал бы новый, и новый адрес был бы недоступен до ручного перезапуска.)

Dev-канал (rolling), с 3.4.2. Кнопка версии панели на «Дашборде» теперь всегда открывает окно обновления (раньше на актуальной стабильной сборке она вела на страницу релизов GitHub), а переключатель **«Dev-канал»** (`devChannelEnable`) показывается всегда — даже на стабильной сборке. Включив его, при следующем обновлении можно перейти на «катящийся» dev-канал (сборки по каждому коммиту в `main`). При включении выводится предупреждение: «Dev-сборки отслеживают каждый коммит в main и не являются стабильными релизами — автоматического отката нет». Без явного действия пользователя ничего не обновляется.

Канал обновления Dev (rolling-сборки по коммитам)

Помимо обычного обновления до стабильного релиза, есть необязательный **«Канал разработки»** (`Dev`). Переключатель появляется в окне обновления панели **только на dev-сборках** (CI-сборки, собранные по отдельному коммиту); на стабильных релизах его не видно. При включении панель будет обновляться до rolling-сборки `dev-latest`, которая отслеживает каждый коммит ветки `main` и не является стабильным релизом — показывается предупреждение, что dev-сборки нестабильны и автоматического отката нет. В режиме dev окно показывает «Текущий коммит» / «Последний коммит» вместо номеров версий. Функция доступна только на Linux с systemd.

На dev-сборках панель показывает свою версию как `dev+<короткий-коммит>` вместо вводящего в заблуждение стабильного номера — в бейдже боковой панели, на карточке «Дашборда», в окне обновления, в отчёте Telegram-бота о статусе и в выводе команды `x-ui -v`. На стабильных релизах вид версии не меняется.

> На узлах (нодах) панель этого же 3x-ui обновляется централизованно через `POST /panel/api/nodes/updatePanel` — см. раздел про узлы. Токену, под которым центральная панель ходит на узел, для этого нужна область `admin` или `node-admin` (см. 13.9).

Остановка службы

Поставляемые юниты `x-ui.service` (Debian/Ubuntu, RHEL, Arch) используют `KillMode=mixed`: при `systemctl stop` и `restart` сигнал `SIGTERM` получает только процесс панели, а она сама останавливает Xray и вспомогательные процессы (например, `mtg`). Всё, что осталось по истечении таймаута остановки, systemd добивает `SIGKILL`. Штатная остановка больше не пишет в журнал ложную ошибку «Server shutdown failed: use of closed network connection | xray is not running» (#6884).

Песочница службы systemd (с 3.10.0)

Поставляемые юниты `x-ui.service` (варианты для Debian/Ubuntu, RHEL и Arch) раньше запускали панель от root без каких-либо ограничений (`systemd-analyze security` оценивал их в 9.6 UNSAFE). С 3.10.0 панель по-прежнему работает от root, но в песочнице (#6718); те же ограничения наследуют её дочерние процессы — Xray, `mtg` и всё, что они запускают. На Alpine (OpenRC), в Docker и на Windows ничего не меняется. Что это значит на практике:

- **Запись в файловую систему.** `ProtectSystem=full` делает `/usr`, `/boot`, `/efi` и `/etc` доступными только для чтения, `ProtectHome=read-only` — домашние каталоги (`/root`, `/home`); читать сертификаты из `/root/cert` панель по-прежнему может. Исключения (`ReadWritePaths`) — каталоги панели: по умолчанию `/etc/x-ui`, `/usr/local/x-ui` и `/var/log/x-ui`. Остальное (`/var`, `/tmp`, `/srv`, отдельные диски) доступно на запись как раньше; `PrivateTmp` намеренно не включён. Если лог Xray или другой файл, который пишет панель или её дочерний процесс, лежит в `/etc`, `/usr`, `/root` или `/home`, запись туда перестанет работать — перенесите файл или добавьте путь в свой drop-in.
- **Перенесённые хранилища.** Каталоги из `XUI_DB_FOLDER`, `XUI_LOG_FOLDER`, `XUI_BIN_FOLDER` (и `XUI_MAIN_FOLDER`) из файла окружения службы `install.sh` и `update.sh` записывают в drop-in `/etc/systemd/system/x-ui.service.d/10-xui-sandbox.conf` и перегенерируют его при каждой установке и обновлении. Поменяв такую переменную позже, запустите `x-ui update` или переустановку, иначе новый каталог в защищённом дереве останется только для чтения. Каталог с пробелами в имени в список не попадает (скрипт предупреждает об этом), а `%` в имени экранируется.
- **Свои изменения** держите в отдельном drop-in, например `/etc/systemd/system/x-ui.service.d/20-x-ui-local.conf`: сам юнит и `10-xui-sandbox.conf` перезаписываются при каждом обновлении, и ручные правки в них теряются.
- **Права root ограничены.** `CapabilityBoundingSet` оставляет только `CAP_NET_ADMIN`, `CAP_NET_BIND_SERVICE`, `CAP_NET_RAW` и `CAP_DAC_OVERRIDE` (последняя — чтобы читать чужие приватные ключи TLS, например выпущенные Caddy или acme.sh); `NoNewPrivileges=yes` не даёт повышать права через setuid-программы (например, `sudo` из дочернего процесса);

`RestrictAddressFamilies` разрешает только сокеты `AF_INET`, `AF_INET6`, `AF_UNIX` и `AF_NETLINK`. Также включены защита параметров и модулей ядра, журнала ядра, часов и имени хоста, `RestrictNamespaces`, `RestrictSUIDSGID`, `LockPersonality` и `RestrictRealtime`.

- **`/proc/sys` — только для чтения, кроме `/proc/sys/net`.** Его WireGuard-outbound в режиме TUN ядра использует при старте Xray для отключения `rp_filter`; без этого исключения ядро не запускалось с ошибкой «failed to disable ipv4 rp_filter for all: … read-only file system» (#6796). Если TUN ядра создать всё же нельзя (нет `/dev/net/tun` или `/proc/sys/net` закрыт вашим drop-in'ом), панель переводит WireGuard-outbound'ы на пользовательский TUN (`noKernelTun`), а не роняет Xray.
- **`UMask=0077`**: новые файлы панели и Xray (логи, конфигурация, база) доступны только владельцу (root). Сторонний агент, читавший, например, access-лог от имени другого пользователя, этот доступ потеряет.
- **Фильтр системных вызовов** `SystemCallFilter=@system-service` записывается в drop-in только при systemd 239 и новее — на более старом systemd панель с ним не запустилась бы. Директивы, неизвестные вашему systemd, он пропускает; установщик и `update.sh` в этом случае выводят «Note: systemd <версия> ignores part of the hardening in x-ui.service; the panel still starts.» со списком неприменённых защит (на systemd старше 231, например CentOS 7, запись в каталоги панели обеспечивает `ReadWriteDirectories=`).
- **Удаление** панели (`x-ui uninstall`) убирает каталог `x-ui.service.d` целиком, вместе с вашими drop-in'ами.

Автоматическая установка fail2ban

Чтобы лимит по количеству IP у клиентов (раздел 16.3) работал из коробки, при установке и обновлении панели на обычном сервере `fail2ban` теперь устанавливается и настраивается автоматически (прежде это происходило только в Docker-образе). Поведением управляет переменная окружения `XUI_ENABLE_FAIL2BAN`: настройка выполняется, если переменная не задана или равна `true`. Ручной запуск доступен командой `x-ui setup-fail2ban`. Сбой настройки fail2ban не прерывает установку или обновление панели. На Debian 12+ и Ubuntu 22.04+ скрипт переключает общий бэкенд fail2ban на `systemd` отдельным файлом `/etc/fail2ban/jail.d/3x-ipl-backend.conf` (секция `[DEFAULT]`) вместо правки пакетного `/etc/fail2ban/jail.conf` — такая настройка переживает обновление пакета fail2ban. Файл создаётся, только если в `jail.conf` осталось штатное `backend = auto`, поэтому собственный бэкенд администратора не перетирается; сам jail `3x-ipl` по-прежнему использует `backend=auto`. При установке старой версии, чей `x-ui.sh` ещё не знает команды `setup-fail2ban` (до 3.4.0), автонастройка пропускается с сообщением «This x-ui.sh predates 'x-ui setup-fail2ban'; skipping Fail2ban auto-setup.».

Установка и обновление на IPv6-only хостах

Скрипты `install.sh` и `update.sh` теперь корректно работают на серверах только с IPv6: загрузка релиза, скрипта `x-ui.sh` и сервис-файлов больше не использует принудительно IPv4 (`curl -4`), а берёт доступный протокол. Поэтому панель можно установить и обновить и на хосте без IPv4-адреса.

Исправления скриптов в 3.5.0

- **RHEL-семейство** (Rocky/Alma/RHEL/Oracle): локальная установка PostgreSQL из меню теперь работает (аутентификация по паролю вместо ident), а fail2ban для лимита IP ставится из **EPEL**.
- **Arch/Manjaro**: установка и обновление больше не запускают полный апгрейд системы `pacman -Syu` — только обновление базы пакетов.

- **32-битный ARM**: скачанный бинарь Xray именуется `xray-linux-arm32` — именно так, как его запускает панель (раньше обновление клало файл под другим именем, и панель продолжала гонять старое ядро).
- **IP-сертификат**: автоопределённый публичный IPv4 показывается для подтверждения (Enter — принять) перед выпуском; при отказе — проверенный ручной ввод.
- **Выбор ACME-порта**: Enter (принять порт 80 по умолчанию) больше не выводит ложное «Your input is invalid».

Переопределение порта панели переменной `XUI_PORT`

Порт прослушивания веб-панели можно переопределить переменной окружения `XUI_PORT` — она действует только на время работы текущего процесса и **не изменяет** сохранённое значение `webPort` в базе. Допустимы значения от `1` до `65535`; пустое, некорректное или выходящее за диапазон значение игнорируется (используется `webPort`) с предупреждением в логе. Это удобно при развёртывании, в первую очередь в Docker: при использовании bridge-сети публикуемый порт контейнера должен совпадать с `XUI_PORT` — например, `XUI_PORT=8080` и `ports: "8080:8080"`.

Обновление и переключение версии Xray-core

Здесь же на «Дашборде» можно управлять версией Xray-core отдельно от панели.

- **Обновления Xray** (`Xray Updates`) / **Выбор версии** (`Version`) — выпадающий список доступных версий. Подсказки: «Выберите нужную версию» и предупреждение «Важно: старые версии могут не поддерживать текущие настройки».
- Установка/смена версии — `POST /panel/api/server/installXray/{version}`. Диалог: «Переключить версию Xray» / «Вы точно хотите сменить версию Xray?». При успехе — «Xray успешно обновлён».

Пример: смена версии Xray-core запросом к API. Версия указывается тегом релиза из XTLS/Xray-core (с префиксом `v`). Например, переключиться на `v1.8.24`:

```
curl -s -b cookies.txt -X POST \
     https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — файл с cookie из примера в разделе 16.1.) После установки Xray перезапустится автоматически с выбранной версией.

На сервере при смене версии Xray сначала останавливается, скачивается архив нужной версии с GitHub (XTLS/Xray-core), бинарник распаковывается и заменяется, после чего Xray перезапускается с проверкой контрольных размеров архива/бинарника.

16.6. Периодические задачи (cron)

Панель регистрирует ряд фоновых задач при запуске. Их расписания фиксированы (не настраиваются в UI, за исключением расписаний Telegram- и Discord-отчётов и LDAP-синхронизации). Ниже — задачи, относящиеся к эксплуатации.

Задача	Расписание	Назначение
Проверка работы Xray	каждую 1 с	Контроль, что процесс Xray запущен
Проверка необходимости перезапуска Xray	каждые 30 с	Перезапуск, если конфигурация помечена как изменённая
Сбор трафика Xray	каждые 5 с (старт через 5 с после запуска)	Учёт трафика inbound/клиентов
Проверка IP клиентов	каждые 10 с	Контроль лимита IP по логу
Heartbeat и синхронизация трафика узлов	каждые 5 с	Обмен с узлами (нодами)
Очистка логов	**ежедневно** (`@daily`)	Очищает IP-limit логи и persistent access-log, ротируя текущий лог в `*.prev.log` . С 3.5.0 ежедневная очистка затрагивает **и error-лог Xray** (раньше — только access)
Ограничение роста логов Xray	**каждые 10 минут** (`@every 10m`)	**Новое в 3.5.0.** Обрезает access- и error-лог Xray, когда любой из них превышает **64 МиБ**; отключённые логи (`none` /пусто) не трогаются
Сброс трафика по периоду	`@hourly` , `@daily` , `@weekly` , `@monthly`	Сбрасывает счётчики трафика тех inbound (и их клиентов), у которых задан соответствующий период автосброса
Telegram-отчёт	задаётся в настройках бота (по умолчанию `@daily`)	Рассылка отчёта администраторам; при включённой опции — с приложенной резервной копией БД (раздел 16.1)
Discord-отчёт	«Частота уведомлений от Discord-бота» в настройках Discord-бота (по умолчанию `@daily`)	Только при включённом Discord-боте и запущенном Xray: отчёт в канал; при включённом «Резервное копирование базы данных» — следом бэкап БД и `config.json` (раздел 16.1)
Сброс hash-хранилища Telegram	каждые 2 м	Только при включённом боте
Контроль нагрузки CPU для Telegram	каждые 10 с	Только если задан порог CPU > 0

Дополнительно:

- **Периодический сброс трафика** срабатывает только для тех inbound, у которых выбран соответствующий режим автосброса (ежечасно/ежедневно/еженедельно/ежемесячно). Задача сбрасывает трафик самого inbound и всех его клиентов.
- **Проверка истечения и исчерпания.** Отключение клиентов по истечении срока и при исчерпании лимита трафика выполняется в рамках учёта трафика: клиенты с истёкшим `expiry_time` либо исчерпанным объёмом помечаются и отключаются, при необходимости рассчитывается следующий срок (для циклических лимитов и режима «отсчёт с первого использования»). На «Дашборде» и в списках это отражается статусами «Истекло»/«Исчерпано»/«Скоро закончится».
- **Автоматический бэкап в Telegram** — это побочный эффект задачи отчёта, отдельного cron-расписания только для бэкапа нет. Поэтому частота автобэкапа равна частоте отчёта бота. Так же устроен автобэкап Discord-бота.

16.7. Консольное меню и CLI (`x-ui`)

На сервере панель управляется командой `x-ui`. Без аргументов открывается интерактивное меню «3X-UI Panel Management Script»; с аргументом выполняется конкретная подкоманда. Пункты меню, относящиеся к эксплуатации:

№ в меню	Пункт	Действие
1	Install	Установка панели (скачивает и запускает `install.sh`)
2	Update	Обновление всех компонентов x-ui до последней версии без потери данных; после — автоматический перезапуск
3	Update to Dev Channel (latest commit)	Обновление до rolling-сборки `dev-latest` (последний коммит ветки `main`) с подтверждением (см. 16.5)
4	Update Menu	Обновление только самого скрипта меню `x-ui` — версии из тега установленной панели (см. 1.6)
5	Legacy Version	Установка указанной (старой) версии панели по введённому номеру (например, `2.4.0`)
6	Uninstall	Полное удаление панели и Xray (см. 16.8)
7	Reset Username & Password	Сброс логина/пароля администратора
8	Reset Web Base Path	Сброс базового пути веб-панели
9	Reset Settings	Сброс настроек к значениям по умолчанию
10	Change Port	Смена порта панели
11	View Current Settings	Просмотр текущих настроек
12–14	Start / Stop / Restart	Запуск, остановка, перезапуск службы панели
15	Restart Xray	Перезапуск только Xray
16	Check Status	Текущий статус службы
17	Logs Management	Просмотр и очистка логов (см. ниже)
18–19	Enable / Disable Autostart	Включить/выключить автозапуск службы при старте ОС
27	Update Geo Files	Обновление гео-файлов (GeoIP/GeoSite)
25	PostgreSQL Management	Управление PostgreSQL

> Нумерация пунктов меню изменилась в 3.4.1: из-за добавления пункта 3 «Update to Dev Channel» все последующие пункты сдвинулись на единицу. Всего пунктов стало 28, выбор вводится в диапазоне `[0-28]`.

Управление логами в CLI (пункт 17)

Подменю «Logs Management» теперь открывается пунктом **17** (раньше — 16):

- **Debug Log** — потоковый просмотр журнала службы: `journalctl -u x-ui -e --no-pager -f -p debug` (на Alpine — `grep` по `/var/log/messages`).
- **Clear All logs** — очистка системного журнала: `journalctl --rotate` + `journalctl --vacuum-time=1s`, после чего служба перезапускается. (Недоступно на Alpine.)

Прямые подкоманды `x-ui`

Все доступные подкоманды:

Команда	Описание
`x-ui`	Открыть административное меню
`x-ui start`	Запустить панель
`x-ui stop`	Остановить панель
`x-ui restart`	Перезапустить панель
`x-ui restart-xray`	Перезапустить Xray
`x-ui status`	Текущий статус
`x-ui settings`	Показать текущие настройки
`x-ui enable`	Включить автозапуск при старте ОС
`x-ui disable`	Отключить автозапуск
`x-ui log`	Просмотр логов
`x-ui banlog`	Просмотр логов банов Fail2ban
`x-ui setup-fail2ban`	Установить и настроить fail2ban для лимита IP (см. 16.5)
`x-ui update`	Обновить панель
`x-ui update-dev`	Обновить панель до канала разработки (rolling-сборка `dev-latest`)
`x-ui update-all-geofiles`	Обновить все гео-файлы (с последующим перезапуском)
`x-ui migrateDB [file]`	Преобразование базы `.db ⇄ .dump` (SQLite)
`x-ui legacy`	Установить устаревшую версию
`x-ui install`	Установить панель
`x-ui uninstall`	Удалить панель

> Команда `x-ui update` скачивает и запускает официальный `update.sh` (тот же, что и веб-обновление из раздела 16.5), запрашивая подтверждение: «This function will update all x-ui

> components to the latest version, and the data will not be lost.» По завершении панель автоматически перезапускается.

> **Флаги `-webCert` / `-webCertKey` в подкоманде `setting`.** Пути к сертификату и приватному ключу веб-панели можно задать прямо в подкоманде `/usr/local/x-ui/x-ui setting -webCert <путь> -webCertKey <путь>` — указание любого из этих флагов сохраняет соответствующий путь (как и отдельная подкоманда `cert`), и панель сразу переходит на HTTPS.

Установка модуля ядра AmneziaWG (`install-awg-kernel`)

Режим ядра у inbound'а AmneziaWG (см. 5.10) требует модуля ядра `amneziawg` и утилит `amneziawg-tools`. Установить их можно командой самого бинарника панели (не подкомандой скрипта меню), от root:

```
sudo /usr/local/x-ui/x-ui install-awg-kernel
```

Без root команда лишь выводит подсказку `run as root: sudo … install-awg-kernel`; вне Linux — сообщает, что модуль только для Linux. Встроенный скрипт рассчитан на Debian и Ubuntu (`apt`): ставит зависимости (`dkms`, `build-essential`, `git`, `libmnl-dev`, `pkg-config`, `nftables`, а для ядра, собранного clang, ещё LLVM), клонирует закреплённые версии модуля и утилит с проверкой коммитов, собирает модуль через DKMS, загружает его (`modprobe amneziawg`) и ставит `awg`. Нужны заголовки ядра `linux-headers-$(uname -r)`; сборка идёт минуты и без таймаута. DKMS пересобирает модуль при обновлении ядра, а на хостах с Secure Boot модуль нужно подписать. На других дистрибутивах модуль и утилиты ставятся вручную. Пока модуля нет, панель не позволяет включить режим ядра и называет причину.

Получение API-токена через CLI

Выпустить API-токен для скриптов можно командой самого бинарника панели. Это не подкоманда скрипта меню, поэтому бинарник вызывается по полному пути (в Docker — `/app/x-ui`):

```
/usr/local/x-ui/x-ui setting -getApiToken                    # перевыпустить токен cli-fallback
/usr/local/x-ui/x-ui setting -getApiToken -tokenName ci-bot   # перевыпустить токен с именем ci-bot
/usr/local/x-ui/x-ui setting -getApiToken -tokenName ci-bot -tokenScope monitor  # задать область токена (3.9.0)
```

Показать уже выданный токен команда не может: API-токены хранятся только в виде хешей. Что она делает:

- **Если токены уже есть**, команда сообщает их количество, советует управлять токенами в панели (**Settings** → **API Tokens**, см. 13.9) и **перевыпускает** токен с именем из `-tokenName` (по умолчанию `cli-fallback`): прежний токен с этим именем удаляется и сразу перестаёт работать, вместо него создаётся новый. Вывод — строка `The API token "<имя>" has been regenerated (any previous one is now invalid):` и затем `apiToken: <токен>`. Повторные вызовы с одним именем не накапливают токены.

- **Если токенов ещё нет** (свежая панель), создаётся токен с именем из `–tokenName`, а без флага — `install`. Именно его сохраняет установщик, и последующие вызовы без `–tokenName` его не трогают: они перевыпускают `cli–fallback`.
- Имя — не длиннее 64 символов. Разным скриптам давайте разные имена: вызовы с одним именем отзывают токены друг друга.
- **Область токена — флаг `–tokenScope`** (с 3.9.0): `admin`, `monitor`, `node–sync` или `node–admin`. При перевыпуске существующего токена область и срок действия **сохраняются** (раньше токен молча становился `admin`); новый токен без флага получает `admin`. Неизвестное значение отклоняется до удаления старого токена, поэтому опечатка не отзовёт перевыпускаемый токен. Уже истёкший токен не перевыпускается.
- Флаги пишутся **до** позиционных аргументов. В форме `–getApiToken true –tokenName ci–bot` разбор флагов останавливается на `true`: выводится предупреждение `warning: ignored "true –tokenName ci–bot" and any flags after it; put flags before positional arguments`, а перевыпущен будет `cli–fallback`.

16.8. Что изменилось в эксплуатации в 3.7.0

- **`/usr/local/x–ui/x–ui setting –getApiToken` больше не плодит токены.** Раньше каждый вызов создавал новый админ-токен, и в панели копились «забытые» ключи с полным доступом. Теперь команда **ротирует единственный токен `cli–fallback`**: выданные ранее CLI-токены после ротации перестают работать — если вы использовали их в скриптах, обновите значение.
- **Обновление панели не трогает ваши файлы в `bin/`.** Вручную положенный `geoip.dat` или другой файл переживает обновление.
- **`update.sh` ходит через прокси панели.** Если у панели настроен прокси, обновление скачивается через него — на серверах без прямого выхода наружу обновление больше не падает.
- **Наполовину применённая стартовая миграция больше не фиксируется молча.** Панель не поднимется с полусломанной схемой, делая вид, что всё в порядке, — ошибка будет видна сразу.
- **Перед обновлением на 3.7.0 сделайте бэкап базы:** при первом старте выполняются миграции (колонки цикла сброса трафика и времени последней выгрузки подписки у клиента, служебные колонки синхронизации узлов, индекс по `LOWER(email)`, область действия и срок у API-токенов, нормализация внешних ссылок).

16.9. Удаление панели

Удаление выполняется из CLI — пункт меню **6 (Uninstall)** или команда `x–ui uninstall`. Перед удалением запрашивается подтверждение (по умолчанию «нет»): «Are you sure you want to uninstall the panel? xray will also uninstalled!».

При подтверждении скрипт:

1. Останавливает службу и отключает её автозапуск (`systemctl stop/disable x–ui`, либо на Alpine — `rc–service` / `rc–update`), удаляет unit-файл службы и (с 3.10.0) каталог его drop-in'ов `x–ui.service.d` — вместе с вашими собственными drop-in'ами, — затем перезагружает конфигурацию systemd.
2. Удаляет каталоги данных и приложения (`/etc/x–ui/`, каталог установки) и env-файл службы (`/etc/default/x–ui`, `/etc/conf.d/x–ui` или `/etc/sysconfig/x–ui` — в зависимости от дистрибутива).

3. Удаляет сам скрипт `x-ui` и выводит сообщение «Uninstalled Successfully.», а также команду для повторной установки.

Если панель использовала PostgreSQL (в env-файле `XUI_DB_TYPE=postgres`), после удаления файлов панели скрипт дополнительно спрашивает, нужно ли удалить и сам сервер PostgreSQL вместе со всеми его базами: «Also purge PostgreSQL and delete all of its data?». Запрос требует явного подтверждения (по умолчанию — отказ) и сопровождается предупреждением: удаление затронет **ВСЕ** базы PostgreSQL на машине, в том числе принадлежащие другим приложениям, и необратимо. При отказе PostgreSQL и его данные остаются нетронутыми. С 3.10.0 на RHEL-семействе удаляются и пакеты PostgreSQL 18 из репозитория PGDG (`postgresql18*`).

> Удаление необратимо: вместе с панелью удаляется Xray и все данные (включая базу). Если данные могут понадобиться, заранее сделайте экспорт базы (раздел 16.1).

16.10. Команда `x-ui migrateDB`

Начиная с версии 3.3.0 управляющий скрипт `x-ui.sh` получил подкоманду `migrateDB` — обёртку вокруг встроенного бинарника `x-ui` (`x-ui migrate-db`) для конвертации базы данных панели SQLite между двумя форматами: бинарным `.db` и переносимым текстовым дампом `.dump` (обычный SQL-текст).

Что делает команда

Команда работает в двух направлениях, причём направление определяется **автоматически** по входному файлу:

Направление	Как называется	Что происходит
`.db → .dump`	dump (выгрузка)	бинарная база SQLite выгружается в текстовый SQL-файл
`.dump → .db`	restore (восстановление)	из текстового SQL-файла заново собирается бинарная база SQLite

Под капотом скрипт вызывает бинарник панели:

- выгрузка: `x-ui migrate-db --src <вход> --dump <выход>`
- восстановление: `x-ui migrate-db --restore <вход> --out <выход>`

Синтаксис вызова

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **`[file.db|file.dump]`** — входной файл (первый аргумент). Если его не указать, берётся установленная база панели по умолчанию: `/etc/x-ui/x-ui.db`.
- **`[output]`** — путь к выходному файлу (второй аргумент). Необязателен: при отсутствии имя выбирается автоматически рядом со входным файлом (см. ниже).

Примеры:

```
x-ui migrateDB                              # выгрузить /etc/x-ui/x-ui.db -> /etc/x-ui/
x-ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # собрать .db из дампа
```

Как определяется направление

Скрипт смотрит на расширение входного файла:

- `*.db`, `*.sqlite`, `*.sqlite3` → режим **dump** (выгрузка в текст);
- `*.dump`, `*.sql` → режим **restore** (сборка базы).

Если расширение не распознано, скрипт читает первые 16 байт файла: сигнатура `SQLite format 3` означает бинарную базу (режим dump), иначе файл считается дампом (режим restore).

Имя выходного файла, если второй аргумент не задан:

- при выгрузке — то же имя, что у входа, с расширением `.dump`;
- при восстановлении — то же имя с расширением `.db`.

Защитные проверки и поведение

- **Наличие бинарника.** Если бинарник `x-ui` не найден или не исполняемый — выводится ошибка «x-ui binary not found … Is the panel installed?».
- **Поддержка функции в сборке.** Скрипт проверяет, что бинарник умеет `migrate-db --dump/--restore` (через `x-ui migrate-db -h`). Если нет — предлагается сначала обновить панель командой `x-ui update`.
- **Существование входного файла.** При отсутствии входного файла печатается ошибка и строка с синтаксисом вызова.
- **Перезапись выхода.** Если выходной файл уже существует, запрашивается подтверждение (по умолчанию «нет»); без подтверждения операция отменяется. При восстановлении старый выходной файл предварительно удаляется.
- **Защита «живой» базы.** При восстановлении в базу по умолчанию `/etc/x-ui/x-ui.db`, когда панель запущена, операция отклоняется с требованием сначала остановить панель (`x-ui stop`) либо выбрать другой выходной путь. Это предотвращает перезапись рабочей базы у работающего сервиса.
- При неуспехе сборки базы недописанный выходной файл удаляется.
- **Права на дамп.** Текстовый `.dump` содержит в открытом виде те же секреты, что и база (UUID клиентов, приватные ключи REALITY, хеш пароля администратора), поэтому записывается с правами `0600` — прочитать его может только владелец.

Зачем это нужно

- **Бэкап.** Текстовый `.dump` — человекочитаемый, удобен для хранения в системах контроля версий и для дифференциального просмотра содержимого базы.
- **Перенос.** Дамп переносим между машинами и устойчив к различиям версий формата файла SQLite — на новом сервере из него собирается рабочая `.db`.

- **Диагностика.** Из `.dump` можно глазами посмотреть структуру и данные панели, не имея под рукой инструментов SQLite.

Интерактивный режим

Помимо прямого вызова, конвертация доступна из интерактивного меню. В подменю PostgreSQL (`x-ui` → раздел работы с PostgreSQL) есть пункт **9. Convert SQLite `.db <-> .dump`**: она спрашивает путь к входному файлу (по умолчанию `/etc/x-ui/x-ui.db`) и к выходному (можно оставить пустым для авто-именования), а направление, как и в CLI-режиме, определяется автоматически.

Документ подготовлен по исходному коду 3X-UI. Если какой-то пункт интерфейса в вашей версии отличается — приоритет у поведения панели и подсказок в самом UI.